



E-BOOK · 2026

POS ร้านอาหารสร้างเองด้วย Claude Code

คู่มือลงมือทำระบบ QR เมนู สั่งอาหาร จอครัว และชำระเงิน ครบวงจร (ฉบับ
2026)

ตั้งแต่ออกแบบฐานข้อมูลเมนู · QR ประจำโต๊ะ · จอครัว realtime · PromptPay และ
payment gateway ไทย · ไปกำกับภาษีอย่างย่อ · จนถึง deploy และอยู่กับร้านจริง

โดย GrowGenius Academy

สารบัญ

01 ระบบ POS ร้านอาหารทำงานยังไง และทำไมถึงสร้างเองได้แล้ว

02 ตั้ง Claude Code ให้ทำงานถูกตั้งแต่บรรทัดแรก

03 สถาปัตยกรรมทั้งระบบ และการตั้งโปรเจกต์

04 ออกแบบเมนูให้รองรับของจริง

05 โຕ้ะ QR และ session ของโຕ้ะ

06 หน้าเมนูฝั่งลูกค้า

07 ตะกร้า ส่งออร์เดอร์ และ state machine ของออร์เดอร์

08 จอครัว (KDS) แบบ realtime

09 หน้า POS ของพนักงาน

10 คิดเงินให้ถูกทุกบาท

11 รับเงิน: PromptPay บัตร และเงินสด

12 ใบเสร็จ ใบกำกับภาษีอย่างย่อ และ e-Tax

13 สิทธิ ความปลอดภัย และการกันโกงหน้าร้าน

14 สต็อกและต้นทุนอาหาร

15 รายงาน ปิดกะ ปิดวัน

16 Deploy และอยู่กับร้านจริง

ระบบ POS ร้านอาหารทำงานยังไง และทำไมถึงสร้างเองได้แล้ว



เที่ยงวันศุกร์ ร้านอาหาร 40 ที่นั่งเต็มทุกโต๊ะ พนักงานเสิร์ฟสองคนวิ่งจดออเดอร์ใส่กระดาษ มือหนึ่งถือปากกา มือหนึ่งถือถาดน้ำ ลายมือที่เขียนตอนรีบ ๆ กลายเป็นปัญหาตอนส่งเข้าครัว — “ผิดกะเพราหมูสับ” อ่านเป็น “ผิดกะเพราหมูกรอบ” ครัวทำผิด เสิร์ฟผิดโต๊ะ ลูกค้าคิ่จจาน พนักงานต้องวิ่งไปขอโทษพร้อมเคลียร์บิลใหม่ ส่วนแคชเชียร์ตอนปิดวันก็ต้องมานั่งกางใบเสร็จกระดาษไล่บวกเลขว่ายอดขายจริงตรงกับเงินในลิ้นชักไหม

ถ้าคุณเคยเห็นภาพนี้ ไม่ว่าจะในฐานะคนกินที่รอนาน คนเสิร์ฟที่วิ่งจนเหนื่อย หรือเจ้าของร้านที่นั่งกางบัญชีตอนดึก — นั่นคือเหตุผลที่ระบบ POS (Point of Sale) ร้านอาหารมีอยู่ และเป็นเหตุผลที่หนังสือเล่มนี้เขียนขึ้น

หนังสือเล่มนี้ไม่ได้สอนให้คุณ “เกลียด” ระบบสำเร็จรูปที่ขายอยู่ในตลาด ตรงกันข้าม เราจะพูดตรง ๆ ว่าเมื่อไหร่ควรซื้อ เมื่อไหร่ควรสร้างเอง และถ้าเลือกสร้างเอง จะสร้างให้ถูกต้องตั้งแต่การออกแบบฐานข้อมูลไปจนถึงการรับเงินจริงได้อย่างไร โดยใช้ Claude Code เป็นตัวเร่งความเร็ว แต่ไม่ใช่ตัวตัดสินใจแทนคุณ

POS ร้านอาหารจริง ๆ คือ 4 หน้าจอที่คุณกัน

คนทั่วไปมักนึกภาพ POS เป็นแค่ “เครื่องคิดเงิน” เครื่องเดียวที่วางอยู่หน้าร้าน แต่ระบบ POS ร้านอาหารที่ใช้งานได้จริงประกอบด้วย 4 ส่วนที่ต้องคุยกันแบบ realtime — ภาพนี้คือแกนกลางที่หนังสือเล่มนี้จะใช้อ้างอิง

หน้าจอ	ใครใช้	ทำอะไร
มือถือลูกค้า (QR)	ลูกค้าที่นั่งโต๊ะ	สแกน QR ประจำโต๊ะ เปิดเมนูดิจิทัล เลือกอาหาร ปรับตัวเลือก ส่งออเดอร์เอง
เครื่องพนักงาน (POS)	พนักงานเสิร์ฟ/แคชเชียร์	เปิดโต๊ะ สั่งแทนลูกค้าที่ไม่นัดสแกน ย้าย/รวมโต๊ะ คิดเงิน
จอครัว (KDS)	ครัว	รับออเดอร์แบบ realtime แยกตามสถานี กดเปลี่ยนสถานะเมื่อทำเสร็จ
หลังร้าน (admin/report)	เจ้าของ/ผู้จัดการ	ดูยอดขาย เมนูขายดี ปิดกะ ปิดวัน จัดการเมนูและสิทธิ์พนักงาน

จุดสำคัญที่ทำให้ระบบนี้ “เป็นระบบ” ไม่ใช่แค่แอปสี่ตัวที่แยกกันทำงาน คือทั้ง 4 หน้าจอต้องอ้างอิงข้อมูลชุดเดียวกัน — ออเดอร์ที่ลูกค้ากดส่งจากมือถือต้องไปโผล่ที่จอครัวภายในไม่กี่วินาที โดยพนักงานไม่ต้องคีย์ซ้ำ และเมื่อครัวกด “เสร็จ” สถานะต้องอัปเดตกลับไป POS ให้พนักงานรู้ว่าเสิร์ฟได้แล้ว ส่วนหลังร้านต้องเห็นทุกธุรกรรมแบบ real time พอที่จะปิดยอดได้โดยไม่ต้องรอสรุบมือ

ตลอดทั้งเล่ม เราจะสร้างระบบนี้ทีละขั้น: เริ่มจากสถาปัตยกรรมและฐานข้อมูล (บทที่ 3-4) ไปที่ฝั่งลูกค้า (บทที่ 5-7) ฝั่งครัวและหน้าร้าน (บทที่ 8-9) การคิดเงินและรับเงิน (บทที่ 10-12) ความปลอดภัยและสต็อก (บทที่ 13-14) จนถึงรายงานและ deploy จริง (บทที่ 15-16)

สิ่งที่ทำให้ 4 หน้าจอนี้ “คุยกันรู้เรื่อง” ในทางเทคนิคคือการมีสัญญาข้อมูล (data contract) ชุดเดียวกันที่ทุกหน้าจอเข้าใจตรงกัน ตัวอย่างง่าย ๆ ที่จะเห็นซ้ำตลอดทั้งเล่ม (รายละเอียดเต็มอยู่ในบทที่ 7-8) คือสถานะของออเดอร์หนึ่งใบ ที่ทั้งฝั่งลูกค้า พนักงาน และครัวต้องอ้างอิงค่าเดียวกันเป๊ะ ๆ:

```
// ภาพรวมก่อนเริ่มลงมือจริง - โครงเดิมอยู่ในบทที่ 7
type OrderStatus =
  | "DRAFT"
  | "PLACED"
  | "IN_PROGRESS"
  | "READY"
  | "SERVED"
  | "PAID"
  | "CANCELLED";

interface OrderUpdateEvent {
  orderId: string;
  tableCode: string;
  status: OrderStatus;
  updatedAt: string; // ISO timestamp
}
```

ส่วนความสัมพันธ์ของข้อมูลเบื้องหลัง — เมนูหนึ่งรายการถูกสั่งได้หลายครั้งในหลายออร์เดอร์ และหนึ่งออร์เดอร์มีได้หลายรายการอาหาร — เป็นความสัมพันธ์แบบ many-to-many ที่ต้องมีตารางกลางมารองรับ รีวิวคร่าว ๆ ก่อนลงรายละเอียดเต็มในบทที่ 3-4:

```
// รีวิวความสัมพันธ์หลักของระบบ (ฉบับเต็มพร้อม Tenant/Branch อยู่ในบทที่ 3-4)
model Order {
  id      String      @id @default(cuid())
  tableId String
  status  String      // จะกลายเป็น enum เต็มรูปแบบในบทที่ 7
  items   OrderItem[]
}

model OrderItem {
  id      String @id @default(cuid())
  orderId String
  menuItemId String
  order   Order  @relation(fields: [orderId], references: [id])
}
```

ตลาด POS ร้านอาหารไทยตอนนี้มีใครบ้าง

ก่อนตัดสินใจสร้างเอง ควรรู้ว่าตลาดมีตัวเลือกอะไรอยู่แล้ว เพราะการสร้างเองที่ดีต้องเริ่มจากการรู้ว่าคู่แข่งทำอะไรได้ และทำอะไรไม่ได้

- **Qashier** — แนวคิด all-in-one เครื่องเดียวจบ มีระบบรับเงินรวมอยู่ในตัว มีแพ็คเกจเริ่มต้นแบบฟรี เหมาะกับร้านขนาดเล็กถึงกลางที่อยากเริ่มเร็ว
- **FoodStory** — เน้นฟีเจอร์เฉพาะร้านอาหารเชิงลึก ทำงานบน iPad และเชื่อมต่อกับ LINE MAN ได้โดยตรง
- **Wongnai POS** — อยู่ในเครือเดียวกับ FoodStory ทำงานบน Android ราคาเยือกเยือกกว่า เริ่มต้นราวปีละ 4,860 บาท (เป็นราคาประกาศของผู้ขาย ณ ช่วงที่อ้าง ต้องไปตรวจกับผู้ให้บริการเองอีกครั้งก่อนตัดสินใจ)
- **StoreHub** — จุดเด่นอยู่ที่การจัดการสต็อกและวัตถุดิบ และทำงานตอนอินเทอร์เน็ตหลุดได้ (offline mode)

ทุกเจ้าที่กล่าวมามีของพื้นฐานครบอยู่แล้ว คือรองรับการรับเงินอิเล็กทรอนิกส์และมีรายงานยอดขายให้ดู คำถามจึงไม่ใช่ “เจ้าไหนดีที่สุด” แต่เป็น “ร้านของคุณ (หรือของลูกค้าคุณ) ควรซื้อของสำเร็จรูป หรือควรสร้างเอง”

เกณฑ์ตัดสินใจ: ซื้อ vs สร้างเอง

หนังสือเล่มนี้ไม่ได้จะบอกว่า “อย่าซื้อของสำเร็จรูป” เพราะสำหรับร้านจำนวนมาก ระบบสำเร็จรูปคือทางเลือกที่คุ้มค่ากว่าจริง ๆ แต่มี 6 สถานการณ์ที่การสร้างเองเริ่มสมเหตุผล ลองไล่เช็คทีละข้อ

1. **ร้านมีขั้นตอนการทำงานเฉพาะตัวที่ระบบสำเร็จรูปบีบให้ทำตามแม่แบบไม่ได้** เช่น ร้านชาบูที่มีขั้นตอนเปิด-ปิดโต๊ะแบบเฉพาะ หรือร้านที่มีระบบคิวครัวหลายสถานีที่ซับซ้อนกว่าค่าเริ่มต้นของซอฟต์แวร์ทั่วไป

2. ต้องเชื่อมกับระบบหลังบ้านที่มีอยู่แล้ว เช่น ระบบบัญชี ERP หรือระบบสต็อกกลางของเครือร้าน ที่ระบบสำเร็จรูปไม่มี API เปิดให้เชื่อมได้ตามต้องการ
3. อยากมีระบบเป็นสินทรัพย์ของตัวเอง ไม่ผูกกับค่าธรรมเนียมรายเดือน/รายปีของผู้ให้บริการตลอดไป และยอมรับภาระดูแลระบบเองในระยะยาว
4. อยากขายระบบต่อให้ร้านอื่น — กลายเป็นสินค้าของตัวเอง ไม่ใช่แค่เครื่องมือใช้ในร้านเดียว
5. มีทีมหรือมีงบประมาณจะดูแลระบบต่อเนื่อง ไม่ใช่แค่เขียนเสร็จแล้วปล่อยทิ้ง เพราะ POS เป็นระบบที่แตะเงินจริงทุกวัน บั๊กเล็ก ๆ กระทบรายได้ทันที
6. สเกลของร้านคุ้มกับการลงทุนสร้าง ไม่ใช่ร้านเดียวเล็ก ๆ ที่ของสำเร็จรูปราคาหลักพันต่อก็ตอบโจทย์ได้แล้ว

กลับกัน ถ้าร้านของคุณ (หรือลูกค้าคุณ) เป็นร้านเดียว ไม่มีความต้องการพิเศษ และที่สำคัญที่สุดคือ **ไม่มีคนดูแลระบบต่อหลังส่งมอบ** — การสร้างเองมักไม่คุ้ม เพราะ POS ไม่ใช่เว็บโพรไฟล์ที่สร้างเสร็จแล้วจบ มันคือระบบที่ต้องรันทุกวัน ทุกมือ และพังไม่ได้ตอนร้านแน่นที่สุด

สำหรับนักพัฒนาหรือฟรีแลนซ์ที่อ่านเล่มนี้อยู่ ข้อ 4 คือโอกาสจริง — ถ้าคุณสร้างระบบที่ออกแบบแบบ multi-tenant ตั้งแต่แรก (เราจะพูดเรื่องนี้ในบทที่ 3) คุณจะเปลี่ยนงานที่ทำครั้งเดียวให้กลายเป็นสินค้าที่ขายซ้ำได้

ทำไมปี 2026 ถึงเป็นจังหวะที่ทำเองไหว

เมื่อสองสามปีก่อน การสร้าง POS เองสำหรับนักพัฒนาคนเดียวหรือทีมเล็กแทบเป็นไปไม่ได้ในเวลาที่คุ้มค่า เพราะปริมาณงานเยอะมาก ทั้งฝั่งลูกค้า ฝั่งครัว ฝั่งพนักงาน ฝั่งรายงาน และเรื่องที่พลาดไม่ได้คือการเงิน แต่เครื่องมือ AI ช่วยเขียนโค้ดในช่วงหลังเปลี่ยนสมการนี้ไปพอสมควร

จากผลสำรวจ Stack Overflow Developer Survey 2025 นักพัฒนา 84% ใช้หรือวางแผนจะใช้ AI ช่วยเขียนโค้ด (เพิ่มจาก 76% ในปี 2024) ขณะที่การศึกษาแบบ controlled study ของ GitHub พบว่างานเขียนโค้ดเสร็จเร็วขึ้นถึง 55% (จาก 2 ชั่วโมง 41 นาที เหลือ 1 ชั่วโมง 11 นาที) — นี่คือเหตุผลเชิงตัวเลขว่าทำไมงานที่เคยใช้ทีมใหญ่หลายเดือนถึงพอลงมือคนเดียวหรือทีมเล็กได้ในเวลาที่คุ้มค่าขึ้นมาก

แต่ตัวเลขพวกนี้มีอีกด้านที่สำคัญไม่แพ้กัน คือความเชื่อมั่นในความแม่นยำของ output AI กลับลดลงในช่วงเดียวกัน (รายละเอียดตัวเลขและกฎการรีวิวที่ตามมาอยู่ในบทที่ 2) — นี่คือเหตุผลที่เล่มนี้ไม่ได้บอกให้ “ปล่อยให้ AI เขียนแล้วจบ” แต่ให้ AI แรงงาน แล้วคุณรีวิวทุกจุดที่แตะเงิน

นี่คือเหตุผลที่เริ่มของเล่มนี้คือ “โค้ดที่ AI เขียนให้เร็วมาก แต่เงินของร้านผิบบาทเดียวก็จบ — คุณคือคนที่ต้องรีวิว” Claude Code จะช่วยคุณร่างสคีมาฐานข้อมูล เขียน server action เจน QR PromptPay หรือแม้แต่ช่วยดีบั๊กปัญหา realtime แต่ตรงไหนที่แตะเงิน แตะภาษี หรือแตะข้อมูลบัตร — คุณต้องอ่านทุกบรรทัดเอง ไม่มีทางลัด

สโคปของเล่มนี้: จะสร้างอะไรจนจบ และจะไม่ทำอะไร

เพื่อให้หนังสือจบจนอ่านไม่จบ เราตีกรอบสโคปให้ชัดตั้งแต่บทแรก

สิ่งที่เราจะสร้างจนจบเล่ม: - ระบบเมนูที่รองรับตัวเลือกซ้อนตัวเลือกได้จริง (เช่น ชานมไข่มุก: เลือกขนาด → เลือกความหวาน → เพิ่มที่อปิ้งได้หลายอัน) - QR สั่งอาหารประจำโต๊ะ พร้อม session ป้องกันคนนอกร้านสั่งจากรูป QR ที่ถ่ายไว้ - จอครัว (KDS) แบบ realtime ด้วย Server-Sent Events (SSE) พร้อม fallback เมื่อจอหลุด - หน้า POS ของพนักงานที่รองรับงานจริง: ย้ายโต๊ะ รวมโต๊ะ แยกบิล ยกเลิกที่ต้องขออนุมัติ - การคิดเงินที่ถูกต้องทุกบาท (ลำดับ service charge → VAT ตามกฎ) ด้วยจำนวนเต็มสตางค์ ไม่ใช่ float - การรับเงินจริงทั้ง PromptPay (สร้าง QR เอง), บัตรผ่าน payment gateway ไทย, และเงินสด - ใบเสร็จและใบกำกับภาษีอย่างย่อ พร้อมพิมพ์ผ่านเครื่องพิมพ์ความร้อนจริง - สิทธิการใช้งาน การกันโกงหน้าร้าน และการทำ audit log - สต็อกวัตถุดิบระดับเริ่มต้น รายงานปิดกะปิดวัน และการ deploy ให้อยู่กับร้านจริงได้

สิ่งที่เล่มนี้จะไม่ทำ (บอกไว้ตรงนี้เพื่อกันความคาดหวังผิด): - การเชื่อมต่อกับแพลตฟอร์มเดลิเวอรี่ของเจ้าอื่น เช่น LINE MAN, Grab, foodpanda — เป็นงานเชื่อม API เฉพาะแต่ละเจ้าที่ควรเป็นหนังสือแยกต่างหาก - ระบบบัญชีเต็มรูปแบบ (งบดุล งบกำไรขาดทุน) — เล่มนี้ให้แค่ตัวเลขที่ป้อนเข้าระบบบัญชีได้ ไม่ได้แทนที่โปรแกรมบัญชี - โมดูล CRM หรือระบบสมาชิกสะสมแต้มแบบเต็มรูปแบบ

สั่ง Claude Code แบบนี้

ก่อนเริ่มบทที่ 2 ลองใช้ Claude Code สืบรวจไฉ่เดียวเบื้องต้นแบบ **plan mode** (กด **Shift+Tab** สลับเป็น plan mode ก่อนพิมพ์ prompt) เพื่อให้ Claude คิดโครงสร้างก่อนแต่ะไ้ค้จริง:

ฉันจะสร้างระบบ POS ร้านอาหารด้วย Next.js (App Router) + TypeScript + Prisma + PostgreSQL ระบบมี 4 ส่วนหลักที่ต้องคุยกันแบบ realtime:

- 1) หน้าเมนูลูกค้าสั่งผ่าน QR ประจำโต๊ะ
- 2) หน้า POS ของพนักงาน
- 3) จอครัว (KDS) ที่ต้องอัปเดตสถานะออร์เดอร์แบบ realtime
- 4) หลังร้านสำหรับดูรายงานและตั้งค่าเมนู

ช่วยร่าง high-level architecture คร่าว ๆ ว่าแต่ละส่วนควรเป็น route group ไหนใน App Router, ข้อมูลไหนต้อง realtime ระหว่างส่วนไหนกับส่วนไหน, และมีจุดไหนบ้างที่เสี่ยงเรื่องความถูกต้องของเงิน (ราคา ส่วนลด VAT) ที่ต้อง validate ที่ server เท่านั้น ห้ามเขียนโค้ดตอนนี้ ขอแค่แผนก่อน

สิ่งที่ต้องรื้อในผลลัพธ์: เช็คว่า Claude เข้าใจถูกว่า Route Handler ของ Next.js ทำ WebSocket ไม่ได้ (ถ้า Claude เสนอ WebSocket ตรง ๆ ผ่าน Route Handler ให้ทักท้วงทันที) เช็คว่าแผนแยกชัดว่าราคาต้องคำนวณที่ server เสมอ ไม่ใช่เชื่อค่าที่ client ส่งมา และเช็คว่า Claude ไม่ได้ชวนคุณข้ามขั้นตอนออกแบบฐานข้อมูลไปเขียน UI ก่อน — ลำดับที่ถูกต้องคือ schema ก่อน แล้วค่อยขึ้น UI

ยังไม่ต้องใช้ subagent หรือ skill ในบทนี้ — บทที่ 2 จะพาไปตั้งค่าเครื่องมือพวกนี้ให้ครบก่อนเริ่มเขียนโค้ดจริงในบทที่ 3

มุมเจ้าของร้าน

ถ้าคุณเป็นเจ้าของร้านที่กำลังอ่านเล่มนี้เพื่อประกอบการตัดสินใจไม่ใช่นักพัฒนา คำถามที่คุณต้องตอบตัวเองก่อนคือ “ถ้าระบบพัง กลางมือเที่ยงวันเสาร์ ใครแก้ไขฉันได้ภายในกี่นาที” ถ้าคำตอบคือ “ไม่รู้” หรือ “ไม่มีใคร” — ระบบสำเร็จรูปที่มีทีมซัพพอร์ตหลังบ้านมักตอบโจทย์คุณดีกว่า เพราะเวลาที่ระบบล่มตอนร้านแน่น คือช่วงเวลาแพงที่สุดของธุรกิจร้านอาหาร

แต่ถ้าคุณมีนักพัฒนาประจำ หรือกำลังจ้างทีมที่จะดูแลระบบต่อเนื่อง การสร้างเองแลกมาด้วยระบบที่ตรงกับขั้นตอนการทำงานจริงของร้านคุณแบบ 100% ไม่ต้องปิดขั้นตอนร้านให้เข้ากับซอฟต์แวร์สำเร็จรูป และในระยะยาวไม่ต้องจ่ายค่าธรรมเนียมรายเดือนแบบไม่มีที่สิ้นสุด — แลกกับความเสี่ยงที่คุณต้องรับผิดชอบเองเมื่อระบบมีปัญหา

จำไว้ว่าตัวเลขอย่าง “ลดเวลารอได้ถึง 30%” ที่คุณจะได้เห็นจากโฆษณาของผู้ให้บริการ QR ordering หลายเจ้า เป็นตัวเลขที่ผู้ขายเคลมเอง ไม่ใช่ผลวัดจากร้านของคุณโดยตรง ให้มองเป็น “แนวโน้มที่เป็นไปได้” ไม่ใช่การันตี

กล่องเตือนที่ต้องรู้ตั้งแต่บทแรก

❌ **พลาดบ่อย:** เริ่มเขียนโค้ด POS ทันทีโดยยังไม่ได้ตัดสินใจว่าระบบต้องรองรับกี่สาขา หรือคิดว่า “ตอนนี้ร้านเดียวก่อน ค่อยไปคิดเรื่อง multi-tenant ทีหลัง” ✅ **ทำแบบนี้:** ตัดสินใจเรื่อง multi-tenant ตั้งแต่การออกแบบฐานข้อมูลในบทที่ 3 แม้จะเริ่มด้วยร้านเดียว เพราะการเพิ่มสาขาทีหลังต้องรื้อ schema แทบทั้งระบบ

❌ **พลาดบ่อย:** เข้าใจว่า Claude Code เขียนโค้ดถูกเพราะ “โค้ดรันได้ไม่ error” แล้วไม่รีวิวนี้อีกต่อ ✅ **ทำแบบนี้:** ใช้กฎที่ว่า “รันได้” กับ “ถูกต้อง” เป็นคนละเรื่องกัน — โดยเฉพาะจุดที่แตะเงิน ต้องอ่านทุกบรรทัดเอง (รายละเอียดกฎรีวิว 7 ข้ออยู่ในบทที่ 2)

จำ 3 ข้อ POS ร้านอาหารที่ใช้งานได้จริง คือ 4 หน้าจอ (ลูกค้า/พนักงาน/ครัว/หลังร้าน) ที่ต้องคุยกันแบบ realtime ไม่ใช่แค่เครื่องคิดเงินเครื่องเดียว การสร้างเองคุ้มก็ต่อเมื่อร้านมีความต้องการเฉพาะตัว ต้องเชื่อมระบบเดิม หรืออยากขายต่อ — และต้องมีคนดูแลระบบต่อเนื่องเสมอ AI ช่วยให้เขียนโค้ดเร็วขึ้นจริง แต่เวลารีวิวโค้ดที่แตะเงินของร้านคือสิ่งที่ลัดไม่ได้ ไม่ว่าเครื่องมือจะเก่งแค่ไหน

ตั้ง Claude Code ให้ทำงานถูกตั้งแต่บรรทัดแรก



ก่อนเปิดไฟล์แรกของโปรเจกต์ POS มีงานที่สำคัญกว่าการเขียน route handler ตัวแรกเสียอีก คืองาน “ตั้งเครื่องมือ” ให้ Claude Code เข้าใจกติกาของโปรเจกต์นี้ตั้งแต่ต้น เพราะระบบที่เราจะสร้างแตะเงินจริงของร้านทุกวัน ความผิดพลาดเล็ก ๆ เช่น ปิดเศษผิด หรือลืม validate ราคาที่ server จะไม่ใช่แค่บัก แต่เป็นเงินที่หายไปจากลิ้นชักจริง ๆ

หนังสือเล่มก่อน ๆ ในซีรีส์นี้พูดถึง Claude Code ในภาพกว้างไปแล้ว บทนี้จะโฟกัสเฉพาะสิ่งที่ต้องตั้งค่าให้โปรเจกต์ POS ร้านอาหารโดยเฉพาะ

Claude Code มีอะไรให้ตั้งค่าบ้าง

ก่อนลงรายละเอียด ทวนสั้น ๆ ว่า Claude Code เป็น AI coding agent ที่ทำงานได้ทั้งใน terminal, เป็น IDE extension (VS Code/JetBrains) และผ่าน desktop/web โดยมี “primitives” หลักที่ทุกโปรเจกต์ควรรู้จักและใช้ให้ถูกจุด:

- **CLAUDE.md** — ธรรมนูญของ repo ที่ Claude อ่านทุก session เป็นที่แรกที่เราใส่กติกาเฉพาะโปรเจกต์
- **skills** — workflow สำเร็จรูปที่เรียกด้วย `/` (ปี 2026 skills กับ slash command ถูกรวมเป็นสิ่งเดียวกันแล้ว)
- **subagents** (`.claude/agents/`) — ผู้ช่วยเฉพาะทางที่ทำงานใน context แยกจาก session หลัก

- **hooks** — สคริปต์ที่รันอัตโนมัติ ณ จุดต่าง ๆ ของ workflow เช่น ก่อน commit — จุดสำคัญคือ hook เป็น **deterministic** (รันแน่นอนทุกครั้ง) ต่างจาก CLAUDE.md ที่เป็นแค่คำแนะนำให้โมเดลอ่าน ซึ่งโมเดลอาจลืมทำตามได้
- **MCP servers** — ต่อเครื่องมือภายนอกเข้ากับ Claude Code ด้วย `claude mcp add` เช่น ต่อฐานข้อมูล PostgreSQL ให้ Claude อ่าน schema จริงได้
- **plugins** — มักรวมทุกอย่างข้างต้นเพื่อติดตั้งทีเดียว

โปรเจกต์ POS ของเราจะใช้ครบทุกตัว แต่เริ่มจากตัวที่สำคัญที่สุดก่อน — `CLAUDE.md`

CLAUDE.md ของโปรเจกต์ POS ควรเขียนอะไร

`CLAUDE.md` ที่ดีสำหรับโปรเจกต์นี้ต้องตอบ 4 เรื่องอย่างชัดเจน: โครง repo หน้าตาเป็นยังไง, กติกาเรื่องเงิน/ทศนิยมที่ห้ามพลาด, กติกาเรื่อง schema migration, และขอบเขตที่ห้ามแตะเด็ดขาด (ข้อมูลบัตร) นี่คือตัวอย่างไฟล์เต็มที่ใช้เป็นจุดเริ่มต้นได้จริง ปรับรายละเอียดตามโปรเจกต์ของคุณ:

```
# CLAUDE.md - POS ร้านอาหาร (โปรเจกต์นี้)

## ภาพรวมระบบ
Next.js (App Router) + TypeScript + Prisma + PostgreSQL
ระบบมี 4 ส่วนที่ต้องคุยกันแบบ realtime: หน้าเมนู QR ลูกค้า, หน้า POS
พนักงาน, จอครัว (KDS), และหลังร้าน (admin/report)

## โครง repo
- `app/(customer)/` - หน้าเมนู QR ของลูกค้า, ไม่มี auth เต็มรูปแบบ
  ใช้ table session token แทน
- `app/(pos)/` - หน้าพนักงาน ต้อง login ด้วย PIN + RBAC
- `app/(kds)/` - จอครัว ใช้ SSE รับอัปเดตออเดอร์แบบ realtime
- `app/(admin)/` - หลังร้าน รายงาน ตั้งค่าเมนู จัดการพนักงาน
- `lib/server/` - โค้ดที่รันฝั่ง server เท่านั้น (import `server-only`
  บังคับทุกไฟล์ในนี้)
- `prisma/schema.prisma` - schema เดียว ห้ามแยกหลายไฟล์
- `claude/agents/` - subagent เฉพาะโปรเจกต์นี้

## กติกาเรื่องเงิน (ห้ามพลาดเด็ดขาด)
1. เก็บเงินทุกจำนวนเป็น **จำนวนเต็มสตางค์ (Int)** ห้ามใช้ Float หรือ
  Decimal แบบ JS number เด็ดขาด - เหตุผลอยู่ใน docs/money.md
2. ลำดับคำนวณบิลคือ ราคาอาหาร → บวก service charge → บวก VAT
  (คิดจากค่าอาหาร + service charge) ห้ามคิด VAT ก่อน service charge
3. ราคาที่ client ส่งมาห้ามเชื่อเด็ดขาด ทุก server action ที่รับ
  ออร์เดอร์ต้องคำนวณราคาใหม่จากฐานข้อมูลเสมอ
4. บัณฑิตทำครั้งเดียวตอนยอดสุดท้ายของบิล ห้ามปิดรายการรหัดแล้วบวก
5. ห้ามเก็บเลขบัตร (PAN) หรือ CVV ลงฐานข้อมูลของเราเด็ดขาด ไม่ว่าจะ
  เข้ารหัสหรือไม่ - ใช้ hosted payment page / token เท่านั้น

ตัวอย่างที่กติกาข้อ 1 พุดถึง หน้าตาจริงใน `schema.prisma` ควรเป็นแบบนี้
เสมอ - ฟิลดราคาทุกตัวเป็น `Int` แทนสตางค์ ไม่ใช่ `Float`:

```prisma
model MenuItem {
 id String @id @default(cuid())
 name String
 priceInSatang Int // เก็บเป็นสตางค์เต็มจำนวน เช่น 4500 = 45.00 บาท
 isAvailable Boolean @default(true)
}
```

ถ้า Claude Code เสนอ `price Float` หรือ `price Decimal` ที่ไหนก็ตาม ให้ถือว่าเป็นการฝ่าฝืนกติกาข้อ 1
ทันที ไม่ว่าโค้ดส่วนนั้นจะรันผ่านหรือไม่ก็ตาม

## กติกาเรื่อง schema migration

- ทุกการแก้ `schema.prisma` ต้องรัน `npx prisma migrate dev --name <ชื่อสื่อความหมาย>` ห้ามแก้ไฟล์
migration ที่ apply ไปแล้วบน production
- ห้าม `prisma migrate reset` บนฐานข้อมูลที่มีข้อมูลร้านจริง
- ทุก schema change ต้องรัน subagent `schema-reviewer` ก่อน commit

. .

## คำสั่งที่ใช้บ่อย

---

- `npm run dev` — รัน dev server (Turbopack)
- `npx prisma studio` — เปิดดูข้อมูลผ่าน UI
- `npm run test:money` — รัน unit test เฉพาะฟังก์ชันคิดเงิน (ต้องผ่าน ก่อน commit ทุกครั้งที่แตะ `lib/server/billing/`)

## ขอบเขตที่ห้ามแตะ

---

- ห้ามเขียนโค้ดที่รับหรือส่งผ่านเลขบัตรเครดิต/เดบิตในระบบของเรา ทุกการรับบัตรต้องผ่าน hosted page หรือ iframe ของ payment gateway
- ห้ามลบหรือแก้ไข record ใน `WebhookEvent` table ด้วยมือ

สังเกตว่าไฟล์นี้ไม่ได้สอน Claude ว่า "TypeScript คืออะไร" – มันไม่จำเป็น เพราะโมเดลรู้เรื่องพื้นฐานอยู่แล้ว สิ่งที่ `CLAUDE.md` ต้องทำคือบอก **\*\*กติกาเฉพาะของโปรเจกต์นี้\*\*** ที่โมเดลเดาเองไม่ได้ เช่น ลำดับการคิดภาษี หรือว่าโฟลเดอร์ไหนคือ server-only

## plan mode ใช้ตอนไหน

Best practice หลักของการทำงานกับ Claude Code คือ **\*\*แยกขั้นตอน research/plan ออกจากขั้นตอนลงมือเขียนโค้ด\*\*** ปล่อยให้ AI พุงเขียนโค้ดทันทีที่จบด้วยการแก้ข้อผิดพลาด เพราะโมเดลเดาความต้องการเอาเองในจุดที่กำกวม

สำหรับโปรเจกต์นี้ ใช้ plan mode (สลับด้วย `Shift+Tab`) เมื่อ:

- เริ่มฟีเจอร์ใหม่ที่แตะหลาย layer พร้อมกัน (เช่น เพิ่มฟีเจอร์แยกบิล ต้องแตะทั้ง schema, server action, และ UI)
- แก้ schema ที่มีข้อมูลอยู่แล้ว (ต้องคิดเรื่อง migration ก่อนแตะโค้ด)
- ทำอะไรก็ตามที่เกี่ยวกับเงินหรือ state machine ของออร์เดอร์ – ให้ Claude เสนอแผนก่อน แล้วคุณอ่านแผนทั้งหมดก่อนอนุมัติให้ลงมือ

ไม่จำเป็นต้องใช้ plan mode กับงานเล็ก ๆ ที่ชัดเจนอยู่แล้ว เช่น แก้ข้อความ label หรือปรับสไลด์ปุ่ม – ใช้ plan mode พรา ๆ เพื่อเก็บไปจะทำให้ workflow เข้าใจง่ายขึ้น

## subagent ที่ควรมีในโปรเจกต์นี้

subagent คือผู้ช่วยที่ทำงานใน context แยก เหมาะกับงานที่ต้อง "ตรวจ" มากกว่า "สร้าง" สองตัวที่โปรเจกต์ POS ควรมีตั้งแต่แรกคือ

**\*\*`schema-reviewer`\*\*** – ตรวจทุกครั้งที่มีการแก้ `schema.prisma` ว่าฟิลด์เงินเป็น `Int` ไม่ใช่ `Float`, ความสัมพันธ์ many-to-many มีตารางกลางถูกต้อง, และ migration ที่ generate ออกมาไม่ทำให้ข้อมูลเก่าหาย

**\*\*`payment-auditor`\*\*** – ตรวจทุกครั้งที่มีการแก้โค้ดในโฟลเดอร์ที่เกี่ยวข้องกับการคิดเงินหรือรับชำระ ว่าไม่มีจุดไหนเชื่อราคาจาก client, ไม่มีการเก็บ PAN/CVV, และ webhook handler เป็น idempotent

ตัวอย่างไฟล์ subagent ตัวแรก:

```
```markdown
---
name: schema-reviewer
description: ตรวจ schema.prisma ทุกครั้งที่มีการแก้ก่อน commit
  โฟกัสที่ชนิดข้อมูลของฟิลด์เงินและความสัมพันธ์ระหว่างตาราง
tools: Read, Grep, Bash
---
คุณคือผู้ตรวจ schema ของระบบ POS ร้านอาหาร ตรวจทุกครั้ง diff และ
prisma/schema.prisma โดยเช็ก:
1. ฟิลด์เกี่ยวกับราคา/เงินต้องเป็น Int (สตริง) ห้ามเป็น Float
2. ความสัมพันธ์ Order-Item ต้องผ่านตารางกลาง OrderItem เสมอ
3. ทุก migration ใหม่ต้องไม่ DROP COLUMN ที่มีข้อมูลอยู่โดยไม่มี
  ขั้นตอน backfill ก่อน
รายงานผลเป็นรายการ ✅/❌ พร้อมเลขบรรทัดที่ตรวจพบปัญหา
```

hooks ที่คุ้มค่าในโปรเจกต์นี้

hooks ต่างจาก **CLAUDE.md** ตรงที่ hook รันแบบ deterministic – ไม่ว่าโมเดลจะ “ลืม” หรือไม่ hook ก็รันเสมอ สำหรับโปรเจกต์ POS ควรตั้ง hook ที่รัน **prisma validate** และ **tsc --noEmit** ก่อน commit ทุกครั้ง เพื่อจับ schema ผิดและ type error ก่อนที่จะหลุดเข้า repo

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/check-money-fields.sh"
          }
        ]
      }
    ]
  }
}
```

ส่วนสคริปต์ตัวอย่างที่ hook เรียกใช้ เขียนเป็น TypeScript ให้รันด้วย `tsx` ตรวจสอบจับ pattern อันตรายง่าย ๆ ก่อน commit เช่น การใช้ `Float` ในฟิลด์ที่ชื่อมีคำว่า price/amount:

```
// .claude/hooks/check-money-fields.ts
import { readFileSync } from "node:fs";

const schema = readFileSync("prisma/schema.prisma", "utf-8");
const lines = schema.split("\n");

const suspicious = lines
  .map((line, i) => ({ line, no: i + 1 }))
  .filter(({ line }) =>
    /(price|amount|total|fee)\w*s+Float/i.test(line)
  );

if (suspicious.length > 0) {
  console.error("พบฟิลด์เงินที่เป็น Float - ต้องเป็น Int (สตริง):");
  for (const s of suspicious) {
    console.error(` บรรทัด ${s.no}: ${s.line.trim()}`);
  }
  process.exit(1);
}
```

MCP ต่อ Postgres ให้ Claude อ่าน schema จริง

ต่อ MCP server สำหรับ PostgreSQL ด้วยคำสั่ง `claude mcp add` ขึ้นที่ฐานข้อมูล dev ของโปรเจกต์ จะทำให้ Claude ตอบคำถามเกี่ยวกับ schema ปัจจุบันได้ถูกต้องแม่นยำกว่าการอ่านจากไฟล์ `.prisma` เพียงๆ เพราะเห็นข้อมูลจริงในตาราง เช่น เห็นว่าตาราง `MenuItem` มีกี่แถว ราคาเฉลี่ยเท่าไร ช่วยตอนสร้าง seed data หรือตรวจสอบว่า migration ที่รันไปแล้วตรงกับ schema.prisma จริงหรือไม่

ข้อควรระวัง: ต่อ MCP กับฐานข้อมูล **dev** เท่านั้น อย่าต่อ MCP เข้าฐานข้อมูล production โดยตรง เพราะ Claude อาจรัน query ที่กระทบข้อมูลจริงโดยไม่ตั้งใจ

สั่ง Claude Code แบบนี้

หลังตั้งค่า `CLAUDE.md`, hooks, และ subagent เสร็จ ให้ Claude Code ช่วยสร้าง subagent ตัวที่สอง (`payment-auditor`) ด้วย prompt แบบนี้ — งานนี้เป็นงาน “สร้างไฟล์ config” ไม่ใช่งานแตะโค้ดจริง จึงไม่จำเป็นต้องใช้ plan mode แต่ควรอ้างอิงไฟล์ `schema-reviewer.md` ที่มีอยู่แล้วเพื่อให้สไตล์สอดคล้องกัน:

```
ช่วยสร้าง subagent ใหม่ชื่อ payment-auditor ใน .claude/agents/
โดยใช้โครงเดียวกับ schema-reviewer.md ที่มีอยู่แล้วเป็นแบบ

payment-auditor มีหน้าที่ตรวจโค้ดทุกครั้ง diff และไฟล์ในโฟลเดอร์
lib/server/billing/ หรือ lib/server/payment/ โดยเช็ค 3 เรื่อง:
1. ไม่มีจุดไหนใช้ราคาที่ client ส่งมาตรง ๆ โดยไม่คำนวณใหม่จาก DB
2. ไม่มีการเก็บเลขบัตรเครดิต (PAN) หรือ CVV ลงตัวแปรที่จะถูกบันทึก
   ลงฐานข้อมูลหรือ log
3. webhook handler ทุกตัวใช้ event ID เป็น dedup key ก่อนประมวลผลซ้ำ

ให้ subagent รายงานผลเป็นรายการ ✓/✗ พร้อมชี้ตำแหน่งไฟล์และบรรทัด
```

สิ่งที่ต้องรีวิวในผลลัพธ์: เปิดไฟล์ `payment-auditor.md` ที่ได้มาอ่านทุกบรรทัด เช็คที่ `tools:` ที่ระบุไว้จำกัดแค่ `Read, Grep, Bash` ไม่ใช่ `Edit` หรือ `Write` (subagent ตัวนี้มีหน้าที่ตรวจ ไม่ใช่แก้ไขตัวเอง) และเช็คที่คำอธิบายใน `description:` สื่อชัดพอที่ Claude หลักจะเรียกใช้ subagent นี้ถูกจังหวะโดยอัตโนมัติ

มุมมองของร้าน

ถ้าคุณเป็นเจ้าของร้านที่ไม่ได้เขียนโค้ดเอง บทนี้อาจดูเป็นเรื่องของนักพัฒนาล้วน ๆ แต่มีจุดหนึ่งที่คุณควรถามทีมพัฒนาของคุณตรง ๆ ก่อนจ้างทำ: “ระบบตรวจสอบตัวเองก่อนขึ้นระบบจริงมีอะไรบ้าง” ถ้าคำตอบคือ “ให้ AI เขียนแล้วก็ใช้ได้เลย” นั่นคือสัญญาณเตือน เพราะจากสถิติในบทที่แล้ว นักพัฒนาที่ใช้ AI ช่วยเขียนโค้ดใช้เวลารีวิวโค้ด AI มากกว่าเวลาที่เขียนโค้ดเองเสียอีก (11.4 ชั่วโมงต่อสัปดาห์ เทียบกับ 9.8 ชั่วโมง) — การรีวิวไม่ใช่ขั้นตอนเสริม แต่เป็นงานหลักพอ ๆ กับการเขียน

ทีมที่ตั้ง hook ให้รันเช็คอัตโนมัติก่อน commit ทุกครั้ง หรือมี subagent เฉพาะทางไว้ตรวจจุดที่เกี่ยวกับเงิน คือทีมที่เข้าใจว่าความเร็วจาก AI ต้องมาคู่กับระบบตรวจสอบ ไม่ใช่แค่ “เขียนเร็วแล้วจบ”

กฎรีวิวโค้ด AI 7 ข้อสำหรับงานที่เกี่ยวกับเงิน

ไม่ว่า Claude Code จะเขียนโค้ดให้เร็วแค่ไหน จุดที่แตะเงินของร้านต้องผ่าน 7 ข้อนี้ก่อนขึ้นระบบจริงเสมอ

- ราคาคำนวณที่ server เท่านั้น — ทุกจุดที่รับค่าราคาหรือยอดรวมจาก client ต้องคำนวณใหม่จากฐานข้อมูล ไม่เชื่อค่าที่ส่งมา
- ฟิลด์เงินทุกตัวเป็น Int (สตริงค์) — ไม่มี Float แอบแฝงอยู่ที่ไหนเลยในทั้งระบบ
- ลำดับ service charge → VAT ถูกทุกจุด — เช็คทุกฟังก์ชันที่คำนวณบิล ไม่ใช่แค่จุดหลัก
- ปิดเศษครึ่งเดียวตอนยอดสุดท้าย — ไม่ปิดรายบรรทัดแล้วบวกจนเศษหายหรือเกิน

5. ไม่มีการเก็บ PAN/CVV ที่ไหนเลยในระบบ — grep หาคำว่า `cardNumber` , `cvv` , `pan` ทั้ง repo ก่อน deploy ทุกครั้ง
6. `webhook handler` เป็น `idempotent` — ใช้ event ID เป็น dedup key เพราะ payment gateway ส่ง event ซ้ำได้ (at-least-once delivery)
7. ทุก `transaction` ที่แก้หลายตารางพร้อมกันใช้ `Prisma transaction` จริง — ไม่ใช่เรียกหลาย query แยกกันแล้วหวังว่าจะสำเร็จพร้อมกัน

จำ 7 ข้อนี้ให้ขึ้นใจ เพราะทุกบทตั้งแต่บทที่ 7 เป็นต้นไป (state machine ของออร์เดอร์) จะย้อนกลับมาที่กฎเหล่านี้ซ้ำแล้วซ้ำเล่า

กล่องเตือนที่ต้องรู้ก่อนไปบทถัดไป

❌ **พลาดบ่อย:** เขียน `CLAUDE.md` แบบทั่วไป เช่น “เขียนโค้ดสะอาด” “ทำตาม best practice” โดยไม่ระบุกติกาเฉพาะของโปรเจกต์นี้ ✅ **ทำแบบนี้:** ระบุกติกาที่โมเดลตัวเองไม่ได้ เช่น ลำดับคำนวณภาษี, ชนิดข้อมูลของฟิลด์เงิน, และโฟลเดอร์ไหนคือ server-only — สิ่งเหล่านี้คือสิ่งที่ `CLAUDE.md` ต้องทำหน้าที่แทน

❌ **พลาดบ่อย:** ให้ subagent ที่มีหน้าที่ “ตรวจ” อย่าง `payment-auditor` มีสิทธิ์ `Edit / Write` ทำให้มันแก้โค้ดเองแทนที่จะแค่รายงานปัญหา ✅ **ทำแบบนี้:** จำกัดสิทธิ์ subagent ตรวจสอบให้เหลือแค่ `Read, Grep, Bash` เพื่อให้มันมีหน้าที่ชัดเจนแค่ “ตรวจแล้วรายงาน” ส่วนการแก้ไขให้เป็นการตัดสินใจของคุณในโฟลว์หลัก

จำ 3 ข้อ `CLAUDE.md` ต้องเขียนกติกาเฉพาะโปรเจกต์ที่โมเดลตัวเองไม่ได้ ไม่ใช่คำแนะนำทั่วไปที่ไม่มี ความหมาย hooks ต่างจาก `CLAUDE.md` ตรงที่รันแบบ deterministic เสมอ — ใช้ hook กับสิ่งที่พลาดไม่ได้จริง ๆ เช่น เช็ค Float ในฟิลด์เงิน จำกฎรีวิวกัด AI 7 ข้อให้ขึ้นใจ เพราะทุกบทที่เตะเงินตั้งแต่นี้ไปจะย้อนกลับมาใช้กฎชุดนี้ซ้ำ

สถาปัตยกรรมทั้งระบบ และการตั้งโปรเจกต์



บทที่แล้วเราตั้งเครื่องมือให้ Claude Code พร้อมทำงานถูกต้องตั้งแต่ต้น บทนี้คือจุดที่เริ่มลงมือจริง — วางโครงสร้างทั้งระบบก่อนเขียนฟีเจอร์แรก เพราะการแก้สถาปัตยกรรมที่หลังแพงกว่าการคิดให้ถูกตั้งแต่วันแรกเสมอ โดยเฉพาะเรื่อง multi-tenant ที่ถ้าไม่ได้วางไว้ตั้งแต่ต้น การเติมทีหลังคือการรื้อ schema เกือบทั้งระบบ

4 หน้าจอ แปลงเป็น route group ใน App Router

จากบทที่ 1 เราเห็นแล้วว่า POS ร้านอาหารมี 4 ส่วนหลัก — App Router ของ Next.js มีฟีเจอร์ **route group** ที่เหมาะกับการแยกส่วนเหล่านี้พอดี route group คือโพลเดอร์ที่ครอบด้วยวงเล็บ **(ชื่อ)** ซึ่งไม่กระทบ URL จริง แต่ช่วยแยก layout, middleware logic และโค้ดของแต่ละส่วนออกจากกันอย่างชัดเจน

```
app/
├── (customer)/           → ลูกค้าสแกน QR เข้ามา ไม่ต้อง login เต็มรูปแบบ
│   └── t/[tableCode]/
├── (pos)/                → พนักงาน ต้อง login ด้วย PIN
│   ├── login/
│   └── tables/
├── (kds)/                → จอดครัว ใช้ SSE รับอัปเดต realtime
│   └── stations/[stationId]/
├── (admin)/              → หลังร้าน รายงาน ตั้งค่าเมนู จัดการพนักงาน
│   ├── reports/
│   └── menu/
└── api/                  → route handler ที่ทั้ง 4 ส่วนเรียกใช้ร่วมกัน
```

แต่ละ route group มี layout, session logic และการตรวจสอบสิทธิ์ของตัวเอง — **(customer)** ตรวจสอบแค่ table session token ใน cookie ส่วน **(pos)**, **(kds)**, **(admin)** ตรวจสอบ session พนักงานที่ login ด้วย PIN และเช็ค role ตามสิทธิ์ที่ควรเข้าถึงหน้านั้น ๆ (รายละเอียดสิทธิ์เต็มรูปแบบอยู่ในบทที่ 13)

เวอร์ชันของ stack ที่ใช้ตลอดเล่ม

ณ ปี 2026 **Next.js 16.2.x** คือเวอร์ชัน **stable** (16.2.6 เผยแพร่ 7 พฤษภาคม 2026 เวอร์ชันล่าสุดที่อ้างอิงคือ 16.2.12) มีการเปลี่ยนแปลงสำคัญหลายจุดที่กระทบวิธีเขียนโค้ดของเราทั้งหมด:

- **Turbopack** เป็น **bundler ดีฟอลต์** — เร็วกว่า Webpack แบบเดิมมาก โดยเฉพาะตอน dev server รีเฟรช
- **React 19.2** — มี **useActionState** และ **useFormStatus** สำหรับจัดการ pending/error state ของฟอร์ม ซึ่งเราจะใช้หนักในบทที่ 6-7 ตอนทำหน้าสั่งอาหารและตะกร้า
- **Cache Components** — โมเดล caching ใหม่ที่ควบคุมได้ละเอียดขึ้นว่าส่วนไหนของหน้าเว็บ cache ได้ ส่วนไหนต้องสดเสมอ (เช่น สถานะออร์เดอร์ต้องไม่ cache)
- **async params** — **params** และ **searchParams** ใน App Router เป็น Promise ต้อง **await** ก่อนใช้เสมอ
- **middleware** เปลี่ยนชื่อไฟล์เป็น **proxy.ts** — ทำหน้าที่เดิมคือดักทุก request ก่อนเข้าถึง route แต่เปลี่ยนชื่อไฟล์และ export

และที่สำคัญมาก: **Pages Router เข้าสู่ maintenance mode แล้ว** — App Router คือค่าเริ่มต้นที่ควรใช้ ทั้งหมดนี้จะไม่แตะ Pages Router เลยแม้แต่บรรทัดเดียว

ตัวอย่าง **proxy.ts** ที่ตรวจสอบ session พนักงานก่อนเข้าหน้า **(pos)** และ **(admin)** (ยังไม่ใช้โค้ดสมบูรณ์ รายละเอียด RBAC เต็มรูปแบบอยู่ในบทที่ 13):

```
// proxy.ts
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

const PROTECTED_PREFIXES = ["/tables", "/reports", "/menu"];

export function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const needsAuth = PROTECTED_PREFIXES.some((p) => pathname.startsWith(p));

  if (!needsAuth) {
    return NextResponse.next();
  }

  const staffSession = request.cookies.get("staff_session");
  if (!staffSession) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/tables/:path*", "/reports/:path*", "/menu/:path*"],
};
```

โครงสร้างโฟลเดอร์เต็มของโปรเจกต์

นอกจาก `app/` เรายังต้องแยกโค้ดที่ไม่ใช่ UI ออกให้ชัดเจน โดยเฉพาะโค้ดที่แตะฐานข้อมูลหรือคำนวณเงิน ต้องอยู่ในโฟลเดอร์ที่บังคับว่า “รันฝั่ง server เท่านั้น” — ห้ามหลุดไปอยู่ใน client bundle เด็ดขาด

```

pos-project/
├─ app/
│  ├─ (customer)/
│  ├─ (pos)/
│  ├─ (kds)/
│  ├─ (admin)/
│  └─ api/
├─ lib/
│  ├─ server/
│  │  ├─ billing/      → คำนวณราคา, ส่วนลด, VAT (server-only)
│  │  ├─ payment/     → เชื่อม gateway, webhook handler
│  │  ├─ db.ts         → Prisma client singleton
│  │  └─ auth.ts       → ตรวจสอบ session พนักงาน/table session
│  ├─ shared/         → type, constant ที่ทั้ง client/server ใช้รวม
│  └─ client/          → hook ผัง client เช่น useOrderStream
├─ prisma/
│  ├─ schema.prisma
│  ├─ migrations/
│  └─ seed.ts
├─ .claude/
│  ├─ agents/
│  │  ├─ schema-reviewer.md
│  │  └─ payment-auditor.md
│  └─ settings.json
├─ docker-compose.yml
├─ proxy.ts
├─ .env.local
├─ CLAUDE.md
└─ package.json

```

ทุกไฟล์ใน `lib/server/` ต้องขึ้นต้นด้วย `import "server-only"` เพื่อให้ Next.js บังคับ build error ทันที ถ้ามีใคร (หรือ Claude Code) เผลอ import โค้ดนี้เข้าไปใน client component โดยไม่ตั้งใจ — เป็นเพราะป้องกันขั้นแรกไม่ให้ logic การคิดเงินหลุดไปวิ่งฝั่ง browser

```

// lib/server/billing/index.ts
import "server-only";
import { prisma } from "@lib/server/db";

export async function getMenuPrice(menuItemId: string) {
  const item = await prisma.menuItem.findUniqueOrThrow({
    where: { id: menuItemId },
  });
  return item.priceInSatang; // ราคาจริงจากฐานข้อมูลเสมอ ไม่รับจาก client
}

```

Docker Compose สำหรับ dev

เพื่อไม่ให้ทุกคนในทีม (หรือคุณคนเดียว) ต้องติดตั้ง PostgreSQL ลงเครื่องเอง ใช้ Docker Compose รัน Postgres สำหรับ dev environment เท่านั้น — production จะใช้ managed database (พูดถึงในบทที่ 16)

```
# docker-compose.yml
services:
  postgres:
    image: postgres:17
    restart: unless-stopped
    environment:
      POSTGRES_USER: pos_dev
      POSTGRES_PASSWORD: pos_dev_password
      POSTGRES_DB: pos_dev
    ports:
      - "5432:5432"
    volumes:
      - pos_pgdata:/var/lib/postgresql/data

volumes:
  pos_pgdata:
```

และ `.env.local` ที่ชี้ไปยัง container นี้ (ไฟล์นี้ไม่ควร commit เข้า git — ใส่ `.env.local` ใน `.gitignore` ตั้งแต่วันแรก):

```
DATABASE_URL="postgresql://pos_dev:pos_dev_password@localhost:5432/pos_dev"
```

ทำไมต้อง multi-tenant ตั้งแต่แรก แม้ร้านเดียว

นี่คือการตัดสินใจที่สำคัญที่สุดของบอทนี่ ถ้าคุณข้ามไป จะเจ็บที่หลังแน่นอน — ออกแบบให้ระบบรองรับหลายร้าน (Tenant) และหลายสาขา (Branch) ตั้งแต่ schema แรก แม้ตอนนี้จะมีแค่ร้านเดียวกก็ตาม

เหตุผลมี 2 ข้อ: (1) ถ้าคุณเป็นนักพัฒนา/ฟรีแลนซ์ที่อยากขายระบบนี้ต่อให้ร้านอื่น การมี `Tenant` ตั้งแต่ต้นทำให้คุณเปลี่ยนจาก “โปรเจกต์เดียว” เป็น “สินค้า SaaS” ได้โดยไม่ต้องรื้อ schema (2) แม้ร้านของคุณจะมีสาขาเดียวตอนนี้ แต่ร้านที่ไปได้มักขยายสาขา และการเติม `Branch` เข้าไปที่หลังหมายถึงต้องรื้อทุกตารางที่ผูกกับ “ร้าน” (เมนู, ออร์เดอร์, พนักงาน, รายงาน) ให้มี foreign key ใหม่ — งานที่เจ็บปวดกว่าการคิดไว้ตั้งแต่ต้นมาก

โครง schema เริ่มต้นที่วาง `Tenant` และ `Branch` เป็นรากของทุกตาราง:

```

// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

/// ร้าน/กิจการหนึ่งราย - ถ้าจะขายระบบต่อให้ร้านอื่น นี่คือหน่วยแยกลูกค้า
model Tenant {
  id          String   @id @default(cuid())
  name        String
  createdAt   DateTime @default(now())

  branches Branch[]
  staff      Staff[]

  @@map("tenants")
}

/// สาขาของร้าน - แม็ดอนนี้มีสาขาเดียว ก็ต้องมีตารางนี้เสมอ
model Branch {
  id          String   @id @default(cuid())
  tenantId    String
  name        String
  createdAt   DateTime @default(now())

  tenant Tenant @relation(fields: [tenantId], references: [id])
  tables RestaurantTable[]

  @@index([tenantId])
  @@map("branches")
}

/// โต๊ะในสาขา - ใช้ผูกกับ QR ประจำโต๊ะในบทที่ 5
model RestaurantTable {
  id          String   @id @default(cuid())
  branchId    String
  tableCode   String   @unique
  seats       Int
  createdAt   DateTime @default(now())

  branch Branch @relation(fields: [branchId], references: [id])

  @@index([branchId])
  @@map("restaurant_tables")
}

/// พนักงาน - ผูกกับ tenant เดียว เข้าได้หลายสาขาตาม assignment (บทที่ 13)
model Staff {
  id          String   @id @default(cuid())
  tenantId    String
  name        String
  pinHash     String

```

```

role      StaffRole
createdAt DateTime @default(now())

tenant Tenant @relation(fields: [tenantId], references: [id])

@@index([tenantId])
@@map("staff")
}

enum StaffRole {
  STAFF
  SHIFT_LEAD
  OWNER
}

```

โมเดล `MenuItem`, `Order`, `OrderItem` และตารางอื่น ๆ ที่จะตามมาในบทที่ 4 เป็นต้นไป ทุกตัวจะมี `branchId` (หรือ `tenantId` แล้วแต่ว่าข้อมูลนั้นควรใช้ร่วมกันทุกสาขาหรือแยกตามสาขา) เป็น foreign key เสมอ — นี่คือกฎที่ต้องยึดไว้ในหัวตลอดทั้งเล่ม

แยก server-only code ให้เด็ดขาด

จุดที่มีเอใหม่ (และบางครั้ง AI ที่รีบเขียนโค้ด) พลาดบ่อยคือเผลอ import โค้ดที่คุยกับฐานข้อมูลเข้าไปใน client component โดยตรง ทำให้ connection string หรือ logic การคิดราคาหลุดไปอยู่ใน JavaScript bundle ที่ browser โหลดได้ ทางแก้ที่ Next.js ให้มาคือแพ็คเกจ `server-only` — import เข้าไปในไฟล์ไหนก็ตาม ถ้าไฟล์นั้นถูกดึงเข้า client bundle จะ build error ทันทีแทนที่จะปล่อยให้หลุดไปโดยไม่รู้ตัว กติกานี้ถูกเขียนไว้แล้วใน `CLAUDE.md` จากบทที่ 2 ว่าทุกไฟล์ใน `lib/server/` ต้องมี import นี้บรรทัดแรกเสมอ

สั่ง Claude Code แบบนี้

งานตั้งโปรเจกต์ครั้งแรกมีหลาย layer ต่อเนื่องกัน (folder structure → docker-compose → schema เริ่มต้น → proxy.ts) เหมาะกับการใช้ **plan mode** ก่อนเพื่อให้ Claude เห็นภาพรวมทั้งหมดก่อนเริ่มสร้างไฟล์จริง:

ช่วยตั้งโครงโปรเจกต์ Next.js 16.2 (App Router) + TypeScript + Prisma + PostgreSQL สำหรับระบบ POS ร้านอาหาร โดยมีเงื่อนไข:

- แบ่ง route group เป็น (customer) (pos) (kds) (admin) ตามที่คุยกัน
- ต้องรองรับ multi-tenant ตั้งแต่ schema แรก มี Tenant, Branch, RestaurantTable, Staff เป็นตารางราก แม้อตอนนี้จะมีร้านเดียว
- โค้ดที่แตะฐานข้อมูลหรือคำนวณเงินทั้งหมดต้องอยู่ใน `lib/server/` และ import "server-only" ทุกไฟล์
- ใช้ `docker-compose.yml` รัน Postgres สำหรับ dev เท่านั้น
- middleware ใช้ `proxy.ts` ตาม convention ของ Next.js 16.2 ไม่ใช่ `middleware.ts` แบบเก่า

ขอแผนโครงสร้างไฟล์ทั้งหมดก่อน ยังไม่ต้องสร้างไฟล์จริง แล้วรอให้ฉันอนุมัติก่อนค่อยลงมือ

สิ่งที่ต้องรีวในผลลัพธ์: เช็คว่าแผนที่ได้ไม่มี `middleware.ts` หลงเหลืออยู่ (ต้องเป็น `proxy.ts` เท่านั้นตามเวอร์ชัน Next.js 16.2), เช็คว่าทุกตารางที่ผูกกับร้านมี `branchId` / `tenantId` ตั้งแต่ตัวแรก ไม่ใช่ “ไว้ค่อยเติมทีหลัง”, และเช็คว่า Claude ไม่ได้เสนอให้ import Prisma client ตรง ๆ ใน client component ที่ไหนเลย — ถ้าเจอจุดไหนที่หลุด ให้สั่งแก้ก่อน apply แผน หลังจากแผนผ่านแล้วค่อยให้ Claude ลงมือสร้างไฟล์จริงทั้งหมด แล้วรัน `npx prisma migrate dev --name init` ด้วยตัวเองเพื่อตรวจว่า migration แรกสร้างตารางถูกต้องตามที่ออกแบบไว้

มุมมองของร้าน

ถ้าคุณเป็นเจ้าของร้านที่กำลังจะจ้างทีมทำระบบนี้ให้ ประเด็น multi-tenant ในบทนี้อาจฟังดูเป็นเรื่องทางเทคนิคที่ไกลตัว แต่จริง ๆ แล้วมันคือคำถามธุรกิจตรง ๆ ว่า “ถ้าร้านฉันขยายสาขา ระบบนี้จะรองรับได้โดยไม่ต้องสร้างใหม่ทั้งหมดไหม” ถามคำถามนี้กับทีมพัฒนาตั้งแต่วันแรกที่คุยงาน เพราะถ้าคำตอบคือ “ตอนนี้ทำแบบร้านเดียวไปก่อน ค่อยคิดทีหลัง” นั้นหมายความว่าวันที่คุณเปิดสาขาสอง คุณอาจต้องจ่ายเงินสร้างระบบใหม่เกือบทั้งหมดอีกรอบ

อีกจุดที่ควรถามคือเรื่อง environment แยกระหว่าง dev กับ production — ถ้าทีมพัฒนาพัฒนาและทดสอบบนฐานข้อมูลเดียวกับที่ร้านใช้งานจริง ความเสี่ยงที่การทดสอบจะไปกระทบข้อมูลขายจริงมีสูงมาก ควรถามให้ชัดว่ามีการแยก dev/production หรือไม่ตั้งแต่ต้น

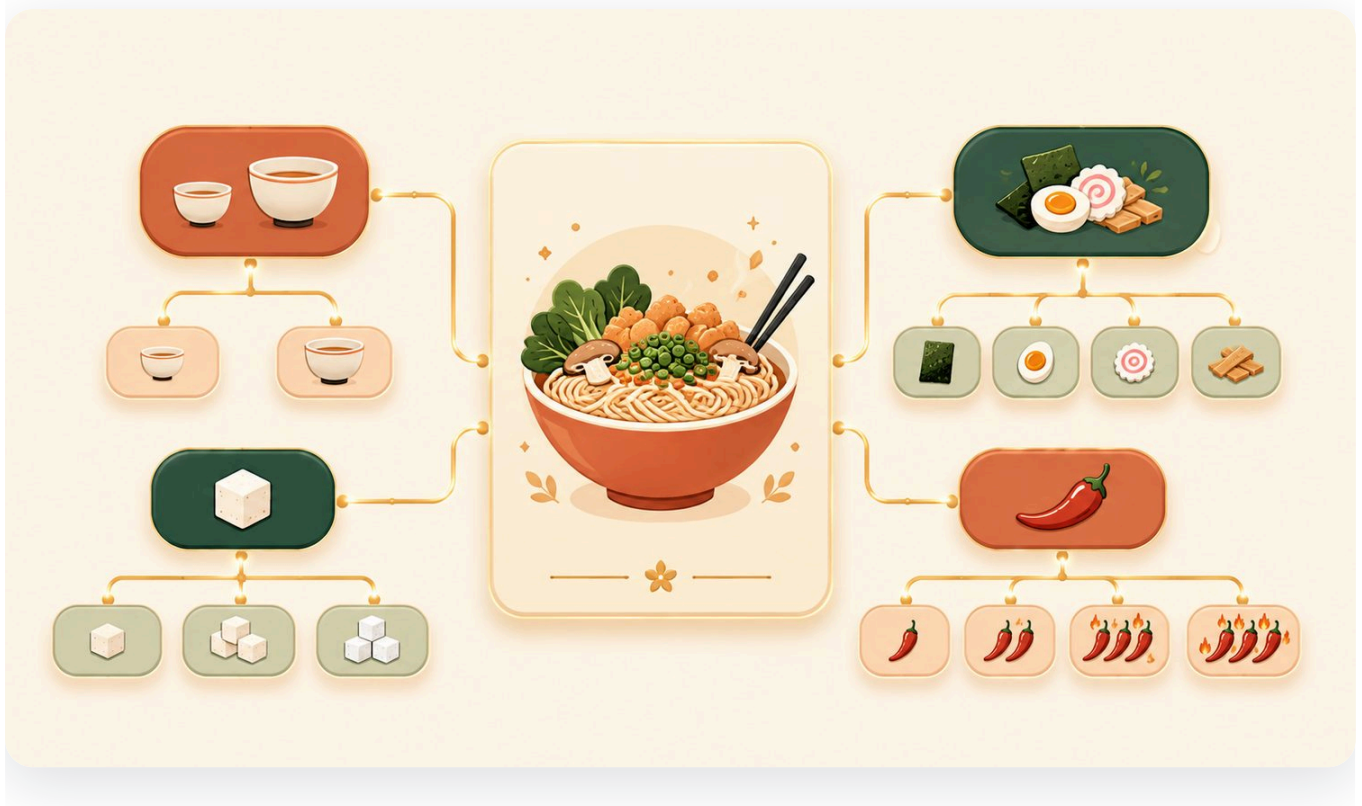
กล่องเตือนที่ต้องรู้ก่อนไปบดไป

❌ **พลาดบ่อย:** เริ่มสร้างตาราง `Order`, `MenuItem` โดยยังไม่มี `Tenant` / `Branch` เพราะคิดว่า “ตอนนี้ร้านเดียวก่อน” แล้วค่อยเติมทีหลัง
✅ **ทำแบบนี้:** วาง `Tenant` และ `Branch` เป็นรากของทุกตารางตั้งแต่ migration แรก แม้จะมีแค่ร้านเดียวและสาขาเดียวตอนนี้ก็ตาม

❌ **พลาดบ่อย:** เขียนไฟล์ `middleware.ts` ตามความเคยชินจากโปรเจกต์เก่า หรือปล่อยให้ Claude Code เขียนตาม pattern เก่าที่เรียนรู้มา
✅ **ทำแบบนี้:** ใช้ `proxy.ts` ตาม convention ของ Next.js 16.2 และเช็กในทุก PR ว่าไม่มี `middleware.ts` หลงเหลืออยู่ในโปรเจกต์

จำ 3 ข้อ 4 หน้าจอของระบบ POS แปลงตรงเป็น route group (`customer`) (`pos`) (`kds`) (`admin`) ใน App Router ได้พอดี พร้อมแยก layout และสิทธิ์ของแต่ละส่วนชัดเจน multi-tenant (`Tenant` / `Branch`) ต้องอยู่ในฐานข้อมูลตั้งแต่ migration แรก แม้ระบบจะเริ่มจากร้านเดียว เพราะเติมทีหลังคือการรื้อ schema เกือบทั้งระบบ โค้ดที่แตะฐานข้อมูลหรือคำนวณเงินต้องอยู่ใน `lib/server/` และ import `server-only` ทุกไฟล์ เพื่อกันไม่ให้ logic การเงินหลุดไปอยู่ฝั่ง client

ออกแบบเมนูให้รองรับของจริง



ลูกค้าคนหนึ่งสั่งชามไข่มุก — แคนัน ฟังดูง่าย จนกระทั่งคุณเปิดหน้าจอสั่งของจริงแล้วเจอว่าต้องเลือก **ขนาด** (M หรือ L) เลือก **ความหวาน** (0% 25% 50% 75% 100%) แล้วยังต้องเลือก **ท็อปปี้ง** ได้มากกว่าหนึ่งอย่าง (ไข่มุก วุ้นมะพร้าว พุดดิง) โดยแต่ละท็อปปี้งบวกราคาเพิ่มไม่เท่ากัน ถ้าร้านมีเมนูคอมโบที่ให้เลือกเครื่องดื่มจากในลิสต์ แล้วเครื่องดื่มแต่ละตัวก็ยังคงต้องเลือกความหวานได้อีก — คุณจะเห็นว่าเมนูร้านอาหารไม่ใช่รายการ “ชื่อ-ราคา” แบบตารางเดียวจบ

ทีมที่เร่งส่งงานมักออกแบบเมนูเป็นตารางเดียว มีคอลัมน์ `size`, `sweetness`, `topping1`, `topping2`, `topping3` แบบ `hardcode` พอเจอเมนูใหม่ที่ต้องการตัวเลือกที่ 4 หรือร้านอยากเพิ่มขนาด S ก็ต้องแก้ `schema` แก่โค้ดหน้าเมนู แก่ทุกที่ที่แตะราคา — และนี่คือจุดที่ AI เขียนโค้ดเร็วเป็นดาบสองคม ถ้า `schema` ตั้งต้นผิด Claude Code จะ generate โค้ดผิดตามไปเรื่อย ๆ เร็วขึ้นเท่านั้นเอง บทนี้จะวางฐานให้ถูกตั้งแต่ตารางแรก เพราะแก้ `schema` เมนูทีหลังคือหนึ่งใน `migration` ที่เจ็บที่สุดของระบบ POS

ปัญหาของ `schema` ผิดไม่ได้จบแค่ตอนแก้โค้ด มันไหลต่อไปทั้งระบบ: หน้าเมนูฝั่งลูกค้า (บทที่ 6) ต้อง `render` ตัวเลือกตามโครงสร้างนี้ ตะกร้าและ `server action` รับออร์เดอร์ (บทที่ 7) ต้อง `validate` ตามกฎ `required` / `minSelect` / `maxSelect` ที่มาจากตารางนี้ จอครัว (บทที่ 8) ต้องพิมพ์รายการ `modifier` ที่ลูกค้าเลือกให้ครัวอ่านออก และรายงานปิดวัน (บทที่ 15) ต้องสรุปยอดขายแยกตามตัวเลือกได้ ถ้าฐานผิดตั้งแต่ต้น ทุกจุดที่วามารถต้องแก้ตามหมด นี่คือเหตุผลที่บทนี้ให้เวลากับการออกแบบมากกว่าบทอื่น ๆ

หน่วยข้อมูล 3 ชั้นที่ต้องรู้จัก

โมเดลข้อมูลเมนูของร้านอาหารสร้างจากหน่วยหลัก 3 อย่าง:

- **Item** — สินค้าที่สิ่งเดียว ๆ ได้ เช่น ชานมไข่มุก, ข้าวผัดกะเพรา, น้ำเปล่า Item หนึ่งตัวอ้างถึง **ModifierGroup** ได้หลายกลุ่ม
- **ModifierGroup** — ชุดตัวเลือก เช่น “เลือกขนาด”, “เลือกความหวาน”, “เพิ่มท็อปปิ้ง” กลุ่มนี้ผูกกับ Item โดยตรงก็ได้ หรือผูกกับ Modifier ตัวอื่นก็ได้ — ซ้อนกันได้ไม่จำกัดชั้นในทางทฤษฎี (ในทางปฏิบัติร้านอาหาร ซ้อน 2-3 ชั้นก็เกินพอแล้ว)
- **Modifier** — ตัวเลือกเดี่ยวภายใน ModifierGroup เช่น “ไซส์ L”, “หวาน 50%”, “เพิ่มไข่มุก” แต่ละตัวปรับราคาได้ (บวกหรือลบก็ได้) และตัว Modifier เองก็ มี **ModifierGroup ซ้อนข้างในได้อีก** — นี่คือกุญแจของเมนูคอมโบ

ตัวอย่างที่ทำให้ภาพชัด: เมนู “ชุดคอมโบไก่ทอด” เป็น Item ที่มี ModifierGroup ชื่อ “เลือกเครื่องดื่ม” (required, เลือกได้ 1) ภายในมี Modifier “ชานมไข่มุก” กับ “โค้ก” — ตัว Modifier “ชานมไข่มุก” เองมี ModifierGroup ซ้อนอยู่ข้างในอีกชั้นชื่อ “เลือกความหวาน” การซ้อนแบบนี้คือเหตุผลที่ ModifierGroup ต้องมีทั้ง **menuItemId** (ผูกกับ Item ตรง ๆ) และ **parentModifierId** (ผูกกับ Modifier) แบบ optional ทั้งคู่

ลองไล่ลำดับการซ้อนของตัวอย่างนี้เป็นขั้น ๆ จะเห็นภาพว่าทำไมโครงสร้างแบบตารางแบน (flat table ที่มีคอลัมน์ **topping1**, **topping2** ตายตัว) ถึงรองรับไม่ได้:

ชั้น	หน่วยข้อมูล	ตัวอย่าง	required?
1	MenuItem	ชุดคอมโบไก่ทอด	—
2	ModifierGroup (ผูกกับ Item)	เลือกเครื่องดื่ม	required, minSelect 1, maxSelect 1
3	Modifier	ชานมไข่มุก	—
4	ModifierGroup (ผูกกับ Modifier)	เลือกความหวาน	required, minSelect 1, maxSelect 1
5	Modifier	หวาน 50%	—

โครงสร้างนี้ไม่มีจุดสิ้นสุดตายตัว — ถ้าร้านอยากให้ “หวาน 50%” มีตัวเลือกน้ำแข็งต่ออีกชั้น ก็แค่ผูก ModifierGroup ใหม่เข้ากับ Modifier “หวาน 50%” โดยไม่ต้องแก้ schema เลยสักบรรทัด นี่คือการต่างสำคัญระหว่างการออกแบบแบบ normalized (แยกตาราง ผูกความสัมพันธ์) กับแบบตารางแบน: ตารางแบนต้องมีคน “เดาล่วงหน้า” ว่าจะซ้อนกี่ชั้น ส่วนแบบ normalized รองรับความลึกเท่าไรก็ได้โดยไม่ต้อง migration ใหม่

ทำไมราคาต้องเป็นจำนวนเต็มสตางค์ ห้ามใช้ Float

ข้อนี้เป็นกฎที่พลาดไม่ได้ทั้งเล่ม: เก็บราคาทุกช่องเป็น **Int** หน่วยสตางค์ (บาท × 100) ห้ามใช้

Float / Decimal แบบลอยตัวเด็ดขาด

เหตุผลไม่ใช่แค่ความสวยงามของโค้ด แต่เป็นเรื่องคณิตศาสตร์ล้วน ๆ: floating point เก็บเลขฐาน 2 ซึ่งไม่สามารถแทนเศษส่วนฐาน 10 อย่าง 0.1 ได้พอดี ลอง **0.1 + 0.2** ใน JavaScript จะได้

0.30000000000000004 ไม่ใช่ **0.3** ถ้าเผอิญเก็บราคาอาหารเป็น Float แล้วบวกลบคูณหารหลายรอบ (ราคา × จำนวน + service charge + VAT – ส่วนลด) ความคลาดเคลื่อนระดับสตางค์จะสะสมจนบิลจริงกับบิลที่ระบบคำนวณไม่ตรงกัน แม้จะเป็นเสี้ยวสตางค์ก็ตาม — และนั่นคือจุดที่ฝ่ายบัญชีร้านจะจับได้ทันทีตอนกระทบยอด

วิธีแก้คือเก็บทุกอย่างเป็นจำนวนเต็ม แล้วหารด้วย 100 เฉพาะตอนแสดงผลบนหน้าจอเท่านั้น เช่น ก๋วยเตี๋ยว ราคา 60 บาท เก็บเป็น **basePriceSatang = 6000** ไม่ใช่ **60.00**

กฎนี้ไม่ได้จำกัดแค่ราคาเมนู แต่ครอบคลุมทุกช่องที่เกี่ยวกับเงินในระบบทั้งเล่ม — **priceDeltaSatang** ของ modifier, **unitPriceSnapshotSatang** ใน OrderItem, ยอดส่วนลด, service charge, VAT ไปจนถึงยอดรวม บิลสุดท้าย ทุกตัวต้องเป็น **Int** หน่วยสตางค์ทั้งหมด เหตุผลคือความคลาดเคลื่อนของ Float ไม่ได้เกิดแค่ตอนบวกราคาอาหารตรง ๆ แต่ยิ่งเห็นชัดตอนคำนวณเปอร์เซ็นต์ เช่น ส่วนลด 15% ของยอด 333 บาท ถ้าคำนวณด้วย Float แล้วปัดเศษผิดจุดแม้แต่จุดเดียว บิลที่พิมพ์ให้ลูกค้ากับยอดที่บันทึกในฐานข้อมูลอาจต่างกันหนึ่งสตางค์ — ฟังดูเล็กน้อย แต่พอคูณด้วยจำนวนบิลต่อวัน ต่อเดือน ก็กลายเป็นตัวเลขที่กระทบยอดไม่ลงจริง ๆ รายละเอียดสูตรคำนวณเต็มรูปแบบ (ลำดับ service charge → VAT → ปัดเศษ) จะอยู่ในบทที่ 10

schema.prisma เต็มของโมดูลเมนู

นี่คือ schema ที่ครอบคลุมทุกความสัมพันธ์ที่พูดถึงข้างบน รวมถึงจุดที่มักถูกมองข้าม: **ความสัมพันธ์ระหว่าง Order กับ Item เป็น many-to-many** (ออเดอร์หนึ่งใบมีหลายเมนู เมนูหนึ่งตัวก็ถูกสั่งในหลายออเดอร์) จึงต้องมีตารางกลาง **OrderItem** ทำหน้าที่ join

```
// schema.prisma - โมเดลเมนู (ต่อยอดจาก Tenant/Branch ที่ตั้งไว้ในบทที่ 3)

model MenuCategory {
  id          String      @id @default(cuid())
  tenantId    String
  branchId    String
  name        String
  nameEn      String?
  sortOrder   Int          @default(0)
  isActive    Boolean      @default(true)
  items       MenuItem[]
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt

  @@index([tenantId, branchId])
}

model MenuItem {
  id          String      @id @default(cuid())
  tenantId    String
  branchId    String
  categoryId   String
  category     MenuCategory @relation(fields: [categoryId], references: [id])
  name        String
  nameEn      String?
  description  String?
  imageUrl     String?
  // ราคาเป็นจำนวนเต็มสตางค์เสมอ (บาท x 100) - ห้ามใช้ Float
  basePriceSatang Int
  // ของหมดวันนี้ ปิดชั่วคราวโดยไมลบทิ้ง (ประวัติออเดอร์เก่ายังต้องอ้างอิง MenuItem นี้ได้)
  isAvailable Boolean      @default(true)
  sortOrder    Int          @default(0)
  modifierGroups MenuItemModifierGroup[]
  orderItems   OrderItem[]
  createdAt    DateTime      @default(now())
  updatedAt    DateTime      @updatedAt

  @@index([tenantId, branchId, categoryId])
}

model ModifierGroup {
  id          String      @id @default(cuid())
  tenantId    String
  name        String
  nameEn      String?
  // required = true ต้องเลือกก่อนบันทึกออเดอร์ได้
  required    Boolean      @default(false)
  minSelect   Int          @default(0)
  maxSelect   Int          @default(1)
  modifiers    Modifier[]
  menuItems   MenuItemModifierGroup[]
  // ผูกกับ Modifier ตัวอื่นได้ (ซ้อนกัน เช่น "เลือกเครื่องดื่ม" → ขานมไข่มุก → "เลือกความหวาน")
  parentModifierId String?
  parentModifier Modifier? @relation("ModifierNestedGroups", fields: [parentModifierId],
references: [id])
  createdAt   DateTime      @default(now())

```

```

    updatedAt    DateTime    @updatedAt
  }

  model Modifier {
    id            String      @id @default(cuid())
    tenantId      String
    groupId       String
    group         ModifierGroup @relation(fields: [groupId], references: [id])
    name          String
    nameEn        String?
    // ปรับราคาได้ทั้งบวกและลบ หน่วยสตางค์ (0 = ไม่บวกเพิ่ม)
    priceDeltaSatang Int      @default(0)
    isAvailable   Boolean     @default(true)
    sortOrder     Int         @default(0)
    // ModifierGroup ที่ซ่อนอยู่ข้างในตัวเลือกนี้ (ถ้ามี)
    nestedGroups  ModifierGroup[] @relation("ModifierNestedGroups")
    orderItemModifiers OrderItemModifier[]
  }

  // join table: MenuItem ↔ ModifierGroup (many-to-many)
  // กลุ่มเดียวกัน เช่น "เพิ่มท็อปปิ้ง" ใช้ซ้ำได้กับหลายเมนู
  model MenuItemModifierGroup {
    id            String      @id @default(cuid())
    menuItemId    String
    menuItem      MenuItem    @relation(fields: [menuItemId], references: [id])
    modifierGroupId String
    modifierGroup ModifierGroup @relation(fields: [modifierGroupId], references: [id])
    sortOrder     Int         @default(0)

    @@unique([menuItemId, modifierGroupId])
  }

  // join table: Order ↔ MenuItem (many-to-many ผ่านตารางกลางนี้)
  model OrderItem {
    id            String      @id @default(cuid())
    orderId       String
    order         Order       @relation(fields: [orderId], references: [id])
    menuItemId    String
    menuItem      MenuItem    @relation(fields: [menuItemId], references: [id])

    // --- snapshot ราคา/ชื่อ ณ เวลาสั่ง (ออกแบบของเราเอง ไม่ใช่มาตรฐาน Prisma หรือมาตรฐานอุตสาหกรรม) ---
    // เก็บไว้เพราะถ้าเมนูขึ้นราคาพวงนี้ บิลของเมื่อวานต้องยังโชว์ราคาเดิมที่ลูกค้าจ่ายจริง
    nameSnapshot  String
    unitPriceSnapshotSatang Int

    quantity      Int         @default(1)
    modifiers      OrderItemModifier[]
    createdAt     DateTime    @default(now())
  }

  // snapshot ของ modifier ที่เลือกในแต่ละ OrderItem - เหตุผลเดียวกับด้านบน
  model OrderItemModifier {
    id            String      @id @default(cuid())
    orderItemId    String
    orderItem      OrderItem @relation(fields: [orderItemId], references: [id])
    modifierId     String
    modifier        Modifier @relation(fields: [modifierId], references: [id])
  }

```

```
nameSnapshot      String
priceDeltaSnapshotSatang  Int
}
```

จุดที่ต้องขีดเส้นใต้: `required`, `minSelect`, `maxSelect` อยู่ที่ `ModifierGroup` ไม่ใช่ที่ `Modifier` เพราะกฎ “ต้องเลือกอย่างน้อย 1 อย่าง” เป็นกฎของกลุ่ม ไม่ใช่ของตัวเลือกใดตัวเลือกหนึ่ง เช่น `ModifierGroup` “เลือกความหวาน” ตั้ง `required = true`, `minSelect = 1`, `maxSelect = 1` (เลือกได้แค่ 1) ส่วน `ModifierGroup` “เพิ่มที่อปั้ง” ตั้ง `required = false`, `minSelect = 0`, `maxSelect = 3` (ไม่บังคับ เลือกได้สูงสุด 3)

รายละเอียดอีกจุดที่มักถูกมองข้ามคือ `tenantId` / `branchId` ที่แปะอยู่แทบทุกโมเดล นี่ไม่ใช่ความซ้ำซ้อนโดยไม่มีเหตุผล แต่เป็นผลจากการตัดสินใจตั้งแต่บทที่ 3 ว่าระบบนี้ต้อง **multi-tenant** ตั้งแต่แรก แม้ร้านของผู้่านจะมีสาขาเดียวก็ตาม เพราะถ้าวันหนึ่งร้านขยายสาขาเพิ่ม การเติม `branchId` เข้าไปที่หลังในตารางที่มีข้อมูลอยู่แล้วเป็นงานที่เจ็บกว่าการใส่ไว้ตั้งแต่ต้นมาก ส่วน `@@index([tenantId, branchId])` ที่ใส่ไว้ก็เพื่อให้ query ที่กรองตามสาขา (ซึ่งเกิดแทบทุก request ของระบบ POS) ทำงานเร็วตั้งแต่วันแรกที่ข้อมูลยังน้อย ไม่ต้องรอให้ตารางโตแล้วค่อยมาแก้ทีหลัง

การใช้ `@default(cuid())` แทน auto-increment integer ก็เป็นการตัดสินใจที่ตั้งใจ — cuid สร้างจาก client ได้โดยไม่ต้องรอฐานข้อมูล generate ID กลับมาก่อน ซึ่งมีประโยชน์ตอนสร้างข้อมูลซ้อนกันหลายชั้นแบบใน seed script ด้านล่าง (สร้าง Modifier พร้อม nested ModifierGroup ในคำสั่งเดียว) และไม่เปิดเผยว่าร้านมีเมนูที่รายการหรือมีออร์เดอร์ก็ไ้แล้วผ่านการเดาจากเลข ID ต่อเนื่องแบบ auto-increment

ส่วนโมเดล `MenuItemModifierGroup` ที่ทำหน้าที่ join ระหว่าง `MenuItem` กับ `ModifierGroup` ก็คุ้มค่าที่จะอธิบายเพิ่ม เพราะมือใหม่มักสงสัยว่าทำไมไม่ผูก `ModifierGroup` เข้ากับ `MenuItem` ตรง ๆ ด้วย foreign key เดียว คำตอบคือ ModifierGroup อย่าง “เพิ่มที่อปั้ง” หรือ “เลือกความหวาน” มักถูกใช้ซ้ำกับเมนูหลายตัว (เครื่องดื่มทุกแก้วในร้านอาจใช้ ModifierGroup ความหวานชุดเดียวกัน) ถ้าผูกแบบ one-to-many ตรง ๆ จะบังคับให้แต่ละเมนูต้องมี ModifierGroup เป็นของตัวเองแยกกันแม้เนื้อหาจะเหมือนกันทุกประการ ทำให้ร้านที่อยากแก้ “เพิ่มความหวาน 125%” ให้ทุกเมนูเครื่องดื่มพร้อมกันต้องไปไล่แก้ทีละเมนู ตารางกลางแบบ many-to-many จึงแก้ปัญหานี้ได้ตรงจุด — แก่ ModifierGroup ที่เดียว มีผลกับทุกเมนูที่ผูกอยู่ทันที

ของหมดวันนี้: `isAvailable` vs ลบทิ้ง

ของหมดกลางวันเป็นเหตุการณ์ปกติของร้านอาหาร ไม่ใช่ของแปลก ระบบต้องรองรับให้พนักงานกดปิดเมนูชั่วคราวได้ในไม่กี่วินาที — นี่คือหน้าที่ของ `isAvailable = false` ห้ามใช้การลบแถวทิ้ง (`DELETE`) เด็ดขาด เพราะ `OrderItem` เรายังมี foreign key อ้างถึง `MenuItem` นั้นอยู่ ถ้าลบทิ้งจริงจะทำให้ประวัติบิลเก่าพังหรือ query error ทันที การปิดเมนูที่ถูกต้องคือ toggle flag แล้วให้หน้าเมนูฝั่งลูกค้ากรอง `isAvailable = true` ออกมาแสดงเท่านั้น ส่วนเมนูที่เลิกขายถาวรจริง ๆ ก็ยังคงเก็บแถวไว้ในฐานข้อมูล เพียงแค่ไม่แสดงผล — การ “ลบ” ของจริงในระบบนี้ควรเกิดขึ้นน้อยมากหรือไม่เกิดขึ้นเลย

ในทางปฏิบัติ การปิดเมนูของหมดต้องทำได้จากหน้าจอที่พนักงานหน้าครัวเข้าถึงง่ายที่สุด ไม่ใช่ซ่อนอยู่ลึกในเมนูแอดมิน เพราะสถานการณ์จริงคือครัวรู้ตัวว่าวัตถุดิบหมดกลางคือออร์เดอร์ ต้องกดปิดได้ทันทีก่อนออร์เดอร์

ถัดไปจะสั่งเมนูเดิมเข้ามาอีก ถ้าระบบทำให้ขั้นตอนนี้ยุ่งยาก พนักงานจะเลี่ยงไปบอกปากเปล่าแทน ซึ่งย้อนกลับ
ไปเป็นปัญหาเดิมที่ระบบ POS ถูกสร้างมาเพื่อแก้ไขตั้งแต่ต้น — ข้อมูลไม่ตรงกันระหว่างครัวกับหน้าจอลูกค้า
เมนูเดียวกันควรมีสวิตช์เปิด-ปิดที่ทั้งพนักงานหน้าร้าน (POS บทที่ 9) และครัว (KDS บทที่ 8) กดได้ ไม่ใช่สิทธิ์
ของแอดมินคนเดียว

รูปเมนูเก็บที่ไหน

เก็บเป็น URL ในคอลัมน์ `imageUrl` ไม่เก็บไฟล์ตรงในฐานข้อมูล ตัวไฟล์จริงควรอยู่บน object storage (เช่น
S3-compatible bucket) แล้ว `imageUrl` เก็บแค่ path หรือ URL เต็ม เหตุผลคือฐานข้อมูล relational ไม่ได้ถูก
ออกแบบมาให้เก็บไฟล์รูปขนาดใหญ่จำนวนมาก การ query และ backup จะช้าลงเรื่อย ๆ ตามจำนวนรูป และ
Next.js เองก็มีระบบ image optimization (`next/image`) ที่ทำงานได้ดีกับ URL ภายนอกอยู่แล้ว รายละเอียด
การอัปโหลดและ optimize รูปจะพูดต่อในบทที่ 6 ตอนทำหน้าเมนูฝั่งลูกค้า

จุดที่ควรวางแผนไว้ตั้งแต่ต้นคือการตั้งชื่อไฟล์รูป ถ้าตั้งชื่อไฟล์ตามชื่อเมนูตรง ๆ (เช่น `pad-krapow.jpg`)
แล้ววันหนึ่งร้านเปลี่ยนรูปเมนูเดิมแต่ตั้งชื่อไฟล์ใหม่ทับชื่อเดิม ระบบ cache ของเบราว์เซอร์หรือ CDN อาจยัง
โชว์รูปเก่าค้างอยู่นานเป็นวัน วิธีที่ปลอดภัยกว่าคือให้ชื่อไฟล์มีส่วนที่เปลี่ยนทุกครั้งทีอัปโหลดใหม่ (เช่น hash
ของเนื้อไฟล์ หรือ timestamp ต่อท้าย) เพื่อให้ URL เปลี่ยนไปด้วยเมื่อรูปเปลี่ยน แล้วปล่อยให้ cache เดิมหมด
อายุตามธรรมชาติโดยไม่ต้องไปเคลียร์ cache ของทุกอุปกรณ์ลูกค้าเอง

seed เมนูตัวอย่างด้วย Claude Code

หลังจากมี schema แล้ว ให้ Claude Code เขียน seed script เพื่อทดสอบว่าความสัมพันธ์ที่ซ้อนกันทำงานถูก
จริง ตัวอย่างที่ครอบคลุมเคสยากสุด (คอมโบที่มี modifier ซ้อน modifier):

```

// prisma/seed.ts
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

async function main() {
  const tenant = await prisma.tenant.create({
    data: { name: "ร้านทดสอบ" },
  });
  const branch = await prisma.branch.create({
    data: { tenantId: tenant.id, name: "สาขาหลัก" },
  });

  const category = await prisma.menuCategory.create({
    data: {
      tenantId: tenant.id,
      branchId: branch.id,
      name: "เครื่องดื่ม",
      nameEn: "Drinks",
    },
  });

  // ModifierGroup: เลือกความหวาน (ซ่อนอยู่ได้ modifier "ขนมไข่มุก")
  const sweetnessGroup = await prisma.modifierGroup.create({
    data: {
      tenantId: tenant.id,
      name: "เลือกความหวาน",
      required: true,
      minSelect: 1,
      maxSelect: 1,
      modifiers: {
        create: [
          { tenantId: tenant.id, name: "หวาน 0%", priceDeltaSatang: 0 },
          { tenantId: tenant.id, name: "หวาน 50%", priceDeltaSatang: 0 },
          { tenantId: tenant.id, name: "หวาน 100%", priceDeltaSatang: 0 },
        ],
      },
    },
  });

  // ModifierGroup: เลือกเครื่องดื่ม (ผูกกับ Item ตรง ๆ) – Modifier ตัวนี้มี nested group
  const drinkGroup = await prisma.modifierGroup.create({
    data: {
      tenantId: tenant.id,
      name: "เลือกเครื่องดื่ม",
      required: true,
      minSelect: 1,
      maxSelect: 1,
      modifiers: {
        create: [
          {
            tenantId: tenant.id,
            name: "ขนมไข่มุก",
            priceDeltaSatang: 0,
            // ตัวเลือกนี้เปิดเมนูย่อยต่อ: ต้องเลือกความหวานอีกชั้น
            nestedGroups: { connect: { id: sweetnessGroup.id } },
          },
        ],
      },
    },
  });
}

```

```

    },
    { tenantId: tenant.id, name: "โค้ก", priceDeltaSatang: 0 },
  ],
},
},
});

// ModifierGroup: เพิ่มท็อปปิ้ง (ไม่บังคับ เลือกได้หลายอัน)
const toppingGroup = await prisma.modifierGroup.create({
  data: {
    tenantId: tenant.id,
    name: "เพิ่มท็อปปิ้ง",
    required: false,
    minSelect: 0,
    maxSelect: 3,
    modifiers: {
      create: [
        { tenantId: tenant.id, name: "ไข่มุก", priceDeltaSatang: 1000 },
        { tenantId: tenant.id, name: "วนมะพร้าว", priceDeltaSatang: 1000 },
        { tenantId: tenant.id, name: "พุดดิ้ง", priceDeltaSatang: 1500 },
      ],
    },
  },
});

await prisma.menuItem.create({
  data: {
    tenantId: tenant.id,
    branchId: branch.id,
    categoryId: category.id,
    name: "ชุดคอมโบไก่ทอด",
    basePriceSatang: 12900, // 129.00 บาท
    modifierGroups: {
      create: [
        { modifierGroupId: drinkGroup.id, sortOrder: 1 },
        { modifierGroupId: toppingGroup.id, sortOrder: 2 },
      ],
    },
  },
});
}

main()
  .catch((e) => {
    console.error(e);
    process.exit(1);
  })
  .finally(() => prisma.$disconnect());

```

รัน `npx prisma db seed` แล้วเปิด Prisma Studio ตรวจสอบว่าความสัมพันธ์ต่อกันถูกต้องจริง — โดยเฉพาะ `nestedGroups` ของ Modifier “ชานมไข่มุก” ต้องเห็น ModifierGroup “เลือกความหวาน” โผล่มา ถ้าไม่เห็น แปลว่า schema หรือ seed ผิดจุดใดจุดหนึ่ง

seed script แบบนี้มีประโยชน์มากกว่าแค่ทดสอบว่า relation ต่อกันถูก มันยังเป็นเอกสารที่ทีมใหม่หรือ Claude Code เองอ่านแล้วเข้าใจโครงสร้างเมนูได้เร็วกว่าอ่าน schema เปล่า ๆ เพราะเห็นตัวอย่างข้อมูลจริงประกอบ แนะนำให้เก็บ seed ที่ครอบคลุมเคสยาก (เช่น เคสซ้อน 3 ชั้นแบบนี้) ไว้ถาวรในโปรเจกต์ ไม่ใช่ลบทิ้งหลังทดสอบผ่านครั้งแรก เพราะทุกครั้งที่แก้ schema เกี่ยวกับเมนู การรัน seed ใหม่แล้วเช็คด้วยตาใน Prisma Studio คือวิธีตรวจสอบที่เร็วและตรงไปตรงมาที่สุด ก่อนจะไปเขียน automated test ที่ซับซ้อนกว่านั้นในบทหลัง ๆ

สั่ง Claude Code แบบนี้

ให้ Claude ออกแบบ schema ในโหมด plan ก่อน อย่าปล่อยให้ฟุ้งเขียนไฟล์ทันที เพราะ schema เมนูเป็นจุดที่แก้ที่หลังแพงที่สุดของทั้งระบบ

```
ช่วยออกแบบ schema.prisma สำหรับโมดูลเมนูร้านอาหาร โดยมีเงื่อนไขตายตัวดังนี้:
1. MenuCategory → MenuItem → ModifierGroup → Modifier
2. ModifierGroup ผูกกับ MenuItem ได้ และผูกกับ Modifier ตัวอื่นได้ (ซ้อนกันได้ไม่จำกัดชั้น)
   ตัวอย่าง: คอมโบ → เลือกเครื่องดื่ม → ชานมไข่มุก → เลือกความหวาน
3. ModifierGroup มี required / minSelect / maxSelect
4. ราคาทุกช่องเป็น Int หน่วยสตางค์ ห้ามใช้ Float หรือ Decimal เด็ดขาด
5. Order กับ MenuItem เป็น many-to-many ผ่านตารางกลาง OrderItem
6. OrderItem และ OrderItemModifier ต้องมีคอลัมน์ snapshot ชื่อและราคา ณ เวลาสั่ง
   แยกจากราคาปัจจุบันใน MenuItem/Modifier
7. ห้ามใช้ onDelete: Cascade กับ MenuItem/Modifier ที่ถูก OrderItem อ้างถึง
   (ของหมด/เลิกขาย ต้อง soft-delete ด้วย isAvailable ไม่ใช่ลบแถว)
```

ก่อนเขียนไฟล์ ให้สรุปโครงสร้างความสัมพันธ์เป็นข้อความให้ผมอนุมัติก่อน แล้วค่อยสร้าง migration

สิ่งที่ต้องรีวิวใน output ของ Claude ทุกครั้ง: (1) เช็คจากราคาทุกคอลัมน์เป็น `Int` ไม่มี `Float` / `Decimal` หลุดมาสักตัว (2) เช็คความสัมพันธ์ระหว่าง `Modifier` กับ `ModifierGroup` (nested) ตั้งเป็น optional ทั้งคู่ ไม่ใช่ required ที่จะบังคับให้ modifier ทุกตัวต้องมี nested group (3) เช็คที่ไม่มี `onDelete: Cascade` จาก `MenuItem` / `Modifier` ไปหา `OrderItem` เพราะจะทำให้ลบเมนูแล้วบิลเก่าหายไปด้วย (4) รัน `npx prisma validate` และ `npx prisma migrate dev` เองแล้วอ่าน SQL ที่ generate ออกมาก่อน apply จริง อย่าเชื่อว่า schema ที่ Claude เขียนถูกต้อง 100% เพียงเพราะ compile ผ่าน

มุมมองของร้าน

เจ้าของร้านมักไม่รู้สึกว่า “โครงสร้างเมนูในระบบ” เป็นเรื่องที่ต้องสนใจ จนกระทั่งวันที่อยากเพิ่มตัวเลือกใหม่แล้วทำไม่ได้ ตัวอย่างจริง: ร้านอยากเพิ่ม “ไซส์ XL” ให้เครื่องดื่มบางตัวเท่านั้น ไม่ใช่ทุกตัว ถ้าระบบ hardcode ขนาดไว้ในโค้ดตรง ๆ การเพิ่มนี้จะกลายเป็นงาน dev ทุกครั้ง แต่ถ้า schema ออกแบบถูกต้องตั้งแต่ต้นแบบในบทนี้ การเพิ่มตัวเลือกใหม่คือแค่กดเพิ่มแถวข้อมูล ไม่ต้องแก้โค้ด

อีกเรื่องที่กระทบเงินโดยตรงคือ **snapshot ราคาในบิลเก่า** — ถ้าระบบไม่เก็บ snapshot แล้ววันหนึ่งร้านขึ้นราคาก๋วยเตี๋ยวจาก 60 เป็น 65 บาท รายงานยอดขายย้อนหลังของเดือนที่แล้วจะโชว์ยอดผิดทันที เพราะระบบไปคำนวณด้วยราคาปัจจุบันแทนราคาที่ลูกค้าจ่ายจริง ณ ตอนนั้น เรื่องนี้เจ้าของร้านต้องถามนักพัฒนาตรง ๆ ก่อนจ้างทำ: “ถ้าผมขึ้นราคาเมนู บิลเก่าจะเปลี่ยนไหม” — ถ้าคำตอบคือ “เปลี่ยน” แปลว่า schema ยังไม่ปลอดภัยพอสำหรับใช้งานจริง

เรื่องนี้สำคัญมากขึ้นไปอีกในวันที่ร้านต้องเปิดสมุดบัญชีให้สำนักงานบัญชีหรือกรมสรรพากรตรวจ ถ้ายอดขายย้อนหลังขยับไปมาได้ตามราคาปัจจุบันของเมนู ผู้สอบบัญชีจะตั้งคำถามทันทีว่าทำไมยอดเดือนที่แล้วไม่เท่าเดิม และร้านจะไม่มีคำตอบที่ชัดเจนให้ นอกจากนี้การเปลี่ยนชื่อเมนู (เช่น เปลี่ยนชื่อจาก “ผัดไทยกุ้งสด” เป็น “ผัดไทยกุ้งสดพิเศษ”) ก็ต้องไม่ทำให้บิลเก่าเปลี่ยนชื่อตามไปด้วยเช่นกัน — นี่คือเหตุผลที่ `nameSnapshot` ต้องอยู่คู่กับ `unitPriceSnapshotSatang` เสมอ ไม่ใช่แค่ราคาอย่างเดียว การลงทุนออกแบบ schema ให้ถูกต้องตั้งแต่บั้นต้นจึงไม่ใช่แค่เรื่องความสะดวกของโค้ด แต่คือการป้องกันปัญหาที่จะโผล่มาตอนร้านเปิดมาแล้วครึ่งปีและข้อมูลในระบบเริ่มมีค่าที่ย้อนกลับไปแก้ไขไม่ได้

กล่องพลาดบ้อย

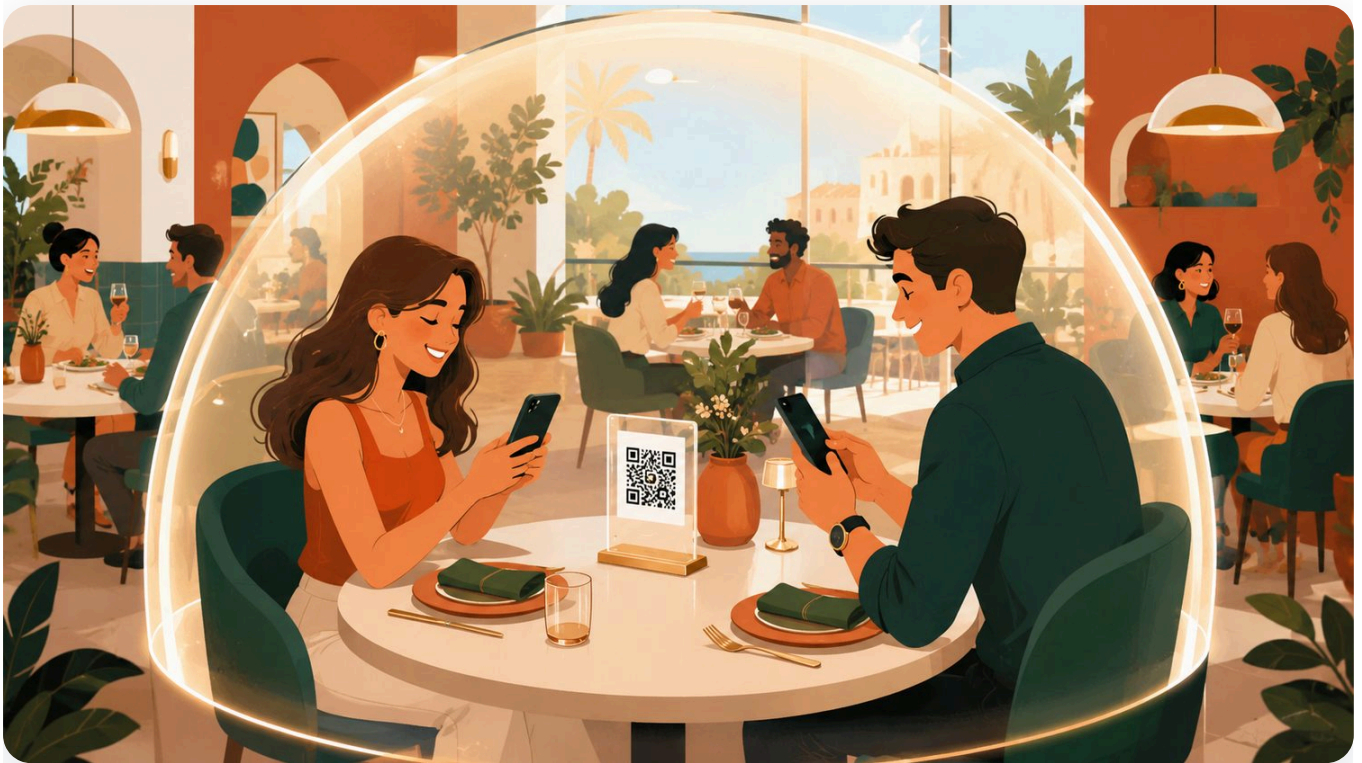
❌ **พลาดบ้อย:** เก็บราคาเป็น `Float` เช่น `price: 60.00` เพราะเขียนง่าย ดูเป็นธรรมชาติ ✅ **ทำแบบนี้:** เก็บเป็น `Int` หน่วยสตางค์เสมอ `basePriceSatang: 6000` แล้วหารด้วย 100 ตอนแสดงผลบนหน้าจอเท่านั้น

❌ **พลาดบ้อย:** อ้างราคาจาก `MenuItem.basePriceSatang` แบบ join สด ๆ ทุกครั้งที่เปิดดูบิลเก่า ✅ **ทำแบบนี้:** `OrderItem` เก็บ `unitPriceSnapshotSatang` และ `nameSnapshot` เป็นของตัวเอง ณ เวลาสั่ง ไม่ต้องพึ่งราคาปัจจุบันของเมนู

❌ **พลาดบ้อย:** กด `DELETE` แถว `MenuItem` ที่ร้านที่เลิกขายเมนูนั้น ทำให้บิลเก่าที่อ้างถึงเมนูนี้ query ไม่ขึ้นหรือ error ✅ **ทำแบบนี้:** ปิดด้วย `isAvailable = false` เก็บแถวไว้ตลอด ให้นำเมนูลูกค้านำกรองแสดงเฉพาะ `isAvailable = true`

จำ 3 ข้อ 1. ราคาทุกช่องในระบบเก็บเป็น `Int` หน่วยสตางค์เท่านั้น ห้ามใช้ `Float / Decimal` เด็ดขาด 2. `ModifierGroup` ซ้อนได้ทั้งบน `MenuItem` และบน `Modifier` — นี่คือนกไกที่รองรับเมนูคอมโบและตัวเลือกหลายชั้น 3. `OrderItem / OrderItemModifier` ต้องเก็บ snapshot ชื่อและราคา ณ เวลาสั่งเสมอ เพราะราคาเมนูเปลี่ยนได้ แต่บิลที่ออกไปแล้วต้องไม่เปลี่ยน

โต๊ะ QR และ session ของโต๊ะ



ร้านหนึ่งเคยเจอปัญหานี้: ลูกค้าคนหนึ่งถ่ายรูป QR โต๊ะ 12 เก็บไว้ในมือถือ กลับบ้านไปแล้วยังเปิดสแกนสั่งอาหารจากที่บ้านได้อีก 3 วันให้หลัง — ออร์เดอร์วิ่งเข้าจอครัวของร้านตามปกติ เพราะระบบไม่เคยตรวจว่า “โต๊ะ 12 ตอนนี้มีลูกค้านั่งอยู่จริงไหม” เอาแค่ tableId ที่ฝังอยู่ใน URL ถูกต้องก็พอ

ความเสียหายที่เกิดขึ้นไม่ได้จบแค่วัตถุดิบที่ครัวทำทิ้งไปเปล่า ๆ พนักงานเสียเวลาทำอาหารที่ไม่มีใครมารับ ครัวสืบสวนว่าออร์เดอร์นี้มาจากไหนทั้งที่โต๊ะ 12 วางอยู่ตรงหน้า และถ้าเกิดขึ้นซ้ำหลายครั้งจนพนักงานเริ่มไม่เชื่อออร์เดอร์ที่มาจากจอครัว ก็จะกลับไปใช้วิธีเดินไปถามลูกค้าทุกโต๊ะเหมือนเดิม ซึ่งขัดกับเป้าหมายทั้งหมดของการทำระบบ QR ตั้งแต่แรก

นี่คือเหตุผลที่ QR โต๊ะไม่ใช่แค่ “ลิงก์ไปหน้าเมนู” แต่ต้องผูกกับแนวคิดที่เรียกว่า **table session** — สถานะที่บอกว่าโต๊ะนี้เปิดให้สั่งอยู่จริงหรือไม่ ใครเป็นคนเปิด และเปิดมาถึงเมื่อไหร่

ทำไมไม่ควรใส่ tableId ตรง ๆ ใน URL

วิธีที่ง่ายที่สุดแต่อันตรายที่สุดคือทำ URL แบบ `/t/12` หรือ `/menu?tableId=12` ตรง ๆ ปัญหาคือ:

- **เดาได้** — ใครเปลี่ยนเลขท้าย URL จาก 12 เป็น 13 ก็เห็นโต๊ะอื่นได้ทันที
- **ไม่หมดอายุ** — QR ที่พิมพ์ติดโต๊ะไว้ถาวรจะใช้ได้ตลอดกาล ไม่ว่าจะมีลูกค้านั่งอยู่จริงหรือไม่

- **ปลอมง่าย** — คนที่เคยมากินแล้วเก็บรูป QR ไว้ กลับมาสั่งซ้ำได้จากที่ไหนก็ได้โดยที่ร้านไม่รู้ตัว

ทางแก้คือแยกสิ่งที่ QR ชี้ไปออกจาก “สิทธิ์ในการสั่งอาหาร” สองเรื่องนี้เป็นคนละชั้นกัน: `tableCode` เป็นแค่ตัวบอกว่า URL นี้คือโต๊ะไหน (ส่มเป็นสตริงยาวพอสมควร ไม่ใช่เลข 1, 2, 3 ที่เดาง่าย) ส่วนสิทธิ์สั่งอาหารจริงมาจาก `table session` ที่ต้องถูกพนักงานเปิดก่อนเท่านั้น สแกน QR แล้วไม่มี session เปิดอยู่ = เข้าไม่ได้ ต้องเรียกพนักงาน

table session ทำงานยังไง

แนวคิดคือโต๊ะแต่ละตัวมีสถานะ “เปิด/ปิด” คล้ายกะทำงานของพนักงาน:

1. **เปิด** — พนักงานกดเปิดโต๊ะจากหน้า POS ตอนลูกค้ามานั่ง ระบบสร้าง `TableSession` ใหม่ พร้อม token สุ่ม
2. **ใช้งาน** — ลูกค้าสแกน QR ที่โต๊ะ ระบบเช็คว่ามี session เปิดอยู่ ถ้ามี ก็ฝัง session token นี้ลง cookie ของเบราว์เซอร์ลูกค้า แล้วพาไปหน้าเมนู ทุกคนที่นั่งโต๊ะเดียวกันสแกน QR แผ่นเดียวกันจะได้ session token เดียวกัน — นี่คือการทำให้ **หลายคนในโต๊ะเดียวกันสั่งพร้อมกันได้** เพราะออร์เดอร์ทุกใบผูกกับ `sessionId` เดียวกัน ไม่ใช่ผูกกับตัวลูกค้าคนใดคนหนึ่ง
3. **ปิด** — พนักงานกดปิดโต๊ะตอนลูกค้าจ่ายเงินจบ session token เดิมถูกทำให้ใช้ไม่ได้อีกทันที ต่อให้มีคนยังเก็บ QR หรือ cookie เดิมอยู่ก็สั่งอาหารเพิ่มไม่ได้แล้ว

session token เก็บอยู่ใน cookie แบบ `httpOnly` (จาวาสคริปต์ฝั่งลูกค้าอ่านไม่ได้ ป้องกัน XSS ขโมย token) และมี `expiresAt` เป็นเพดานเวลาอีกชั้นหนึ่ง เพื่อพนักงานลืมกดปิดโต๊ะ

เรื่องที่ควรพูดให้ชัดคือ “หลายคนสั่งพร้อมกัน” ในที่นี้ไม่ได้แปลว่าระบบต้องรู้จักว่าใครเป็นใครในโต๊ะนั้น ระบบไม่ได้บังคับให้ลูกค้าล็อกอินหรือกรอกชื่อก่อนสั่ง — ทุกคนที่ถือมือถือแล้วสแกน QR แผ่นเดียวกันของโต๊ะจะได้ session token ตัวเดียวกันโดยอัตโนมัติ ออร์เดอร์ที่แต่ละคนกดส่งจึงถูกรวมเข้ากับบิลของโต๊ะนั้นโต๊ะเดียว ไม่ต้องมีขั้นตอนยืนยันตัวตนเพิ่ม ถ้าร้านอยากรู้ว่าใครสั่งอะไรในโต๊ะเดียวกัน (เช่น เพื่อแยกจ่ายทีหลัง) ก็ทำได้ด้วยการเพิ่มช่องชื่อเล่นแบบไม่บังคับตอนกดสั่งแต่ละครั้ง ไม่ใช่ระบบ authentication เต็มรูปแบบ — ความซับซ้อนของการยืนยันตัวตนลูกค้าไม่คุ้มกับปัญหาที่แก้ได้ในระดับ session ของโต๊ะอยู่แล้ว

ป้องกันคนนอกสแกนจากรูปเก่า

การป้องกันไม่ได้อยู่ที่ QR ยากเดาอย่างเดียว แต่คือการออกแบบให้ QR เป็นแค่ “กุญแจไปหาโต๊ะ” ส่วน “สิทธิ์สั่งอาหาร” มาจาก session ที่มีอายุจำกัดเสมอ สรุปเป็น 3 ชั้น:

- **หมดอายุ** — `TableSession.expiresAt` บังคับ (เช่น 3 ชั่วโมงต่อ session) ต่อให้พนักงานลืมปิด ลูกค้าเก่าก็สั่งต่อไม่ได้เกินเวลาที่กำหนด
- **ผูกกับ session ที่พนักงานเปิดเท่านั้น** — สแกน QR ตอนไม่มี session เปิดอยู่ = ขึ้นหน้า “กรุณาเรียกพนักงาน” ไม่ใช่หน้าเมนู ป้องกันกรณีลูกค้าเก็บรูป QR เก่ากลับไปสั่งจากบ้าน เพราะพอร้านปิด session ของโต๊ะนั้นไปแล้ว รูปเก่าก็ใช้ไม่ได้อีก

- **rate limit** — จำกัดจำนวนครั้งที่ IP หนึ่งยิงมาที่ endpoint สแกนโต๊ะเดียวกันได้ต่อนาที กันการยิง brute-force เด่า `tableCode` หรือกันบอทสแกนรัว ๆ

ทั้ง 3 ขั้นนี้ทำงานร่วมกัน ไม่ใช่แยกกันทำงานเดี่ยว ๆ — สมมติมีคนถ่ายรูป QR โต๊ะ 12 เก็บไว้ตอนมากินเมื่อสัปดาห์ที่แล้ว แล้ววันนี้กลับมาสแกนรูปเดิมอีกครั้ง ต่อให้ session ตอนนั้นยังไม่หมดอายุตามเวลา

(`expiresAt`) พนักงานก็ปิด session ไปแล้วตั้งแต่ลูกค้าคนก่อนจ่ายเงินเสร็จ ระบบจะเช็คสถานะ session กับฐานข้อมูลแล้วพบว่าไม่มี session ที่ `OPEN` ให้โต๊ะนั้นอีกต่อไป จึงพาไปหน้า “เรียกพนักงาน” แทนที่จะเข้าเมนูได้ทันที การพึ่งพาแค่ชั้นใดชั้นหนึ่งเพียงอย่างเดียว เช่นตั้งเวลาหมดอายุยาว ๆ ไว้เพื่อลูกค้ากินนาน จะทำให้ QR เก่ามีช่วงเวลาที่ยังใช้งานได้ยาวนานเกินจำเป็น ส่วนการพึ่ง session อย่างเดียวโดยไม่มี rate limit ก็เปิดช่องให้มีคนลองเดา `tableCode` ของโต๊ะอื่นแบบวนซ้ำเป็นพัน ๆ ครั้งต่อวินาทีได้

โค้ด: route handler สร้าง QR ประจำโต๊ะ

endpoint นี้ให้พนักงาน/แอดมินกดสร้างรูป QR สำหรับพิมพ์ติดโต๊ะ ใช้ `tableCode` แบบสุ่มยาว ไม่ใช่เลขโต๊ะที่เดาง่าย

```
// app/api/admin/tables/[tableId]/qr-code/route.ts
import { NextRequest, NextResponse } from "next/server";
import { randomBytes } from "crypto";
import QRCode from "qrcode";
import { prisma } from "@lib/prisma";
import { requireStaffSession } from "@lib/auth";

export async function POST(
  req: NextRequest,
  { params }: { params: Promise<{ tableId: string }> }
) {
  // เฉพาะพนักงาน/แอดมินที่ล็อกอินแล้วเท่านั้นถึงสร้าง QR ใหม่ได้
  const staff = await requireStaffSession(req);
  if (!staff) {
    return NextResponse.json({ error: "unauthorized" }, { status: 401 });
  }

  const { tableId } = await params;
  const table = await prisma.table.findUnique({ where: { id: tableId } });
  if (!table) {
    return NextResponse.json({ error: "table not found" }, { status: 404 });
  }

  // สุ่ม tableCode ใหม่ทุกครั้งที่เกิด "ออก QR ใหม่" - เพื่อ QR เก่ารั่วไหลหรือถูกถ่ายไปเผยแพร่
  const tableCode = randomBytes(12).toString("base64url");

  await prisma.table.update({
    where: { id: tableId },
    data: { tableCode },
  });

  const menuUrl = `${process.env.PUBLIC_BASE_URL}/t/${tableCode}`;
  const qrPngBuffer = await QRCode.toBuffer(menuUrl, {
    type: "png",
    width: 512,
    margin: 2,
  });

  return new NextResponse(qrPngBuffer, {
    headers: { "Content-Type": "image/png" },
  });
}
```

ตั้ง **tableCode** ใหม่ทุกครั้งที่เกิดสร้าง QR ใหม่แปลว่า QR แผ่นเก่าที่หลุดไปก็ใช้ไม่ได้ทันที ไม่ต้องรอให้ session หมดอายุ

สังเกตว่า endpoint นี้ต้องผ่าน **requireStaffSession** ก่อนเสมอ ไม่เปิดให้ใครก็ตามยิงมาสร้าง QR ใหม่ได้ตามใจ เพราะการเปลี่ยน **tableCode** มีผลข้างเคียงจริง — ถ้าคนนอกเข้าถึง endpoint นี้ได้โดยไม่ต้องล็อกอิน เขาสามารถสั่งสร้าง QR ใหม่ให้ทุกโต๊ะพร้อมกันได้ ซึ่งจะทำให้ QR ที่พิมพ์ติดโต๊ะไว้ของจริงใช้ไม่ได้ทันที กลายเป็นการโจมตีแบบ denial-of-service ต่อระบบสั่งอาหารทั้งร้านโดยไม่ต้องแฮ็กอะไรซับซ้อนเลย นี่คือเหตุผลที่การ authorize endpoint บริหารจัดการโต๊ะสำคัญไม่แพ้การป้องกัน endpoint ที่รับเงิน

โค้ด: เก็บและตรวจ table session

เมื่อลูกค้าสแกน QR แล้วเปิด `/t/<tableCode>` route handler นี้ทำหน้าที่ตรวจว่ามี session เปิดอยู่จริงไหม แล้วค่อยฝัง cookie:

```
// app/t/[tableCode]/route.ts
import { NextRequest, NextResponse } from "next/server";
import { prisma } from "@/lib/prisma";
import { checkScanRateLimit } from "@/lib/rate-limit";

const SESSION_COOKIE = "ds_table_session";

export async function GET(
  req: NextRequest,
  { params }: { params: Promise<{ tableCode: string }> }
) {
  const { tableCode } = await params;
  const ip = req.headers.get("x-forwarded-for") ?? "unknown";

  // กัน brute-force เดา tableCode หรือมอหสแกนรัว ๆ
  const allowed = await checkScanRateLimit({ ip, tableCode });
  if (!allowed) {
    return NextResponse.json({ error: "too many requests" }, { status: 429 });
  }

  const table = await prisma.table.findUnique({
    where: { tableCode },
    include: {
      sessions: {
        where: { status: "OPEN", expiresAt: { gt: new Date() } },
        orderBy: { openedAt: "desc" },
        take: 1,
      },
    },
  });

  const activeSession = table?.sessions[0];

  if (!table || !activeSession) {
    // ไม่มีโต๊ะ หรือไม่มีใครเปิด session ให้ - ต้องเรียกพนักงาน ไม่ใช่เดินตรงเข้าหน้าเมนู
    return NextResponse.redirect(new URL("/t/not-open", req.url));
  }

  const res = NextResponse.redirect(new URL("/menu", req.url));
  res.cookies.set(SESSION_COOKIE, activeSession.token, {
    httpOnly: true,
    secure: true,
    sameSite: "lax",
    // อายุ cookie ถึงตาม expiresAt ของ session ไม่ยาวเกินอายุจริงของ session
    expires: activeSession.expiresAt,
    path: "/",
  });
  return res;
}
```

ฝั่งเก็บสถานะใน Prisma ต้องมีสองโมเดลคู่กัน — `Table` เก็บ `tableCode` ปัจจุบัน และ `TableSession` เก็บประวัติการเปิด-ปิดทุกครั้ง (ไม่ overwrite ทับ เพราะร้านต้องย้อนดูได้ว่าโต๊ะนี้เปิด-ปิดกี่รอบต่อวัน):

```
model Table {
  id          String      @id @default(cuid())
  branchId    String
  label       String      // เช่น "โต๊ะ 12"
  tableCode   String      @unique
  sessions    TableSession[]
  createdAt   DateTime    @default(now())
}

model TableSession {
  id          String      @id @default(cuid())
  tableId     String
  table       Table        @relation(fields: [tableId], references: [id])
  token       String      @unique
  status      String      // "OPEN" | "CLOSED"
  openedByStaffId String
  openedAt    DateTime    @default(now())
  closedAt    DateTime?
  expiresAt   DateTime

  @@index([tableId, status])
}
```

การตรวจ session cookie สำหรับทุก request ในโซนลูกค้า (`(customer)` route group) ควรอยู่ใน `proxy.ts` (ชื่อไฟล์ middleware ตั้งแต่ Next.js 16.2) เพื่อกันไม่ให้หลุดเข้าหน้าสั่งอาหารได้โดยไม่มี cookie ที่ valid เลยแม้แต่ตอนเปิด URL หน้าเมนูตรง ๆ

เหตุผลที่ต้องเช็คซ้ำใน `proxy.ts` แม้ route handler ของ `/t/[tableCode]` จะเช็คไปแล้วชั้นหนึ่ง คือ URL ของหน้าเมนูจริง (`/menu`) เป็นคนละ URL กับ URL ที่สแกน (`/t/<tableCode>`) — ถ้าลูกค้าเผลอ bookmark หน้า `/menu` ไว้ หรือกด refresh ค้างไว้นาน ๆ จนเน็ตหลุดแล้วเปิดใหม่ Next.js จะ route ตรงไปที่ `page.tsx` ของ `/menu` โดยไม่ผ่าน route handler ของ `/t/[tableCode]` อีก ถ้าไม่มี `proxy.ts` มาดักตรวจ cookie ซ้ำอีกชั้น หน้าเมนูจะพยายาม render โดยไม่รู้เลยว่า session ที่อ้างถึงยังใช้ได้จริงหรือไม่ ระบบตรวจสอบที่ดีจึงไม่ควรพึ่งจุดเดียว — เช็คที่ทางเข้า (`/t/[tableCode]`) เพื่อสร้าง cookie ที่ถูกต้อง และเช็คซ้ำที่ `proxy.ts` ทุกครั้งที่มี request เข้าโซนลูกค้า เพื่อปิดช่องที่ลูกค้าเข้าถึงหน้าเมนูโดยข้ามขั้นตอนแรกไป

จุดที่ต้องระวังอีกอย่างคือประวัติของ `TableSession` ต้องไม่ถูก overwrite ทับ ต้องสร้างแถวใหม่ทุกครั้งที่พนักงานเปิดโต๊ะ ไม่ใช่ update แถวเดิมซ้ำ เพราะร้านต้องย้อนดูได้ว่าโต๊ะนี้เปิด-ปิดกี่รอบต่อวัน ใครเป็นคนเปิดแต่ละรอบ — ข้อมูลนี้มีประโยชน์ตอนสืบหาปัญหาย้อนหลัง เช่น ลูกค้าโต้แย้งว่า “ไม่เคยสั่งเมนูนี้” ทีมงานสามารถย้อนดูได้ว่า session ไหนเปิดช่วงเวลาไหน พนักงานคนไหนเป็นคนเปิดให้ และออร์เดอร์ไหนผูกกับ session นั้นบ้าง

สั่ง Claude Code แบบนี้

--

จุดนี้เกี่ยวกับความปลอดภัยของสิทธิ์การสั่งอาหาร ควรให้ Claude อธิบาย threat model ก่อนเขียนโค้ด แทนที่จะสั่งเขียน route handler ตรง ๆ

ช่วยออกแบบระบบ table session สำหรับ QR สั่งอาหารที่โต๊ะ ข้อกำหนด:

1. QR เข้ารหัสด้วย tableCode แบบสุ่ม ไม่ใช่ tableId ตรงๆ และเปลี่ยนใหม่ได้ทุกครั้งที่พนักงานกด "ออก QR ใหม่"
2. ลูกค้าสั่งอาหารได้ก็ต่อเมื่อมี TableSession สถานะ OPEN และยังไม่หมดอายุเท่านั้น
ถ้าไม่มี session เปิดอยู่ ให้พาไปหน้า "กรุณาเรียกพนักงาน" ไม่ใช่หน้าเมนู
3. session token เก็บใน cookie แบบ httpOnly, secure, sameSite=lax
4. ต้องมี rate limit บน endpoint ที่รับการสแกน (ต่อ IP ต่อ tableCode)
5. อธิบายก่อนว่าถ้ามีคนถ่ายรูป QR เก็บไว้แล้วเอากลับมาสแกนหลังร้านปิดโต๊ะไปแล้ว ระบบที่ออกแบบนี้ป้องกันได้ตรงจุดไหน

ขอ threat model สั้นๆ ก่อน แล้วค่อยเขียน route handler กับ Prisma schema ของ Table/TableSession

สิ่งที่ต้องรีวิวใน output: (1) cookie ต้องเป็น `httpOnly` และ `secure` เสมอ ห้าม client-side JavaScript อ่าน token ได้ (2) endpoint สแกนต้องเช็ค `expiresAt` จริง ไม่ใช่แค่ `status = OPEN` เพราะพนักงานอาจลืมกดปิด (3) เช็คว่าการปิด session (พนักงานกดจ่ายเงินจบ) ทำให้ cookie เดิมใช้ไม่ได้จริง — ต้อง invalidate ที่ฝั่ง server (เทียบ token กับ DB ทุกครั้ง) ไม่ใช่พึ่งแค่วันหมดอายุของ cookie ฝั่งเบราว์เซอร์ (4) เช็คว่า rate limit ไม่ไปบล็อกลูกค้าโต๊ะข้างกันที่ใช้ IP วงเดียวกันเปลว ๆ (Wi-Fi ร้านตัวเดียวกัน) — ควร rate limit แยกตาม `tableCode` ด้วย ไม่ใช่ IP อย่างเดียว

มุมมองของร้าน

QR โต๊ะที่ไม่มีระบบ session อยู่เบื้องหลังคือความเสี่ยงที่มองไม่เห็นจนกว่าจะเกิดปัญหา เช่น เคสที่ยกตัวอย่างต้นบท ลูกค้าสั่งจากบ้านหลังจ่ายเงินและออกจากร้านไปแล้ว 3 วัน คราวนี้ทำอาหารตามออเดอร์ที่เข้ามาโดยไม่รู้ว่ามีลูกค้านั่งอยู่จริง ร้านเสียทั้งวัตถุดิบและเวลาครัว

อีกมุมที่เจ้าของร้านควรถามทีมพัฒนา: “ถ้าโต๊ะ 5 ยังไม่มีลูกค้านั่ง มีคนสแกน QR จากรูปเก่าได้ไหม” คำตอบที่ถูกต้องคือ “ไม่ได้ เพราะไม่มี session เปิดให้” — ถ้าคำตอบคือ “ได้ แต่ order จะไปเข้าคิวรอพนักงานยืนยัน” นั่นก็เป็นทางเลือกที่พอรับได้เหมือนกัน แต่ต้องรู้ล่วงหน้าว่าระบบเลือกออกแบบมาแบบไหน ไม่ใช่มารู้ตอนเกิดปัญหาแล้ว

ยังมีอีกมุมที่เกี่ยวกับต้นทุนที่มองไม่เห็นในทีแรก คือค่าใช้จ่ายและเวลาในการพิมพ์ QR ใหม่ทุกครั้งที่ต้อง invalidate โต๊ะที่ QR หลุด ถ้าร้านมี 30 โต๊ะและออกแบบให้ต้องพิมพ์สติ๊กเกอร์ใหม่ทุกครั้งที่ทำ QR ให้หมดอายุ นั่นคืองานปฏิบัติการที่ต้องมีคนทำจริง เจ้าของร้านควรคุยกับทีมพัฒนาว่าจะ balance ระหว่าง “QR คงที่ พิมพ์ครั้งเดียวติดโต๊ะถาวร” (ปลอดภัยน้อยกว่าเล็กน้อยแต่ปฏิบัติงานง่าย) กับ “QR หมุนเวียนบ่อย” (ปลอดภัยกว่าแต่ต้องพิมพ์ใหม่บ่อย) ยังไง เพราะทั้งสองแบบใช้กลไก table session แบบเดียวกันได้ — ความถี่ในการหมุน `tableCode` เป็นแค่พารามิเตอร์ที่ปรับได้ตามความเสี่ยงที่ร้านยอมรับได้ ไม่ใช่กฎตายตัว

ในทางปฏิบัติ ร้านส่วนใหญ่เลือกทางสายกลาง: พิมพ์ QR แบบตั้งโต๊ะ (table tent) หรือแผ่นอะคริลิกที่เปลี่ยนแผ่นกระดาษข้างในได้ง่าย ไม่ต้องพิมพ์สติ๊กเกอร์ติดถาวร แล้วหมุน `tableCode` เฉพาะตอนที่มิมีเหตุผลจริง เช่น สงสัยว่า QR รั่วไหล หรือทำความสะอาดใหญ่ประจำเดือน ส่วนกลไกป้องกันหลักที่ใช้งานจริงในทุกวันคือ session ที่อธิบายไปข้างบน ไม่ใช่การหมุน `tableCode` บ่อย ๆ — นี่คือจุดที่ควรอธิบายให้เจ้าของร้านเข้าใจตั้งแต่ต้น เพื่อไม่ให้คาดหวังผิดว่าต้องเปลี่ยน QR ทุกวันถึงจะปลอดภัย

กล่องพลาดบ้อย

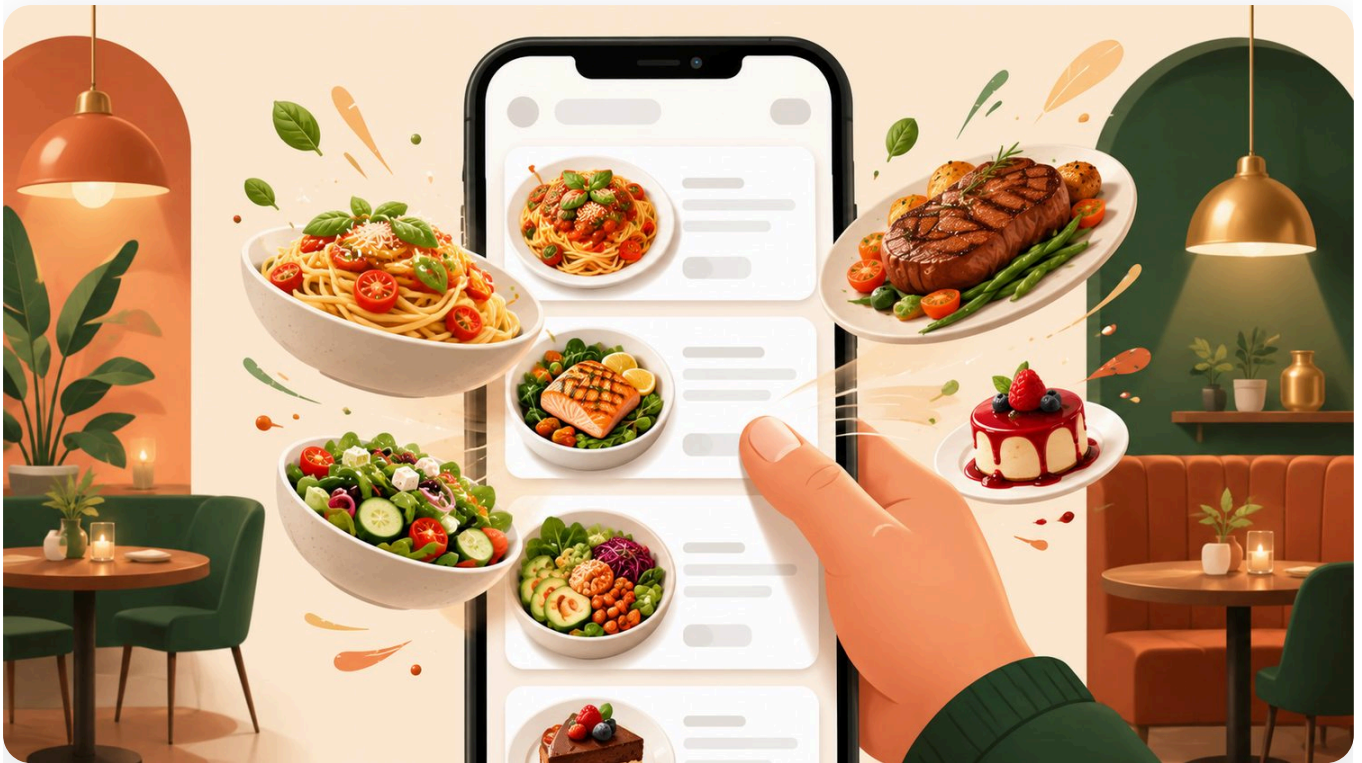
❌ พลาดบ้อย: ใส่ `tableId` ตรง ๆ ใน URL เช่น `/menu?tableId=5` ทำให้เดาโต๊ะอื่นได้ง่ายด้วยการเปลี่ยนเลข ✅ ทำแบบนี้: ใช้ `tableCode` แบบสุ่มยาว (เช่น `randomBytes(12)`) และสร้างใหม่ได้ทุกครั้งที่ต้องการ invalidate QR เก่า

❌ พลาดบ้อย: ให้สแกน QR แล้วเข้าหน้าเมนูได้ทันทีโดยไม่เช็คว่ามี session เปิดอยู่จริง ✅ ทำแบบนี้: บังคับให้ต้องมี `TableSession` สถานะ `OPEN` และยังไม่หมดอายุก่อน ไม่งั้นพาไปหน้า “เรียกพนักงาน”

❌ พลาดบ้อย: เปิด endpoint สร้าง/หมุน QR ให้ยิงได้โดยไม่ต้องล็อกอิน เพราะคิดว่าเป็นแค่ endpoint สร้างรูปภาพธรรมดา ✅ ทำแบบนี้: บังคับ `requireStaffSession` ก่อนทุกครั้ง เพราะการหมุน `tableCode` ใหม่ทำให้ QR เดิมที่พิมพ์ติดโต๊ะใช้ไม่ได้ทันที

จำ 3 ข้อ 1. แยก `tableCode` (ตัวชี้ URL) ออกจากสิทธิ์สั่งอาหารจริง (`TableSession` ที่พนักงานต้องเปิดก่อน) เสมอ 2. session token เก็บใน cookie แบบ `httpOnly` + `secure` และต้องเทียบกับสถานะใน DB ทุกครั้ง ไม่ใช่เช็คว่าวันหมดอายุของ cookie อย่างเดียว 3. การป้องกัน QR เก่าถูกนำกลับมาใช้ต้องมีอย่างน้อย 3 ชั้น: หมดอายุ, ผูกกับ session ที่พนักงานเปิด, และ rate limit

หน้าเมนูฝั่งลูกค้า



เที่ยงวันศุกร์ ร้านเต็มทุกโต๊ะ ลูกค้าคนหนึ่งสแกน QR ด้วย 4G ที่สัญญาณอ่อนเพราะคนแน่นร้าน หน้าเมนูโหลดช้า รูปอาหารตัวใหญ่โหลดไม่จบสักที กดเลือกที่อปบั้งแล้วปุ่มขยับไปมาเพราะ layout ยังไม่นิ่ง สุดท้ายกดผิดสั่งผิด — นี่คือการสแกนจริงที่หน้าเมนูฝั่งลูกค้าต้องรับมือให้ได้ทุกวัน ไม่ใช่แค่ตอน demo ในห้องประชุมที่ Wi-Fi แรงและมีมือถือเครื่องใหม่

หน้าเมนูฝั่งลูกค้าคือหน้าจอที่ ลูกค้าเห็นก่อนตัดสินใจสั่งอาหารทุกจาน ถ้าโหลดช้าหรือกดพลาดง่าย ผลกระทบไม่ใช่แค่ประสบการณ์ที่แย่ แต่กระทบยอดขายตรง ๆ — ลูกค้าที่รอนานเกินไปมักปิดแอปแล้วเรียกพนักงานแทน ซึ่งขัดกับเป้าหมายทั้งหมดของการทำ QR ordering ตั้งแต่แรก

หน้าจอนี้ยังมีข้อจำกัดที่หน้าจออื่นในระบบไม่มี: มันคือหน้าจอเดียวที่ **คนแปลกหน้าถือมือถือของตัวเองเข้าถึงได้โดยตรง** ไม่ผ่านการเทรนของพนักงานเหมือนหน้า POS หรือ KDS ลูกค้าไม่รู้ว่าจะโครงสร้างเมนูข้างหลังซับซ้อนแค่ไหน ไม่รู้ว่ามี ModifierGroup ซ้อนกันกี่ชั้น สิ่งที่เขาเห็นมีแค่สิ่งที่ปรากฏบนจอ ถ้า UX งง เขาไม่มีทางถามใครได้ทันทีนอกจากเรียกพนักงาน — ซึ่งย้อนกลับไปจุดเริ่มต้นของปัญหาที่ระบบนี้ถูกสร้างมาเพื่อลด

หลักการ UX มือถือที่พลาดไม่ได้

- **โหลดเร็ว** — ใช้ React Server Component ดึงข้อมูลเมนูจาก database ตรงบน server แล้ว render เป็น HTML ส่งมาเลย ไม่ต้องรอ client ยิ่ง fetch ช้าหลังโหลดหน้าเสร็จ ลด round trip ที่ไม่จำเป็นบนเน็ตมือถือที่

ไม่แน่นอน

- **รูปต้อง optimize** — ใช้ `next/image` แทน `<img>` ธรรมดา เพราะจะ resize/แปลง format (WebP/AVIF) และ lazy-load รูปที่ยังไม่อยู่ในจอให้อัตโนมัติ ป้องกันไม่ให้รูปเมนูตัวใหญ่กินแบนด์วิดท์จนหน้าเมนูค้าง
- **หมวดหมู่เลื่อนแนวนอน** — แถบหมวดหมู่ (เครื่องดื่ม/อาหารจานหลัก/ของหวาน) ควรอยู่บนสุดแบบ sticky แล้วเลื่อนแนวนอนได้ด้วยนิ้วเดียว ไม่บังคับให้ลูกค้าเลื่อนขึ้นลงไล่หาหมวดยาว ๆ
- **หน้าเลือกตัวเลือกไม่ทำให้กดพลาด** — ปุ่มเลือก modifier ต้องมีขนาดเตะง่าย (พื้นที่เตะอย่างน้อยราว 44x44px ตามแนวทาง accessibility ทั่วไป) เว้นระยะห่างจากปุ่มข้างเคียงพอ ไม่ให้กดโดนตัวเลือกผิดตอนรับ ๆ ในร้านที่คนพลุกพล่าน
- **สองภาษา TH/EN** — ใช้ฟิลด์คู่ `name / nameEn` ที่ตั้งไว้ตั้งแต่ schema บทที่ 4 สลับภาษาได้จากปุ่มเดียว ไม่ต้องแยกหน้าเมนูคนละชุด
- **ปุ่มใหญ่พอ** — โดยเฉพาะปุ่ม “เพิ่มลงตะกร้า” กับ “ยืนยันสั่ง” ต้องเป็นปุ่มที่เด่นและกดพลาดยากที่สุดในหน้า เพราะเป็นจุดที่ error สร้างความเสียหายสูงสุด

หลักการ “โหลดเร็ว” ในทางปฏิบัติหมายความว่าแค่เลือกใช้ server component เพียงอย่างเดียว การจัด layout ให้ **นิ่งตั้งแต่เฟรมแรก** ก็สำคัญไม่แพ้กัน — ถ้าโครงหน้าเมนูโหลดมาก่อนแล้วรูปค่อยโผล่ตามทีหลัง ทำให้ตำแหน่งปุ่มขยับไปมาระหว่างที่ลูกค้ากำลังจะกด (สิ่งที่วงการ frontend เรียกกันว่า layout shift) นี่คือสาเหตุตรง ๆ ของ “กดผิดสั่งผิด” ในฉากเปิดบท ทางแก้คือกำหนดสัดส่วนพื้นที่ของรูปไว้ล่วงหน้าเสมอ (เช่น กำหนด `width / height` ให้ `next/image` ไว้ชัดเจน) เพื่อให้เบราว์เซอร์จองพื้นที่ไว้ก่อนรูปโหลดเสร็จ ปุ่มด้านล่างจะได้ไม่ขยับตำแหน่งหนีไปมา

อีกเทคนิคที่ช่วยเรื่องความรู้สึก “เร็ว” ได้มากคือการใช้ loading skeleton ระหว่างที่ server component กำลังดึงข้อมูล แทนที่จะปล่อยหน้าจอว่างเปล่าค้างไว้ — ลูกค้าเห็นโครงหมวดหมู่และช่องรูปเลื่อนราง ๆ ปรากฏขึ้นทันที แล้วค่อยเติมเนื้อหาจริงตามมาเมื่อข้อมูลพร้อม ความรู้สึกของการรอจะสั้นลงมากแม้เวลาที่ใช้จริงจะเท่าเดิม

สิ่งที่ร้านชอบขอเพิ่ม

ร้านมักขอฟีเจอร์แนว “แนะนำเมนู” “เมนูแนะนำวันนี้” “รูปสวยเด่น” ซึ่งทำได้โดยไม่ต้องซับซ้อน — เพิ่มฟิลด์ `isRecommended` หรือ `isFeatured` แบบ boolean ใน `MenuItem` แล้วให้หน้าเมนูจัดกลุ่มรายการที่ติดแท็กนี้ขึ้นแสดงเป็นแถบพิเศษด้านบนสุด ไม่ต้องสร้างตารางหรือโมดูลใหม่ทั้งระบบ ส่วน “ของแนะนำวันนี้” ที่เปลี่ยนทุกวัน ใช้กลไกเดียวกับ `isAvailable` ที่ทำไว้แล้วในบทที่ 4 คือให้พนักงานตั้งค่าแพลิกเองได้จากหน้าแอดมิน ไม่ต้องรอ dev แก่โค้ดทุกครั้งที่ร้านอยากเปลี่ยนเมนูแนะนำ

คำแนะนำจากประสบการณ์ตรง: อย่ารีบสร้างระบบแนะนำเมนูแบบซับซ้อน เช่น อัลกอริทึมแนะนำตามประวัติการสั่งของลูกค้า หรือ A/B test ว่าจัดวางแบบไหนขายดีกว่า — ร้านอาหารทั่วไปที่เพิ่งเริ่มใช้ QR ordering แทบไม่มีทรัพยากรมากพอที่ตัวเลขพวกนี้จะมีนัยสำคัญทางสถิติ สิ่งที่คุณค่ากว่าคือให้เจ้าของร้านหรือหัวหน้าครัวตั้งค่าเมนูแนะนำได้เองจากประสบการณ์ตรงหน้าจอ (เช่น “วันนี้กุ้งสดเข้าเยอะ อยากดันเมนูกุ้ง”) ซึ่ง

ใช้แค่ boolean flag ธรรมดาตามที่อธิบายไปก็เพียงพอแล้ว ค่อยพัฒนาไปเป็นระบบซับซ้อนกว่านี้เมื่อร้านมีข้อมูลมากพอจริง ๆ

โค้ด: server component ดึงเมนู

หน้านี้ทำงานฝั่ง server ทั้งหมด ดึงหมวดหมู่พร้อมเมนูและ modifier group จาก Prisma ในคำสั่งเดียว แล้วส่งเป็น props ให้ client component ที่ต้องมี interactivity เท่านั้น:

```
// app/(customer)/menu/page.tsx
import { prisma } from "@lib/prisma";
import { getActiveTableSession } from "@lib/table-session";
import { redirect } from "next/navigation";
import { MenuBrowser } from "../menu-browser";

export default async function MenuPage() {
  const session = await getActiveTableSession();
  if (!session) {
    // ไม่มี table session ที่ valid - กันเหมือนกับที่ proxy.ts เช็กไว้แล้ว แต่กันซ้ำอีกชั้นตรงนี้
    redirect("/t/not-open");
  }

  const categories = await prisma.menuCategory.findMany({
    where: {
      branchId: session.branchId,
      isActive: true,
    },
    orderBy: { sortOrder: "asc" },
    include: {
      items: {
        where: { isAvailable: true },
        orderBy: { sortOrder: "asc" },
        include: {
          modifierGroups: {
            include: {
              modifierGroup: {
                include: {
                  modifiers: {
                    where: { isAvailable: true },
                    orderBy: { sortOrder: "asc" },
                    include: {
                      nestedGroups: {
                        include: { modifiers: true },
                      },
                    },
                  },
                },
              },
            },
          },
        },
      },
    },
  });

  return <MenuBrowser categories={categories} />;
}
```

จุดสำคัญของ query นี้คือกรอง `isAvailable: true` ทั้งที่ระดับ `MenuItem` และ `Modifier` ตั้งแต่ฝั่ง server — ของหมดที่พนักงานปิดไปเมื่อครั้งจะไม่โผล่ให้ลูกค้าเห็นเลย ไม่ต้องพึ่ง client กรองเอง

อีกจุดที่ควรสังเกตคือ `getActiveTableSession()` ถูกเรียกซ้ำที่นี้แม้ `proxy.ts` (บทที่ 5) จะเช็กไปแล้วชั้นหนึ่ง เพราะ server component นี้ยังต้องใช้ `session.branchId` เพื่อกรองเมนูให้ตรงสาขา — ร้านที่มีหลาย

สาขาต้องแสดงเมนูของสาขาที่โตะนั้นตั้งอยู่เท่านั้น ไม่ใช่เมนูรวมทุกสาขานั่น การเช็คซ้ำตรงนี้จะไม่ใช่ความซ้ำซ้อนที่สิ้นเปลือง แต่เป็นการดึงข้อมูลที่ต้องใช้ต่อในหน้านี้อยู่แล้ว

โค้ด: client component เลือก modifier

ส่วนนี้ต้องเป็น client component (`"use client"`) เพราะต้องเก็บ state ว่าลูกค้าเลือกอะไรไปแล้วบ้าง และคำนวณราคาที่ **แสดงผล** แบบ real-time ตามตัวเลือก โครงสร้างข้อมูลที่ได้รับเข้ามามาตรงกับ

`ModifierGroup` / `Modifier` ที่ออกแบบไว้ในบทที่ 4 ทุกประการ — component นี้ไม่ต้องรู้เรื่อง nested group ซ้อนกี่ชั้นเลย เพราะแต่ละครั้งที่ลูกค้าเลือก modifier ที่มี nested group อยู่ข้างใน หน้าเมนูจะเปิด

`ModifierPicker` อีกชุดซ้อนขึ้นมาสำหรับกลุ่มนั้นโดยเฉพาะ (เรียก component เดิมซ้ำแบบ recursive) ไม่ต้องเขียน logic แยกสำหรับแต่ละระดับความลึก:

```

// app/(customer)/menu/modifier-picker.tsx
"use client";

import { useMemo, useState } from "react";

type Modifier = {
  id: string;
  name: string;
  priceDeltaSatang: number;
};

type ModifierGroup = {
  id: string;
  name: string;
  required: boolean;
  minSelect: number;
  maxSelect: number;
  modifiers: Modifier[];
};

export function ModifierPicker({
  basePriceSatang,
  groups,
  onConfirm,
}: {
  basePriceSatang: number;
  groups: ModifierGroup[];
  onConfirm: (selectedModifierIds: string[]) => void;
}) {
  const [selected, setSelected] = useState<Record<string, string[]>>({});

  function toggle(group: ModifierGroup, modifierId: string) {
    setSelected((prev) => {
      const current = prev[group.id] ?? [];
      const isSelected = current.includes(modifierId);

      if (group.maxSelect === 1) {
        // เลือกได้อันเดียว: กดใหม่แทนที่อันเดิมเสมอ
        return { ...prev, [group.id]: isSelected ? [] : [modifierId] };
      }

      if (isSelected) {
        return { ...prev, [group.id]: current.filter((id) => id !== modifierId) };
      }

      if (current.length ≥ group.maxSelect) {
        return prev; // ครบ maxSelect แล้ว กดเพิ่มไม่ได้
      }

      return { ...prev, [group.id]: [...current, modifierId] };
    });
  }

  // ราคานี้เป็นแค่ตัวอย่างให้ลูกค้าดูก่อนกดยืนยัน - ราคจริงที่ตัดเงินคำนวณใหม่ที่ server เสมอ
  const displayPriceSatang = useMemo(() => {
    let total = basePriceSatang;
    for (const group of groups) {
      const ids = selected[group.id] ?? [];

```

```

    for (const modifierId of ids) {
      const modifier = group.modifiers.find((m) => m.id === modifierId);
      if (modifier) total += modifier.priceDeltaSatang;
    }
  }
  return total;
}, [selected, groups, basePriceSatang]);

const allRequiredFilled = groups
  .filter((g) => g.required)
  .every((g) => (selected[g.id]?.length ?? 0) ≥ g.minSelect);

return (
  <div>
    {groups.map((group) => (
      <fieldset key={group.id}>
        <legend>
          {group.name}
          {group.required ? " (ต้องเลือก)" : " (ไม่บังคับ)"}
        </legend>
        {group.modifiers.map((modifier) => (
          <button
            key={modifier.id}
            type="button"
            className="min-h-11 min-w-11 px-4"
            aria-pressed={(selected[group.id] ?? []).includes(modifier.id)}
            onClick={() => toggle(group, modifier.id)}
          >
            {modifier.name}
            {modifier.priceDeltaSatang > 0 &&
              ` (+${(modifier.priceDeltaSatang / 100).toFixed(2)} บาท)`}
          </button>
        ))}
      </fieldset>
    ))}
    <p>ราคาโดยประมาณ: {(displayPriceSatang / 100).toFixed(2)} บาท</p>

    <button
      type="button"
      disabled={!allRequiredFilled}
      onClick={() => onConfirm(Object.values(selected).flat())}
    >
      เพิ่มลงตะกร้า
    </button>
  </div>
);
}

```

ราคบน client เป็นแค่ตัวอย่าง — ราคาจริงคำนวณที่ server เสมอ

ข้อนี้ต้องย้ำให้ชัดเพราะเป็นจุดที่ระบบรั่วเรื่องเงินได้ง่ายที่สุดถ้าออกแบบผิด: `displayPriceSatang` ในโค้ดข้างบนมิได้ให้ลูกค้าดูตัวเลขคร่าว ๆ ก่อนกดยืนยันเท่านั้น ไม่ใช่ตัวเลขที่ระบบเชื่อถือ เพราะ client-side

JavaScript แก้ไขได้ทุกบรรทัด — ใครก็ตามที่เปิด DevTools แล้วแก้ไข `priceDeltaSatang` ในหน่วยความจำก่อนกดสั่ง จะเห็นราคาที่ผิดจากความจริงได้ทันที แต่นั่นไม่เป็นปัญหาถ้าระบบฝั่ง server ไม่เชื่อตัวเลขที่ client ส่งมาเลย

หน้าที่ของหน้าเมนูในบอทนี้จบแค่ “ส่ง `menuItemId` และ `modifierId` ที่ลูกค้าเลือก” ไปให้ server เท่านั้น ส่วนการคำนวณราคาจริงที่จะตัดเข้าออร์เดอร์ ต้องให้ server ไป join กับ `MenuItem.basePriceSatang` และ `Modifier.priceDeltaSatang` ในฐานข้อมูลเองใหม่ทั้งหมด ไม่ใช้รับตัวเลขราคาจาก request body มาใช้ตรง ๆ — รายละเอียดเต็มเรื่องการ validate ฝั่ง server และห้ามเชื่อราคาที่ client ส่งมาจะอยู่ในบทที่ 7 ตอนทำ server action รับออร์เดอร์

หลายคนสงสัยว่าถ้าไม่เชื่อราคาที่ client คำนวณอยู่แล้ว ทำไมยังต้องคำนวณราคาแสดงผลบน client ให้อย่างยากด้วย คำตอบคือมันเป็นคนละหน้าที่กัน — ราคาที่แสดงบนหน้าเมนูมีไว้เพื่อ **ประสบการณ์ผู้ใช้** ให้ลูกค้ารู้ว่ากดเลือกที่อปิ้งเพิ่มแล้วราคาขยับขึ้นเท่าไรก่อนตัดสินใจกดยืนยัน ถ้าไม่มีตัวเลขนี้เลยลูกค้าจะไม่รู้ที่กำลังจะจ่ายเท่าไรจนกว่าจะเห็นบิลตอนจบมือ ซึ่งเป็นประสบการณ์ที่แย่มาก ส่วนราคาที่ตัดเงินจริงเป็นหน้าที่ของ **ความปลอดภัยของระบบ** ซึ่งต้องพึ่งพา server เท่านั้นเพราะเป็นจุดเดียวที่ client ไม่มีทางเข้าไปแก้ไขข้อมูลได้ สองหน้าที่นี้แยกกันชัดเจนและต้องอยู่คนละชั้นของสถาปัตยกรรมเสมอ

สั่ง Claude Code แบบนี้

หน้าเมนูมีทั้งส่วนที่ควรเป็น server component (ดึงข้อมูล) และ client component (interactivity) ให้ Claude แยกให้ชัดตั้งแต่ prompt ไม่ให้ปนกันจนทั้งหน้ากลายเป็น client component หมด ซึ่งจะทำให้โหลดช้าลง

ช่วยสร้างหน้าเมนูลูกค้าสำหรับ QR ordering ที่ `/app/(customer)/menu` ข้อกำหนด:

- หน้า `page.tsx` เป็น React Server Component ดึงข้อมูล `MenuCategory` → `MenuItem` → `ModifierGroup` → `Modifier` จาก Prisma โดยกรอง `isAvailable = true` ทุกระดับ
- เฉพาะส่วนที่ต้องมี state (เลือก modifier, คำนวณราคาแสดงผล) ให้แยกเป็น client component ต่างหาก มี `"use client"` ชัดเจน
- ใช้ `next/image` สำหรับรูปเมนูทุกจุด ห้ามใช้ `<img>` ตรงๆ
- ราคาที่คำนวณและแสดงบน client ให้กำกับด้วยคอมเมนต์ชัดเจนว่าเป็นราคาประมาณการ ไม่ใช่ราคาที่ตัดเงินจริง เพราะราคาจริงจะคำนวณที่ server อีกทีตอนส่งออร์เดอร์
- ปุ่มเลือก modifier และปุ่ม "เพิ่มลงตะกร้า" ต้องมีพื้นที่แต่ไม่เล็กกว่า `44x44px`
- รองรับสองภาษา TH/EN โดยสลับตามฟิลด์ `name/nameEn` ที่มีอยู่แล้วใน schema

ช่วยแยกให้ชัดว่าไฟล์ไหนเป็น server component ไฟล์ไหนเป็น client component ก่อนเขียนโค้ด

สิ่งที่ต้องรีวิวใน output: (1) เช็คกว่า `page.tsx` ไม่มี `"use client"` หลุดขึ้นไปอยู่บนสุดของไฟล์ ถ้ามีแปลว่าทั้งหน้ากลายเป็น client-rendered หมด เสียประโยชน์ของ server component ไปเลย (2) เช็คกว่า query กรอง `isAvailable` ทุกระดับ (`MenuItem` และ `Modifier`) ไม่ใช่กรองแค่ระดับบนสุด (3) เช็คกว่ารูปทุกจุดใช้ `next/image` ไม่ใช่ `<img>` ธรรมดาที่หลุดมา (4) คั่นในโค้ดว่ามีจุดไหนที่ราคาจาก client ถูกส่งตรงไปบันทึกลง database โดยไม่ผ่านการคำนวณซ้ำที่ server หรือไม่ — ถ้ามีคือบักร้ายแรงที่ต้องแก้ก่อน merge

มุมมองของร้าน

หน้าเมนูคือ “หน้าร้าน” เวอร์ชันดิจิทัล ลูกค้าตัดสินใจสั่งเพิ่มหรือไม่สั่งเพิ่มจากตรงนี้ ร้านที่จัดวางเมนูแนะนำดีรูปสวย โหลดไว มักเห็นยอดขายเฉลี่ยต่อบิลสูงขึ้นเพราะลูกค้ากดสั่งเพิ่มได้เองโดยไม่ต้องรอเรียกพนักงาน

(หมวด B) — แต่ตัวเลขนี้เป็นสิ่งที่ผู้ให้บริการ QR ordering เผลอไว้ ร้านควรวัดผลจริงจากยอดขายของตัวเอง หลังใช้งาน ไม่ใช่เชื่อตัวเลขกลางที่ไม่ได้มาจากร้านตัวเอง

สิ่งที่เจ้าของร้านควรถามทีมพัฒนาก่อนขึ้นระบบจริง: “ถ้าเน็ตร้านช้าตอนพีค ลูกค้าจะรอนานแค่ไหน” และ “ถ้าพนักงานปิดเมนูของหมดจากหลังบ้าน ลูกค้าที่กำลังดูเมนูอยู่ตอนนั้นจะยังเห็นเมนูนั้นค้างอยู่ไหม” คำตอบที่ดีคือรอบถัดไปที่ลูกค้ากด refresh หรือกดสั่ง เมนูที่หมดจะไม่ผ่าน เพราะ server เช็ก `isAvailable` ซ้ำอีกครั้งตอนรับออเดอร์เสมอ — รายละเอียดเรื่องนี้จะอยู่ในบทที่ 7

เรื่องรูปเมนูก็เป็นจุดที่เจ้าของร้านมักประเมินความสำคัญต่ำเกินไป หลายร้านใช้รูปที่ถ่ายด้วยมือถือแบบเร่งรีบ แสงไม่พอ มุมไม่สวย แล้วนำมาขึ้นหน้าเมนูดิจิทัลที่ลูกค้าซูมดูใกล้ ๆ ได้ก่อนตัดสินใจสั่ง ต่างจากเมนูกระดาษที่ลูกค้าเห็นแค่ตัวหนังสือ การลงทุนถ่ายรูปเมนูให้ดีสักครั้ง (หรือใช้ทีมงานที่ถ่ายรูปอาหารเป็น) ให้ผลตอบแทนคุ้มค่ากว่าที่คิด เพราะรูปคือปัจจัยแรกที่ลูกค้าใช้ตัดสินใจก่อนอ่านชื่อเมนูหรือราคาด้วยซ้ำ

ประเด็นสุดท้ายที่เจ้าของร้านมักไม่ทันคิดคือภาษาที่สอง — ร้านที่อยู่ในย่านนักท่องเที่ยวมากได้ลูกค้าต่างชาติที่อ่านเมนูภาษาไทยไม่ออกเลย ถ้าไม่มีตัวเลือกภาษาอังกฤษ ลูกค้ากลุ่มนี้จะเลือกอาหารแบบเดายาก หรือต้องเรียกพนักงานมาอธิบายทีละจาน ซึ่งย้อนกลับไปสร้างภาระงานให้พนักงานเหมือนเดิม การกรอก `nameEn` ให้ครบทุกเมนูตั้งแต่ตอนกรอกข้อมูลครั้งแรก (ไม่ใช่ปล่อยว่างแล้วมาใส่กรอกทีหลัง) จึงเป็นงานเล็ก ๆ ที่ควรทำให้เสร็จตั้งแต่ก่อนเปิดร้านจริง ไม่ใช่ฟีเจอร์ที่ค่อยเพิ่มทีหลัง

กล่องพลาดบอย

❌ **พลาดบอย:** ใส่ `"use client"` ไว้บนสุดของ `page.tsx` ทั้งไฟล์เพื่อความสะดวก ทำให้ต้อง fetch ข้อมูลเมนูจาก client อีกรอบหลังโหลดหน้าเสร็จ
✅ **ทำแบบนี้:** ให้ `page.tsx` เป็น server component ดึงข้อมูลตรงจาก Prisma แล้วแยกเฉพาะส่วนที่ต้องมี interactivity ออกเป็น client component ย่อย

❌ **พลาดบอย:** คำนวณราคารวมฝั่ง client แล้วส่งตัวเลขราคานั้นแนบไปกับออเดอร์ให้ server บันทึกตรง ๆ
✅ **ทำแบบนี้:** client ส่งแค่ `menuItemid / modifierid` ที่เลือก ส่วนราคาให้ server คำนวณใหม่จากฐานข้อมูลเองทุกครั้งก่อนบันทึก

❌ **พลาดบอย:** ใช้ `` ตรง ๆ กับรูปเมนูทุกจาน ทำให้หน้าโหลดช้าบนเน็ตมือถือช่วงพีค
✅ **ทำแบบนี้:** ใช้ `next/image` ให้ Next.js จัดการ resize/format/lazy-load ให้อัตโนมัติ

จำ 3 ข้อ 1. หน้าเมนูควรเป็น server component เป็นหลัก แยก client component เฉพาะจุดที่ต้องมี state จริง ๆ 2. ราคาที่คำนวณและแสดงผลบนหน้าเมนูฝั่งลูกค้าเป็นแค่ตัวอย่างให้ดูก่อนกดยืนยัน ราคาที่ใช้จริงคำนวณที่ server เสมอ 3. กรอง `isAvailable` ทั้งระดับเมนูและระดับ modifier ตั้งแต่ query บน server ไม่ปล่อยให้ client กรองเอง

ตะกร้า ส่งออร์เดอร์ และ state machine ของออร์เดอร์



ร้านหนึ่งที่ผมเคยวางระบบให้ เปิดใช้ QR ordering ได้สามวันก็โทรมาด่า — ลูกค้าโต๊ะ 12 กดส่งออร์เดอร์ตอนเน็ดในร้านสะดุด กดซ้ำสองครั้งเพราะจอค้าง คราวได้ผัดไทยสองจานทั้งที่สั่งจานเดียว ลูกค้าไม่ยอมจ่ายจานที่สอง ร้านเสียทั้งวัตถุดิบทั้งหน้าตา

ปัญหานี้ไม่ได้อยู่ที่ “โค้ดพัง” แต่อยู่ที่ตอนออกแบบไม่มีใครถามคำถามว่า “ถ้าปุ่มนี้ถูกกดสองครั้งจะเกิดอะไรขึ้น” บทนี้เป็นบทที่เชื่อมหน้าเมนูลูกค้า (บทที่ 6) เข้ากับหลังบ้านจริง ๆ — ตะกร้าสินค้า การส่งออร์เดอร์แบบกันพลาด และวงจรชีวิตของออร์เดอร์หนึ่งใบตั้งแต่เกิดจนจบ

ตะกร้าอยู่ที่ไหน

คำถามแรกที่ต้องตอบคือ ตะกร้าเก็บอยู่ที่ไหน — client state (React state/localStorage) หรือฝั่ง server ตั้งแต่แรก

แนวทางที่เล่นนี้แนะนำคือ **แบบผสม**: ระหว่างที่ลูกค้ากำลังเลือกของ (ยังไม่กดส่ง) ให้เป็น client state ล้วน ๆ เพื่อความไว ไม่ต้องรอ round-trip ทุกครั้งที่กดเพิ่ม/ลดจำนวน แต่ **ทันทีที่กดปุ่ม “สั่งอาหาร”** ตะกร้าทั้งใบต้อง

ถูกส่งไปตรวจสอบและบันทึกที่ server ภายในธุรกรรมเดียว — จากจุดนั้นเป็นต้นไป server คือความจริงหนึ่งเดียว (single source of truth) ไม่ใช่ client อีกต่อไป

ระหว่างที่ตะกร้ายังอยู่ฝั่ง client มีรายละเอียด UX เล็ก ๆ ที่ร้านชอบมองข้ามแต่กระทบจริง เช่น ถ้าลูกค้าพลิกจอปิดแอปกลางคันหรือสลับไปดูแอปธนาคารแล้วกลับมา ตะกร้าต้องยังอยู่ครบ — วิธีที่ตรงไปตรงมาคือ sync ตะกร้าลง `sessionStorage` ผูกกับ table session token ทุกครั้งที่มีการเปลี่ยนแปลง แล้วโหลดกลับตอนเปิดหน้าใหม่ ไม่ใช่เก็บไว้ใน React state เฉย ๆ ที่หายทันทีที่ refresh อีกจุดที่ควรมีคือปุ่ม “สั่งอาหาร” ต้อง disable ทันทีที่กด และเปลี่ยนข้อความเป็นสถานะกำลังส่ง (ผูกกับ `useActionState` / `useFormStatus` ของ React 19) เพื่อลดโอกาสที่ผู้ใช้จะกดซ้ำตั้งแต่ต้นทาง — ถึงจะมี idempotency key รองรับอยู่แล้วก็ตาม การกันไม่ให้เกิดเหตุการณ์ตั้งแต่ต้นย่อมดีกว่าต้องพึ่งกลไกสถานการณ์ที่หลังเสมอ

เหตุผลที่ต้องยึดแบบนี้เข้มงวด: ราคาที่ client แสดงมาจาก state ที่โหลดตอนเปิดหน้าเมนู เมนูอาจเพิ่งเปลี่ยนราคา ของอาจเพิ่งหมด ตัวเลือกที่บังคับเลือกอาจถูกข้ามเพราะบั๊กฝั่ง UI — ถ้า server เชื่อตัวเลขจาก client ตรง ๆ ร้านเสี่ยงขาดทุนทุกครั้งที่มีช่องโหว่แบบนี้ และที่แย่กว่าคือมันเปิดช่องให้คนที่แกล้ง request ด้วยมือ (เปิด dev tools แล้วยิง POST เอง) สั่งอาหารราคา 1 บาทได้จริง ๆ

Server Action รับออร์เดอร์: validate ทุกอย่างใหม่

หลักการที่ต้องท่องให้ขึ้นใจก่อนเขียนโค้ดบล็อคนี้อือ ห้ามเชื่อราคาที่ client ส่งมาเด็ดขาด client ส่งมาได้แค่ `menuItemId`, `modifierIds`, และ `quantity` — ราคาทุกบาททุกสตางค์ server ต้องคำนวณเองจากฐานข้อมูล ณ เวลานั้น

สิ่งที่ server action ต้อง validate ก่อนบันทึกออร์เดอร์ มีอย่างน้อย:

- โตะ/session ยังเปิดอยู่จริง ไม่ใช่ session ที่หมดอายุหรือปิดไปแล้ว
- ทุก `menuItemId` มีอยู่จริงและ `isAvailable = true`
- ทุก ModifierGroup ที่ `required = true` ถูกเลือกครบ และจำนวนที่เลือกอยู่ในช่วง `minSelect` – `maxSelect`
- ราคาทุกบรรทัดคำนวณใหม่จากราคาปัจจุบันของ MenuItem + Modifier ในฐานข้อมูล ไม่ใช่ตัวเลขจาก client
- คำขอนี้ไม่ใช่การส่งซ้ำ (idempotency — อธิบายถัดไป)

```

// app/(customer)/actions/place-order.ts
"use server";

import { prisma } from "@lib/prisma";
import { getTableSession } from "@lib/table-session";
import { z } from "zod";

const CartItemSchema = z.object({
  menuItemId: z.string().cuid(),
  quantity: z.number().int().min(1).max(20),
  modifierIds: z.array(z.string().cuid()),
  note: z.string().max(200).optional(),
});

const PlaceOrderSchema = z.object({
  items: z.array(CartItemSchema).min(1),
  idempotencyKey: z.string().uuid(),
});

export type PlaceOrderResult =
  | { ok: true; orderId: string; totalAmount: number }
  | { ok: false; reason: string };

export async function placeOrder(
  input: z.infer<typeof PlaceOrderSchema>
): Promise<PlaceOrderResult> {
  const parsed = PlaceOrderSchema.safeParse(input);
  if (!parsed.success) {
    return { ok: false, reason: "ข้อมูลออเดอร์ไม่ถูกต้อง" };
  }

  const session = await getTableSession();
  if (!session || session.status !== "OPEN") {
    return { ok: false, reason: "โต๊ะนี้ยังไม่เปิด หรือปิดไปแล้ว กรุณาเรียกพนักงาน" };
  }

  // idempotency: ถ้ามี order ที่ใช้ key นี้แล้ว คืนผลเดิมไปเลย ไม่สร้างซ้ำ
  const existing = await prisma.order.findUnique({
    where: { idempotencyKey: parsed.data.idempotencyKey },
  });
  if (existing) {
    return { ok: true, orderId: existing.id, totalAmount: existing.totalAmount };
  }

  try {
    const order = await prisma.$transaction(async (tx) => {
      const menuItemIds = parsed.data.items.map((i) => i.menuItemId);
      const menuItems = await tx.menuItem.findMany({
        where: { id: { in: menuItemIds } },
        include: { modifierGroups: { include: { modifiers: true } } },
      });

      let totalAmount = 0;
      const orderItemsData = [];

      for (const line of parsed.data.items) {

```

```

const menuItem = menuItems.find((m) => m.id === line.menuItemId);
if (!menuItem || !menuItem.isAvailable) {
  throw new Error(`สินค้า "${menuItem?.name ?? line.menuItemId}" หหมดหรือไม่มีในเมนูแล้ว`);
}

// ตรวจสอบ ModifierGroup ที่บังคับ
for (const group of menuItem.modifierGroups) {
  const selectedInGroup = group.modifiers.filter((m) =>
    line.modifierIds.includes(m.id)
  );
  if (group.required && selectedInGroup.length < group.minSelect) {
    throw new Error(`"${menuItem.name}" ต้องเลือก "${group.name}" อย่างน้อย ${group.minSelect} รายการ`);
  }
  if (selectedInGroup.length > group.maxSelect) {
    throw new Error(`"${menuItem.name}" เลือก "${group.name}" เกินจำนวนที่กำหนด`);
  }
}

const allModifiers = menuItem.modifierGroups.flatMap((g) => g.modifiers);
const chosenModifiers = allModifiers.filter((m) => line.modifierIds.includes(m.id));

// ราคาต่อหน่วย = ราคารฐาน + ราคา modifier ทั้งหมด - ค่าลบจาก DB เท่านั้น
const unitPrice =
  menuItem.priceCents + chosenModifiers.reduce((sum, m) => sum + m.priceCents, 0);
const lineTotal = unitPrice * line.quantity;
totalAmount += lineTotal;

orderItemsData.push({
  menuItemId: menuItem.id,
  // snapshot ชื่อ/ราคา ณ เวลาสั่ง - เมนูขึ้นราคาทีหลัง บิลเก่าต้องไม่เปลี่ยน
  nameSnapshot: menuItem.name,
  unitPriceCentsSnapshot: unitPrice,
  quantity: line.quantity,
  note: line.note,
  modifierNamesSnapshot: chosenModifiers.map((m) => m.name),
});

const created = await tx.order.create({
  data: {
    tableSessionId: session.id,
    status: "PLACED",
    totalAmount,
    idempotencyKey: parsed.data.idempotencyKey,
    items: { create: orderItemsData },
  },
});

return created;
});

return { ok: true, orderId: order.id, totalAmount: order.totalAmount };
} catch (err) {
  return { ok: false, reason: err instanceof Error ? err.message : "บันทึกออเดอร์ไม่สำเร็จ" };
}

```

```
}  
}
```

สังเกตว่าทุกอย่างอยู่ใน `prisma.$transaction` ก้อนเดียว — ถ้าบรรทัดใดบรรทัดหนึ่งพัง (เช่นของหมดกลางทาง) ทั้งออร์เดอร์ต้อง rollback ไม่ใช่บันทึกไปครั้งเดียว

idempotency key: กันกดซ้ำ กันเน็ตช้า

โต๊ะ 12 ที่เล่าไปตอนต้นบท ปัญหาคือกดปุ่ม “สั่งอาหาร” สองครั้งเพราะจอค้าง ระบบเดิมไม่มีอะไรกันไว้เลย — ทุกครั้งที่ POST เข้ามาคือสร้างออร์เดอร์ใหม่เสมอ

วิธีแก้มาตรฐานที่วงการ payment ใช้กันมานาน (และ Omise/Stripe ก็บังคับสำหรับ webhook เช่นกัน — ดูบทที่ 11) คือ **idempotency key**: ฝั่ง client สร้าง UUID ขึ้นมาหนึ่งตัวตอนที่ผู้ใช้เริ่มกดปุ่ม แล้วส่ง key เดียวกันนี้แนบไปทุกครั้งที่ retry (ไม่ว่าจะ retry เพราะ timeout หรือเพราะ user กดซ้ำ) ฝั่ง server เก็บ key นี้เป็น **unique constraint** บนตาราง `Order` — ถ้าเจอ key ซ้ำ ให้คืนผลของออร์เดอร์เดิมกลับไปแทนที่จะสร้างใหม่

```
model Order {  
  id          String    @id @default(cuid())  
  idempotencyKey String  @unique  
  // ...  
}
```

จุดสำคัญ: key ต้องถูกสร้างครั้งเดียวตอนเริ่ม “ความตั้งใจ” ของผู้ใช้ (เช่นตอน component ตะกร้า mount หรือตอนกดปุ่มครั้งแรก) ไม่ใช่สร้างใหม่ทุกครั้งที่ฟังก์ชัน retry ทำงาน ไม่งั้น key จะไม่ซ้ำและ constraint ก็ช่วยอะไรไม่ได้

Order state machine

ออร์เดอร์หนึ่งใบไม่ได้มีแค่ “สั่งแล้ว” กับ “จ่ายแล้ว” — มันมีวงจรชีวิตที่คร่าว หน้าร้าน และเจ้าของร้านต้องเห็นตรงกัน เล่มนี้ใช้ state machine ดังนี้:

```
DRAFT → PLACED → IN_PROGRESS → READY → SERVED → PAID  
      ↓       ↓       ↓  
    CANCELLED VOID   VOID
```

ความหมายของแต่ละสถานะ:

สถานะ	ความหมาย	ใครเปลี่ยนได้
DRAFT	อยู่ในตะกร้า ยังไม่ส่งเข้าระบบ (มีแค่ฝั่ง client)	ลูกค้า (client-only ไม่มีแถวใน DB)
PLACED	บันทึกในฐานข้อมูลแล้ว ครัวยังไม่เริ่มทำ	ระบบ (server action) สร้างอัตโนมัติ
IN_PROGRESS	ครัวว์ได้รับแล้ว กำลังปรุง	พนักงานครัว (บทที่ 8)
READY	ทำเสร็จ รอเสิร์ฟ	พนักงานครัว
SERVED	เสิร์ฟถึงโต๊ะแล้ว	พนักงานหน้าร้าน (บทที่ 9)
PAID	จ่ายเงินแล้ว ปิดออร์เดอร์	ระบบชำระเงิน (บทที่ 11) หลังยืนยันยอด
CANCELLED	ยกเลิกก่อนครัวว์เริ่มทำ	พนักงาน/ลูกค้า (ยกเลิกได้เอง เพราะยังไม่เสียของ)
VOID	ยกเลิกหลังครัวว์เริ่มทำหรือหลังเสิร์ฟแล้ว	หัวหน้ากะขึ้นไปเท่านั้น (บทที่ 9 มี guard สิทธิ์ + audit log)

กฎที่ต้องบังคับในโค้ด ไม่ใช่แค่เขียนไว้ในเอกสาร:

- **ยกเลิกฟรีได้เฉพาะตอน PLACED** — เพราะครัวยังไม่แตะวัตถุดิบ พอเข้า IN_PROGRESS แล้วต้องเปลี่ยนไปใช้ VOID ซึ่งมีการอนุมัติ (บทที่ 9 อธิบายเรื่อง guard สิทธิ์และ audit log ต่อ)
- **ห้ามข้ามสถานะ** เช่นจาก PLACED ไป SERVED ตรง ๆ โดยไม่ผ่าน IN_PROGRESS / READY — เพราะรายงานเวลาปรุงอาหาร (บทที่ 8) จะพังถ้าไม่มี timestamp ของแต่ละช่วง
- **PAID เป็นสถานะปลายทางที่ห้ามย้อนกลับ** — ถ้าต้องคืนเงินหรือแก้ไขหลังจ่ายแล้ว ต้องออกเอกสารยกเลิก/คืนเงินแยกต่างหาก (บทที่ 12) ไม่ใช่แก้สถานะออร์เดอร์เดิม

โค้ด: discriminated union ของ state

แทนที่จะปล่อยให้สถานะเป็น string ลอย ๆ แล้วเขียน if/else เดาเอาว่าฟิลด์ไหนควรมีค่าตอนไหน เล่มนี้แนะนำให้จำลอง state เป็น discriminated union ใน TypeScript ตั้งแต่ขั้น domain logic (แยกจาก Prisma model ที่เก็บแค่ status: OrderStatus enum ธรรมดา) เพื่อให้ compiler ช่วยจับเคสที่ลืมทำ

```
// lib/order/state.ts

type BaseOrder = {
  id: string;
  tableSessionId: string;
  totalAmount: number;
};

export type OrderState =
  | (BaseOrder & { status: "PLACED"; placedAt: Date })
  | (BaseOrder & { status: "IN_PROGRESS"; placedAt: Date; startedAt: Date })
  | (BaseOrder & { status: "READY"; placedAt: Date; startedAt: Date; readyAt: Date })
  | (BaseOrder & { status: "SERVED"; placedAt: Date; startedAt: Date; readyAt: Date; servedAt: Date })
  | (BaseOrder & { status: "PAID"; placedAt: Date; paidAt: Date })
  | (BaseOrder & { status: "CANCELLED"; placedAt: Date; cancelledAt: Date; cancelledBy: string })
  | (BaseOrder & { status: "VOID"; placedAt: Date; voidedAt: Date; voidedBy: string; voidReason:
string });

type Actor = { role: "KITCHEN" | "STAFF" | "SHIFT_LEAD" | "OWNER" | "SYSTEM" };

const ALLOWED_TRANSITIONS: Record<OrderState["status"], OrderState["status"][]> = {
  PLACED: ["IN_PROGRESS", "CANCELLED"],
  IN_PROGRESS: ["READY", "VOID"],
  READY: ["SERVED", "VOID"],
  SERVED: ["PAID", "VOID"],
  PAID: [],
  CANCELLED: [],
  VOID: [],
};

export function canTransition(
  from: OrderState["status"],
  to: OrderState["status"],
  actor: Actor
): boolean {
  if (!ALLOWED_TRANSITIONS[from].includes(to)) return false;

  // VOID ต้องหัวหน้ากะขึ้นไปเท่านั้น
  if (to === "VOID" && actor.role !== "SHIFT_LEAD" && actor.role !== "OWNER") {
    return false;
  }
  // CANCELLED ทำได้ตั้งแต่พนักงานทั่วไป เพราะยังไม่เสียวัตถุดิบ
  if (to === "CANCELLED" && actor.role === "KITCHEN") {
    return false; // คราวไม่มีสิทธิ์ยกเลิก แคร็บ/ทำเสร็จ
  }
  return true;
}

```

การแยก type ตาม status แบบนี้ทำให้เวลาเขียนโค้ดฝั่ง UI (เช่นปุ่มในจอครัวบทที่ 8) TypeScript จะบังคับให้ตรวจสอบ `status` ก่อนถึงจะอ่าน `startedAt` ได้ — ไม่มีทางเขียนโค้ดที่อ่าน `readyAt` ของออเดอร์ที่ยังเป็น `PLACED` หลุดผ่าน compile ไปได้ ต่างจากการปล่อยทุกฟิลด์เป็น optional ทั้งก้อนซึ่งจะซ่อนบั๊กไว้นานกว่าจะรันจริง

เปลี่ยนสถานะใน transaction พร้อม audit trail

```
// app/(kds)/actions/advance-order.ts
"use server";

import { prisma } from "@lib/prisma";
import { canTransition } from "@lib/order/state";
import { requireStaff } from "@lib/auth";

export async function advanceOrderStatus(orderId: string, nextStatus: string) {
  const actor = await requireStaff();

  return prisma.$transaction(async (tx) => {
    const order = await tx.order.findUniqueOrThrow({ where: { id: orderId } });

    if (!canTransition(order.status as any, nextStatus as any, { role: actor.role })) {
      throw new Error(`เปลี่ยนสถานะจาก ${order.status} ไป ${nextStatus} ไม่ได้ หรือไม่มีสิทธิ์`);
    }

    const timestampField =
      nextStatus === "IN_PROGRESS" ? "startedAt" :
      nextStatus === "READY" ? "readyAt" :
      nextStatus === "SERVED" ? "servedAt" : undefined;

    const updated = await tx.order.update({
      where: { id: orderId },
      data: {
        status: nextStatus,
        ... (timestampField ? { [timestampField]: new Date() } : {}),
      },
    });

    await tx.auditLog.create({
      data: {
        actorId: actor.id,
        action: "ORDER_STATUS_CHANGE",
        targetId: orderId,
        metadata: { from: order.status, to: nextStatus },
      },
    });

    return updated;
  });
}
```

หลังส่งออเดอร์แล้ว ลูกค้าเห็นอะไร

ส่ง `placeOrder` สำเร็จแล้วไม่ได้แปลว่าจบหน้าที่ของบหน้า — ลูกค้าที่กดปุ่มไปจะรอฟังอะไรบางอย่างเสมอ อย่างน้อยที่สุดคือ “ระบบได้รับออเดอร์แล้วจริงไหม” หน้าจอหลังกดส่งควรเปลี่ยนไปแสดงสถานะออเดอร์ทันที ไม่ใช่กลับไปหน้าจอเมนูเฉย ๆ เหมือนไม่มีอะไรเกิดขึ้น เพราะจุดที่ลูกค้าไม่มั่นใจว่าออเดอร์เข้าไปหรือยัง คือ

จุดที่มีจะไปกดปุ่มซ้ำอีกรอบ ซึ่งแม้ idempotency key จะกันความเสียหายฝั่งข้อมูลไว้ได้ แต่ก็ยังทำให้ลูกค้ากังวลอยู่ดีถ้าไม่มี feedback อะไรกลับมาเลย

วิธีที่เชื่อมกับบทที่ 8 ได้ตรงที่สุดคือให้หน้าออร์เดอร์ของลูกค้าฟัง SSE stream ตัวเดียวกับที่จอครัวฟัง (กรองด้วย `tableSessionId` แทน `branchId`) เพื่อให้เห็นสถานะไล่ตาม state machine แบบ realtime — “ครัวรับออร์เดอร์แล้ว” → “กำลังทำ” → “พร้อมเสิร์ฟ” โดยไม่ต้องกดรีเฟรชเอง ระบบที่ทำแบบนี้ได้ลดจำนวนครั้งที่ลูกค้าเรียกพนักงานมาถามว่า “อาหารจะมาไหม” ได้จริง ซึ่งเป็นผลข้างเคียงที่ร้านชอบมากกว่าที่คาดไว้ตอนแรก

ทดสอบ state machine ด้วย unit test

`canTransition` เป็นฟังก์ชัน pure function ล้วน ๆ ไม่แตะฐานข้อมูล ไม่แตะ network — นี่คือจุดที่ควรมี unit test ครอบให้ครบทุกเส้นทางที่อนุญาตและทุกเส้นทางที่ต้องห้าม เพราะถ้าเผลอแก้กฎวันไหนแล้วลืมเคสใดเคสหนึ่ง test จะจับได้ทันทีก่อนที่จะไปพึ่งพนักงานจริง

```
// lib/order/state.test.ts
import { describe, expect, it } from "vitest";
import { canTransition } from "../state";

describe("canTransition", () => {
  it("อนุญาตให้ครัวเริ่มทำออร์เดอร์ที่เพิ่ง PLACED", () => {
    expect(canTransition("PLACED", "IN_PROGRESS", { role: "KITCHEN" })).toBe(true);
  });

  it("ห้ามข้ามจาก PLACED ไป SERVED ตรง ๆ โดยไม่ผ่าน IN_PROGRESS/READY", () => {
    expect(canTransition("PLACED", "SERVED", { role: "STAFF" })).toBe(false);
  });

  it("ห้ามพนักงานทั่วไปสั่ง VOID ออร์เดอร์ที่ครัวเริ่มทำแล้ว", () => {
    expect(canTransition("IN_PROGRESS", "VOID", { role: "STAFF" })).toBe(false);
  });

  it("อนุญาตให้หัวหน้ากะ VOID ออร์เดอร์ที่ครัวเริ่มทำแล้ว", () => {
    expect(canTransition("IN_PROGRESS", "VOID", { role: "SHIFT_LEAD" })).toBe(true);
  });

  it("ห้ามยกเลิกออร์เดอร์ที่จ่ายเงินแล้ว (PAID เป็นสถานะปลายทาง)", () => {
    expect(canTransition("PAID", "CANCELLED", { role: "OWNER" })).toBe(false);
  });

  it("ครัวไม่มีสิทธิ์ CANCELLED เพราะเป็นสิทธิ์ของฝั่งหน้าร้าน", () => {
    expect(canTransition("PLACED", "CANCELLED", { role: "KITCHEN" })).toBe(false);
  });
});
```

ข้อดีของการแยก `canTransition` ออกมาเป็นฟังก์ชันล้วนแบบนี้คือทดสอบได้เร็วมาก (ไม่ต้อง spin ฐานข้อมูลจำลอง) และเวลาสั่ง Claude Code ให้เพิ่มกฎใหม่ในอนาคต (เช่นเพิ่ม role `DELIVERY_STAFF`) ก็สั่งให้รัน test ชุดนี้ก่อนแล้วค่อยเพิ่มเคสใหม่ตามที่ต้องการ ไม่ต้องกลัวว่าจะไปแก้ฟังก์ชันเดิมที่มีอยู่แล้วโดยไม่รู้ตัว

สั่ง Claude Code แบบนี้

ช่วยเขียน server action 'placeOrder' สำหรับระบบ POS ร้านอาหารที่ผมทำอยู่
บริบท: ลูกค้าสั่งอาหารผ่าน QR ที่โต๊ะ ตะกร้าอยู่ฝั่ง client แต่การยืนยันออเดอร์ต้อง
validate ใหม่ทั้งหมดฝั่ง server ตาม schema ใน prisma/schema.prisma (ดูตาราง MenuItem,
ModifierGroup, Modifier, Order, OrderItem)

กติกาที่ต้องทำตามเป็น:

- ห้ามซ้ำราคาที่ client ส่งมาเด็ดขาด ต้องคำนวณราคาใหม่จาก DB ทุกบรรทัด
- ต้อง validate ว่า ModifierGroup ที่ required เลือกครบตาม minSelect/maxSelect
- ต้องมี idempotency key เป็น unique constraint กันการสร้างออเดอร์ซ้ำ
- ทุกอย่างต้องอยู่ใน prisma.\$transaction เดียว
- เก็บ snapshot ชื่อและราคา ณ เวลาสั่งลงใน OrderItem

ให้ทำงานใน plan mode ก่อน อธิบายลำดับการตรวจก่อนเขียนโค้ดจริง แล้วค่อย apply

ต้องรีวิวอะไรในผลลัพธ์: (1) เช็คว่าราคาทุกบรรทัดคำนวณจากค่าที่ query จาก DB จริง ไม่มีจุดไหนแอบอ่านค่าจาก input ที่ client ส่งมาโดยตรง (2) unique constraint ของ idempotency key ต้องอยู่ใน schema.prisma จริง ไม่ใช่แค่เช็คด้วยโค้ดแอปพลิเคชัน เพราะ race condition สองคำขอพร้อมกันต้องให้ database เป็นคนตัดสิน (3) เช็คว่า error ที่ throw ออกจาก transaction สื่อสารกับผู้ใช้ได้ ไม่ใช่ stack trace ดิบ ๆ

มุมมองของร้าน

ทุกครั้งที่เน็ตในร้านสะดุดหรือลูกค้ากดปุ่มไม่มั่นใจว่าส่งไปหรือยัง จุดนี้คือจุดที่ระบบ “ทำงานเสียเงินร้าน” ได้จริงถ้าไม่มี idempotency key — คราวจะได้ใบสั่งซ้ำ ทำอาหารซ้ำ วัตถุดิบหาย และถ้าลูกค้าไม่ยอมจ่ายจานที่ซ้ำร้านต้องรับผิดชอบเอง

อีกจุดที่เจ้าของร้านควรรู้คือ state machine ไม่ใช่แค่เรื่องเทคนิค — มันคือกฎการทำงานของร้านที่เขียนเป็นโค้ด เช่น “ใครมีสิทธิ์ยกเลิกออเดอร์ที่ครัวเริ่มทำไปแล้ว” ถ้าตอบไม่ได้ตั้งแต่ตอนออกแบบ ระบบจะปล่อยให้พนักงานคนไหนก็ดกเลิกได้ ซึ่งเป็นช่องโหว่ที่คนโกงหน้าร้านชอบใช้ (รายละเอียดเรื่องการตรวจจับความผิดปกติแบบนี้อยู่ในบทที่ 13)

❌ พลาดบ่อย / ✅ ทำแบบนี้

❌ **พลาดบ่อย:** คำนวณราคารวมจากตัวเลขที่ client ส่งมาใน request body โดยตรง เพราะ “ก็หน้าเมนูคำนวณให้แล้วนี่” ✅ **ทำแบบนี้:** ส่งเฉพาะ `menuItemId` + `modifierIds` + `quantity` จาก client แล้วให้ server query ราคาจริงจากฐานข้อมูลมาคำนวณเองทุกครั้ง ราคาที่ client แสดงเป็นแค่ preview

❌ **พลาดบ่อย:** สร้าง idempotency key ใหม่ทุกครั้งที่ฟังก์ชัน retry ทำงาน (เช่น generate `uuid()` อยู่ใน try/catch ที่ retry เอง) ✅ **ทำแบบนี้:** สร้าง key ครั้งเดียวตอนผู้ใช้เริ่มกดปุ่ม เก็บไว้ใน state แล้วใช้ key เดิมซ้ำในทุกความพยายามส่งจนกว่าจะสำเร็จหรือ user ยกเลิกเอง

❌ **พลาดบ่อย:** ปล่อยให้ทุก role เปลี่ยนสถานะออเดอร์ได้ทุกสถานะผ่าน endpoint เดียวกัน ✅ **ทำแบบนี้:** ตรวจสอบสิทธิ์คู่กับ state machine ทุกครั้ง (`canTransition` รับทั้ง state ปัจจุบัน สถานะปลายทาง และ role

ของผู้เรียก) และบันทึก audit log ทุกครั้งที่มีการเปลี่ยนสถานะที่กระทบเงินหรือวัตถุดิบ

จำ 3 ข้อ ราคาที่ client ส่งมาเป็นแค่คำขอ ไม่ใช่คำสั่ง — server ต้องคำนวณใหม่เสมอ ปุ่มถูกกดซ้ำได้เสมอทั้งจากคนและจากเน็ตที่สะดุด ต้องมี idempotency key เป็น unique constraint จริงในฐานข้อมูลสถานะออเดอร์คือกฎการทำงานของร้าน ไม่ใช่แค่ label — ต้องผูกกับสิทธิ์ของผู้เปลี่ยนและมี audit log ทุกจุดที่กระทบเงิน

จอครัว (KDS) แบบ realtime



ครัวคือจุดที่ทนความซ้ำไม่ได้ที่สุดในร้าน ลูกค้ารอเมื่อนาน 2 นาทีไม่รู้สึกอะไร แต่ถ้าจอครัวอัปเดตซ้ำ 2 นาที อาหารไหม้ คิวยาว และพนักงานเสิร์ฟยืนงงว่าจานไหนเสร็จหรือยัง บทนี้พาไปสร้างจอครัว (Kitchen Display System — KDS) ที่รับออเดอร์จากบทที่ 7 มาแสดงแบบ realtime แทนใบกระดาษหรือเครื่องพิมพ์ครัวแบบเดิม

ทำไมเลือก SSE ไม่ใช่ WebSocket

ก่อนเขียนโค้ดสักบรรทัด ต้องตอบคำถามสถาปัตยกรรมให้ชัดก่อน เพราะเลือกผิดจะรื้อที่หลังเจ็บ

Next.js Route Handler เขียน SSE ได้ตรง ๆ เพราะ SSE คือ HTTP response ธรรมดาที่ server ค่อย ๆ stream ข้อมูลออกไปทีละก้อน (`text/event-stream`) บน connection เดียวที่เปิดค้างไว้ แต่ **Route Handler ของ Next.js อัปเดตเป็น WebSocket ไม่ได้** — WebSocket ต้องมีขั้นตอน HTTP upgrade ที่ต้องควบคุม TCP socket ระดับต่ำกว่าที่ Route Handler เปิดให้ทำ ซ้ำร้ายไปกว่านั้น ถ้าจะ deploy บน Vercel ปัญหาจะใหญ่ขึ้นอีกชั้น เพราะ **Vercel ถือ TCP connection ค้างไว้แบบยาว ๆ ไม่ได้** (serverless function มีเวลาทำงานจำกัด) ซึ่งเป็นข้อจำกัดที่ตัด WebSocket ออกจากตัวเลือกไปเลยถ้าจะ host บน Vercel

SSE เป็น protocol ทางเดียว (server → client) ซึ่งพอดีกับงานของ KDS เป๊ะ — จอครัวไม่เคยต้องส่งข้อมูลกลับไปหา server ผ่าน connection เดียวกันที่รับ event มันแค่รอฟังว่า “มีออเดอร์ใหม่” หรือ “สถานะเปลี่ยน”

การกดปุ่มเปลี่ยนสถานะของครัว (เช่น “เริ่มทำ” หรือ “เสร็จแล้ว”) เป็นคนละ request — เป็น server action ธรรมดา (บทที่ 7) ที่ยิง mutation แล้วปล่อยให้ SSE stream กระจายผลลัพธ์กลับไปหาทุกจอที่ฟังอยู่

สรุปข้อสรุปที่เล่มนี้ใช้: ใช้ **SSE เป็นค่าเริ่มต้น** สำหรับทั้ง KDS และหน้า POS พนักงาน (บทที่ 9) เพราะเข้ากับ Next.js Route Handler ได้ตรง ๆ ไม่ต้องแยก server ต่างหาก และ deploy บน Vercel ได้จริง ส่วนเมื่อไหร่ที่ต้องขยับไป WebSocket หรือ managed realtime service (เช่น Pusher, Ably, หรือ Cloudflare Durable Objects) คือตอนที่ต้องการสื่อสารสองทางความถี่สูงจริง ๆ เช่น จอครัวหลายสาขาต้องคุยกันแบบ collaborative editing, หรือระบบเรียกคิวที่ต้องส่ง state กลับไปกลับมาถี่กว่าที่ HTTP request แยกทำได้ทัน — สำหรับร้านอาหารทั่วไปที่ SSE ทำงานฝ่ายเดียวพอ ยังไม่ถึงจุดนั้น

Route Handler ที่ stream SSE

Route Handler ที่ทำ SSE ต้องคืน **Response** ที่ body เป็น **ReadableStream** และตั้ง header **Content-Type: text/event-stream** ให้ถูก โครงสร้าง event ของ SSE เป็นข้อความล้วน ขึ้นต้นด้วย **data:** แล้วปิดท้ายด้วยบรรทัดว่างสองครั้ง

จุดที่มักลืมคือ **heartbeat** — ถ้าไม่มี traffic นานเกินไป reverse proxy บางตัว (nginx, load balancer) จะตัด connection ที่ทิ้งเพราะคิดว่าตายแล้ว วิธีกันคือส่ง comment event (**: heartbeat\n\n**) เป็นจังหวะทุก ๆ สัก 15-20 วินาที

```
// app/(kds)/api/orders/stream/route.ts
import { subscribeToOrderEvents } from "@lib/order/event-bus";

export const dynamic = "force-dynamic";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const branchId = searchParams.get("branchId");
  if (!branchId) {
    return new Response("missing branchId", { status: 400 });
  }

  const encoder = new TextEncoder();
  let unsubscribe: (() => void) | undefined;
  let heartbeatTimer: ReturnType<typeof setInterval> | undefined;

  const stream = new ReadableStream({
    start(controller) {
      const send = (event: string, data: unknown) => {
        controller.enqueue(
          encoder.encode(`event: ${event}\ndata: ${JSON.stringify(data)}\n\n`)
        );
      };

      // ส่ง snapshot แรกทันทีที่ต่อ เพื่อไม่ต้องรอ event ถัดไปจึงจะเห็นออร์เดอร์ค้าง
      send("connected", { branchId, at: new Date().toISOString() });

      unsubscribe = subscribeToOrderEvents(branchId, (evt) => {
        send(evt.type, evt.payload);
      });

      // heartbeat กับ proxy ตัด connection ตอนไม่มี event นาน ๆ
      heartbeatTimer = setInterval(() => {
        controller.enqueue(encoder.encode(`: heartbeat\n\n`));
      }, 15000);
    },
    cancel() {
      unsubscribe?.();
      if (heartbeatTimer) clearInterval(heartbeatTimer);
    },
  });

  return new Response(stream, {
    headers: {
      "Content-Type": "text/event-stream",
      "Cache-Control": "no-cache, no-transform",
      Connection: "keep-alive",
    },
  });
}
```

subscribeToOrderEvents ในตัวอย่างนี้เป็น event bus ฝั่ง server ที่ผูกกับ event ของแต่ละสาขา (**branchId** — จำไว้ว่าเราออกแบบ multi-tenant ตั้งแต่บทที่ 3) ในระบบเล็ก ๆ ระดับร้านเดียว/สาขาเดียว การใช้ **EventEmitter** ของ Node ในหน่วยความจำ process เดียวก็เพียงพอ แต่ถ้า deploy หลาย instance

(เช่น serverless ที่สเกลเป็นหลาย process พร้อมกัน) event ที่เกิดใน instance หนึ่งจะไปไม่ถึง client ที่ connection เปิดอยู่กับอีก instance — ตรงนี้ต้องเปลี่ยนไปใช้ pub/sub กลาง (เช่น Postgres **LISTEN/NOTIFY** หรือ Redis pub/sub) แทน in-memory emitter

client hook: useOrderStream() พร้อม reconnect และ backoff

จอร์จเปิดทิ้งไว้ทั้งกะ — connection หลุดได้จากหลายสาเหตุ (wifi สะดุด, router รีเซ็ต, deploy ใหม่ทำให้ server restart) hook ฝั่ง client ต้องจัดการ reconnect เองแบบไม่ต้องให้พนักงานกด refresh

หลักการ backoff ที่ใช้: reconnect ทันทีครั้งแรก ถ้ายังหลุดอีกให้ค่อย ๆ เพิ่มเวลาหน่วง (exponential backoff) จนถึงเพดานหนึ่ง เพื่อไม่ให้ยิง request ถี่เกินไปตอน server ล่มจริง ๆ

```

// hooks/use-order-stream.ts
"use client";

import { useEffect, useRef, useState } from "react";

type OrderEvent =
  | { type: "order.placed"; payload: { orderId: string } }
  | { type: "order.status_changed"; payload: { orderId: string; status: string } };

type ConnectionState = "connecting" | "open" | "reconnecting" | "closed";

const MAX_BACKOFF_MS = 15000;
const BASE_BACKOFF_MS = 1000;

export function useOrderStream(branchId: string, onEvent: (evt: OrderEvent) => void) {
  const [state, setState] = useState<ConnectionState>("connecting");
  const attemptRef = useRef(0);
  const onEventRef = useRef(onEvent);
  onEventRef.current = onEvent; // กัน stale closure โดยไม่ต้อง reconnect ทุกครั้งที่ callback เปลี่ยน

  useEffect(() => {
    let es: EventSource | null = null;
    let retryTimer: ReturnType<typeof setTimeout> | undefined;
    let stopped = false;

    function connect() {
      if (stopped) return;
      setState(attemptRef.current === 0 ? "connecting" : "reconnecting");

      es = new EventSource(`/api/orders/stream?branchId=${branchId}`);

      es.addEventListener("connected", () => {
        attemptRef.current = 0; // ต่อสำเร็จ รีเซ็ต backoff
        setState("open");
      });

      es.addEventListener("order.placed", (e) => {
        onEventRef.current({ type: "order.placed", payload: JSON.parse(e.data) });
      });

      es.addEventListener("order.status_changed", (e) => {
        onEventRef.current({ type: "order.status_changed", payload: JSON.parse(e.data) });
      });

      es.onerror = () => {
        es?.close();
        if (stopped) return;
        setState("reconnecting");

        const delay = Math.min(BASE_BACKOFF_MS * 2 ** attemptRef.current, MAX_BACKOFF_MS);
        attemptRef.current += 1;
        retryTimer = setTimeout(connect, delay);
      };
    }

    connect();
  }, [branchId]);
}

```

```

return () => {
  stopped = true;
  es?.close();
  if (retryTimer) clearTimeout(retryTimer);
};
}, [branchId]);

return { state };
}

```

หน้าจอครัวเอาไปใช้ง่าย ๆ แบบนี้ พร้อมแสดงสถานะการเชื่อมต่อให้พนักงานเห็นชัด ๆ ว่าตอนนี้จอ “สด” อยู่หรือหลุดไปแล้ว:

```

// app/(kds)/kitchen/page.tsx
"use client";

import { useOrderStream } from "@/hooks/use-order-stream";
import { useState } from "react";

export default function KitchenPage({ branchId }: { branchId: string }) {
  const [orders, setOrders] = useState<Record<string, { id: string; status: string }>>({});

  const { state } = useOrderStream(branchId, (evt) => {
    if (evt.type === "order.placed") {
      setOrders((prev) => ({ ...prev, [evt.payload.orderId]: { id: evt.payload.orderId, status:
"PLACED" } }));
    }
    if (evt.type === "order.status_changed") {
      setOrders((prev) => ({
        ...prev,
        [evt.payload.orderId]: { ...prev[evt.payload.orderId], status: evt.payload.status },
      }));
    }
  });

  return (
    <div>
      {state === "open" && (
        <div className="kds-offline-banner">
          จอหลุดการเชื่อมต่อ กำลังต่อใหม่ ... ระหว่างนี้ให้เช็กใบครัวที่พิมพ์สำรอง
        </div>
      )}
      {/* การดออร์เดอร์แต่ละใบ ตัวใหญ่ กดยาก ไม่พาดง่าย */}
    </div>
  );
}

```

routing รายการไปสถานีครัว

ร้านที่มีมากกว่าหนึ่งสถานี (เตา ทอด เครื่องดื่ม ของหวาน) ต้องกระจายรายการในออร์เดอร์เดียวไปยังจอที่ถูกต้อง ไม่ใช่โยนออร์เดอร์ทั้งไปไปทุกจอ วิธีที่ตรงไปตรงมาคือผูก `station` เข้ากับ `MenuItem` (หรือ `MenuCategory` ถ้าอยากตั้งเป็นกลุ่ม) แล้วให้ query ของแต่ละจอกรองเฉพาะ `OrderItem` ที่ `menuItem.station` ตรงกับตัวเอง — event ที่ยิงผ่าน SSE ก็ยังแบบ per-station ได้เช่นกัน ไม่จำเป็นต้องให้ client กรองเองทั้งหมด เพื่อลด payload ที่ส่งไปจอที่ไม่เกี่ยวข้อง

จุดที่ต้องคิดเพิ่มคือ **expo station** — จอที่รวมสถานะของทุกสถานีในออร์เดอร์เดียว เพื่อให้รู้ว่างานทั้งหมดของโต๊ะนี้พร้อมเสิร์ฟพร้อมกันหรือยัง (กันเคสอาหารจานหนึ่งเสร็จก่อนแล้วเย็นรอเพื่อน) ออร์เดอร์จะถือว่า **READY** ในภาพรวมได้ก็ต่อเมื่อทุก `OrderItem` ของออร์เดอร์นั้น **READY** ครบทุกสถานีแล้ว

จับเวลาปรุงอาหารและเตือนเมื่อเกินเกณฑ์

เพราะ state machine ในบทที่ 7 บันทึก timestamp ของทุกช่วง (`placedAt` , `startedAt` , `readyAt`) การคำนวณเวลาที่ใช้ต่อออร์เดอร์จึงเป็นแค่การลบเวลา — สิ่งที่ต้องเพิ่มในจอครัวคือการแจ้งเตือนเมื่อออร์เดอร์ค้างนานเกินเกณฑ์ที่ร้านตั้งไว้ (เช่นเกิน 12 นาทีนับจาก `placedAt` ยังไม่ **READY**) ระบบส่วนใหญ่ในวงการ KDS ใช้สถานะพื้นฐานราว `placed` → `in-progress` → `ready` → `completed` ซึ่งตรงกับ state machine ที่ขยายไว้ในบทที่ 7 พอดี

เกณฑ์เวลาที่ใช้เตือนไม่ควรตั้งค่าเดียวทั้งร้าน เพราะเมนูแต่ละอย่างใช้เวลาปรุงไม่เท่ากัน — สเต็กใช้เวลานานกว่าน้ำเปล่าเป็นเรื่องปกติ ไม่ควรขึ้นแดงเตือนพร้อมกัน วิธีที่ใช้ได้จริงคือผูก `targetMinutes` ไว้ที่ระดับ `MenuItem` หรือ `station` แล้วให้ query ฝั่งจอครัวคำนวณเทียบเป็นเปอร์เซ็นต์ของเวลาที่ตั้งไว้ (เช่นเกิน 100% ของ target ให้ขึ้นเหลือง เกิน 150% ให้ขึ้นแดง) แทนที่จะใช้ตัวเลขนาทีตายตัวทีเดียวทั้งเมนู ตัวเลขพวกนี้ยังเป็นวัตถุดิบสำคัญของรายงานเวลาเฉลี่ยต่อออร์เดอร์ที่เจ้าของร้านอยากเห็นตอนปิดวัน (บทที่ 15)

สังเกตสุขภาพของ connection ทั้งระบบ

จอครัวหลายจอ (สถานีเตา ทอด เครื่องดื่ม) หมายความว่า `EventSource` เปิดค้างพร้อมกันหลายสิบ connection ในแต่ละสาขา ถ้าร้านมีหลายสาขาก็คูณขึ้นไปอีก สิ่งที่ต้องมี dashboard ฝั่ง admin (แม้จะเรียบง่ายแค่ query เดียว) คือจำนวน connection ที่เปิดอยู่ต่อสาขา ณ ขณะนี้ กับเวลาที่แต่ละ connection ต่อเข้ามาล่าสุด เพื่อให้รู้ทันทีถ้าสาขาไหน “จอครัวเจียบผิดปกติ” (เช่นจำนวน connection เป็นศูนย์ทั้งที่ร้านเปิดอยู่) ก่อนที่พนักงานหน้างานจะรู้ตัวเองด้วยซ้ำว่าจอหลุดไปแล้วนานแค่ไหน ในระบบที่ใช้ pub/sub กลางอย่าง Redis ข้อมูลนี้ดึงได้ตรง ๆ จากจำนวน subscriber ของ channel นั้น ๆ

UI ครัว: ตัวใหญ่ มือเปียกก็แตะไม่พลาด

ครัวไม่ใช่ office — มือเปียก มือมัน ใส่ถุงมือ จอเปื้อนไอน้ำ ปุ่มกดต้องใหญ่พอที่จะแตะแม่นยำไม่ได้มองตรง ๆ (ปุ่มเปลี่ยนสถานะควรใหญ่กว่ามาตรฐานเว็บทั่วไปหลายเท่า), สีต้องแยกจากกันชัดเจนแม้แสงในครัวจะจ้าหรือ

มิด, และควรมีเสียงเตือนเมื่อมีออเดอร์ใหม่เข้ามา ไม่ใช่พึ่งการมองจอเพียงอย่างเดียว — พ่อครัวก้มหน้าทำงานอยู่ ไม่มีเวลาเหลือบดูจอตลอด

รายละเอียดที่ทำให้จอครัวใช้งานได้จริงในสถานการณ์ยุ่ง ๆ มีเพิ่มอีกสามข้อที่ควรใส่ตั้งแต่รอบแรก ไม่ใช่ค่อยแก้ไขทีหลังตอนพนักงานบ่น:

- **ป้องกันการกดพลาดซ้ำ** — เมื่อกดปุ่ม “เสร็จแล้ว” ควรมีติลย์สั้น ๆ ก่อนปุ่มกลับมากดได้อีกครั้ง หรือเปลี่ยนเป็นปุ่ม “เลิกทำ” ชั่วคราวสัก 3-5 วินาที เพื่อให้กดผิดแล้วยกเลิกทันได้โดยไม่ต้องเรียกพนักงานเข้ามาแก้ผ่านระบบหลังบ้าน
- **จัดเรียงตามความเร่งด่วน ไม่ใช่ตามเวลาที่เข้ามา** — ออร์เดอร์ที่ใกล้เกินเกณฑ์เวลาควรลอยขึ้นบนสุดอัตโนมัติ ไม่ใช่เรียงเป็นคิวเรียบ ๆ ตามเวลาเข้าอย่างเดียว เพราะครัวต้องตัดสินใจเร็วว่าจานไหนต้องรีบก่อน
- **แยกมุมมองต่อสถานี vs มุมมองรวม (expo)** — พ่อครัวแต่ละสถานีอยากเห็นแค่ของตัวเอง ตัวใหญ่ ไม่รบกวนหัวหน้าครัวหรือ expo อยากเห็นภาพรวมทั้งโต๊ะเพื่อประสานเวลาเสิร์ฟ การยืดสองความต้องการนี้ไว้ในจอเดียวกันมักจบด้วยจอที่ไม่มีใครใช้แล้วดี

fallback: จอหลุดต้องพิมพ์ใบครัวได้

นี่คือกฎที่พลาดไม่ได้เด็ดขาด — ถ้าจอครัวหลุดหรือเน็ตในร้านล่ม ครัวต้องมีทางรู้ว่าออเดอร์อะไรเข้ามาโดยไม่ต้องพึ่งจอ เพราะเมื่อ SSE หลุด (ไม่ว่าจะจาก wifi ร้าน หรือ server restart ตอน deploy) ครัวจะโดนตัดขาดจากออเดอร์ใหม่ทันทีถ้าไม่มี fallback

วิธีที่ใช้ได้จริงคือคู่กับเครื่องพิมพ์ครัว (kitchen printer) แบบ ESC/POS — ทุกครั้งที่ server action

`placeOrder` (บทที่ 7) บันทึกออเดอร์สำเร็จ ให้ยิงคำสั่งพิมพ์ใบครัวควบคู่ไปกับการส่ง SSE event เสมอ ไม่ใช่พิมพ์เฉพาะตอนจอ SSE ตรวจพบว่าตัวเองหลุด เพราะการตรวจจับ “จอหลุดหรือยัง” ที่แม่นยำร้อยเปอร์เซ็นต์ทำยาก การพิมพ์คู่ขนานเสมอปลอดภัยกว่า ต้นทุนที่เสียไปคือกระดาษบางใบที่ซ้ำกับจอตอนทุกอย่างทำงานปกติ ซึ่งถูกกว่าความเสี่ยงออเดอร์หายตอนจอหลุดมาก รายละเอียดการต่อเครื่องพิมพ์ด้วย `node-thermal-printer` และปัญหาภาษาไทยกับ codepage อยู่ในบทที่ 12 ที่พูดเรื่องใบเสร็จเต็ม ๆ แต่หลักการเดียวกันนี้ใช้กับใบครัวได้ตรง ๆ

สั่ง Claude Code แบบนี้

ช่วยเขียน Route Handler สำหรับ Server-Sent Events (SSE) ในโปรเจกต์ Next.js App Router ของผม สำหรับจอครัว (KDS) ของระบบ POS ร้านอาหาร

บริบท: ผมตัดสินใจแล้วว่าจะใช้ SSE ไม่ใช่ WebSocket เพราะ Route Handler ของ Next.js อัปเดตเป็น WebSocket ไม่ได้ และผมมีแผนจะ deploy บน Vercel ซึ่งคือ TCP connection ค้างไม่ได้ ช่วยเขียน:

- route handler ที่ stream event ประเภท `order.placed` และ `order.status_changed` ต้องมี heartbeat กับ proxy ตัด connection และต้องรองรับ multi-tenant (กรองด้วย `branchId`)
- client hook `useOrderStream()` ที่ reconnect อัตโนมัติเมื่อหลุด ด้วย exponential backoff (เริ่ม 1 วิ สูงสุด 15 วิ) และรีเซ็ต backoff เมื่อต่อสำเร็จ
- แสดง state การเชื่อมต่อ (`connecting/open/reconnecting`) ให้ UI โชว์แบนเนอร์เตือน พนักงานเมื่อหลุด

ให้ใช้ subagent หรือ skill ถ้ามีสำหรับรีวิว realtime code โดยเฉพาะ แล้วอธิบายทางเลือกเรื่อง in-memory event emitter vs pub/sub กลาง (Postgres LISTEN/NOTIFY หรือ Redis) ว่ากระทบยังไงถ้าผมสเกลเป็นหลาย instance

ต้องรีวิวอะไรในผลลัพธ์: (1) เช็คว่า Route Handler ไม่ได้เขียนโค้ดที่พยายาม upgrade เป็น WebSocket หรือใช้ raw socket ใด ๆ (2) เช็ก cleanup — `cancel()` ของ `ReadableStream` และ `useEffect` cleanup ของ hook ต้อง unsubscribe/clear timer ทุกตัวจริง ไม่งั้นเปิดจอครัวทิ้งไว้นาน ๆ จะรั่ว memory/connection (3) ถ้า Claude เสนอ in-memory `EventEmitter` ให้ถามกลับว่าระบบจะรันกี่ instance พร้อมกัน ถ้ามากกว่า 1 ต้องเปลี่ยนไปใช้ pub/sub กลางไม่ใช่ปล่อยผ่าน

มุมมองของร้าน

จอครัวคือจุดที่ “ข้าแต่ไม่กั๊วนาที” แปลว่าเงินจริง — อาหารใหม่ ลูกค้านาน คิวค่าง เจ้าของร้านที่ตัดสินใจทำ KDS เองต้องรู้ว่าต้นทุนที่มองไม่เห็นคือเครื่องพิมพ์ใบครัวสำรองที่ต้องมีติดร้านเสมอ — ไม่ใช่ทางเลือก แต่เป็นของบังคับ เพราะวันไหนเน็ตร้านล่ม (ซึ่งเกิดขึ้นแน่นอนสักวัน) ถ้าไม่มีใบครัวสำรอง ครัวจะไม่รู้เลยว่ามือออร์เดอร์อะไรเข้ามาบ้าง

อีกจุดที่ควรถามทีมพัฒนาตรง ๆ ก่อนจ่ายเงินจ้างทำ คือ “ถ้าจอครัวหลุดกลางพีค พนักงานต้องทำอะไร” ถ้าคำตอบคือ “รอเน็ตกลับมา” นั่นคือความเสี่ยงที่รับไม่ได้สำหรับร้านที่มีลูกค้าเข้าคิว

❌ พลาดบ่อย / ✅ ทำแบบนี้

❌ **พลาดบ่อย:** ปล่อยให้ SSE connection ไม่มี heartbeat เลย พอไม่มี event นานเกิน 1-2 นาที reverse proxy หรือ load balancer ตัด connection ทั้งเจียบ ๆ โดยที่ client ไม่รู้ตัวว่าหลุดไปแล้ว
✅ **ทำแบบนี้:** ส่ง heartbeat comment event ทุก 15-20 วินาที และให้ client ตรวจสอบผ่าน `onerror` ของ `EventSource` เพื่อ reconnect ทันทีที่หลุดจริง

❌ **พลาดบ่อย:** คิดว่าจอครัว realtime แทนใบครัวได้ 100% แล้วถอดเครื่องพิมพ์ครัวออกไปเลยเพื่อประหยัด
✅ **ทำแบบนี้:** พิมพ์ใบครัวคู่ขนานกับทุกออร์เดอร์เสมอ ไม่ว่าจะทำงานปกติหรือไม่ ต้นทุนกระดาษถูกกว่าความเสี่ยงออร์เดอร์หายตอนจอหลุด

❌ **พลาดบ่อย:** ยิง SSE event ทั้งออร์เดอร์ไปทุกจอครัวโดยไม่กรองตามสถานี ทำให้จอเตาเห็นออร์เดอร์ของจอเครื่องต้มปนกันจนพนักงานสับสน ✅ **ทำแบบนี้:** ผูก **station** เข้ากับ MenuItem/Category แล้วกรอง event เป็น per-station ตั้งแต่ฝั่ง server พร้อมมี expo station แยกที่รวมภาพรวมทั้งออร์เดอร์

จำ 3 ข้อ Route Handler ของ Next.js ทำ WebSocket ไม่ได้ และ Vercel ถือ TCP ค้างไม่ได้ — SSE คือค่าเริ่มต้นที่ถูกต้องสำหรับ KDS จนกว่าจะต้องสื่อสารสองทางความถี่สูงจริง ๆ client ต้อง reconnect เองด้วย exponential backoff และมี heartbeat กันหลุดเจียบ — จอครัวเปิดทิ้งไว้ทั้งกะ พังให้พนักงานกด refresh เองไม่ได้ จอหลุดได้เสมอ ครัวต้องมีใบพิมพ์สำรองคู่ขนานทุกออร์เดอร์ ไม่ใช่ทางเลือกเสริม แต่เป็นของบังคับ

หน้า POS ของพนักงาน



หน้าเมนูลูกค้า (บทที่ 6) กับจอครัว (บทที่ 8) เป็นแค่สองในสี่หน้าจอของระบบ ยังมีอีกหน้าที่งานหนักที่สุดและถูกมองข้ามบ่อยที่สุดตอนออกแบบ — หน้าจอที่พนักงานหน้าร้านใช้ทั้งกะ เปิดโต๊ะ สั่งแทนลูกค้าที่ไม่ถนัดสแกน QR ย้ายโต๊ะตอนลูกค้าขอเปลี่ยนที่นั่ง รวมโต๊ะตอนกลุ่มใหญ่มาเพิ่ม แยกบิลตอนแต่ละคนอยากจ่ายเอง และบางครั้งต้องยกเลิกรายการที่ครัวเริ่มทำไปแล้วเพราะลูกค้าเปลี่ยนใจ

ร้านที่ทีมผมเคยคุยด้วยเจ้าหนึ่งเล่าว่าเคยใช้ระบบสำเร็จรูปที่ “ย้ายโต๊ะ” ทำแค่เปลี่ยนเลขโต๊ะในหน้าจอ แต่ไม่ย้ายออเดอร์ที่ยังไม่จ่ายไปด้วย พนักงานกดผิดสองครั้งจนบิลกระจายอยู่คนละโต๊ะ ปิดกะไม่ลง เงินขาดหาย 300 กว่าบาทหาที่มาไม่เจอ — บทนี้คือบทที่แก้ปัญหาแบบนี้ตั้งแต่ต้นด้วยการมองงานหน้าร้านเป็น **transaction** ที่ต้องถูกทั้งชุดหรือไม่ทำเลย ไม่ใช่การอัปเดตทีละฟิลด์

งานจริงหน้าร้านที่ระบบต้องรองรับ

ก่อนเขียนโค้ด ต้องแยกให้ชัดว่างานหน้าร้านมีกี่แบบ เพราะแต่ละแบบมีความเสี่ยงเรื่องเงินไม่เท่ากัน:

- **เปิดโต๊ะ** — สร้าง table session ใหม่ (รายละเอียด session อยู่ในบทที่ 5) ความเสี่ยงต่ำ
- **สั่งแทนลูกค้า** — ใช้ server action เดียวกับบทที่ 7 (`placeOrder`) แค่เรียกจากฝั่งพนักงานแทนฝั่งลูกค้า ผ่าน validate ชุดเดียวกันทุกอย่าง ไม่มีทางลัด
- **เพิ่มรายการที่หลัง** — เหมือนสั่งใหม่ แต่ผูกกับออเดอร์/โต๊ะที่เปิดอยู่แล้ว

- **ย้ายโต๊ะ** — ย้ายออร์เดอร์ที่ยังไม่จ่ายทั้งหมดจากโต๊ะเดิมไปโต๊ะใหม่ ความเสี่ยงกลาง (ถ้าทำไม่ atomic บิลจะหลุดหาย)
- **รวมโต๊ะ** — สองโต๊ะขึ้นไปกลายเป็นบิลเดียว ความเสี่ยงสูง (ต้องรู้ว่ารวมแล้วยอดใครเป็นใคร)
- **แยกบิล** — บิลเดียวแตกเป็นหลายใบ ความเสี่ยงสูงที่สุด (เศษสตางค์หายได้ง่ายที่สุด — รายละเอียดการปิดเศษหลังแยกบิลอยู่ในบทที่ 10)
- **ยกเลิกรายการที่ครัวเริ่มทำแล้ว** — ต้องขออนุมัติ (state machine บทที่ 7 บังคับให้เข้า **VOID** ไม่ใช่ **CANCELLED**)
- **เปลี่ยนกะ** — ผูกกับ audit log และรายงานปิดกะ (บทที่ 15)

แต่ละงานข้างต้นดูเหมือนเป็นแค่ “ปุ่มบนหน้าจอ” แต่ในความเป็นจริงแต่ละปุ่มแต่ละข้อมูลคนละชุด — เปิดโต๊ะแต่ละแค่ **TableSession** ตารางเดียว ในขณะที่รวมโต๊ะแต่ละทั้ง **TableSession** หลายแถว, **Order** ทุกใบที่ผูกอยู่กับโต๊ะเหล่านั้น, และต้องคำนึงถึงว่าถ้าโต๊ะใดโต๊ะหนึ่งมีบิลที่จ่ายไปแล้วบางส่วนจะรวมได้หรือไม่ (คำตอบคือไม่ได้ — บิลที่ **PAID** แล้วต้องแยกออกจากกระบวนการรวมเสมอ เพราะแก้ยอดที่จ่ายเงินไปแล้วไม่ได้) ความซับซ้อนที่ซ่อนอยู่หลังปุ่มแต่ละปุ่มนี้เองที่ทำให้ทีมพัฒนาหลายทีมมองข้ามตอนประเมินงาน แล้วมาเจอปัญหาจริงตอน user acceptance test กับพนักงานร้าน

สิทธิ์: ใครทำอะไรได้บ้าง

งานหน้าร้านทั้งหมดต้องผูกกับ role ไม่ใช่ปล่อยให้ทุกคนกดได้ทุกอย่าง เพราะนี่คือจุดที่คนโกงหน้าร้านมักใช้ประโยชน์ (รายละเอียดรูปแบบการโกงอยู่ในบทที่ 13) เล่มนี้ใช้ role ขั้นต่ำสามระดับ:

Role	ทำได้
STAFF (พนักงานทั่วไป)	เปิดโต๊ะ, สั่งแทนลูกค้า, เพิ่มรายการ, ยกเลิกก่อนครัวเริ่มทำ (PLACED → CANCELLED)
SHIFT_LEAD (หัวหน้ากะ)	ทุกอย่างของ STAFF + ย้ายโต๊ะ + รวมโต๊ะ + แยกบิล + ยกเลิกหลังครัวเริ่มทำ (VOID) + ให้ส่วนลด
OWNER (เจ้าของร้าน)	ทุกอย่าง + ดูรายงาน audit log ทั้งหมด + จัดการสิทธิ์คนอื่น

กฎนี้ต้องบังคับที่ server action ทุกจุด ไม่ใช่แค่ซ่อนปุ่มใน UI — ซ่อนปุ่มกันแค่คนไม่ตั้งใจ ไม่กันคนที่เปิด dev tools ยิง request ตรง

โค้ด: ย้ายโต๊ะและรวมโต๊ะแบบ transaction พร้อม guard สิทธิ์

หลักการเดียวกับบทที่ 7 — ทุกการเปลี่ยนแปลงที่กระทบมากกว่าหนึ่งแถวต้องอยู่ใน **prisma.\$transaction** เดียว เพื่อไม่ให้เกิดสภาพ “ย้ายไปครึ่งทาง” ถ้ามีอะไรพังกลางคัน

```

// app/(pos)/actions/move-table.ts
"use server";

import { prisma } from "@lib/prisma";
import { requireStaff } from "@lib/auth";
import { logAudit } from "@lib/audit";

export async function moveTable(fromSessionId: string, toTableId: string) {
  const actor = await requireStaff(["STAFF", "SHIFT_LEAD", "OWNER"]);

  return prisma.$transaction(async (tx) => {
    const fromSession = await tx.tableSession.findUniqueOrThrow({
      where: { id: fromSessionId },
      include: { orders: true },
    });

    if (fromSession.status !== "OPEN") {
      throw new Error("โต๊ะต้นทางไม่ได้เปิดอยู่ ย้ายไม่ได้");
    }

    const toTable = await tx.table.findUniqueOrThrow({ where: { id: toTableId } });
    const toActiveSession = await tx.tableSession.findFirst({
      where: { tableId: toTable.id, status: "OPEN" },
    });
    if (toActiveSession) {
      throw new Error("โต๊ะปลายทางมีบิลเปิดอยู่แล้ว ต้องรวมโต๊ะแทนการย้าย");
    }

    // ย้าย session ทั้งก้อนไปยังโต๊ะใหม่ - ออร์เดอร์ที่ผูกกับ session นี้ย้ายตามไปเองโดยอัตโนมัติ
    const moved = await tx.tableSession.update({
      where: { id: fromSessionId },
      data: { tableId: toTable.id },
    });

    await logAudit(tx, {
      actorId: actor.id,
      action: "TABLE_MOVED",
      targetId: fromSessionId,
      metadata: { fromTableId: fromSession.tableId, toTableId: toTable.id },
    });

    return moved;
  });
}

export async function mergeTables(sessionIds: string[]) {
  // รวมโต๊ะกระหนยอดเงินรวมของหลายบิล - ต้องหัวหนากะขึ้นไปที่เท่านั้น
  const actor = await requireStaff(["SHIFT_LEAD", "OWNER"]);

  if (sessionIds.length < 2) {
    throw new Error("ต้องเลือกอย่างน้อย 2 โต๊ะถึงจะรวมได้");
  }

  return prisma.$transaction(async (tx) => {
    const sessions = await tx.tableSession.findMany({
      where: { id: { in: sessionIds } },
    });
  });
}

```

```

});

if (sessions.length !== sessionIds.length || sessions.some((s) => s.status !== "OPEN")) {
  throw new Error("มีโต๊ะที่ไม่ได้เปิดอยู่ปนอยู่ในรายการที่จะรวม");
}

const [primary, ...rest] = sessions;

// ย้ายออเดอร์ทั้งหมดของโต๊ะรองเข้าโต๊ะหลัก แล้วปิด session ของโต๊ะรอง
for (const s of rest) {
  await tx.order.updateMany({
    where: { tableSessionId: s.id },
    data: { tableSessionId: primary.id },
  });
  await tx.tableSession.update({
    where: { id: s.id },
    data: { status: "MERGED", mergedIntoId: primary.id },
  });
}

await logAudit(tx, {
  actorId: actor.id,
  action: "TABLES_MERGED",
  targetId: primary.id,
  metadata: { mergedSessionIds: rest.map((s) => s.id) },
});

return tx.tableSession.findUniqueOrThrow({
  where: { id: primary.id },
  include: { orders: true },
});
}
}

```

สังเกตสามจุดสำคัญ: (1) `mergeTables` เรียก `requireStaff(["SHIFT_LEAD", "OWNER"])` ปิดสิทธิ์ `STAFF` ธรรมดาไว้ตั้งแต่ต้นฟังก์ชัน ไม่ใช่เช็คทีหลัง (2) เช็ค `status !== "OPEN"` ก่อนทุกครั้งเพื่อป้องกันการย้าย/รวมโต๊ะที่ปิดไปแล้วหรือถูก merge ไปแล้วซ้ำ (3) `logAudit` ถูกเรียกอยู่ *ในธุรกรรมเดียวกัน* กับการเปลี่ยนแปลงจริง — ถ้าธุรกรรม rollback audit log ก็หายตามไปด้วย ไม่ทิ้ง log ลอย ๆ ที่ไม่ตรงกับสิ่งที่เกิดขึ้นจริง

แยกบิลและยกเลิกรายการที่ครัวเริ่มทำแล้ว

```
// app/(pos)/actions/void-item.ts
"use server";

import { prisma } from "@lib/prisma";
import { requireStaff } from "@lib/auth";
import { logAudit } from "@lib/audit";

export async function voidOrderItem(orderItemId: string, reason: string) {
  // ยกเลิกหลังครัวเริ่มทำแล้วต้องขออนุมัติจากหัวหน้ากะขึ้นไปเท่านั้น
  const actor = await requireStaff(["SHIFT_LEAD", "OWNER"]);

  if (!reason || reason.trim().length < 3) {
    throw new Error("ต้องระบุเหตุผลการยกเลิกทุกครั้ง");
  }

  return prisma.$transaction(async (tx) => {
    const item = await tx.orderItem.findUniqueOrThrow({
      where: { id: orderItemId },
      include: { order: true },
    });

    if (item.order.status === "PAID") {
      throw new Error("บิลนี้จ่ายเงินแล้ว ต้องออกเอกสารคืนเงินแยกต่างหาก ไม่ใช่ยกเลิยก่อนหลัง");
    }

    const updated = await tx.orderItem.update({
      where: { id: orderItemId },
      data: { voidedAt: new Date(), voidedBy: actor.id, voidReason: reason },
    });

    // ปรับยอดรวมของออเดอร์ใหม่หลังตัดรายการที่ถูก void ออก
    const remaining = await tx.orderItem.findMany({
      where: { orderId: item.orderId, voidedAt: null },
    });
    const newTotal = remaining.reduce(
      (sum, i) => sum + i.unitPriceCentsSnapshot * i.quantity,
      0
    );
    await tx.order.update({ where: { id: item.orderId }, data: { totalAmount: newTotal } });

    await logAudit(tx, {
      actorId: actor.id,
      action: "ORDER_ITEM_VOIDED",
      targetId: orderItemId,
      metadata: { orderId: item.orderId, reason, refundAmount: item.unitPriceCentsSnapshot *
item.quantity },
    });

    return updated;
  });
}
```

หลักการที่ต้องยึดคือ ทุกการยกเลิก/แก้ไข/ให้ส่วนลดต้องลง audit log เสมอ ไม่มีข้อยกเว้น เพราะนี่คือหลักฐานเดียวที่เจ้าของร้านมีไว้ตรวจสอบภายหลังว่าเกิดอะไรขึ้นจริง — ไม่ใช่แค่ “บันทึกไว้เพื่อมีปัญหา” แต่คือกลไก

--

ป้องกันการโกงหน้าร้านโดยตรง (บทที่ 13 อธิบายว่าคั่นหารูปแบบผิดปกติจาก log พวกนี้ได้ยังไง เช่น พนักงานคนเดียวกัน void ถีผิดปกติในกะเดียว)

แยกบิล: จุดที่เศษสตางค์หายง่ายที่สุด

แยกบิลเสี่ยงที่สุดในบรรดางานหน้าร้านทั้งหมด เพราะเป็นจุดที่ยอดรวมเดิมต้องถูกตัดแบ่งออกเป็นหลายก้อน โดยผลรวมของทุกก้อนต้องเท่ากับยอดตั้งต้นเป๊ะ — ไม่ใช่แค่แบ่งราคาสินค้าเท่า ๆ กัน แต่ต้องคิดด้วยว่า service charge และ VAT ของบทที่ 10 ผูกไว้ที่ระดับบิลไหน ก่อนแบ่ง ต้องคำนวณยอดสุทธิของบิลเดิมให้เสร็จก่อน แล้วค่อยตัดแบ่งยอดที่คิดภาษีแล้วลงในแต่ละบิลย่อย ไม่ใช่แบ่งยอดก่อนภาษีแล้วไปคิด VAT แยกทีละบิลย่อยซึ่งจะพิเศษไม่ตรงกับยอดรวมเดิม

```

// app/(pos)/actions/split-bill.ts
"use server";

import { prisma } from "@lib/prisma";
import { requireStaff } from "@lib/auth";
import { logAudit } from "@lib/audit";

type SplitGroup = { orderItemIds: string[] };

export async function splitBill(orderId: string, groups: SplitGroup[]) {
  const actor = await requireStaff(["SHIFT_LEAD", "OWNER"]);

  if (groups.length < 2) {
    throw new Error("ต้องแบ่งอย่างน้อย 2 บิลถึงจะเรียกว่าแยกบิล");
  }

  return prisma.$transaction(async (tx) => {
    const original = await tx.order.findUniqueOrThrow({
      where: { id: orderId },
      include: { items: true },
    });

    if (original.status === "PAID") {
      throw new Error("บิลนี้จ่ายเงินแล้ว แยกย้อนหลังไม่ได้");
    }

    const allAssignedIds = groups.flatMap((g) => g.orderItemIds);
    const uncovered = original.items.filter((i) => !allAssignedIds.includes(i.id));
    if (uncovered.length > 0) {
      throw new Error("มีรายการที่ยังไม่ถูกจัดลงบิลย่อยใดเลย ต้องแบ่งให้ครบทุกรายการ");
    }

    const newOrders = [];
    for (const group of groups) {
      const groupItems = original.items.filter((i) => group.orderItemIds.includes(i.id));
      const groupTotal = groupItems.reduce(
        (sum, i) => sum + i.unitPriceCentsSnapshot * i.quantity,
        0
      );

      const newOrder = await tx.order.create({
        data: {
          tableSessionId: original.tableSessionId,
          status: original.status,
          totalAmount: groupTotal,
          splitFromOrderId: original.id,
        },
      });

      await tx.orderItem.updateMany({
        where: { id: { in: group.orderItemIds } },
        data: { orderId: newOrder.id },
      });

      newOrders.push(newOrder);
    }
  });
}

```

```

await tx.order.update({
  where: { id: original.id },
  data: { status: "VOID", voidedAt: new Date(), voidedBy: actor.id, voidReason: "SPLIT_BILL" },
});

await logAudit(tx, {
  actorId: actor.id,
  action: "BILL_SPLIT",
  targetId: original.id,
  metadata: { newOrderIds: newOrders.map((o) => o.id), originalTotal: original.totalAmount },
});

return newOrders;
});
}

```

จุดที่ห้ามลืมคือการเช็ค **uncovered** — ถ้ามีรายการหลุดไม่ถูกจัดลงบิลย่อยไหนเลย ต้อง throw ทันทีก่อนที่จะสร้างบิลย่อยไปครึ่งทาง ไม่ใช่ปล่อยให้แยกบิลสำเร็จโดยมีรายการลอยหายไปเงียบ ๆ ซึ่งเป็นสาเหตุที่ปิดกะแล้วยอดไม่ลงตัวบ่อยที่สุด

เปลี่ยนกะ ก็ต้องมีรอยดินบนกระดาด

เปลี่ยนกะดูเหมือนงานธุรการเล็ก ๆ แต่จริง ๆ คือจุดที่ต้อง “ปิดบัญชี” ความรับผิดชอบของคนหนึ่งแล้วส่งต่อให้อีกคน ระบบควรบันทึกอย่างน้อยว่าใครเข้ากะ ใครออกกะ เวลาไหน และเงินสดในลิ้นชักตอนส่งกะมีเท่าไร (ผูกกับรายงานปิดกะเต็มรูปแบบในบทที่ 15) ทุกครั้งที่มีการเปลี่ยนกะ ควรเขียน audit log ประเภท

SHIFT_CHANGED ไปด้วย เพื่อให้ตอนตรวจสอบ log ของ **TABLE_MOVED**, **BILL_SPLIT**, **ORDER_ITEM_VOIDED** ย้อนหลัง รู้ได้ทันทีว่าเหตุการณ์นั้นเกิดในกะของใคร ไม่ต้องไปนั่งเทียบเวลาเอาเอง

สั่ง Claude Code แบบนี้

ช่วยเขียน server action สำหรับหน้า POS พนักงานในระบบร้านอาหารของผม 3 ฟังก์ชัน: moveTable (ย้ายโต๊ะ), mergeTables (รวมโต๊ะ), voidOrderItem (ยกเลิกรายการที่ครัวเริ่มทำไปแล้ว)

กติกาที่ต้องทำตามเป๊ะ อ้างอิง schema ใน prisma/schema.prisma:

1. ทุกฟังก์ชันต้องเช็คสิทธิ์ก่อนทำงานจริง ไม่ใช่เช็คทีหลัง — STAFF ทำได้แค่เปิดโต๊ะ/สั่ง/ยกเลิกก่อนครัวเริ่มทำ ส่วนย้ายโต๊ะ/รวมโต๊ะ/void ต้อง SHIFT_LEAD ขึ้นไป
2. ทุกฟังก์ชันต้องอยู่ใน prisma.\$transaction เดียว ห้ามอัปเดตทีละคำสั่งแยกกัน
3. ทุกการยกเลิก/รวม/ย้ายต้องเขียน audit log อยู่ในธุรกรรมเดียวกับการเปลี่ยนแปลงจริง (ถ้า rollback ต้อง rollback log ด้วย)
4. voidOrderItem ต้องบังคับให้กรอกเหตุผล และห้าม void ออร์เดอร์ที่จ่ายเงินแล้ว

ใช้ plan mode ก่อนเพื่อวางแผนการตรวจเงื่อนไขในแต่ละฟังก์ชัน แล้วค่อย apply โค้ด

ต้องรีวิวอะไรในผลลัพธ์: (1) เช็คกว่า **requireStaff** (หรือฟังก์ชันเช็คสิทธิ์ที่ Claude เขียน) ถูกเรียกเป็นบรรทัดแรก ๆ ของทุกฟังก์ชัน ไม่ใช่แทรกอยู่กลางท่อ logic ที่อาจมีทางหลุดผ่านได้ (2) ตรวจสอบว่า **mergeTables** / **voidOrderItem** ปิดสิทธิ์ **STAFF** จริง ไม่ใช่แค่คอมเมนต์บอกไว้เฉย ๆ (3) เช็คกว่า audit log

เขียนอยู่ ในตัวแปร `tx` เดียวกับการเปลี่ยนแปลงข้อมูล ไม่ใช่เรียก `prisma.auditLog.create` แยกนอก transaction ซึ่งจะทำให้ log ไม่ sync กับข้อมูลจริงถ้า transaction fail

มุมมองของร้าน

ก่อนอื่นต้องพูดตรง ๆ ว่าหน้า POS พนักงานเป็นหน้าจอที่พนักงานเปิดค้างไว้ทิ้งกะ ไม่เคยปิด — ต่างจากหน้าเมนูลูกค้าที่เปิดแค่ตอนสั่งอาหารแล้วก็หายไป การออกแบบที่ไม่ดีในหน้านี้จึงถูกใช้ซ้ำแล้วซ้ำเล่านับร้อยครั้งต่อวัน ความผิดพลาดเล็ก ๆ ที่ดูไม่สำคัญตอนทดสอบครั้งแรก จะถูกขยายด้วยความถี่ของการใช้งานจนกลายเป็นปัญหาใหญ่ในเวลาไม่นาน

หน้า POS พนักงานคือหน้าที่ “เงินหาย” บ่อยที่สุดในร้าน ไม่ใช่เพราะโกงเสมอไป แต่เพราะเป็นหน้าที่พนักงานใช้เร็ว ใช้บ่อย ใช้ตอนงานยุ่ง — พลาดง่ายที่สุด เจ้าของร้านควรถามทีมพัฒนาให้ขีดสามข้อก่อนใช้งานจริง: (1) ย้ายโต๊ะแล้วบิลเดิมย้ายไปด้วยจริงไหม หรือแค่เปลี่ยนเลขโต๊ะเฉย ๆ (2) ใครมีสิทธิ์กดยกเลิกรายการที่ครัวทำไปแล้ว และมีเหตุผลบันทึกไว้ให้ตรวจสอบย้อนหลังได้ไหม (3) พนักงานลาออกไปแล้ว บัญชียังใช้ได้อยู่ไหม

สิทธิ์ที่ตั้งไว้หลวมเกินไปคือความเสี่ยงเงียบ ๆ ที่ไม่แสดงผลจนกว่าจะมีคนใช้ประโยชน์จากมันจริง — ตอนนั้นมักสายเกินจะย้อนกลับไปแก้ไขได้ทันที

เรื่องที่มักคุ้ยกันซ้ำเกินไปคือ “พนักงานพาร์ทไทม์ที่มาแค่วันเสาร์-อาทิตย์ ควรมีสิทธิ์ทำพนักงานประจำไหม” คำตอบไม่มีถูกผิดตายตัว แต่ประเด็นคือระบบต้องเปิดให้ตั้งสิทธิ์ได้ละเอียดระดับบุคคล ไม่ใช่ผูกสิทธิ์ไว้กับตำแหน่งงานตายตัวจนปรับไม่ได้ — ร้านหนึ่งอาจอยากให้พนักงานพาร์ทไทม์สั่งอาหารได้แต่ยกเลิก/void ไม่ได้เลยแม้จะทำงานมานานแค่ไหนก็ตาม ซึ่งเป็นการตัดสินใจเชิงธุรกิจของเจ้าของร้าน ไม่ใช่เรื่องที่ทีมพัฒนาควรตัดสินใจแทน

❌ พลาดบ่อย / ✅ ทำแบบนี้

❌ **พลาดบ่อย:** ย้ายโต๊ะด้วยการอัปเดตแค่เลขโต๊ะในตาราง session โดยไม่ตรวจว่าโต๊ะปลายทางมีบิลเปิดอยู่แล้วหรือเปล่า ทำให้บิลสองใบไปทับกัน ✅ **ทำแบบนี้:** ตรวจสอบสถานะโต๊ะปลายทางก่อนย้ายทุกครั้ง ถ้ามีบิลเปิดอยู่แล้วต้องบังคับให้ใช้ฟังก์ชันรวมโต๊ะแทน ไม่ใช่ย้ายทับ

❌ **พลาดบ่อย:** ซ่อนปุ่ม “ยกเลิก/void” ไว้เฉพาะ UI ของหัวหน้ากะ แต่ server action เดียวกันเปิดให้ role ไหนก็เรียกได้ถ้ารู้ endpoint ✅ **ทำแบบนี้:** เช็กละเอียดที่ server action ทุกจุดเป็นเงื่อนไขบังคับ (ไม่ใช่แค่ซ่อน UI) เพราะการซ่อนปุ่มกันแค่คนไม่ตั้งใจ ไม่กันคนที่ยิง request ตรง

❌ **พลาดบ่อย:** เขียน audit log แยกเป็น query ต่างหากนอก transaction หลักหลังจากอัปเดตข้อมูลเสร็จแล้ว (“เดี๋ยวค่อยบันทึก log”) ✅ **ทำแบบนี้:** เขียน audit log ในตัวแปร transaction เดียวกับการเปลี่ยนแปลงข้อมูลเสมอ เพื่อให้ log กับข้อมูลจริง sync กันร้อยเปอร์เซ็นต์ ไม่ว่าจะ commit หรือ rollback

จำ 3 ข้อ งานหน้าร้าน (ย้ายโต๊ะ/รวมโต๊ะ/แยกบิล) คือ transaction ที่ต้องถูกทั้งชุดหรือไม่ทำเลย ไม่ใช่การอัปเดตทีละฟิลด์ สิทธิ์ต้องเช็คที่ server action ทุกจุด ไม่ใช่แค่ซ่อนปุ่มใน UI — role ที่หลวมเกินไปคือ ความเสี่ยงเจ็บที่ไม่โผล่จนกว่าจะสาย ทุกการยกเลิก แก้ไข หรือให้ส่วนลด ต้องลง audit log ในธุรกรรมเดียวกับการเปลี่ยนแปลงจริงเสมอ ไม่มีข้อยกเว้น

คิดเงินให้ถูกทุกบาท



บ่ายวันหนึ่งพนักงานเสิร์ฟบิลให้ลูกค้าโต๊ะ 12 ยอด 1,177.30 บาท ลูกค้าหยิบใบเสร็จขึ้นมาดู แล้วถามเบา ๆ ว่า “ทำไมยอดมันไม่ลงตัวแบบเดิมที่เคยกินคะ” พนักงานตอบไม่ได้ เจ้าของร้านตอบไม่ได้ เพราะไม่มีใครรู้วาระบบคิดเลขยังไง — และนี่คือปัญหาที่อันตรายที่สุดของ POS ที่สร้างเอง ไม่ใช่เพราะมันทำให้ร้านขาดทุนวันละกี่บาท แต่เพราะ **ทุกบาทที่ผิดคือความเชื่อมั่นที่หายไป** ทั้งจากลูกค้าและจากเจ้าของร้านที่จ่ายเงินจ้างให้คุณสร้างระบบนี้ขึ้นมา

บทนี้เป็นบทที่ “ไม่มีพีเจอรี่ใหม่ให้ดูตื่นเต้น” แต่เป็นบทที่ถ้าทำผิดจะเจ็บที่สุด เพราะการคำนวณเงินที่ผิดจะไม่มี error สีแดงขึ้นมาเตือน มันจะเนียนอยู่ในระบบ แสดงตัวเลขให้ดูสมเหตุสมผล จนกว่าจะมีคนสังเกตเห็นความต่างเล็ก ๆ ที่สะสมมาเป็นเดือน

ที่อันตรายกว่านั้นคือ Claude Code เขียนฟังก์ชันคิดเงินให้ได้ในไม่กี่วินาที และโค้ดที่ได้มามักจะ “ดูถูกต้อง” เพราะมันคอมไพล์ผ่าน รันได้ ตัวเลขออกมาดูสมเหตุสมผลในเคสที่ลองไม่กี่เคส แต่โค้ดที่ดูถูกต้องกับโค้ดที่ถูกต้องจริงเป็นคนละเรื่องกันโดยสิ้นเชิงเมื่อพูดถึงเงิน — งานของคุณในบทนี้ไม่ใช่แค่สั่งให้ AI เขียนฟังก์ชัน แต่คือ การรู้ล่วงหน้าว่ากับดักอยู่ตรงไหนบ้าง เพื่อตรวจ output ให้ทันก่อนที่มันจะไปอยู่หน้าเคาน์เตอร์จริง

ลำดับการคำนวณที่ถูกต้อง: อาหาร → service charge → VAT

ร้านอาหารไทยที่เก็บทั้ง service charge และ VAT ต้องคำนวณตามลำดับนี้เท่านั้น:

1. ยอดอาหาร (ราคาสุทธิหลังหักส่วนลดรายการแล้ว)
2. บวก service charge 10% — คิดจากยอดอาหาร
3. บวก VAT 7% — คิดจากยอดอาหาร บวกกับ service charge ที่คำนวณไปแล้วในข้อ 2

ตัวอย่างที่ง่ายที่สุด: อาหาร 100 บาท → +10% (service charge 10 บาท) = 110 บาท → +7% ของ 110 บาท (VAT 7.70 บาท) = **117.70 บาท**

เหตุผลที่ VAT ต้องคิดทีหลังและคิดจากฐานที่รวม service charge เข้าไปแล้ว เพราะ **service charge ถือเป็นรายได้ค่าบริการของร้าน** ไม่ใช่เงินที่แยกออกไปจากรายได้ค่าอาหาร มันจึงต้องถูกนับรวมเข้าไปในฐานภาษีก่อนคิด VAT เหมือนกับที่ค่าอาหารเองก็เป็นฐานภาษี — พุดง่าย ๆ คือ VAT คือภาษีที่คิดจาก “สิ่งที่ร้านขาย” ทั้งก่อนไม่ใช่คิดแยกทีละส่วนแล้วเอามารวมกันทีหลัง

จุดที่ dev มือใหม่พลาดบ่อยที่สุดคือสลับลำดับ เอา VAT คิดก่อน แล้วเอา service charge มาบวกทับ ผลลัพธ์จะได้ตัวเลขใกล้เคียงกันมาก (117.70 เท่ากันทั้งสองวิธีในเคสนี้พอดี เพราะ 10% และ 7% คุณสลับที่กันได้ในทางคณิตศาสตร์แบบง่าย) แต่พอมีส่วนลด มีรายการที่ยกเว้น VAT บางส่วน หรือมีการพิเศษเข้ามาเกี่ยวข้อง ลำดับที่ผิดจะทำให้ตัวเลขเพี้ยนทันที และที่สำคัญกว่านั้นคือ **ใบเสร็จต้องแจกแจงให้เห็นว่า service charge คือฐานของ VAT** ไม่ใช่แค่ตัวเลขรวมท้ายบิลที่บังเอิญตรง

ใบเสร็จร้านอาหารจึงต้องแสดงอย่างน้อย 3 บรรทัด: ยอดอาหาร, ค่าบริการ (service charge), และ VAT — แยกกันให้เห็นชัด ไม่ใช่โยนเป็นยอดเดียว

มีอีกจุดที่ควรระวังคือร้านบางแห่งไม่เก็บ service charge เลย (โดยเฉพาะร้านเล็กหรือร้านสตรีทฟู้ดที่จดทะเบียน VAT) เพราะฉะนั้นอัตรา service charge ไม่ควรถูก hardcode ไว้ในโค้ดเป็นค่าคงที่ตายตัวที่แก้ไม่ได้ ให้ออกแบบเป็นค่าที่ตั้งได้ต่อร้าน (per-tenant configuration) ตั้งแต่แรก เพราะถ้าไปฝังไว้ในฟังก์ชันคำนวณตรง ๆ วันหนึ่งที่ร้านต้องปรับอัตรา หรือระบบต้องรองรับร้านที่สองที่ไม่เก็บ service charge เลย จะกลายเป็นงาน refactor ใหญ่ที่แตะทุกจุดที่คิดเงิน

เก็บเงินเป็นจำนวนเต็มสตางค์ ห้ามใช้ float

นี่คือกฎที่สำคัญที่สุดของบทนี้ และควรเขียนไว้ใน **CLAUDE.md** ตั้งแต่บทที่ 2 ด้วยซ้ำ: **ห้ามเก็บหรือคำนวณเงินด้วยชนิดข้อมูลทศนิยมแบบ float เด็ดขาด** ให้เก็บทุกยอดเป็นจำนวนเต็มหน่วยสตางค์ (1 บาท = 100 สตางค์) แล้วค่อยหารด้วย 100 ตอนแสดงผลเท่านั้น

ปัญหาของ float ไม่ใช่เรื่องทฤษฎี ลองรันโค้ดนี้ใน Node.js ดูจริง ๆ:

```
console.log(0.1 + 0.2);  
// 0.30000000000000004  
  
console.log(29.4 * 3);  
// 88.19999999999999 (ไม่ใช่ 88.2)  
  
console.log(1177.7 === 100 + 10 + 7.7);  
// false
```

เหตุผลคือคอมพิวเตอร์เก็บเลขทศนิยมแบบ binary floating point (IEEE 754) ซึ่งไม่สามารถแทนค่าเลขฐานสิบอย่าง 0.1 หรือ 0.2 ได้แม่นยำร้อยเปอร์เซ็นต์ — เหมือนที่เลข 1/3 เขียนเป็นทศนิยมฐานสิบไม่ลงตัวฉันใด เลข 0.1 ในฐานสองก็ไม่ลงตัวฉันนั้น ผลคือทุกครั้งที่บวก ลบ คูณ หารเลขทศนิยม จะมีเศษความคลาดเคลื่อนเล็ก ๆ หลุดเข้ามา และเมื่อร้านมีหลายร้อยบิลต่อวัน เศษเหล่านี้จะสะสมจนยอดขายในระบบไม่ตรงกับเงินสดในลิ้นชักหรือยอดโอนจริง

วิธีแก้คือเปลี่ยนหน่วยคิดจาก “บาท (มีทศนิยม)” เป็น “สตางค์ (จำนวนเต็ม)” ตลอดทั้งระบบ ตั้งแต่ schema ฐานข้อมูล (`unitPriceSatang Int` ตามที่วางไว้ในบทที่ 4) ไปจนถึงตัวแปรในโค้ด การคำนวณของจำนวนเต็มด้วย `+` `-` `*` ไม่มีปัญหาความคลาดเคลื่อนแบบนี้เลย มีแค่การหารเท่านั้นที่ต้องปัดเศษ ซึ่งนำไปสู่กฎข้อถัดไป

กติกานี้ต้องบังคับใช้ทุกจุดที่แตะเงิน ไม่ใช่แค่ฐานข้อมูล — API ที่รับ/ส่งยอดเงินก็ควรส่งเป็นจำนวนเต็มสตางค์เช่นกัน ไม่ใช่ส่ง `117.70` เป็น JSON number แล้วให้ฝั่งรับเดาว่าหน่วยคืออะไร เพราะ JSON เองก็เข้ารหัสตัวเลขทศนิยมด้วย IEEE 754 เหมือนกัน ปัญหาเดียวกันจะเกิดซ้ำแค่ย้ายจุดที่ฟังจากฝั่ง server ไปฝั่ง client เท่านั้น ส่วนการแสดงผลบนหน้าจอหรือใบเสร็จ ให้มีฟังก์ชันกลางฟังก์ชันเดียวทำหน้าที่แปลงสตางค์เป็นข้อความบาทที่อ่านง่าย เช่น `formatSatangToBaht(11770) → "117.70"` เพื่อไม่ให้แต่ละหน้าจอเขียนสูตรหาร 100 ซ้ำกันคนละที่คนละแบบ ซึ่งเป็นอีกจุดหนึ่งที่ error เล็ก ๆ ซบอบแอบแฝงเข้ามา

ส่วนลด: ต่อยาการ vs ทั้งบิล

ร้านอาหารมีส่วนลด 2 ระดับที่ต้องแยกให้ชัดเพราะจุดที่หักไม่เหมือนกัน:

- **ส่วนลดต่อยาการ** (line discount) เช่น “เมนูนี้ลด 20 บาท”, “ท็อปปิ้งฟรี” — หักตรงที่บรรทัดนั้นก่อนรวมยอด
- **ส่วนลดทั้งบิล** (bill discount) เช่น “คูปองลด 10%”, “สมาชิกลด 50 บาท” — หักหลังรวมยอดทุกรายการแล้ว แต่ต้องหักก่อนคิด service charge เสมอ เพราะ service charge ควรคิดจากยอดที่ลูกค้าจ่ายจริงหลังส่วนลด ไม่ใช่ยอดเต็มก่อนลด

ส่วนลดแบบเปอร์เซ็นต์ต้องระวังเรื่องการปัดเศษเป็นพิเศษ เพราะ 10% ของ 333 สตางค์ คือ 33.3 สตางค์ ซึ่งไม่ใช่จำนวนเต็ม ต้องปัดเศษตรงจุดนี้ — และนี่คือจุดที่การปัดเศษ “เกิดขึ้นจริง” อย่างมีเหตุผล ต่างจากการปัดเศษ VAT ที่ควรเกิดครั้งเดียว (อธิบายต่อในหัวข้อถัดไป)

โปรโมชั่นที่ซับซ้อนกว่านั้น เช่น “ซื้อ 1 แถม 1” หรือ “ลด 20% เฉพาะเครื่องดื่ม” ให้มองเป็นกรณีพิเศษของส่วนลดต่อยาการเสมอ — แถมฟรีคือส่วนลด 100% ของรายการนั้น ส่วนลดเฉพาะหมวดคือส่วนลดต่อยาการที่กรองด้วยเงื่อนไขหมวดหมู่ก่อนคำนวณ อย่าสร้างระบบคิดเงินคู่ขนานสำหรับโปรโมชั่น เพราะจะทำให้มีจุดคำนวณ VAT/service charge สองที่ที่ต้องดูแลให้ตรงกันตลอดไป

อีกเรื่องที่ต้องตัดสินใจตั้งแต่ตอนออกแบบคือ ส่วนลดควรถูกบันทึกเป็น “เหตุผล” เสมอ ไม่ใช่แค่ตัวเลข — เช่น เลือกจากรายการที่ตั้งไว้ล่วงหน้า (“ส่วนลดพนักงาน”, “ค่าอาหารไม่ได้มาตรฐาน”, “โปรโมชั่นวันเกิด”) แทนที่จะให้พนักงานพิมพ์จำนวนเงินลดเองแบบอิสระโดยไม่ระบุเหตุผล เพราะช่องว่างตรงนี้คือช่องทางโกงที่พบบ่อยที่สุดในร้านอาหาร (พนักงานให้ส่วนลดตัวเองแล้วเก็บส่วนต่างเข้ากระเป๋า) — รายละเอียดการตรวจจับความผิดปกตินี้จะอยู่ในบทที่ 13 แต่จุดเริ่มต้นคือการบังคับให้ทุกส่วนลดมีเหตุผลตั้งแต่ตอนออกแบบฟังก์ชันนี้

กฎการปิดเศษ: ปิดครั้งเดียวที่ยอดสุดท้าย

นี่คือกฎที่ dev พลาดบ่อยที่สุดเป็นอันดับสองรองจากลำดับ VAT: ห้ามคำนวณภาษีหรือค่าบริการแยกทีละรายการแล้วปิดเศษทีละบรรทัดก่อนเอามารวมกัน ให้รวมยอดทุกรายการเป็นก้อนเดียวก่อน แล้วปิดเศษที่ยอดรวมนั้นครั้งเดียว

เหตุผลคือการปิดเศษแต่ละครั้งมีโอกาสดคลื่อนได้สูงสุด 0.5 หน่วย ถ้าบิลมี 10 รายการแล้วปิดเศษ VAT ทีละรายการ ความคลาดเคลื่อนสูงสุดสะสมได้ถึง 5 หน่วย ในขณะที่ปิดครั้งเดียวที่ยอดรวมความคลาดเคลื่อนสูงสุดคงที่ที่ 0.5 หน่วยเท่านั้นไม่ว่าบิลจะมีกี่รายการ ตัวอย่างจริงจะเห็นในชุดทดสอบด้านล่าง ซึ่งพิสูจน์ว่าสองวิธีให้ผลต่างกันจริงแม้จะเป็นบิลง่าย ๆ

ปัญหาอีกจุดที่เกี่ยวข้องกันคือ แยกบิลแล้วเศษหาย — เวลาลูกค้า 3 คนขอแยกจ่ายบิล 100.01 บาทเท่า ๆ กัน หางตรง ๆ จะได้ 33.336... บาทต่อคน ถ้าไม่จัดการเศษให้ดี อาจปิดลงทั้งสามคนแล้วยอดรวมขาดไป 1 สตางค์ วิธีที่ถูกต้องคือหารเป็นจำนวนเต็มสตางค์แบบ floor แล้วเอาเศษที่เหลือ (remainder) ไปแจกให้คนแรก ๆ ทีละสตางค์จนหมด เพื่อให้ผลรวมของทุกส่วนเท่ากับยอดบิลเป๊ะเสมอ ไม่มีสตางค์ไหนหายไปจากระบบ

โค้ด: `calculateBill()` และชุดทดสอบ

ก่อนดูโค้ด ให้ตั้งสติก่อนว่าฟังก์ชันนี้มี “จุดปิดเศษ” อยู่แค่ 2 จุดเท่านั้นตามกฎที่วางไว้ข้างบน คือตอนหักส่วนลดเปอร์เซ็นต์ทั้งบิล (ถ้ามี) และตอนคิด service charge/VAT จากยอดรวม — ถ้าอ่านโค้ดที่ Claude Code เขียนมาแล้วนับจุดที่มี `Math.round` หรือการปิดเศษได้มากกว่านี้ ให้สงสัยไว้ก่อนว่ามีการปิดเศษรายบรรทัดหลุดเข้ามาโดยไม่ตั้งใจ

ไฟล์ `lib/money.ts` — ฟังก์ชันคำนวณบิลตามกติกาทั้งหมดข้างต้น:

```

// lib/money.ts
// กติกาตายตัว: ทุกยอดเงินเป็นจำนวนเต็มสตางค์ (Satang) เท่านั้น ห้ามมีทศนิยม
export type Satang = number;

export interface BillLineInput {
  id: string;
  unitPriceSatang: Satang; // snapshot ราคา ณ เวลาสั่ง (ดูบทที่ 4)
  quantity: number;
  lineDiscountSatang?: Satang; // ส่วนลดเฉพาะรายการนี้
}

export interface BillDiscountInput {
  type: 'PERCENT' | 'AMOUNT';
  value: number; // PERCENT: 0-100, AMOUNT: สตางค์
}

export interface CalculateBillOptions {
  serviceChargeRatePerMille?: number; // ค่าเริ่มต้น 100 = 10%
  vatRatePerMille?: number; // ค่าเริ่มต้น 70 = 7%
  billDiscount?: BillDiscountInput;
}

export interface BillResult {
  lines: { id: string; lineTotalSatang: Satang }[];
  itemSubtotalSatang: Satang; // ก่อนหักส่วนลดทั้งบิล
  subtotalSatang: Satang; // หลังหักส่วนลดทั้งบิล = ฐานของ service charge
  serviceChargeSatang: Satang;
  vatBaseSatang: Satang; // subtotal + serviceCharge
  vatSatang: Satang;
  grandTotalSatang: Satang;
}

const DEFAULT_SERVICE_CHARGE_PER_MILLE = 100; // 10%
const DEFAULT_VAT_PER_MILLE = 70; // 7%

export function calculateBill(
  items: BillLineInput[],
  options: CalculateBillOptions = {},
): BillResult {
  const serviceChargeRate =
    options.serviceChargeRatePerMille ?? DEFAULT_SERVICE_CHARGE_PER_MILLE;
  const vatRate = options.vatRatePerMille ?? DEFAULT_VAT_PER_MILLE;

  // 1) รวมยอดแต่ละรายการ - คุณ/ลบเลขจำนวนเต็มล้วน ไม่มีการปัดเศษเกิดขึ้นตรงนี้
  const lines = items.map((item) => {
    if (!Number.isInteger(item.unitPriceSatang) || !Number.isInteger(item.quantity)) {
      throw new Error(`ราคา/จำนวนต้องเป็นจำนวนเต็มสตางค์เท่านั้น: ${item.id}`);
    }

    const rawTotal = item.unitPriceSatang * item.quantity;
    const discount = item.lineDiscountSatang ?? 0;
    const lineTotalSatang = Math.max(0, rawTotal - discount);
    return { id: item.id, lineTotalSatang };
  });

  const itemSubtotalSatang = lines.reduce((sum, l) => sum + l.lineTotalSatang, 0);

```

```

// 2) ส่วนลดทั้งบิล หักหลังรวมรายการ ก่อนคิด service charge เสมอ
let subtotalSatang = itemSubtotalSatang;
if (options.billDiscount) {
  const { type, value } = options.billDiscount;
  subtotalSatang -=
    type === 'PERCENT'
      ? Math.round((itemSubtotalSatang * value) / 100) // บั๊ตเศษ ณ จุดที่ส่วนลดเกิดขึ้นจริง
      : value;
  subtotalSatang = Math.max(0, subtotalSatang);
}

// 3) service charge คิดจากยอดอาหารหลังส่วนลด - บั๊ตเศษ "ครั้งแรก"
const serviceChargeSatang = Math.round((subtotalSatang * serviceChargeRate) / 1000);

// 4) ฐาน VAT = อาหาร + service charge ตามลำดับที่กรมสรรพากรกำหนด
const vatBaseSatang = subtotalSatang + serviceChargeSatang;

// 5) VAT คิดจากฐานรวมก่อนเดียว บั๊ตเศษ "ครั้งที่สอง" ที่นี่ทีเดียว
// ห้ามคิด VAT แยกรายบรรทัดแล้วบั๊ตเศษทีละบรรทัดก่อนบวกกัน (ดูทดสอบด้านล่าง)
const vatSatang = Math.round((vatBaseSatang * vatRate) / 1000);

const grandTotalSatang = vatBaseSatang + vatSatang;

return {
  lines,
  itemSubtotalSatang,
  subtotalSatang,
  serviceChargeSatang,
  vatBaseSatang,
  vatSatang,
  grandTotalSatang,
};
}

// แยกบิล N ทางเท่า ๆ กัน - เฉพาะสตางค์ที่หารไม่ลงตัวต้องมีคนรับไป ไม่ใช่หายจากระบบ
export function splitBillEqually(totalSatang: Satang, parts: number): Satang[] {
  if (!Number.isInteger(parts) || parts ≤ 0) {
    throw new Error('parts ต้องเป็นจำนวนเต็มมากกว่า 0');
  }
  const base = Math.floor(totalSatang / parts);
  const remainder = totalSatang - base * parts;
  return Array.from({ length: parts }, (_, i) => base + (i < remainder ? 1 : 0));
}

```

ชุด unit test ด้วย vitest — ไฟล์ `lib/money.test.ts`:

```

// lib/money.test.ts
import { describe, it, expect } from 'vitest';
import { calculateBill, splitBillEqually } from './money';

describe('calculateBill', () => {
  it('คำนวณลำดับ อาหาร → service charge 10% → VAT 7% ตามตัวอย่างมาตรฐาน', () => {
    const result = calculateBill([
      { id: 'i1', unitPriceSatang: 10000, quantity: 1 }
    ]);
    expect(result.subtotalSatang).toBe(10000); // 100.00 บาท
    expect(result.serviceChargeSatang).toBe(1000); // 10.00 บาท
    expect(result.vatBaseSatang).toBe(11000); // 110.00 บาท
    expect(result.vatSatang).toBe(770); // 7.70 บาท
    expect(result.grandTotalSatang).toBe(11770); // 117.70 บาท
  });

  it('รวมหลายรายการและหักส่วนลดต่อรายการก่อนคิด service charge', () => {
    const result = calculateBill([
      { id: 'i1', unitPriceSatang: 12000, quantity: 2 }, // 240.00
      { id: 'i2', unitPriceSatang: 4500, quantity: 1, lineDiscountSatang: 500 }, // 40.00
    ]);
    expect(result.itemSubtotalSatang).toBe(28000); // 280.00
  });

  it('หักส่วนลดทั้งบิลแบบเปอร์เซ็นต์หลังรวมรายการ ก่อนคิด service charge', () => {
    const result = calculateBill([
      { id: 'i1', unitPriceSatang: 20000, quantity: 1 }
    ], {
      billDiscount: { type: 'PERCENT', value: 10 },
    });
    expect(result.subtotalSatang).toBe(18000); // 200 - 10% = 180.00
    expect(result.serviceChargeSatang).toBe(1800); // คิดจาก 180.00 ไม่ใช่ 200.00
  });

  it('ส่วนลดทั้งบิลแบบจำนวนเงินที่เกินยอด ต้อง clamp 'ไม่ให้ติดลบ', () => {
    const result = calculateBill([
      { id: 'i1', unitPriceSatang: 5000, quantity: 1 }
    ], {
      billDiscount: { type: 'AMOUNT', value: 999999 },
    });
    expect(result.subtotalSatang).toBe(0);
    expect(result.grandTotalSatang).toBe(0);
  });

  it('ปฏิเสธราคาที่ไม่ใช่จำนวนเต็ม กัน float หลุดเข้าระบบ', () => {
    expect(() =>
      calculateBill([
        { id: 'i1', unitPriceSatang: 99.5, quantity: 1 }
      ])
    ).toThrow();
  });

  it('บัตพิเศษ VAT ที่ละรายการแล้วบวก ให้ผลต่างจากบัตครั้งเดียวที่ยอดรวม (นี่คือบั๊กที่ต้องรีวิว)', () => {
    const pricesSatang = [3333, 3333, 3334]; // รวม 10000 สตางค์ = 100.00 บาทพอดี

    // วิธีถูก: รวมยอดก่อน แล้วบัตพิเศษครั้งเดียวตอนคิด VAT
    const totalFirst = pricesSatang.reduce((a, b) => a + b, 0);
    const correctVat = Math.round(totalFirst * 0.07);

    // วิธีผิด: คิด VAT แยกทีละรายการแล้วบัตพิเศษทีละบรรทัดก่อนบวก
    const wrongVat = pricesSatang.reduce((sum, p) => sum + Math.round(p * 0.07), 0);

    expect(correctVat).toBe(700); // 7.00 บาทพอดี ตรงกับ 100 * 7%
    expect(wrongVat).toBe(699); // ขาดไป 1 สตางค์เพราะบัตพิเศษสามครั้งแทนที่จะบัตครั้งเดียว
  });
});

```

```

    expect(wrongVat).not.toBe(correctVat);
  });
});

describe('splitBillEqually', () => {
  it('แบ่งบิลที่ลงตัวได้เท่ากันทุกคน', () => {
    expect(splitBillEqually(30000, 3)).toEqual([10000, 10000, 10000]);
  });

  it('แบ่งบิลไม่ลงตัว เศษสตางค์ต้องไม่หายไปจากผลรวม', () => {
    const shares = splitBillEqually(10001, 3);
    expect(shares).toEqual([3334, 3334, 3333]);
    expect(shares.reduce((a, b) => a + b, 0)).toBe(10001); // ผลรวมต้องเท่ายอดเดิมเป๊ะ
  });

  it('ปฏิเสธการแบ่งเป็น 0 หรือค่าติดลบ', () => {
    expect(() => splitBillEqually(10000, 0)).toThrow();
  });
});

```

นี่คือบทที่ควรมีทดสอบมากที่สุดในเล่ม เพราะทุก edge case ที่ทดไว้ในนี้คือเหตุการณ์ที่เกิดขึ้นจริงในร้าน ไม่ใช่ทฤษฎี — ลูกค้าขอแยกบิลจริง มีคูปองส่วนลดจริง มีบิลที่ตัวเลขลงตัวพอดีไม่บ่อยเท่าที่คิด

ข้อควรระวังอีกอย่างเวลาให้ AI เขียนเทสให้คือ อย่าปล่อยให้ AI คิดค่าคาดหวัง (expected value) เองทั้งหมด จากการรันโค้ดจริงแล้วก็อปผลลัพธ์มาใส่ในเทส เพราะถ้าโค้ดมีบั๊กอยู่แล้ว เทสแบบนี้จะ “ยืนยันบั๊ก” แทนที่จะจับบั๊ก ค่าคาดหวังของกรณีสำคัญอย่างตัวอย่างมาตรฐาน 100 บาทต้องได้ 117.70 บาท ควรคำนวณด้วยมือหรือเครื่องคิดเลขแยกต่างหากก่อน แล้วค่อยเอาไปเทียบกับสิ่งที่โค้ดคำนวณได้ไม่ใช่กลับกัน

สั่ง Claude Code แบบนี้

ช่วยเขียนฟังก์ชัน `calculateBill()` ใน TypeScript สำหรับคำนวณบิลร้านอาหาร
รับ input เป็น array ของรายการอาหาร (ราคาต่อหน่วยเป็นจำนวนเต็มสตางค์, จำนวน, ส่วนลดต่อรายการถ้ามี) และ options สำหรับส่วนลดทั้งบิล

กติกาที่ต้องทำตามเป๊ะ ห้ามยืดหยุ่นเอง:

- เก็บและคำนวณเงินเป็นจำนวนเต็มสตางค์ (Int) ตลอดทาง ห้ามใช้ float/ตัวเลขทศนิยมเด็ดขาด
- ลำดับคำนวณคือ ยอดอาหาร (หักส่วนลดแล้ว) → service charge 10% → VAT 7%
จากยอดอาหารบวก service charge (ไม่ใช่คิด VAT ก่อน)
- ปัดเศษ (Math.round) เฉพาะตอนสรุปยอด service charge และยอด VAT เท่านั้น
ห้ามคิดภาษี/ค่าบริการแยกรายการแล้วปัดเศษทีละบรรทัดก่อนค่อยบวกรวม
- เขียน unit test ด้วย vitest ให้ครอบคลุมอย่างน้อย: กรณีมาตรฐาน (อาหาร 100 บาท ต้องได้ 117.70 บาทพอดี), ส่วนลดต่อรายการ, ส่วนลดทั้งบิลทั้งแบบ % และแบบจำนวนเงิน, กรณีส่วนลดเกินยอดต้องไม่ติดลบ, กรณีพิสูจน์ว่าปัดเศษทีละบรรทัดให้ผลต่างจากปัดครั้งเดียว, และฟังก์ชันแยกบิลที่ผลรวมของทุกส่วนต้องเท่ายอดเดิมเป๊ะไม่มีสตางค์หาย

เพราะนี่คือฟังก์ชันที่กระทบเงินทุกบิลของร้าน ให้เข้า plan mode ก่อน แล้วอธิบายลำดับการคำนวณและจุดที่จะปัดเศษให้ฉันดูก่อนเริ่มเขียนโค้ดจริง

ตอน review output ให้เช็ก 4 อย่างนี้เป็นอย่างน้อย: (1) ทุกจุดที่แตะราคาเป็น `Int` จริงไหม มี `toFixed()` หรือ `parseFloat()` หลุดเข้ามาที่ไหนหรือเปล่า (2) `Math.round` เกิดกี่จุด เกิดตรงไหนบ้าง ตรงกับกติกาการปัดเศษครั้งเดียวไหม (3) ค่าติดลบจากส่วนลดเกินยอดถูก clamp ทุกจุดไหม (4) เทสครอบคลุมทุก branch ของ

โค้ดจริงหรือแค่ happy path — ถ้า Claude Code เขียนเทสมาน้อยกว่าที่สั่ง ให้สั่งเพิ่มจนครบ อย่ายอมรับแค่เพราะเทสที่มีอยู่ผ่านหมด

มุมมองจากร้าน

เรื่องนี้ฟังดูเป็นรายละเอียดทางเทคนิค แต่ผลกระทบตกที่กระเป๋าเงินของร้านโดยตรง ลองนึกภาพร้านที่มี 150 บิลต่อวัน ถ้าการคำนวณ VAT ผิดพลาดสะสมเฉลี่ยบิลละ 1 สตางค์ (เพราะปัดเศษผิดวิธี) วันหนึ่งดูไม่เท่าไร แต่สะสมเป็นปีคือหลักพันบาทที่ต้องยื่นภาษีผิดจากที่ควรจะเป็น และที่สำคัญกว่าตัวเงินคือ **ยอด VAT ที่แจ้งกรมสรรพากรต้องตรงกับที่เก็บจากลูกค้าจริง** ความเพี้ยนเล็ก ๆ ที่ระบบสร้างขึ้นเองโดยไม่มีใครตั้งใจ อาจกลายเป็นปัญหาตอนสอบบัญชีปลายปี

อีกเรื่องที่เจ้าของร้านต้องรู้คือ **การแยกบิล** เป็นฟีเจอร์ที่ลูกค้าขบถและพนักงานหน้าร้านมักทำเองด้วยเครื่องคิดเลข ถ้าระบบไม่รองรับให้ลูกค้าตั้งตั้งแต่แรก (เศษสตางค์ต้องไม่หาย ไม่ใช่ปัดลงทุกส่วนจนยอดขาด) พนักงานจะหันไปใช้วิธีมั่ว ๆ นอกระบบ ซึ่งพาไปสู่การโกงหรือความผิดพลาดที่ตรวจสอบย้อนหลังไม่ได้เลย — เรื่องนี้จะกลับมาอีกครั้งในบทที่ 13 เรื่องการกันโกงหน้าร้าน

สุดท้าย เจ้าของร้านควรถามผู้พัฒนาให้ชัดเจนตั้งแต่วันแรกว่า “ฟังก์ชันคิดเงินมีเทสไหม เทสครอบคลุมกรณีไหนบ้าง” เพราะนี่คือคำถามเดียวที่แยกได้ว่าโค้ดเรื่องเงินถูกเขียนอย่างระมัดระวังหรือแค่เขียนให้ผ่านตาบนหน้าจอ ระบบที่ไม่มีเทสสำหรับการคำนวณเงินเลยคือระบบที่ไม่มีใครรู้ว่าจะพังตอนไหน จนกว่าจะมีลูกค้าหรือกรมสรรพากรมาเจอเข้าเอง

❌ พลาดบ่อย / ✅ ทำแบบนี้

❌ **พลาดบ่อย:** เก็บราคาสินค้าในฐานข้อมูลเป็น `Float` หรือ `Decimal` แบบบาทตรง ๆ (เช่น `99.50`) แล้วคำนวณ VAT/service charge ด้วยเลขทศนิยมตลอดทาง

✅ **ทำแบบนี้:** เก็บทุกยอดเงินเป็น `Int` หน่วยสตางค์ (`9950` แทน `99.50`) คำนวณด้วยจำนวนเต็มทั้งหมด แล้วหารด้วย 100 เฉพาะตอนจะแสดงผลบนหน้าจอหรือใบเสร็จเท่านั้น

❌ **พลาดบ่อย:** คำนวณ VAT และ service charge แยกทีละรายการอาหารในบิล แล้วปัดเศษทีละบรรทัดก่อนรวมยอด เพราะดูเหมือนจะ “แม่นยำกว่า” ที่ให้แต่ละเมนูมีภาษีของตัวเอง

✅ **ทำแบบนี้:** รวมยอดอาหารทุกรายการเป็นก้อนเดียวก่อน แล้วคำนวณ service charge และ VAT จากยอดรวมนั้น ปัดเศษแค่ 2 จุด (ตอนได้ service charge และตอนได้ VAT) ไม่ใช่ปัดเศษทุกบรรทัด

❌ **พลาดบ่อย:** คิด VAT จากยอดอาหารอย่างเดียว แล้วเอา service charge มาบวกทีหลังโดยไม่รวมเข้าฐานภาษี

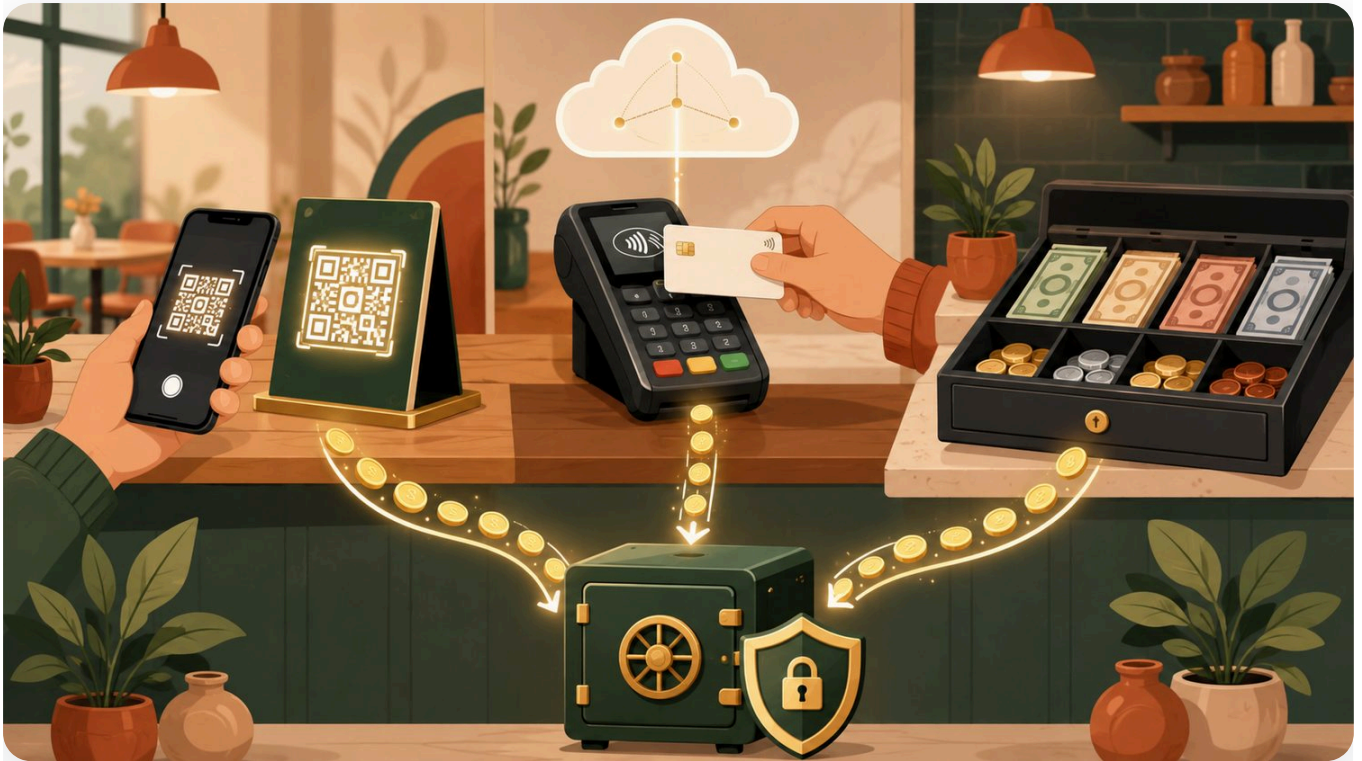
✅ **ทำแบบนี้:** บวก service charge เข้ากับยอดอาหารก่อน แล้วค่อยคิด VAT 7% จากยอดรวมนั้น ตามที่ service charge ถือเป็นรายได้ค่าบริการที่ต้องอยู่ในฐานภาษีด้วย

ก่อนปิดบท มีเรื่องหนึ่งที่เราควรทำความเข้าใจไว้ล่วงหน้า คือฟังก์ชัน `calculateBill()` ในบทนี้เป็น “แกนคำนวณ” (pure function) ที่ไม่แตะฐานข้อมูลหรือ side effect ใด ๆ เลย รับ input แล้วคืนผลลัพธ์ตรง ๆ ซึ่งเป็นการออกแบบที่ตั้งใจ เพราะฟังก์ชันที่แตะเงินควรทดสอบได้ง่ายที่สุดเท่าที่จะทำได้ ไม่ต้อง mock ฐานข้อมูล ไม่

ต้องรอ network เวลาจะเรียกใช้จริงในบทที่ 9 (หน้า POS) หรือบทที่ 11 (รับเงิน) ก็แค่ดึงข้อมูลจาก Prisma มาแปลงเป็น input ของฟังก์ชันนี้ แล้วค่อยเอาผลลัพธ์ไปบันทึกหรือไปแสดงผลต่อ การแยกส่วนแบบนี้ทำให้เวลาสงสัยว่ายอดเงินผิดตรงไหน สามารถตัดปัญหาเรื่องคำนวณกับปัญหาเรื่องดึงข้อมูลออกจากกันได้ทันที

จำ 3 ข้อ เงินทุกบาทเก็บเป็นจำนวนเต็มสตางค์ ห้ามใช้ float เด็ดขาด ลำดับคำนวณคือ อาหาร → service charge 10% → VAT 7% จากยอดรวม ห้ามสลับ ปิดเศษครั้งเดียวที่ยอดสุดท้าย ไม่ปัดรายบรรทัดแล้วค่อยบวก — และทดสอบกรณีที่มีการปิดเศษ

รับเงิน: PromptPay บัตร และเงินสด



ออร์เดอร์เสร็จ คราวทำเสร็จ ลูกค้ากินอ้อม — งานที่เหลือฟังดูง่ายที่สุดคือ “รับเงิน” แต่ในทางปฏิบัติ นี่คือจุดที่ระบบ POS ทำเองพังบ่อยที่สุด เพราะการรับเงินไม่ได้จบที่วินาทีที่ลูกค้าสแกน QR หรือรูดบัตร มันมีขั้นตอนที่มองไม่เห็นอีกหลายชั้นอยู่เบื้องหลัง ตั้งแต่มาตรฐาน QR ของธนาคารแห่งประเทศไทย ไปจนถึง webhook ที่ธนาคารหรือ payment gateway ส่งมาบอกว่า “จ่ายแล้วนะ” ซึ่งถ้าเขียนรับมือไม่ถูก บิลของร้านจะค้างสถานะ “รอชำระ” ทั้งที่เงินเข้าบัญชีไปแล้วจริง ๆ

บทนี้หนาที่สุดในเล่มด้วยเหตุผลเดียว: เรื่องเงินเข้าออกจริงคือจุดที่ให้อภัยไม่ได้ ผิดเรื่องอื่นแก้ทีหลังได้ ผิดเรื่องรับเงินคือลูกค้าไวหนี้ร้านทันที

และต่างจากบทที่ 10 ที่ทุกอย่างเกิดในระบบของเราเอง ในบทนี้ระบบของเราต้องคุยกับ “คนนอก” อย่างน้อยสามฝ่าย คือแอปธนาคารของลูกค้า, payment gateway ที่เราเลือกใช้, และธนาคารของร้านเอง — แต่ละฝ่ายมีมาตรฐาน มีจังหวะเวลา และมีความน่าเชื่อถือของตัวเองที่เราควบคุมไม่ได้ สิ่งที่เราควบคุมได้มีแค่ฝั่งเดียวคือโค้ดของเรา ต้องเขียนให้รับมือกับความไม่แน่นอนของอีกสามฝ่ายนั้นได้อย่างปลอดภัย

PromptPay QR ทำงานยังไงจริง ๆ

PromptPay เป็นมาตรฐาน QR การชำระเงินที่ธนาคารแห่งประเทศไทย (ธปท.) ออกแบบไว้ให้ใช้ได้กับแอป mobile banking แทบทุกธนาคารในไทย โดยอิงสเปกสากล **EMV QRCPs (QR Code Payment)**

Specification) แบบ Merchant Presented Mode หมายความว่า ร้านเป็นฝ่ายแสดง QR ให้ลูกค้าสแกนจากแอปธนาคารของตัวเอง (ตรงข้ามกับ Consumer Presented Mode ที่ลูกค้าโชว์ QR ให้ร้านสแกน)

QR ของ PromptPay ไม่ใช่แค่รูปสี่เหลี่ยมสุ่ม ๆ แต่คือการเข้ารหัสข้อความ (payload) ตามโครงสร้าง tag-length-value (TLV) ที่ตายตัว โดย tag ที่ต้องรู้จักมีดังนี้:

Tag	ความหมาย	ตัวอย่างค่า
00	Payload Format Indicator	01 (เวอร์ชันของสเปก)
01	Point of Initiation Method	11 = static (ไม่ระบุยอด), 12 = dynamic (ระบุยอดตายตัว)
29	Merchant Account Information	ข้อมูลบัญชี PromptPay ของร้าน
53	Transaction Currency	764 = รหัสสกุลเงินบาทตามมาตรฐาน ISO 4217
54	Transaction Amount	ยอดเงิน (มีเฉพาะ QR แบบ dynamic)
58	Country Code	TH
63	CRC	checksum ตรวจสอบความถูกต้องของ payload ทั้งก้อน

Tag 29 กับ Tag 30 — ห้ามสลับกันเด็ดขาด

จุดที่สับสนบ่อยที่สุดและเป็นจุดที่ทำให้ร้านเสียหายจริงถ้าเขียนผิดคือความต่างระหว่าง tag 29 กับ tag 30:

- **Tag 29 = PromptPay credit transfer** — โอนเงินตรงเข้าบัญชีที่ผูก PromptPay ไปด้วย เบอร์มือถือ หรือ เลขประจำตัวผู้เสียภาษี ของร้าน โดย **ไม่มี reference data** ใด ๆ แขนงมากับเงินที่โอน เงินเข้าบัญชีเฉย ๆ เหมือนโอนเงินให้เพื่อน ร้านอาหารทั่วไปที่รับเงินผ่าน QR หน้าร้านใช้ tag นี้เป็นหลัก
- **Tag 30 = QR Cross Bank / Bill Payment** — มี **reference data** แนบไปด้วย ใช้เมื่อร้านลงทะเบียนเป็น **Biller** กับธนาคาร โดย Biller ID ประกอบด้วย เลขประจำตัวผู้เสียภาษี 13 หลัก + suffix อีก 2 หลัก วิธีนี้เหมาะกับธุรกิจที่ต้องผูกเลขบิล/เลขใบแจ้งหนี้เข้ากับยอดที่จ่ายโดยตรง เช่น การจ่ายค่าน้ำค่าไฟ ไม่ใช่ QR หน้าเคาน์เตอร์ร้านอาหารทั่วไป

ทำไมห้ามสลับ: ถ้าโค้ดใส่ payload ผิด tag ไป เช่นเอาโครงสร้างของ tag 30 มาใช้ทั้งที่ร้านไม่ได้ลงทะเบียน Biller ไว้จริง แอปธนาคารของลูกค้าจะปฏิเสธ QR ทันทีหรือแยกว่านั่นคือสแกนผ่านแต่เงินไปผิดที่ ส่วนถ้าร้านมี Biller ID จริงแต่ดันสร้าง QR แบบ tag 29 (ไม่มี reference) ระบบจะเสีย reference ที่ควรใช้จับคู่บิลอัตโนมัติไปฟรี ๆ ต้องกลับไปทำ reconcile มือเหมือนไม่มีระบบ

CRC16 คืออะไร ทำไมต้องมี

Tag 63 ปิดท้าย payload ด้วยค่า CRC (Cyclic Redundancy Check) แบบ 16 บิต ซึ่งเป็นค่า checksum ที่คำนวณจากอักขระทั้งหมดของ payload ตั้งแต่ tag **00** ไปจนถึง tag+length ของ tag **63** เอง (ไม่รวมค่า checksum ที่กำลังจะได้) แล้วแนบผลลัพธ์เป็นเลขฐาน 16 จำนวน 4 หลักปิดท้าย จุดประสงค์คือให้แอปธนาคารตรวจสอบได้ว่า payload ที่อ่านมาจากภาพ QR ไม่ถูกอ่านผิดหรือเสียหายระหว่างทาง (เช่น กล้องอ่าน QR ผิดตัวอักษรบางตัว) ถ้า checksum ไม่ตรง แอปจะปฏิเสธ QR นั้นทันทีโดยไม่พยายามโอนเงิน

ข้อควรระวังในทางปฏิบัติ: **อย่าเขียนอัลกอริทึม CRC16 เองใหม่** ต่อให้เข้าใจหลักการดีแค่ไหน เพราะ CRC16 มีหลายสูตรย่อย (ต่าง polynomial ต่าง initial value) และ EMV QR spec ต้องการสูตรที่ตรงเป๊ะ ถ้าเลือกสูตรผิดนิดเดียว QR จะถูกแอปธนาคารปฏิเสธหมดทุกครั้งแบบหาสาเหตุยาก — ให้พึ่งไลบรารีที่ผ่านการทดสอบมาแล้ว เช่นที่ **nodejs-promptpay** ใช้ **crc16-xmodem** เป็นตัวคำนวณ

เจน QR ด้วยไลบรารีจริง

อย่าประกอบ payload TLV เองมือ ให้ใช้ไลบรารีที่มีอยู่แล้วในวงการ ที่ตรวจสอบว่ามีจริงและใช้กันแพร่หลาย ได้แก่ **promptpay-qr** (โดย dtinth ซึ่งมีทั้ง web app, CLI และ JS library), **promptpay-js**, **promptparse** (all-in-one รองรับทั้ง PromptPay และ EMVCo QR ทั่วไป) และ **nodejs-promptpay**

ตัวอย่างการเจน payload ด้วย **promptpay-qr** แล้ว render เป็นรูป QR ด้วยไลบรารี **qrcode**:

```
// lib/promptpay.ts
import generatePayload from 'promptpay-qr';
import QRCode from 'qrcode';

interface GeneratePromptPayQrOptions {
  promptPayId: string; // เบอร์มือถือ หรือเลขผู้เสียภาษีผูก PromptPay ของร้าน
  amountBaht?: number; // ใส่ = dynamic QR (ยอดตายตัว), 'ไม่ใส่' = static QR
}

export async function generatePromptPayQrDataUrl(
  opts: GeneratePromptPayQrOptions,
): Promise<string> {
  // ห้ามส่ง amount เป็น float ที่มีเศษเกิน 2 ตำแหน่ง - มีดจากสตริงก่อนแปลงเป็นบาทเสมอ
  const payload = generatePayload(opts.promptPayId, {
    amount: opts.amountBaht,
  });
  // payload คือสตริง TLV ที่มี tag 00/01/29/53/54/58/63 ครบตามสเปก EMV QRCPs
  return QRCode.toDataURL(payload);
}
```

โค้ดฝั่งเรียกใช้จริงในบิล — ต้องแปลงจากสตริง (หน่วยที่เราคำนวณตามบทที่ 10) เป็นบาทเฉพาะตอนสร้าง QR เท่านั้น:

```
// app/(pos)/bill/[billId]/qr/route.ts
import { generatePromptPayQrDataUrl } from '@lib/promptpay';

export async function GET(req: Request, { params }: { params: Promise<{ billId: string }> }) {
  const { billId } = await params;
  const bill = await prisma.bill.findUniqueOrThrow({ where: { id: billId } });

  const dataUrl = await generatePromptPayQrDataUrl({
    promptPayId: process.env.PROMPTPAY_ID!, // เบอร์/เลขผู้เสียภาษีของร้าน ไม่ใช่ของลูกค้า
    amountBaht: bill.grandTotalSatang / 100, // dynamic QR ต้องระบุยอดเสมอสำหรับบิลจริง
  });

  return Response.json({ qrDataUrl: dataUrl });
}
```

static QR vs dynamic QR — ปัญหาการกระทบยอด

QR แบบ **static** (**tag 01 = 11**) ไม่ระบุยอดเงินตายตัว ลูกค้าต้องพิมพ์จำนวนเงินเองในแอปธนาคาร ข้อดีคือพิมพ์ QR แผ่นเดียวแปะหน้าร้านใช้ได้ตลอดไป แต่ข้อเสียที่ร้ายแรงคือ **reconcile** ยากมาก เพราะระบบไม่มีทางรู้เลยว่ายอดเงินที่โอนเข้ามาแต่ละรายการเป็นของบิลไหน โดยเฉพาะร้านที่มีหลายโต๊ะจ่ายพร้อมกันในเวลาใกล้เคียงกัน หรือมีบิลยอดเท่ากันพอดีสองใบ

QR แบบ **dynamic** (**tag 01 = 12**) ระบุยอดเงินตายตัวลงใน tag **54** เงินขึ้นใหม่ทุกครั้งต่อหนึ่งบิล ลูกค้าสแกนแล้วยอดขึ้นให้ทันทีแก้ไม่ได้ วิธีนี้ช่วยลดความเสี่ยงเรื่องยอดผิดได้มาก แต่ **ก็ยังไม่ได้แก้ปัญหา reconcile ให้จบร้อยเปอร์เซ็นต์** เพราะ QR ที่เงินเองผ่าน tag 29 (credit transfer ธรรมดา) ไม่มี reference ผูกกับบิลอยู่ดี ระบบยังต้องอาศัยการจับคู่ “ยอดเงิน + ช่วงเวลา” กับรายการที่ธนาคารแจ้งเข้ามา ซึ่งเสี่ยงชนกันถ้ามีสองบิลยอดเท่ากันพอดีในเวลาใกล้เคียงกัน

นี่คือเหตุผลที่ร้านที่ต้องการ reconcile อัตโนมัติแม่นยำจริง ๆ มักหันไปใช้ **payment gateway** ที่มี webhook แจ้งกลับพร้อมเลขอ้างอิงบิลของเราเอง แทนที่จะเงิน PromptPay QR ตรงด้วยตัวเอง — เนื้อหาหัวข้อถัดไป

ตัวอย่างการใช้ **promptpay-js** ซึ่งเปิดพารามิเตอร์ให้เห็นความต่างระหว่าง static กับ dynamic ชัดกว่า **promptpay-qr** เล็กน้อย เพราะบังคับให้ระบุ **method** ตรง ๆ:

```
// lib/promptpay-alt.ts - ตัวอย่างเทียบกับ promptpay-js (เลือกใช้ตัวใดตัวหนึ่งพอ)
import { generatePayload } from 'promptpay-js';

export function buildStaticQrPayload(mobileNumber: string) {
  // static: ไม่ระบุยอด ลูกค้าพิมพ์เอง → reconcile ยาก ใช้เฉพาะกรณีจำเป็นจริง ๆ
  return generatePayload({
    method: 'QR_STATIC',
    application: 'PROMPTPAY_CREDIT_TRANSFER', // = tag 29 เสมอ ไม่ใช่ tag 30
    mobileNumber,
    currencyCode: '764',
    countryCode: 'TH',
  });
}

export function buildDynamicQrPayload(mobileNumber: string, amountBaht: number) {
  // dynamic: ระบุยอดตายตัวต่อบิล ใช้เป็นค่าเริ่มต้นของระบบ POS เสมอ
  return generatePayload({
    method: 'QR_DYNAMIC',
    application: 'PROMPTPAY_CREDIT_TRANSFER',
    mobileNumber,
    amount: amountBaht,
    currencyCode: '764',
    countryCode: 'TH',
  });
}
```

ไม่ว่าจะเลือกไลบรารีไหน หลักการต้องเหมือนกันเสมอคือ ปล่อยให้ไลบรารีเป็นคนประกอบ TLV และคำนวณ CRC16 ให้ทั้งหมด โค้ดของเรามีหน้าที่แค่ส่งพารามิเตอร์ที่ถูกต้องเข้าไป ไม่ใช่ไปยุ่งกับการต่อสตริง payload เอง

ต่อ payment gateway ไทย

ตัวเลือก gateway ที่ถูกจัดอันดับบ่อยในตลาดไทยมี **2C2P, Opn Payments (ชื่อเดิม Omise), Pay Solutions, PayPal, TrueMoney** โดยแต่ละเจ้าจะมี flow การทำงานที่คล้ายกันคือ 4 ขั้นตอน:

1. create charge → เรียก API ของ gateway สร้างรายการชำระเงินผูกกับ billId ของเรา
2. redirect/iframe → ลูกค้าถูกพาไปหน้า/กล่องของ gateway เพื่อกรอกข้อมูลบัตรหรือสแกน QR
3. webhook → gateway ยิงกลับมาที่ server เราเมื่อสถานะเปลี่ยน (สำเร็จ/ล้มเหลว)
4. อัปเดตบิล → handler ของเรา validate webhook แล้วเปลี่ยนสถานะบิลเป็น PAID

จุดสำคัญคือ **บิลจะเปลี่ยนเป็น PAID ก็ต่อเมื่อ webhook ยืนยันเท่านั้น** ห้ามอัปเดตสถานะบิลตอนที่ลูกค้าถูก redirect กลับมาที่หน้าเว็บเราเฉย ๆ เพราะการ redirect กลับมาไม่ได้แปลว่าเงินเข้าจริง (ลูกค้าอาจปิดแท็บกลางทาง เน็ตหลุดตอนธนาคารกำลังยืนยัน ฯลฯ) ต้องรอสัญญาณจาก webhook ซึ่งมาจากฝั่ง gateway ที่ยืนยันกับธนาคารแล้วเท่านั้นถึงจะเชื่อถือได้

สิ่งที่ UI ฝั่งลูกค้าทำได้ระหว่างรอ webhook คือแสดงสถานะ “กำลังตรวจสอบการชำระเงิน” แล้ว poll สถานะบิลจาก server เป็นระยะ หรือถ้าสถาปัตยกรรมรองรับ SSE ตามที่วางไว้ในบทที่ 8 ก็ใช้ช่องทางเดียวกันนั้น push

สถานะ “จ่ายสำเร็จ” กลับไปหน้าจอลูกค้าทันทีที่ webhook มาถึงและประมวลผลเสร็จ วิธีนี้ทำให้ประสบการณ์ลูกค้าดูรวดเร็วโดยที่ระบบหลังบ้านยังคงยึดหลัก “เชื่อ webhook เท่านั้น” อยู่เหมือนเดิม

webhook เป็น at-least-once — handler ต้อง idempotent

ข้อเท็จจริงที่สำคัญที่สุดของหัวข้อนี้: ทั้ง Stripe และ Omise การันตีการส่ง webhook แบบ at-least-once ไม่ใช่ exactly-once หมายความว่า endpoint ของเรา มีโอกาสได้รับ event เดิมซ้ำมากกว่าหนึ่งครั้งเสมอ (เพราะ gateway ยิงซ้ำเมื่อไม่ได้รับ response 200 ทันเวลา, เครือข่ายสะดุด, หรือ retry ตามนโยบายของผู้ให้บริการ) ถ้า handler ของเราสมมติว่า event มาแค่ครั้งเดียวแล้วอัปเดตยอดหรือสถานะบิลตรง ๆ ทุกครั้งที่ได้รับ จะเกิดปัญหาซ้ำซ้อน เช่น ส่งใบเสร็จซ้ำ แจ้งครัวซ้ำ หรือแย่ที่สุดคือ บวกยอดขายซ้ำสองครั้งในรายงาน

วิธีแก้คือทำให้ handler idempotent โดยใช้ event ID ที่ gateway ส่งมาเป็น dedup key — บันทึกลงตารางก่อนประมวลผลทุกครั้ง ถ้าเจอ event ID ซ้ำให้ตอบ 200 แล้วจบทันทีโดยไม่ประมวลผลซ้ำ

Prisma schema:

```
// prisma/schema.prisma
model WebhookEvent {
  id          String    @id @default(cuid())
  provider    String    // 'omise' | '2c2p' | 'xendit' ฯลฯ
  eventId     String    // event id จาก gateway - ใช้เป็น dedup key
  eventType   String    // เช่น 'charge.complete'
  payload     Json
  billId      String?
  processedAt DateTime?
  createdAt   DateTime  @default(now())

  @@unique([provider, eventId]) // กัน event ซ้ำระดับฐานข้อมูล ไม่ใช่แค่ระดับโค้ด
}
```

Route handler:

```
// app/api/webhooks/omise/route.ts
import { prisma } from '@lib/prisma';
import { verifyOmiseSignature } from '@lib/omise';

export async function POST(req: Request) {
  const rawBody = await req.text();
  if (!verifyOmiseSignature(req.headers, rawBody)) {
    return new Response('invalid signature', { status: 401 });
  }

  const payload = JSON.parse(rawBody);
  const eventId: string = payload.id; // Omise ส่ง event.id มาเสมอ

  // บันทึกก่อนประมวลผลเสมอ - ถ้า insert ชนกับ unique constraint แปลว่าเจอ event ซ้ำ
  try {
    await prisma.webhookEvent.create({
      data: {
        provider: 'omise',
        eventId,
        eventType: payload.key,
        payload,
      },
    });
  } catch (err) {
    if (isUniqueConstraintViolation(err)) {
      // at-least-once delivery: event นี้เคยประมวลผลไปแล้ว ตอบ 200 แล้วจบ ห้ามซ้ำ
      return new Response('duplicate event, already processed', { status: 200 });
    }
    throw err;
  }

  await handleOmiseChargeEvent(payload);

  await prisma.webhookEvent.update({
    where: { provider_eventId: { provider: 'omise', eventId } },
    data: { processedAt: new Date() },
  });

  return new Response('ok', { status: 200 });
}
```

ข้อสังเกตสำคัญ: การกันซ้ำต้องพึ่ง **unique constraint** ระดับฐานข้อมูล (`@@unique([provider, eventId])`) ไม่ใช่แค่เช็คด้วยโค้ด `SELECT` ก่อน `INSERT` เพราะถ้า webhook สองก้อนมาถึงพร้อมกันเป๊ะ (race condition) การเช็คด้วยโค้ดฝั่งเดียวจะแพ้ race นั้นได้ ในขณะที่ unique constraint ของฐานข้อมูลรับประกันว่ามีแถวเดียวต่อ `(provider, eventId)` เสมอไม่ว่าจะมีกี่ request แข่งกันมา

เงินสด: ลีนซัก เงินทอน ยอดไม่ตรง

ต่อให้ระบบดิจิทัลสมบูรณ์แค่ไหน ร้านอาหารไทยส่วนใหญ่ก็ยังรับเงินสดเป็นสัดส่วนสำคัญ ระบบต้องรองรับ 3 เรื่องนี้เป็นอย่างน้อย:

--

- **เงินทอนในลิ้นชัก (float)** — ต้นกะต้องมีการนับและบันทึกยอดเริ่มต้นก่อนเปิดร้าน เพื่อให้ปลายกะรู้ว่าควรมีเงินสดเหลือเท่าไร
- **การคำนวณเงินทอน** — POS ต้องรับยอดที่ถูกค้ายื่นมา แล้วคำนวณเงินทอนให้พนักงานดูตัวเลขชัดเจน ลดโอกาสทอนผิดด้วยมือ
- **ยอดไม่ตรง (over/short)** — ปลายกะต้องนับเงินสดจริงในลิ้นชัก เทียบกับยอดที่ระบบคำนวณว่าควรมี (ยอดตั้งต้น + ยอดขายเงินสดที่บันทึกไว้) ส่วนต่างที่เกิดขึ้นต้องถูกบันทึกเป็นตัวเลขในระบบ ไม่ใช่แค่จำไว้ปากเปล่า เพราะนี่คือข้อมูลที่บอกได้ว่าพนักงานคนไหนมีรูปแบบยอดขาดบ่อยผิดปกติ (รายละเอียดการตรวจจับความผิดปกตินี้จะอยู่ในบทที่ 13)

ทุกครั้งที่มีการเปิดลิ้นชักโดยไม่ผูกกับการขาย (เช่น เปิดเพื่อทอนแบงก์ หรือแก้ไขยอด) ต้องถูกบันทึก event ไว้ พร้อมชื่อพนักงานเสมอ ลิ้นชักที่เปิดได้โดยไม่มี log คือช่องโหว่อันดับหนึ่งของการโกงหน้าร้าน

ระบบ POS ที่สร้างเองมักมองข้ามเงินสดเพราะรู้สึกว่าเป็นเรื่องกายภาพ ไม่เกี่ยวกับซอฟต์แวร์” แต่ในความเป็นจริงบิลที่จ่ายด้วยเงินสดต้องผ่าน flow เดียวกับบทที่ 10 ทุกอย่าง (คำนวณยอดตามลำดับ service charge → VAT ให้ถูก) แต่ไม่มี gateway หรือ webhook มาช่วยยืนยันเท่านั้น ความรับผิดชอบเรื่องความถูกต้องจึงตกอยู่ที่พนักงานหน้าร้านล้วน ๆ ระบบจึงต้องช่วยลดโอกาสผิดพลาดของมนุษย์ให้มากที่สุดเท่าที่ทำได้ เช่น แสดงเงินทอนเป็นตัวเลขใหญ่ชัดเจน และไม่ปล่อยให้บิลเงินสดได้จนกว่าจะกรอกยอดที่ถูกค้ายื่นมาก่อน

บัตร: EDC ธนาकार vs gateway

ร้านอาหารมักมีเครื่องรูดบัตรอยู่ 2 แบบที่ทำงานต่างกันโดยสิ้นเชิง:

- **EDC (Electronic Data Capture) ของธนาकार** — เครื่องรูดบัตรที่ธนาकारติดตั้งให้ ตัดเงินตรงเข้าบัญชีร้านผ่านระบบธนาकार มี slip กระดาษเป็นหลักฐาน สลิปยอดขายจะต้องกระทบยอดกับ statement ที่ธนาकारส่งมาแยกต่างหาก ระบบ POS ของเราไม่มีทาง “รู้” ผลการรูดบัตรแบบ real-time ได้เอง นอกจากพนักงานกดยืนยันว่ารูดผ่านแล้วด้วยมือ
- **บัตรผ่าน payment gateway** (เช่น Omise/2C2P ผัง hosted page หรือ iframe) — ตัดเงินผ่านระบบของ gateway ตัดเข้าบัญชีร้านตามรอบ payout ของ gateway (ไม่ใช่ทันที) และมี webhook แจ้งกลับให้ระบบอัปเดตสถานะบิลอัตโนมัติแบบเดียวกับ PromptPay ผ่าน gateway

ความต่างนี้สำคัญตอนกระทบยอดปลายวัน เพราะ **ยอดที่ EDC ตัดกับยอดที่ gateway ตัด ไปโผล่ในบัญชีคนละช่องทาง คนละรอบเวลา** ระบบรายงานของร้านต้องแยกให้ชัดว่ายอดวันนี้มาจากช่องทางไหนบ้าง ไม่ใช่โยนรวมเป็น “ยอดขายผ่านบัตร” ก้อนเดียว เพราะเวลากระทบยอดจริงต้องไปเทียบกับ statement คนละใบ

อีกจุดที่ POS ต้องออกแบบให้รองรับคือ **บัตรถูกปฏิเสธ (declined) หรือ timeout ระหว่างทำการรายการ** ทั้งฝั่ง EDC และฝั่ง gateway ต่างมีโอกาสนี้เกิดขึ้นได้เสมอ ระบบต้องมีสถานะ “รอผลบัตร” ที่แยกจาก “จ่ายแล้ว” อย่างชัดเจน และไม่ควรปล่อยให้บิลค้างอยู่ในสถานะนี้นานเกินไปโดยไม่มีใครสังเกต เพราะโตะที่ลูกค้ากลับไปแล้วแต่บิลยังไม่ถูกปิดสถานะให้ถูกต้อง คือรายการที่มักถูกลืมตรวจสอบตอนปิดกะ

การกระทบยอดปลายวัน

ปลายวัน (หรือปลายกะ) ระบบต้องสรุปยอดแยกตามช่องทางชำระเงินทั้งหมด แล้วให้ผู้ปิดกะเทียบกับหลักฐานจริงของแต่ละช่องทาง:

ช่องทาง	ยอดที่ระบบบันทึก	เทียบกับ
เงินสด	ยอดขายเงินสดสะสมในกะ	เงินสดที่นับได้จริงในลิ้นชัก
PromptPay ผ่าน gateway	ยอด webhook ที่ยืนยัน PAID	รายงาน settlement ของ gateway
PromptPay QR เเงินเอง (ไม่ผ่าน gateway)	ยอดที่ผู้ใช้ยืนยันจ่ายเองในระบบ	statement ธนาคารของร้าน (ต้องจับคู่มือถ้าไม่มี reference)
บัตรผ่าน EDC ธนาคาร	สลิปที่พนักงานยืนยันจากเครื่อง	statement ธนาคารรอบตัดบัตร
บัตรผ่าน gateway	ยอด webhook ที่ยืนยัน PAID	รายงาน settlement ของ gateway

ส่วนต่างที่หาสาเหตุไม่ได้ต้องถูกบันทึกเป็นรายการ “ยอดไม่ตรง” ในระบบเสมอ ห้ามปิดตกหรือแก้ตัวเลขให้ลงตัวมือ เพราะข้อมูลนี้คือสิ่งที่บอกเจ้าของร้านได้ว่าจุดไหนของกระบวนการรับเงินกำลังรั่ว

ระบบที่ดีควรทำขั้นตอนนี้เป็น “ปิดกะไม่ได้จนกว่าจะกรอกยอดเงินสดจริง” แทนที่จะเป็นแค่รายงานให้ดูเฉย ๆ เพราะการบังคับให้กรอกทุกกะสร้างวินัยการนับเงินให้พนักงาน และทำให้ข้อมูลส่วนต่างสะสมเป็นชุดข้อมูลย้อนหลังที่มีความหมาย ใช้ดูแนวโน้มได้ว่ากะไหน พนักงานคนไหน มียอดขาดเกินปกติซ้ำ ๆ ซึ่งเป็นข้อมูลตั้งต้นที่สำคัญที่สุดของบทที่ 13 เรื่องการกันโกงหน้าร้าน

ตารางเทียบ payment gateway ไทย

Gateway	ค่าธรรมเนียมที่ประกาศ	จุดเด่น
Omise / Opn Payments	บัตรเครดิต 3.65%, PromptPay 1.65% (บวก VAT 7%)	ติดตั้งง่าย มี REST API หลายภาษา + SDK มือถือ + ปลั๊กอินสำหรับ e-commerce platform หลัก เหมาะร้านเล็ก-กลาง
2C2P	ต่อรองได้	ระบบครบวงจร เหมาะองค์กรใหญ่ที่มีอำนาจต่อรองค่าธรรมเนียม
GB Prime Pay (ปัจจุบันคือ Xendit)	ต้องตรวจสอบราคาปัจจุบันภายใต้แบรนด์ Xendit	เดิมถูกแนะนำสำหรับร้านออนไลน์เล็ก-กลางเช่นเดียวกับ Omise

ตัวเลขค่าธรรมเนียมทั้งหมดในตารางนี้เป็น **ราคาประกาศ ณ ช่วงที่อ้างอิงเท่านั้น** ผู้ให้บริการปรับราคาได้ตลอดเวลา **ต้องไปตรวจสอบราคาล่าสุดกับผู้ให้บริการเองก่อนตัดสินใจใช้งานจริงทุกครั้ง**

สั่ง Claude Code แบบนี้

ช่วยสร้าง webhook handler รับ event จาก payment gateway (เช่น Omise) เป็น Next.js route handler พร้อม Prisma model WebhookEvent สำหรับกันประมวลผลซ้ำ และช่วยเงิน PromptPay QR แบบ dynamic ผูกกับบิล

กติกาที่ต้องทำตามนี้ ห้ามยืดหยุ่นเอง:

- Gateway ส่ง webhook แบบ at-least-once การันตี ไม่ใช่ exactly-once – event เดิมมาซ้ำได้เสมอ handler ต้อง idempotent โดยใช้ event.id จาก gateway เป็น dedup key ผ่าน unique constraint ในฐานข้อมูล ไม่ใช่แค่เช็คด้วยโค้ดก่อน insert
- ถ้าเจอ event.id ซ้ำ ให้ตอบ 200 ทันทีโดยไม่ประมวลผลซ้ำ ห้ามอัปเดตยอดเงินหรือสถานะบิลซ้ำสอง
- ห้ามเก็บเลขบัตร (PAN) หรือ CVV ลงฐานข้อมูลของเราเด็ดขาดไม่ว่าจะเข้ารหัสหรือไม่ เก็บได้แค่ token/charge id ที่ gateway ส่งกลับมา
- เงิน PromptPay QR ด้วยไลบรารี promptpay-qr เท่านั้น ห้ามเขียน payload builder หรือ CRC16 เอง และต้องใช้ tag 29 (credit transfer ผ่านเบอร์/เลขผู้เสียภาษี ไม่มี reference) ไม่ใช่ tag 30 (bill payment ที่มี reference/Biller ID)
- บิลเปลี่ยนสถานะเป็น PAID ได้ก็ต่อเมื่อ webhook ยืนยันแล้วเท่านั้น ห้ามอัปเดตสถานะตอนลูกค้าถูก redirect กลับมาที่หน้าเว็บเลย ๆ

เพราะนี่คือโค้ดที่แตะเงินจริงและข้อมูลอ่อนไหว ให้ใช้ subagent แยกสำหรับรีวิว payment/security โดยเฉพาะก่อน merge เข้า main และเข้า plan mode ก่อนเขียนโค้ดจริง

ตอน review output ให้เช็ค 4 อย่างนี้เป็นอย่างน้อย: (1) `@@unique([provider, eventId])` ครอบจริงไหม และเคส race condition ที่ event ซ้ำมาถึงพร้อมกันถูกจัดการด้วย constraint ของฐานข้อมูล ไม่ใช่แค่ if-check ในโค้ด (2) payload ที่ log หรือเก็บลง `WebhookEvent.payload` มีเลขบัตรหรือ CVV หลุดมาไหม — บาง gateway ส่ง masked PAN ติดมาด้วย ต้องกรองก่อนเก็บถ้าจำเป็น (3) flow อัปเดตสถานะบิลผูกกับ webhook เท่านั้นจริงไหม ไม่มี path ไหนที่อัปเดตจาก redirect ตรง ๆ (4) tag ที่ใช้เงิน QR คือ 29 ไม่ใช่ 30 และ currency/country code ถูกต้องตรงตามสเปก (764/TH)

มุมมองจากร้าน

การรับเงินคือจุดเดียวในระบบทั้งหมดที่เจ้าของร้านสัมผัสได้ตรง ๆ ทุกวัน — ระบบสั่งอาหารพังยังพอมีเวลาแก้ แต่ระบบรับเงินพังคือลูกค้ายื่นรอนหน้าเคาน์เตอร์ทันที เจ้าของร้านควรรู้ 3 เรื่องหลักจากบทนี้: หนึ่ง QR ที่แปะหน้าร้านแบบ static (ไม่ระบุยอด) สะดวกก็จริงแต่ทำให้ปลายวันต้องนั่งไล่จับคู่ยอดโอนกับบิลเอง ถ้าร้านมีหลายโต๊ะพร้อมกันควรพิจารณาลงทุนกับ gateway ที่มี webhook แทน สอง ค่าธรรมเนียมของแต่ละ gateway ไม่เท่ากันและเปลี่ยนได้ตลอด ต้องเช็คราคาจริงก่อนตัดสินใจทุกครั้งไม่ใช่เชื่อตัวเลขเก่าที่เคยได้ยินมา สาม การกระทบยอดปลายวันไม่ใช่ขั้นตอนที่ตัดทิ้งได้เพื่อประหยัดเวลา เพราะเป็นเครื่องมือเดียวที่ทำให้รู้ทันทีว่าเงินรั่วตรงไหนก่อนที่จะสายเกินไป

เจ้าของร้านที่กำลังเปรียบเทียบว่าจะใช้ gateway เจ้าไหน ควรคิดเลยไปไกลกว่าค่าธรรมเนียมเปอร์เซ็นต์ที่เห็นในตาราง เพราะสิ่งที่กระทบธุรกิจจริงในระยะยาวคือรอบการโอนเงินเข้าบัญชี (payout cycle) — บาง gateway

โอนเข้าบัญชีร้านทุกวัน บางเจ้าโอนทุกสัปดาห์ ถ้าร้านมีกระแสเงินสดต้องใช้เงินหมุนเวียนเร็ว ความถี่ของ payout สำคัญไม่แพ้ค่าธรรมเนียม และเป็นคำถามที่ต้องถามผู้ให้บริการตรง ๆ ก่อนเซ็นสัญญา ไม่ใช่แค่ดูตัวเลขเปอร์เซ็นต์บนหน้าเว็บ

❌ พลาดบ่อย / ✅ ทำแบบนี้

❌ **พลาดบ่อย:** อัปเดตสถานะบิลเป็น “จ่ายแล้ว” ทันทีที่ลูกค้าถูก redirect กลับมาที่หน้าเว็บของร้านหลังกดจ่ายเงินผ่าน gateway

✅ **ทำแบบนี้:** รอ webhook จาก gateway ยืนยันสถานะการชำระเงินก่อนเปลี่ยนสถานะบิลเสมอ เพราะ redirect กลับมาไม่ได้แปลว่าเงินเข้าจริง อาจปิดแท็บหรือเน็ตหลุดกลางทางได้

❌ **พลาดบ่อย:** เขียน webhook handler ที่เชื่อว่า event จาก gateway จะมาถึงแค่ครั้งเดียว แล้วอัปเดตยอดขาย/สถานะบิลตรง ๆ ทุกครั้งที่ endpoint ถูกเรียก

✅ **ทำแบบนี้:** บันทึก event ID ลงตารางที่มี unique constraint ก่อนประมวลผลทุกครั้ง ถ้าเจอ event ID ซ้ำให้ตอบ 200 แล้วข้ามการประมวลผล เพราะ webhook เป็น at-least-once เสมอ

❌ **พลาดบ่อย:** สลับใช้โครงสร้าง tag 30 (bill payment ที่มี reference) มาเจน QR หน้าร้านทั้งที่ร้านไม่ได้ลงทะเบียนเป็น Biller กับธนาคารจริง

✅ **ทำแบบนี้:** ร้านอาหารทั่วไปที่รับ QR หน้าเคาน์เตอร์ใช้ tag 29 (PromptPay credit transfer ผ่านเบอร์มือถือ/เลขผู้เสียภาษี ไม่มี reference) เป็นมาตรฐาน และเจนด้วยไลบรารีที่ทดสอบมาแล้วเท่านั้น

จำ 3 ข้อ Tag 29 คือ PromptPay credit transfer ไม่มี reference, tag 30 คือ bill payment ที่มี Biller ID — ห้ามสลับกัน Webhook เป็น at-least-once เสมอ handler ต้อง idempotent ด้วย event ID เป็น dedup key ผ่าน unique constraint ของฐานข้อมูล ห้ามเก็บเลขบัตรหรือ CVV ลงฐานข้อมูลของเราเด็ดขาด ใช้ hosted page/iframe/token แล้วเก็บได้แค่ token

ใบเสร็จ ใบกำกับภาษีอย่างย่อ และ e-Tax



ร้านที่ผมเคยวางระบบให้ เจอเหตุการณ์นี้ในเดือนที่สอง: ลูกค้าโต๊ะมุมขวาสั่งอาหารมือเที่ยงเป็นทีมบริษัท จ่ายเงินเสร็จ แล้วยื่นบัตรพนักงานพร้อมพูดประโยคที่ทำให้เคสเชียร์หน้าซีดว่า “ขอใบกำกับภาษีเต็มรูปด้วยครับ ต้องเอาไปเบิก” ระบบ POS ที่ผมส่งมอบพิมพ์ได้แค่ใบเสร็จธรรมดา ไม่มีเลขผู้เสียภาษี ไม่มีเลขที่เอกสารต่อเนื่อง เจ้าของร้านต้องนั่งเขียนใบกำกับภาษีด้วยมือหลังร้านแล้ววิ่งเอาไปให้ลูกค้าที่รอในรถ

บทนี้ไม่ได้สอนบัญชีภาษีให้ผู้อ่านเป็นนักบัญชี แต่สอนว่าระบบต้อง “ออกเอกสารได้ถูกโครงสร้าง” ตั้งแต่ตอนออกแบบฐานข้อมูล เพราะแก้ทีหลังคือของานหนักกว่าออกแบบตั้งแต่แรกมาก ก่อนไปต่อ ขอพูดให้ชัดเจนตรงนี้ครั้งเดียว: เนื้อหาภาษีในบทนี้มาจากข้อมูลที่ตรวจสอบได้ ณ ช่วงที่เขียน ไม่ใช่คำแนะนำทางกฎหมาย ร้านของคุณต้องยืนยันกับกรมสรรพากรหรือผู้สอบบัญชีของร้านเองทุกครั้งก่อนใช้งานจริง ผมจะไม่แต่งข้อกำหนดเพิ่มเติมนอกเหนือจากที่ตรวจสอบมา และคุณก็ไม่ควรให้ Claude Code แต่งเองเช่นกัน

ใบกำกับภาษีอย่างย่อ (ABB) ต้องมีอะไรบ้าง

ร้านอาหารส่วนใหญ่ที่จดทะเบียน VAT จะออก “ใบกำกับภาษีอย่างย่อ” (Abbreviated Tax Invoice — ABB) แทนใบกำกับภาษีเต็มรูป เพราะขายให้ผู้บริโภคทั่วไปเป็นหลัก ไม่ใช่ทุกบิลต้องมีชื่อ-ที่อยู่ผู้ซื้อแบบใบกำกับภาษีเต็มรูป แต่เอกสารต้องมีข้อมูลขั้นต่ำต่อไปนี้ครบ:

1. ข้อความ “ใบกำกับภาษีอย่างย่อ” ต้องปรากฏบนเอกสารจริง ไม่ใช่แค่ในใจผู้ออก

2. ชื่อหรือชื่อย่อของผู้ออก (ชื่อร้าน/ชื่อนิติบุคคล)

3. เลขประจำตัวผู้เสียภาษีของผู้ออก — ตัวเลข 13 หลักที่ผูกกับร้าน

4. หมายเลขลำดับของใบกำกับภาษี และ หมายเลขเล่ม (ถ้ามีการแบ่งเล่ม)

สี่ข้อนี้คือ minimum ที่ต้องมีบนกระดาษ ส่วนรายละเอียดอื่น เช่น รายการอาหาร ราคาต่อรายการ ยอดรวม VAT แยกออกมา ก็ควรมีเพื่อความชัดเจนกับลูกค้า แต่ที่ “ห้ามขาด” ตามหมวดนี้คือสี่ข้อข้างต้น

ทีนี้มาดูว่าแปลงเป็นโครงสร้างข้อมูลยังไง สิ่งที่เราอยากพลาดคือไปเก็บ “เลขที่ใบกำกับภาษี” เป็น string ว่าง ๆ แล้วให้แอปฝั่ง client เดาเลขต่อไปเอง ซึ่งฟังก์ชันที่มีสองเครื่องคิดเงินพร้อมกัน

```

model Branch {
  id          String   @id @default(cuid())
  tenantId    String
  name        String
  taxId       String   // เลขประจำตัวผู้เสียภาษี 13 หลักของนิติบุคคล/ร้าน
  legalName   String   // ชื่อที่ใช้ออกใบกำกับภาษี (อาจยาวกว่าชื่อเดิม)

  receiptCounters ReceiptCounter[]
  taxInvoices     TaxInvoice[]
}

// ตัวนับเลขที่เอกสารแบบ "ต่อสาขา ต่อวัน" - 1 แลวต่อ 1 สาขา 1 วัน
model ReceiptCounter {
  id          String   @id @default(cuid())
  branchId    String
  branch      Branch   @relation(fields: [branchId], references: [id])
  bookNumber  String   // หมายเลขเล่ม เช่น "01"
  forDate     DateTime @db.Date // วันที่ (แยกนับใหม่ทุกวัน)
  lastSeq     Int      @default(0) // เลขลำดับล่าสุดที่ออกไปแล้ว

  @@unique([branchId, bookNumber, forDate])
}

model TaxInvoice {
  id          String   @id @default(cuid())
  branchId    String
  branch      Branch   @relation(fields: [branchId], references: [id])
  orderId     String   @unique
  bookNumber  String
  sequenceNo  Int      // เลขลำดับที่ออกจริง ห้ามซ้ำ ห้ามข้าม
  issuedAt    DateTime @default(now())
  status      TaxInvoiceStatus @default(ISSUED)
  voidedById  String?  // userId ของคนกดยกเลิก (ถ้ามี)
  voidedAt    DateTime?
  voidReason  String?
  supersedesId String? // ชี้นำเอกสารเดิมถ้านี่คือเอกสารที่ออกแทนของที่ถูกยกเลิก

  @@unique([branchId, bookNumber, sequenceNo])
}

enum TaxInvoiceStatus {
  ISSUED
  VOIDED
}

```

จุดสำคัญคือ `@@unique([branchId, bookNumber, sequenceNo])` — ฐานข้อมูลเป็นคนบังคับว่าเลขซ้ำไม่ได้ ไม่ใช่แค่ logic ฝั่งแอปคอยเช็ค เพราะถ้าพนักงานสองคนกดพิมพ์ใบเสร็จพร้อมกันจากสองเครื่อง race condition จะทำให้ได้เลขซ้ำถ้าไม่มี constraint ระดับ database คุณไว้

ให้เลขเอกสารแบบไม่ซ้ำ ไม่ข้าม

กฎของเลขที่เอกสารคือ ต้องเรียงต่อเนื่อง ไม่ซ้ำ ไม่ข้าม ต่อสาขา ต่อเล่ม ต่อวัน (หรือแล้วแต่ระบบเล่มที่ร้านตั้งไว้) นั่นแปลว่าออกแบบผิดจุดเดียวคือปัญหาใหญ่: ถ้าเลข 00045 ถูกจองไว้แต่ transaction ล้มเหลว (บัตรเครดิตถูกปฏิเสธ, เน็ตหลุดกลางทาง) แล้วเลขนั้น “หาย” ไปเฉย ๆ นั่นคือเลขข้าม ซึ่งเป็นปัญหาเวลาตรวจสอบวิธีที่ปลอดภัยคือ ออกเลขพร้อมกับการยืนยันว่าบิลจ่ายเงินสำเร็จจริงเท่านั้น และทำทั้งหมดใน transaction เดียว ไม่ใช่จองเลขล่วงหน้าตอนเปิดบิล

```
// lib/tax-invoice.ts
import { prisma } from "@lib/prisma";

export async function issueTaxInvoice(orderId: string, branchId: string) {
  const today = new Date();
  today.setHours(0, 0, 0, 0);
  const bookNumber = "01"; // ร้านเล็กมักมีเล่มเดียว ปรับตามการตั้งค่าจริงของร้าน

  return prisma.$transaction(async (tx) => {
    const order = await tx.order.findUniqueOrThrow({ where: { id: orderId } });

    if (order.paymentStatus !== "PAID") {
      // ห้ามออกเลขเอกสารให้บิลที่ยังไม่ยืนยันว่าจ่ายเงินสำเร็จ
      throw new Error("ORDER_NOT_PAID_YET");
    }

    // upsert ตัวนับ แล้วเพิ่มเลขแบบ atomic ภายใน transaction เดียว
    const counter = await tx.receiptCounter.upsert({
      where: {
        branchId_bookNumber_forDate: { branchId, bookNumber, forDate: today },
      },
      create: { branchId, bookNumber, forDate: today, lastSeq: 1 },
      update: { lastSeq: { increment: 1 } },
    });

    return tx.taxInvoice.create({
      data: {
        branchId,
        orderId: order.id,
        bookNumber,
        sequenceNo: counter.lastSeq,
        status: "ISSUED",
      },
    });
  });
}
```

เหตุผลที่ต้อง **upsert** + **increment** อยู่ใน transaction เดียวกับการเช็ค **paymentStatus** คือ Postgres จะล็อกแถว **ReceiptCounter** ระหว่าง transaction ทำงาน ถ้ามีสองคำขอออกเลขพร้อมกันสำหรับสาขาเดียวกัน คำขอที่สองจะรอคำขอแรกจบก่อน ไม่มีทางได้เลขซ้ำกัน — นี่ก็จุดเดียวกับที่บทที่ 7 พูดถึงเรื่อง idempotency ของออร์เดอร์ แค่อ้ายมาใช้กับเลขเอกสารแทน

ยกเลิกหรือคืนเงิน ต้องออกเอกสารยังไง

จุดที่ระบบ POS ทำผิดพลาดที่สุดคือ “ลบ” ใบกำกับภาษีที่ออกผิดทิ้งไปเลย ซึ่งเป็นความคิดที่อันตราย เพราะเอกสารทางบัญชีที่ออกไปแล้วไม่ควรหายไปจากระบบ ไม่ว่าจะออกผิดหรือลูกค้าขอคืนเงิน

หลักการออกแบบที่ปลอดภัยคือ: **ไม่ลบของเดิม เปลี่ยนสถานะเป็น VOIDED พร้อมบันทึกว่าใครกด เมื่อไหร่ เพราะอะไร** แล้วถ้าต้องออกเอกสารใหม่ให้ถูกต้อง ก็สร้างแถวใหม่ที่ **supersedesId** ชี้กลับไปยังเอกสารเดิม เพื่อให้ตรวจสอบย้อนหลังได้ครบทั้งสายว่าเอกสารเลขไหนถูกยกเลิก ถูกแทนที่ด้วยเลขไหน

ส่วนรูปแบบเอกสารที่ถูกต้องตามกฎหมายเวลาต้องคืนเงินหรือลดหนี้ให้ลูกค้าที่ถือใบกำกับภาษีไปแล้ว (เช่น ต้องออกเอกสารประเภทไหน มีข้อกำหนดอะไรเพิ่มเติม) เป็นเรื่องที่ต้องไปยืนยันกับกรมสรรพากรหรือผู้สอบบัญชีของร้านโดยตรง ผมจะไม่เดาให้ เพราะเขียนผิดแล้วร้านจะเข้าใจผิดตามไปด้วย สิ่งที่ผมยืนยันได้คือ “ฝั่งซอฟต์แวร์” ต้องรองรับการที่เอกสารหนึ่งใบสามารถถูก mark ว่ายกเลิกและมีเอกสารทดแทนได้ ไม่ใช่ลบทิ้งเฉยๆ

e-Tax Invoice by Time Stamp

ร้านที่อยากเลิกพิมพ์กระดาษแล้วส่งใบกำกับภาษีเป็นไฟล์อิเล็กทรอนิกส์ มีระบบของกรมสรรพากรที่ชื่อ **e-Tax Invoice by Time Stamp** รองรับอยู่ ข้อเท็จจริงที่ยืนยันได้มีดังนี้:

- ผู้ออกเอกสารต้อง **จดทะเบียนภาษีมูลค่าเพิ่ม (VAT)** เป็นเงื่อนไขแรกก่อน — ร้านที่ไม่ได้จด VAT ยังไม่เข้าเกณฑ์นี้
- ต้อง **ยื่นคำขอใช้งานผ่าน rd.go.th** และ **รอกกรมสรรพากรอนุมัติ** ก่อนเริ่มออกเอกสารแบบนี้ได้จริง ไม่ใช่เปิดใช้ได้ทันทีที่อยากใช้
- ตัวระบบให้บริการอยู่ที่ **etax.rd.go.th**
- มีธุรกิจบางประเภทที่ **ยกเว้น** ไม่เข้าเงื่อนไขนี้ เช่น ธุรกิจสุรา ยาสูบ

สำหรับหนังสือเล่มนี้ สิ่งที่ระบบ POS ควรทำคือออกแบบให้ **TaxInvoice** ของเรามีข้อมูลครบพอที่จะ “ส่งต่อ” เข้าสู่กระบวนการ e-Tax ได้ในอนาคต (เลขที่เอกสารต่อเนื่อง, taxId, เวลาที่ออก, รายการและยอดเงินที่ตรงกับสิ่งที่ขาย) แต่ การเชื่อมต่อเข้าระบบ e-Tax จริงต้องผ่านขั้นตอนการขออนุมัติกับกรมสรรพากรก่อนเสมอ ไม่ใช่สิ่งที่ Claude Code เขียนโค้ดแล้วเปิดใช้ได้เอง อย่าไปสัญญากับเจ้าของร้านว่า “เขียนเสร็จพุงนี้ใช้ e-Tax ได้เลย” เพราะระยะเวลาการขออนุมัติไม่ได้อยู่ในมือทีมพัฒนา

พิมพ์ใบเสร็จจริงด้วยเครื่องพิมพ์ความร้อน

ต่อให้วางแผนจะไป e-Tax ในอนาคต วันแรกที่เปิดร้านก็ต้องพิมพ์ใบเสร็จกระดาษได้ก่อน ไลบรารีที่ใช้งานได้จริงฝั่ง Node.js คือ **node-thermal-printer** ซึ่งคุยกับเครื่องพิมพ์ความร้อนด้วยชุดคำสั่ง ESC/POS พิมพ์ได้ทั้งข้อความ รูป barcode และ QR ตั้ง font style/alignment ได้ และรองรับการเชื่อมต่อได้ทั้ง **USB, Bluetooth, และ network** — สำหรับร้านอาหารที่เครื่องพิมพ์ใบเสร็จมักตั้งอยู่หน้าเคาน์เตอร์และเครื่องพิมพ์ครัวอยู่คนละ

จุด การต่อผ่าน **network (TCP)** สะดวกที่สุดเพราะเซิร์ฟเวอร์ POS ยิงคำสั่งพิมพ์ข้ามเครื่องได้โดยไม่ต้องเสียบสาย USB เข้าเครื่องแคชเชียร์โดยตรง

```

// lib/printer.ts
import { ThermalPrinter, PrinterTypes, CharacterSet } from "node-thermal-printer";

type ReceiptLine = { name: string; qty: number; amount: number };

export async function printReceipt(params: {
  printerIp: string;
  branchName: string;
  taxId: string;
  bookNumber: string;
  sequenceNo: number;
  items: ReceiptLine[];
  serviceCharge: number;
  vat: number;
  total: number;
}) {
  const printer = new ThermalPrinter({
    type: PrinterTypes.EPSON,
    interface: `tcp://${params.printerIp}`,
    characterSet: CharacterSet.WPC1252, // ต้องยืนยันกับสเปกเครื่องจริงว่ารองรับภาษาไทยแบบไหน
    removeSpecialCharacters: false,
  });

  printer.alignCenter();
  printer.println(params.branchName);
  printer.println(`เลขผู้เสียภาษี ${params.taxId}`);
  printer.println("ใบกำกับภาษีอย่างย่อ");
  printer.println(`เล่มที่ ${params.bookNumber} เลขที่ ${params.sequenceNo}`);
  printer.drawLine();

  printer.alignLeft();
  for (const line of params.items) {
    printer.println(`${line.name} x${line.qty}`);
    printer.alignRight();
    printer.println(`${line.amount / 100}.toFixed(2)`);
    printer.alignLeft();
  }

  printer.drawLine();
  printer.alignRight();
  printer.println(`ค่าบริการ ${(params.serviceCharge / 100).toFixed(2)}`);
  printer.println(`VAT 7% ${(params.vat / 100).toFixed(2)}`);
  printer.bold(true);
  printer.println(`รวม ${(params.total / 100).toFixed(2)} บาท`);
  printer.bold(false);

  printer.cut();

  const connected = await printer.isPrinterConnected();
  if (!connected) throw new Error("PRINTER_UNREACHABLE");
  await printer.execute();
}

```

โค้ดนี้ยึดหลักที่บทที่ 10 วางไว้แล้ว: ตัวเลขเงินทั้งหมดเก็บเป็นจำนวนเต็มสตางค์ (Int) แล้วค่อยหารด้วย 100 ตอนแสดงผลบนใบเสร็จเท่านั้น ไม่มีการคำนวณ VAT ใหม่ตรงจุดพิมพ์ — ใบเสร็จแค่ “แสดงผล” ตัวเลขที่คำนวณจบแล้วจากฝั่ง server

ภาษาไทยกับ codepage ปัญหาที่พบบ่อยที่สุด

จุดที่เครื่องพิมพ์สลิปพบบ่อยที่สุดในทุกโปรเจกต์ที่ผมเคยทำคือ **ตัวอักษรไทยกลายเป็นเครื่องหมายคำถามหรือขยะ** สาเหตุคือเครื่องพิมพ์ความร้อนรุ่นเก่าไม่ได้ทำงานด้วย UTF-8 โดยตรง แต่ใช้ตาราง **codepage** ภายในตัวเครื่อง (เช่น CP437, 858, 852, 866 สำหรับภาษาอื่น ๆ) ถ้าส่งข้อความ UTF-8 ไปตรง ๆ โดยไม่ตั้งค่า เครื่องจะตีความไปตั่ว

ทางแก้มีสองทาง:

1. **เลือกเครื่องพิมพ์รุ่นที่รองรับ UTF-8 โดยตรง** — ปัจจุบันมีรุ่นที่รองรับแล้ว วิธีนี้ตัดปัญหาไปเลยตั้งแต่ต้นทาง แนะนำให้เช็สเปกก่อนซื้อเครื่องให้ร้าน ไม่ใช่ไปแก้ที่โค้ดทีหลัง
2. **ถ้าใช้เครื่องรุ่นเก่าที่ไม่รองรับ UTF-8 ต้อง set codepage ให้ถูกต้องก่อนเขียนข้อความภาษาไทยทุกครั้ง** ตามที่คู่มือเครื่องรุ่นนั้นระบุ แล้วแปลงสตริงให้ตรงกับ codepage นั้นก่อนส่ง

เวลาซื้อเครื่องให้ร้านจริง ผมแนะนำให้ทดสอบพิมพ์ชื่อเมนูภาษาไทยที่มีสระบนล่างซับซ้อน (เช่น “กล้วยเตี๋ย”, “แกงเขียวหวาน”) ก่อนติดตั้งจริงเสมอ อย่าเชื่อสเปกบนกล่องเฉย ๆ

พิมพ์จากเว็บโดยไม่ลงไดรเวอร์

อีกทางเลือกหนึ่งที่ไม่ต้องพึ่ง Node.js server เขียนไปเครื่องพิมพ์โดยตรง คือให้ **หน้าเว็บ** (บนเบราว์เซอร์ของเครื่อง POS) คุยกับเครื่องพิมพ์เองผ่าน **WebUSB, Web Serial, หรือ WebHID** ซึ่งใช้งานได้เฉพาะบน **HTTPS** เท่านั้น (ข้อจำกัดความปลอดภัยของเบราว์เซอร์) วิธีนี้เหมาะกับกรณีที่ไม่อยากติดตั้งไดรเวอร์เพิ่มบนเครื่อง POS แต่ละเครื่อง

flow คร่าว ๆ คือ: เรนเดอร์ใบเสร็จด้วย JS/CSS ตามข้อมูลบิล → แปลงเป็นชุดคำสั่ง ESC/POS ด้วยโมดูล JS ที่รองรับ (มีโมดูลที่รองรับทั้งกระดาษขนาด **80mm และ 58mm** ซึ่งเป็นสองขนาดที่ร้านอาหารไทยใช้กันเป็นหลัก) → ส่งไบต์ผ่าน WebUSB/Web Serial/WebHID ไปที่เครื่องพิมพ์โดยตรงจากเบราว์เซอร์ ข้อดีคือไม่ต้องมี background service คอยรันอยู่บนเครื่อง POS ข้อเสียคือผูกกับเบราว์เซอร์ที่รองรับ API เหล่านี้ (ส่วนใหญ่คือตระกูล Chromium) และต้องขอสสิทธ์จากผู้ใช้ตอนจับคู่กับอุปกรณ์ครั้งแรก

โดยทั่วไปแนะนำให้เริ่มจากแนวทาง `node-thermal-printer` ผ่าน network ก่อน เพราะ debug ง่ายกว่าและไม่ผูกกับเบราว์เซอร์ตัวใดตัวหนึ่ง แล้วค่อยพิจารณา WebUSB/Web Serial เป็นทางเลือกเสริมถ้าร้านมีข้อจำกัดเรื่องโครงสร้างเครือข่ายที่ทำให้เดินสาย/ตั้ง IP เครื่องพิมพ์ยาก

สั่ง Claude Code แบบนี้

ช่วยเขียนฟังก์ชัน `issueTaxInvoice(orderId, branchId)` ใน `lib/tax-invoice.ts`

โดยยึดกฎเหล่านี้เคร่งครัด:

1. ออกเลขที่เอกสารได้ก็ต่อเมื่อ `order.paymentStatus === "PAID"` เท่านั้น
2. การอ่านค่า `ReceiptCounter` ล่าสุดและ `increment` ต้องอยู่ใน `prisma.$transaction` เดียวกัน เพื่อกันเลขซ้ำเวลามีค่าขอพร้อมกันจากสองเครื่องคิดเงิน
3. ใช้ `constraint` ระดับฐานข้อมูล (`@unique`) เป็นตัวคุมสุดท้าย ไม่ใช่แค่ `logic` ผังแอป
4. ถ้าต้องยกเลิกเอกสาร ห้ามลบแถวเดิม ให้เปลี่ยน `status` เป็น `VOIDED` พร้อมบันทึก `voidedBy`, `voidedAt`, `voidReason`
5. ห้ามเด้งหรือเพิ่มกฎที่ไม่ได้ระบุในนี้ – ถ้าจำเป็นต้องมีจุดที่ยังไม่ชัดเจน ให้ใส่ `TODO comment` บอกว่าต้องยืนยันกับผู้สอบบัญชีของร้าน

ทำงานใน `plan mode` ก่อน เพื่อให้ฉันเห็นก่อนว่าจะแก้ `schema.prisma` ที่จุด แล้วค่อยให้ `implement`

ต้องรีวิอะไรใน output: เช็กว่า Claude ใส่การ `increment counter` ไว้ “ใน” `transaction` เดียวกับการเช็คสถานะจ่ายเงินจริงหรือไม่ (ไม่ใช่เช็คก่อนแล้วค่อยเปิด `transaction` แยก เพราะจะเกิด `race condition` ได้) เช็กว่า `unique constraint` ครอบคลุมทั้ง `branchId + bookNumber + sequenceNo` ไม่ใช่แค่ `sequenceNo` เฉย ๆ และเช็กว่าไม่มีจุดไหนที่โค้ด “ลบ” ใบกำกับภาษีที่ออกไปแล้วทิ้งจริง ๆ

มุมมองของร้าน

เรื่องเลขที่เอกสารฟังดูเป็นเรื่องเทคนิคน่าเบื่อ แต่คือจุดที่กระทบร้านตรงที่สุดตอนถูกตรวจสอบภาษี ถ้าเลขที่ใบกำกับภาษีของร้านกระโดดข้าม (เช่น 00044 ไปโผล่ 00046 โดยไม่มี 00045) นั่นเป็นสัญญาณที่ทำให้ผู้ตรวจสอบตั้งคำถามทันทีว่าเอกสารหายไปไหน ต่อให้ร้านไม่ได้ทำอะไรผิด การอธิบายทีหลังก็เสียเวลาและความน่าเชื่อถือ

อีกเรื่องที่เจ้าของร้านต้องตัดสินใจเองคือจะไป `e-Tax Invoice by Time Stamp` เมื่อไหร่ เพราะมีขั้นตอนขออนุมัติจากกรมสรรพากรที่ทีมพัฒนาควบคุมเวลาไม่ได้ ถ้าร้านมีลูกค้าองค์กรที่ขอใบกำกับภาษีเป็นประจำ (เช่น ร้านแถวย่านออฟฟิศ) การเริ่มขอตั้งแต่เนิ่น ๆ ค่ะกว่าไปเริ่มตอนลูกค้าเรียกร้องแล้ว

❌ พลาดบ่อย / ✅ ทำแบบนี้

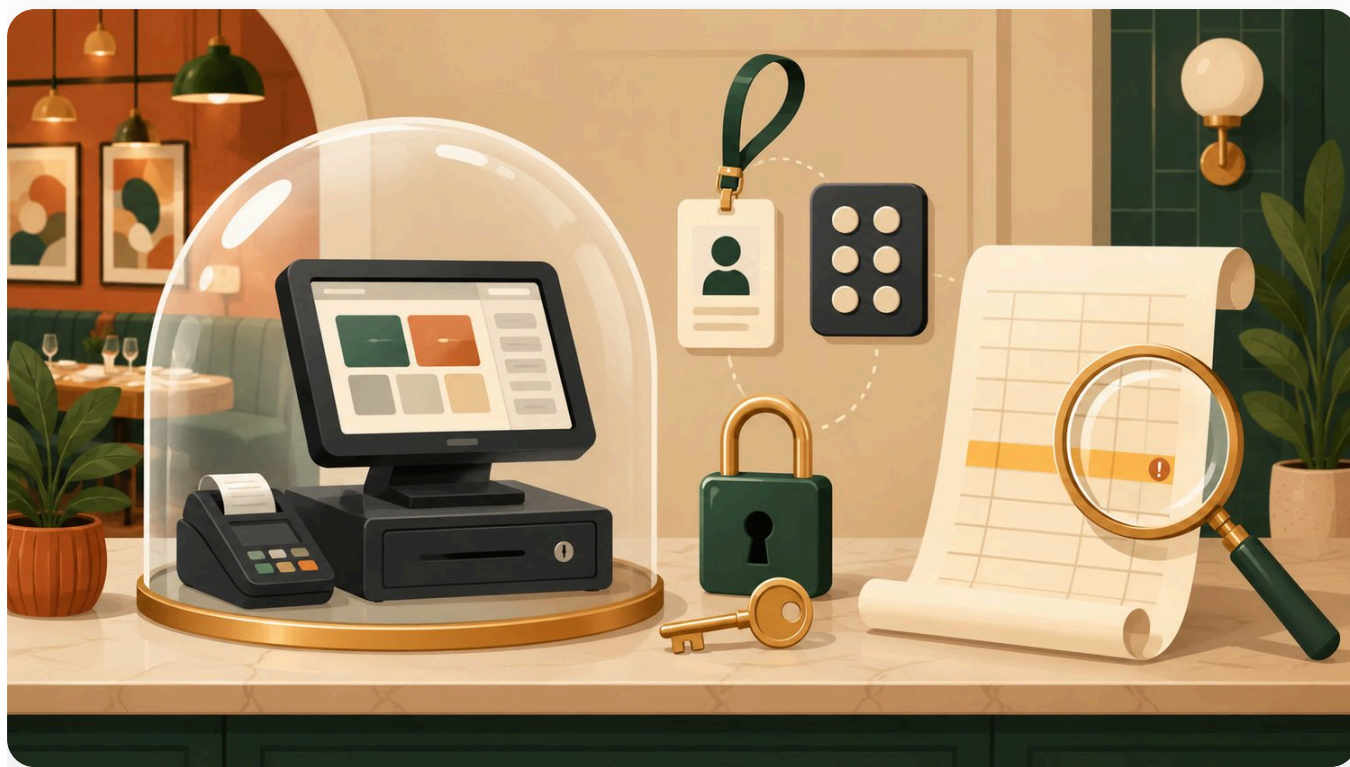
❌ พลาดบ่อย: จองเลขที่เอกสารทันทีที่พนักงานกด “เปิดบิล” แล้วถ้าลูกค้ายกเลิกออเดอร์หรือบัตรเครดิตถูกปฏิเสธ เลขนั้นก็หายไปเฉย ๆ กลายเป็นเลขข้าม ✅ ทำแบบนี้: ออกเลขที่เอกสารเฉพาะตอนที่ยืนยันแล้วว่า `paymentStatus === "PAID"` เท่านั้น และทำใน `transaction` เดียวกับการอัปเดตสถานะจ่ายเงิน

❌ พลาดบ่อย: ลบใบกำกับภาษีที่ออกผิดออกจากฐานข้อมูลไปเลย เพื่อ “เคลียร์” ให้ดูสะอาด ✅ ทำแบบนี้: เก็บทุกแถวไว้ เปลี่ยนสถานะเป็น `VOIDED` พร้อมเหตุผลและผู้กระทำ แล้วให้เอกสารใหม่ (ถ้ามี) ชี้อ้างอิงกลับไปยังเอกสารเดิมด้วย `supersedesId`

❌ พลาดบ่อย: ซื้อเครื่องพิมพ์ความร้อนราคาถูกโดยไม่ทดสอบพิมพ์ภาษาไทยก่อน แล้วมาพบว่าเมนู “ก๋วยเตี๋ยว” ออกมาเป็นตัวอักษรขยุกขยุ้นงานจริง ✅ ทำแบบนี้: ทดสอบพิมพ์ชื่อเมนูไทยที่มีสระซับซ้อนก่อนซื้อ เลือกรุ่นที่รองรับ `UTF-8` โดยตรงถ้าเป็นไปได้ ถ้าไม่ใช่ต้อง `set codepage` ให้ถูกต้องตามคู่มือเครื่องก่อนใช้งานจริง

จำ 3 ข้อ เลขที่เอกสารต้องออกพร้อมกับการยืนยันว่าจ่ายเงินสำเร็จ ในทรานแซกชันเดียวกัน ห้ามจONGL่งหน้า ยกเลิกหรือคืนเงินคือการเปลี่ยนสถานะและออกเอกสารใหม่ที่อ้างอิงกลับ ไม่ใช่การลบของเดิมทิ้ง e-Tax Invoice by Time Stamp ต้องจด VAT ก่อน ยืนยันขอผ่าน rd.go.th แล้วรอกกรมสรรพากรอนุมัติ — ไม่มีทางลัด และทุกเรื่องภาษีต้องยืนยันกับผู้สอบบัญชีของร้านเสมอ

สิทธิ์ ความปลอดภัย และการกันโกงหน้าร้าน



ร้านอาหารที่ผมเคยดูแลระบบให้มีเดือนหนึ่งที่ยอดขายตามระบบกับเงินสดในลิ้นชักไม่ตรงกันต่อเนื่อง 3 สัปดาห์ ไม่มากพอที่เจ้าของจะสังเกตเห็นทันที แต่พอผมดึงรายงานย้อนหลังออกมาดู เจอแพทเทิร์นเดียวกันซ้ำ ๆ: บิลถูก “ยกเลิก” หลังเวลาที่ POS บันทึกว่าจ่ายเงินเสร็จไปแล้วไม่กี่นาที ทุกครั้งเป็นกะเดียวกัน พนักงานคนเดียวกัน เคสนี้ไม่ใช่บั๊กซอฟต์แวร์ — คือการโกงหน้าร้านแบบคลาสสิกที่ระบบเดิมไม่มี audit log ให้สืบสาวเลย กว่าจจะรู้ตัวก็เสียเงินไปหลักหมื่นแล้ว

บทนี้ไม่ได้พูดเรื่องข้อมูลบัตรเครดิตเป็นหลัก (นั่นคุยไปแล้วเรื่องหลักการในบทที่ 11) แต่พูดถึงสามเรื่องที่ถูกมองข้ามตอนออกแบบ POS: ใครทำอะไรได้บ้างในระบบ (RBAC), ระบบของเราอยู่ตรงไหนในภาพ PCI DSS, และจะจับพนักงานโกงยังไงจากข้อมูลที่ระบบเก็บไว้เองอยู่แล้ว

Auth พนักงานหน้าร้าน: PIN + session สั้น + ล็อกจอ

หน้าร้านไม่ใช่ที่ที่พนักงานจะนั่งพิมพ์อีเมลกับรหัสผ่านทุกครั้งที่ต้องคิดเงินลูกค้า สิ่งที่ต้องโฟกัสที่ความเร็วหน้างานคือ PIN สั้น ๆ (4-6 หลัก) ต่อพนักงานแต่ละคน กดปุ่มเข้าใช้งานปั๊บ แต่ความเร็วต้องแลกมาพร้อมการควบคุมความเสี่ยงสามชั้น:

1. **session สั้น** — ล็อกอินด้วย PIN แล้วได้ session ที่หมดอายุเร็ว (เช่นจบกะ หรือไม่มีการใช้งานเกินช่วงเวลาที่กำหนด) ไม่ใช่ login ค้างไว้ทั้งวันจนใครก็หยิบเครื่องไปคิดเงินแทนได้

2. ล็อกหน้าจอทันทีเมื่อไม่มีการใช้งาน — เครื่อง POS วางอยู่หน้าร้านที่ลูกค้าเดินผ่านตลอดเวลา ถ้าไม่ล็อกอัตโนมัติ ใครก็เปิดบิล ให้ส่วนลด หรือดูข้อมูลออเดอร์แทนพนักงานที่ล็อกอินค้างไว้ได้ทันที
3. ทุก action สำคัญผูกกับ userId ที่ล็อกอินจริง ณ ขณะนั้น — ไม่ใช่ผูกกับ “เครื่อง” POS เพราะถ้าผูกกับเครื่อง จะสืบไม่ได้ว่าใครเป็นคนกดจริง ๆ ตอนเครื่องเดียวกันถูกใช้หลายกะ

PIN ควรเก็บแบบ hash เหมือนรหัสผ่านทั่วไป (ห้ามเก็บเป็น plain text) และควรมี rate limit กันการลองสุ่ม PIN ซ้ำ ๆ เหมือนกับระบบล็อกอินทั่วไป

RBAC: ใครทำอะไรได้บ้าง

ร้านอาหารมีบทบาทน้อยกว่าระบบ enterprise ทั่วไป แต่ก็ต้องแยกให้ชัดตั้งแต่ต้น เพราะสิทธิ์ที่หลวมเกินไปคือช่องให้โกงได้ง่าย ตารางสิทธิ์แบบง่ายที่ใช้เป็นจุดเริ่มต้นได้:

การกระทำ	พนักงานทั่วไป	หัวหน้ากะ	เจ้าของ/ผู้จัดการ
เปิดบิล / รับออเดอร์	✓	✓	✓
ย้ายโต๊ะ / รวมโต๊ะ	✓	✓	✓
ยกเลิกรายการก่อนครัวเริ่มทำ	✓	✓	✓
ยกเลิกรายการหลังครัวเริ่มทำแล้ว	✗ ต้องขออนุมัติ	✓	✓
ยกเลิกทั้งบิลหลังรับเงินแล้ว	✗	✗ ต้องขออนุมัติ	✓
ให้ส่วนลดเกินเพดานที่ตั้งไว้	✗	✗ ต้องขออนุมัติ	✓
เปิดคืนชักรงเงินสดนอกรอบขาย	✗	✓	✓
ดูรายงานยอดขาย/กำไร	✗	เฉพาะกะตัวเอง	✓ ทั้งหมด
แก้ไขเมนู/ราคา	✗	✗	✓

ประเด็นสำคัญคือ “ยกเลิกทั้งบิลหลังรับเงินแล้ว” กับ “ให้ส่วนลดเกินเพดาน” ควรเป็นสิทธิ์ที่แคบที่สุดในตาราง เพราะทั้งสองอย่างนี้คือช่องทางโกงที่พบบ่อยที่สุดในร้านอาหาร ยิ่งพนักงานทั่วไปทำได้เองโดยไม่ต้องขออนุมัติยิ่งตรวจสอบยาก

```

enum StaffRole {
  STAFF
  SHIFT_LEAD
  OWNER
}

model Staff {
  id      String    @id @default(cuid())
  branchId String
  name    String
  pinHash String
  role    StaffRole @default(STAFF)
  active  Boolean    @default(true)

  auditLogs AuditLog[]
}

model AuditLog {
  id      String    @id @default(cuid())
  branchId String
  staffId String
  staff   Staff      @relation(fields: [staffId], references: [id])
  action  AuditAction
  orderId String?
  amount  Int?       // สตางค์  ใช้กับ action ที่เกี่ยวเงิน เช่น DISCOUNT_APPLIED
  reason  String?
  approvedBy String? // staffId ของคนอนุมัติ ถ้า action นี้ต้องขออนุมัติ
  createdAt DateTime @default(now())

  @@index([branchId, action, createdAt])
  @@index([staffId, createdAt])
}

enum AuditAction {
  ORDER_VOIDED_BEFORE_KITCHEN
  ORDER_VOIDED_AFTER_KITCHEN
  BILL_VOIDED_AFTER_PAYMENT
  DISCOUNT_APPLIED
  DRAWER_OPENED
  TABLE_MOVED
  TABLE_MERGED
}

```

ระบบของเราอยู่ตรงไหนใน PCI DSS

ต้องพูดให้ชัดตั้งแต่บรรทัดแรก: ระบบที่สร้างในเล่มนี้ไม่ได้ “ผ่าน PCI DSS” และผมจะไม่มีวันเขียนแบบนั้น สิ่งที่ได้และควรทำคือ ออกแบบให้ระบบอยู่ในขอบเขตของ SAQ A (Self-Assessment Questionnaire A) ซึ่งเป็นระดับความรับผิดชอบที่เบาที่สุดสำหรับ merchant

SAQ A ใช้ได้กับร้านที่ outsource ทุกฟังก์ชันที่แตะข้อมูลบัตรให้ผู้ให้บริการที่ผ่าน PCI DSS compliant แล้ว และตัวร้านเอง ไม่เก็บ ไม่ประมวลผล ไม่ส่งผ่านข้อมูลบัตรบนระบบหรือสถานที่ของตัวเองเลย เงื่อนไขนี้

ครอบคลุมทั้งสองรูปแบบการรับชำระที่พบบ่อย:

- **redirect** ไปหน้าของผู้ให้บริการ (payment gateway) ให้ลูกค้ากรอกเลขบัตรที่นั่นโดยตรง
- **ฝัง iframe** ของผู้ให้บริการไว้ในหน้าเว็บของเรา แต่ฟิลด์กรอกเลขบัตรทั้งหมดยังเป็นของ iframe ไม่ใช่ของเรา

ทั้งสองแบบทำให้ **โครงสร้างพื้นฐานของเราไม่เคยเห็น PAN (เลขบัตร) เลยแม้แต่ครั้งเดียว** เพราะลูกค้าพิมพ์เลขบัตรลงหน้าของ payment gateway ตรง ๆ ไม่ผ่านเซิร์ฟเวอร์ของเรา

วิธีที่ใช้ได้จริงคือ **hosted payment page, iframe, หรือ tokenization SDK** ของ gateway ที่เลือกใช้ (ตามที่คุยไว้ในบทที่ 11) เซิร์ฟเวอร์ของเราจะเห็นแค่ **token กับ status** เท่านั้น ไม่เคยเห็น PAN หรือ CVV ตัวเลขที่ถูกอ้างถึงบ่อยในวงการคือ **tokenization ช่วยตัด PCI scope ได้ราว 80–90%** เพราะ vault ที่เก็บข้อมูลบัตรจริงอยู่ฝั่ง processor เราเก็บแค่ token ที่ใช้อ้างอิงกลับไปเรียกเก็บเงินซ้ำ ไม่ใช่ตัวข้อมูลบัตรจริง

แต่ SAQ A ไม่ได้แปลว่าไม่ต้องทำอะไรเลย ยังมีข้อกำหนดพื้นฐานที่ต้องมีอยู่ดี ได้แก่ หน้า checkout ต้องปลอดภัย (HTTPS ทุกจุดที่เกี่ยวข้องกับการชำระเงิน), มี access control กับระบบที่เกี่ยวข้องกับการชำระเงิน, มีการทำ vulnerability scan กับระบบที่เปิดสู่อินเทอร์เน็ต, มีนโยบายความปลอดภัยและการอบรมพนักงาน, และมีกระบวนการรับมือเมื่อเกิดเหตุข้อมูลรั่วไหล

สรุปสำหรับผู้อ่าน: หนังสือเล่มนี้สอนให้ออกแบบระบบเพื่อให้ “อยู่ในขอบเขต SAQ A” ได้ ไม่ใช่การรับรองว่าระบบผ่าน PCI DSS แล้ว การยืนยันความสอดคล้องจริงต้องทำผ่านกระบวนการของ payment gateway และ acquiring bank ที่ร้านใช้งานอยู่ ไม่ใช่สิ่งที่ทีมพัฒนาประกาศเองได้

รูปแบบการโกงหน้าร้านที่ระบบควรจับได้

จากประสบการณ์เคสที่เล่าไปตอนต้นบท มีรูปแบบการโกงหน้าร้านที่พบซ้ำ ๆ ในร้านอาหารไทยอยู่สามแบบหลัก:

1. **ยกเลิกบิลหลังรับเงินแล้ว** — พนักงานเก็บเงินสดจากลูกค้าจริง แล้วกดยกเลิกบิลในระบบทีหลังเพื่อทำให้ระบบไม่นับยอดขายบิลนั้น เงินสดส่วนนั้นก็หายเข้ากระเปาะโดยที่ยอดขายกับยอดเงินสดในสิ้นชั้กดูสมดุลกัน ในตอนแรก (เพราะเงินก็ถูกหักออกจากยอดที่ต้องส่งมอบไปพร้อมกัน)
2. **ส่วนลดที่ผิดปกติ** — ให้ส่วนลดเกินเพดานปกติบ่อยเกินไป หรือให้ส่วนลด 100% กับบิลของ “เพื่อน” ที่มากินจริง
3. **void ถูผิดปกติ** — พนักงานคนหนึ่งมีอัตราการยกเลิกรายการ/บิลสูงกว่าเพื่อนร่วมกะอย่างเห็นได้ชัด โดยเฉพาะถ้ากระจุกตัวอยู่ในช่วงเวลาเดียวกันซ้ำ ๆ

ทั้งสามแบบนี้จับด้วยสายตาคนเดียวไม่ทันหรอก เพราะแต่ละครั้งดูเป็นเรื่องเล็กน้อย ปกติของการทำงานหน้าร้าน (ลูกค้าเปลี่ยนใจ พนักงานคิดผิดต้องแก้ ฯลฯ) สิ่งที่ทำให้จับได้คือ **มองเป็น pattern สะสม ไม่ใช่มองทีละเหตุการณ์**

```

-- หา staff ที่ "ยกเลิกบิลหลังรับเงินแล้ว" บ่อยผิดปกติ ในช่วง 30 วันล่าสุด
SELECT
  s.id AS staff_id,
  s.name,
  COUNT(*) AS void_after_payment_count
FROM "AuditLog" al
JOIN "Staff" s ON s.id = al."staffId"
WHERE al.action = 'BILL_VOIDED_AFTER_PAYMENT'
  AND al."createdAt" ≥ NOW() - INTERVAL '30 days'
GROUP BY s.id, s.name
HAVING COUNT(*) ≥ 3
ORDER BY void_after_payment_count DESC;

-- หา staff ที่ให้ส่วนลดรวมสูงผิดปกติเทียบกับค่าเฉลี่ยของกะเดียวกัน
WITH discount_by_staff AS (
  SELECT
    al."staffId",
    DATE_TRUNC('day', al."createdAt") AS work_day,
    SUM(al.amount) AS total_discount
  FROM "AuditLog" al
  WHERE al.action = 'DISCOUNT_APPLIED'
    AND al."createdAt" ≥ NOW() - INTERVAL '30 days'
  GROUP BY al."staffId", DATE_TRUNC('day', al."createdAt")
)
SELECT
  d."staffId",
  s.name,
  d.work_day,
  d.total_discount,
  AVG(d.total_discount) OVER () AS avg_discount_all_staff
FROM discount_by_staff d
JOIN "Staff" s ON s.id = d."staffId"
WHERE d.total_discount > (
  SELECT AVG(total_discount) * 3 FROM discount_by_staff
)
ORDER BY d.total_discount DESC;

```

คิววีอาร์แรกหาคนที่ยกเลิกบิลหลังรับเงินตั้งแต่ 3 ครั้งขึ้นไปใน 30 วัน ซึ่งปกติควรใกล้ศูนย์อยู่แล้ว (ยกเลิกหลังรับเงินควรเป็นเหตุการณ์หายาก ไม่ใช่กิจวัตร) คิววีอาร์ที่สองหาคนที่ยอดส่วนลดรวมต่อวันสูงกว่าค่าเฉลี่ยของทั้งร้านถึง 3 เท่า ตัวเลขเกณฑ์เหล่านี้ (3 ครั้ง, 3 เท่า) เป็นจุดเริ่มต้นที่ปรับได้ตามพฤติกรรมจริงของแต่ละร้าน ไม่ใช่มาตรฐานตายตัว — ร้านอาหารแต่ละประเภทมีความถี่ปกติของการยกเลิก/ส่วนลดไม่เหมือนกัน

รายงานที่เจ้าของร้านควรได้เห็นจากข้อมูลนี้คือ “รายชื่อ staff ที่มีค่าผิดปกติ” ไม่ใช่ raw log ทั้งหมดที่อ่านไม่ไหว ระบบควรสรุปเป็นรายงานรายสัปดาห์ให้เจ้าของร้านเปิดดูง่าย ๆ ไม่ต้องเปิด SQL เอง

สั่ง Claude Code แบบนี้

ช่วยสร้าง middleware สำหรับตรวจสอบสิทธิ์ (RBAC guard) ในโฟลเดอร์ lib/auth/
โดยยึดตารางสิทธิ์นี้ (แนบตารางสิทธิ์จากบทนี้ให้ Claude อ่านด้วย):

- STAFF: ยกเลิกทั้งบิลหลังรับเงินแล้ว ทำไม่ได้เลย
- SHIFT_LEAD: ทำได้แต่ต้องมี approvedBy เป็น staffId ของ OWNER หรือ SHIFT_LEAD คนอื่น
- OWNER: ทำได้เสมอ

requirement:

1. guard ต้องอ่าน role จาก session ปัจจุบัน ไม่ใช่จาก body ที่ client ส่งมา
2. ทุกครั้งที่ action สำเร็จ (ไม่ว่าจะผ่าน guard เองหรือผ่านการอนุมัติ)
ต้องเขียนแถวลง AuditLog เสมอ ห้ามมี action ไหนที่ผ่านโดยไม่มี log
3. ถ้า action ต้องขออนุมัติ ให้คืน error แบบเจาะจงว่า "NEEDS_APPROVAL"
ไม่ใช่ error ทั่วไป เพื่อให้ UI แสดงหน้าจอขอ PIN คนอนุมัติได้ถูก
4. เขียน unit test อย่างน้อย 1 เคสต่อ role ต่อ action สำคัญ (void, discount, drawer)

ใช้ plan mode ก่อนเริ่ม ให้เห็นรายชื่อไฟล์ที่จะแก้ทั้งหมดก่อน implement

ต้องรื้ออะไรใน output: เช็คว่า guard อ่าน role จาก session ที่ verify แล้วฝั่ง server จริง ๆ ไม่ใช่เชื่อค่าที่ client ส่งมาในช่อง header/body เอง (เหมือนกับหลักการ “ห้ามเชื่อ client” ที่ใช้กับราคาในบทที่ 7) เช็คว่าทุก path ที่ action ผ่าน guard ไปสำเร็จมีการเขียน **AuditLog** จริง ไม่ใช่แค่บาง path และเช็คว่า test ครอบคลุม เคส “SHIFT_LEAD พยายามอนุมัติให้ตัวเอง” ด้วย เพราะเป็นช่องโหว่ที่ AI มักลืมนึกเวลาสร้าง approval flow

มุมมองของร้าน

เงินที่หายจากการโกงหน้าร้านมักไม่ได้หายเป็นก้อนใหญ่ครั้งเดียว แต่หายทีละเล็กทีละน้อยจนสะสมเป็นก้อนโต ในหลายเดือน สิ่งที่ทำให้จับได้เร็วไม่ใช่กล้องวงจรปิดหรือการจับผิดพนักงานรายวัน แต่คือการมีรายงานที่สรุป พฤติกรรมผิดปกติมาให้คุณเป็นประจำ เพราะคนที่โกงมักรู้ตัวว่าไม่มีใครมานั่งไล่ดู log ทีละบรรทัด

อีกเรื่องที่เจ้าของร้านต้องเข้าใจให้ตรงกันคือ RBAC ไม่ใช่การไม่ไว้วางใจพนักงาน แต่คือการปกป้องพนักงานที่ ทำงานตรงไปตรงมาด้วย เพราะถ้าไม่มีระบบแยกสิทธิ์และ log ชัดเจน เวลาเงินขาดแล้วสงสัยกันเอง ก็ไม่มีหลักฐานพิสูจน์ว่าใครทำหรือไม่ได้ทำ ระบบที่ดีช่วยให้ทั้งเจ้าของและพนักงานสบายใจทั้งคู่

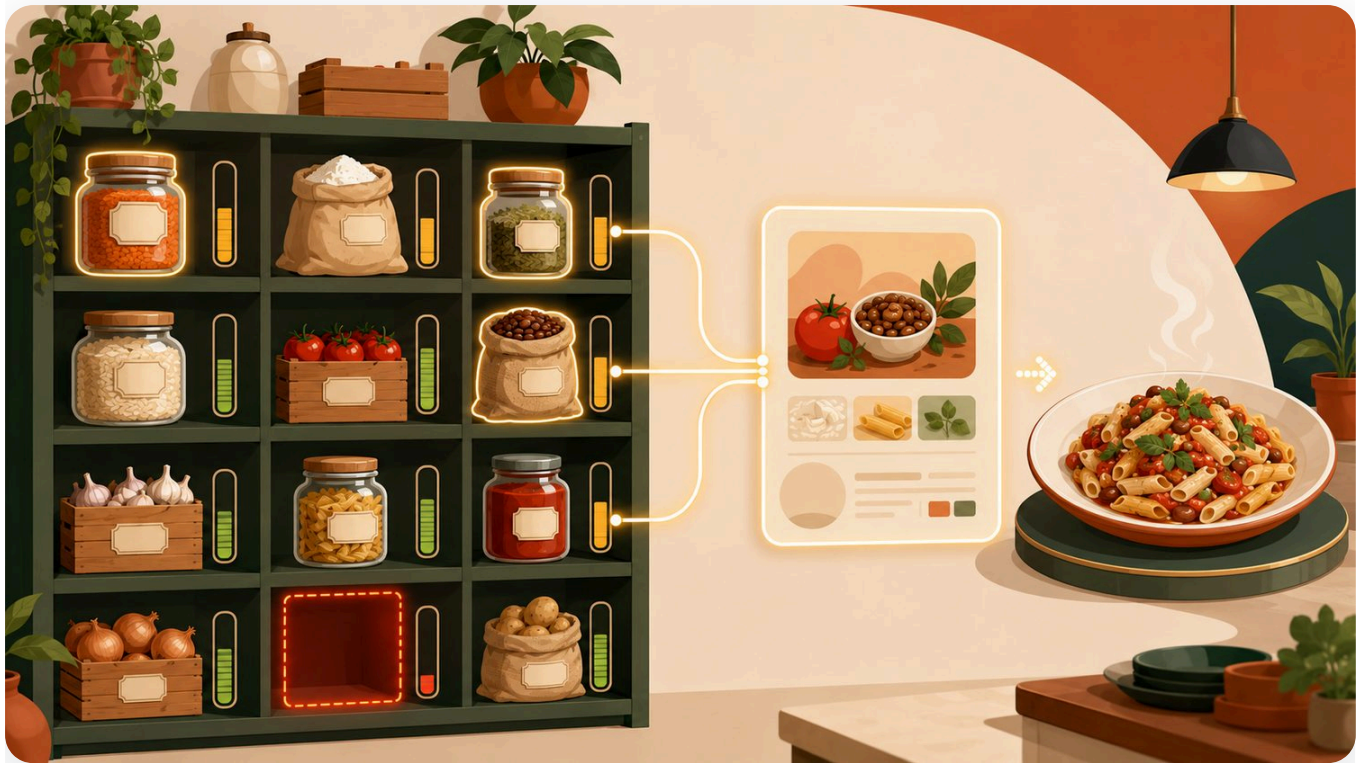
❌ พลาดบ่อย / ✅ ทำแบบนี้

❌ พลาดบ่อย: ให้พนักงานทุกคนล็อกอินด้วยบัญชีเดียวกันของร้าน (เช่น username “staff” รหัสเดียวกันหมด) เพราะตั้งค่าเร็วกว่า
✅ ทำแบบนี้: ทุกคนมี PIN ของตัวเอง และทุก action ที่บันทึกลง AuditLog ผูกกับ **staffId** ที่ล็อกอินจริง ณ ขณะนั้น ไม่ใช่ผูกกับเครื่อง POS

❌ พลาดบ่อย: เขียนในเอกสารขายระบบว่า “POS ของเราผ่านมาตรฐาน PCI DSS” เพื่อให้ดูน่าเชื่อถือ
✅ ทำแบบนี้: บอกตามจริงว่าออกแบบให้อยู่ในขอบเขต SAQ A โดยไม่เก็บ/ประมวลผล/ส่งผ่านข้อมูลบัตรบนระบบของร้านเลย ส่วนการยืนยันความสอดคล้องจริงเป็นกระบวนการที่ทำผ่าน payment gateway และ acquiring bank

จำ 3 ข้อ ยกเลิกทั้งบิลหลังรับเงินแล้ว และการให้ส่วนลดเกินเพดาน ควรเป็นสิทธิ์แคบสุดในระบบ เพราะเป็นสองช่องทางโก่งหน้าร้านที่พบบ่อยที่สุด ใช้ hosted page/iframe/tokenization ให้เซิร์ฟเวอร์ของเราเห็นแค่ token กับ status เพื่อให้ระบบอยู่ในขอบเขต SAQ A — และห้ามพูดว่าระบบ “ผ่าน PCI DSS แล้ว” จับการโก่งหน้าร้านด้วยการมองเป็น pattern สะสมจาก audit log ไม่ใช่มองที่ละเหตุการณ์เดียว ๆ

สต็อกและต้นทุนอาหาร



ร้านที่ผมเคยดูแลมีคืนหนึ่งที่ครัวขายกุ้งแม่น้ำเผาไปเรื่อย ๆ ทั้งที่กุ้งในช่องแช่แข็งเหลืออยู่จนสุดท้ายตั้งแต่สองทุ่ม พนักงานเสิร์ฟไม่มีทางรู้ เพราะระบบ POS ที่ใช้ตอนนั้นไม่เชื่อมกับสต็อกเลย มันรู้แค่ว่า “เมนูนี้มี” หรือ “เมนูนี้ไม่มี” ตามที่มีคนไปกดปิดเอง ผลคือออเดอร์ที่ 47 ของคืนนั้นเข้าไปในครัว แล้วเชฟต้องเดินออกมาขอโทษลูกค้าโต๊ะ 12 ด้วยตัวเอง

บทนี้เป็นบทที่ผมอยากให้คุณอ่านด้วยความคาดหวังที่ถูกต้องตั้งแต่บรรทัดแรก: **โมดูลสต็อกและต้นทุนอาหาร** คือส่วนที่ทำให้เต็มรูปแบบยากที่สุดในเล่มนี้ ยากกว่าการเจน QR PromptPay ยากกว่า state machine ของออเดอร์ เพราะมันไม่ใช่ปัญหาซอฟต์แวร์ล้วน ๆ — มันคือปัญหาความเป็นจริงของครัวที่ตัวเลขไม่นิ่งอยู่กับที่ (น้ำแข็งละลาย ผักช้ำ เศษที่ตัดทิ้ง คนละมาตรฐานการตวง) ที่ต้องพยายามยัดลงระบบที่ต้องการความแน่นอน เราจะเริ่มจากโมเดลข้อมูล ไปที่การตัดสต็อกตอนขาย ของหมดกลางวัน การนับสต็อก และปิดท้ายด้วยว่าทำไมถึงควรเริ่มจากรุ่นเรียบง่ายก่อน

สูตรอาหาร (Recipe/BOM) ผูกกับเมนู

แนวคิดหลักคือทุกเมนู (`MenuItem`) มี “สูตร” (Recipe) ที่บอกว่าใช้วัตถุดิบอะไรบ้าง ปริมาณเท่าไร นี่คือ Bill of Materials แบบเดียวกับที่โรงงานผลิตใช้ ต่างกันตรงที่หน่วยของร้านอาหารมันสับสนกว่ามาก — สิ่ง “ข้าวผัด

กึ่ง 1 จาน” ตัดข้าวเป็นกรัม ตัดกึ่งเป็นตัว ตัดน้ำมันเป็นมิลลิลิตร แต่ละวัตถุดิบมีหน่วยซื้อกับหน่วยใช้ไม่เท่ากัน (ซื้อน้ำมันเป็นถัง 18 ลิตร แต่ใช้ต่อจานเป็นช้อนโต๊ะ) เรื่องแปลงหน่วยนี้แหละที่ทำให้โมดูลนี้ซับซ้อนกว่าที่คิด

```

model Ingredient {
    id          String   @id @default(cuid())
    tenantId    String
    branchId    String
    name        String
    stockUnit   String   // หน่วยที่เก็บสต็อก เช่น "g", "ml", "ชิ้น"
    currentStock Decimal @db.Decimal(12, 3)
    reorderPoint Decimal @db.Decimal(12, 3) // ต่ำกว่านี้ต้องสั่งซื้อ
    avgCostPerUnit Int    // สตางค์ต่อ 1 stockUnit - เก็บเป็นจำนวนเต็มสตางค์เสมอ
    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt

    recipeLines RecipeIngredient[]

    @@unique([tenantId, branchId, name])
}

model Recipe {
    id          String @id @default(cuid())
    menuItemId String @unique
    menuItem    MenuItem @relation(fields: [menuItemId], references: [id])
    yieldQty    Int     @default(1) // สูตรนี้ทำได้กี่จานต่อรอบ (รองรับ batch cooking)

    lines RecipeIngredient[]
}

model RecipeIngredient {
    id          String   @id @default(cuid())
    recipeId    String
    recipe      Recipe   @relation(fields: [recipeId], references: [id], onDelete: Cascade)
    ingredientId String
    ingredient   Ingredient @relation(fields: [ingredientId], references: [id])
    quantity     Decimal @db.Decimal(12, 3) // ปริมาณต่อ yieldQty ของสูตร ในหน่วย stockUnit ของวัตถุดิบนั้น
}

model StockMovement {
    id          String   @id @default(cuid())
    tenantId    String
    branchId    String
    ingredientId String
    ingredient   Ingredient @relation(fields: [ingredientId], references: [id])
    type        StockMovementType
    quantity     Decimal @db.Decimal(12, 3) // ติดลบ = ตัดออก, บวก = เข้า
    orderId      String? // ถ้าตัดจากการขาย ผูกกับบิลนี้
    note         String?
    createdBy    String
    createdAt    DateTime @default(now())
}

enum StockMovementType {
    SALE_DEDUCT    // ตัดจากขาย
    PURCHASE_IN    // รับของเข้า
    COUNT_ADJUST   // ปรับจากการนับสต็อก
    WASTE          // ทิ้ง/เสียหาย
}

```

สังเกตว่าราคาต้นทุน (`avgCostPerUnit`) ผมยังยึดกฎเดิมจากบทที่ 10 คือเก็บเป็น **จำนวนเต็มสตางค์** ส่วนปริมาณวัตถุดิบใช้ **Decimal** เพราะนี่ไม่ใช่เงิน แต่เป็นปริมาณจริงที่มีทศนิยม (0.125 ลิตร, 2.5 กรัม) ห้ามใช้ **Float** เด็ดขาดเหมือนกัน เพราะการบวกลบสะสมหลายพันครั้งต่อวันจะพารวมคลาดเคลื่อนสะสม (floating-point drift) จนสต็อกในระบบกับของจริงเพี้ยนไปเรื่อย ๆ โดยไม่มีใครรู้ว่าเพี้ยนตอนไหน

ตัดสต็อกตอนขาย — ต้องอยู่ใน transaction เดียวกับปิดบิล

หัวใจของบทนี้คือประโยคเดียว: การตัดสต็อกวัตถุดิบต้องเกิดขึ้นใน transaction เดียวกับตอนบิลถูกปิด (สถานะเปลี่ยนเป็น **PAID**) ไม่ใช่ตอนสั่งอาหาร (เพราะออร์เดอร์อาจถูกยกเลิกก่อนทำ) และไม่ใช่เป็น job แยกที่รันทีหลัง (เพราะถ้า job ล้มกลางทาง สต็อกกับยอดขายจะไม่ตรงกันแบบตามหาสาเหตุไม่เจอ) ถ้าคุณแยกสองอย่างนี้ออกจากกัน สิ่งที่จะเกิดขึ้นแน่ ๆ คือวันหนึ่งเซิร์ฟเวอร์ล่มกลางที่ตัดสต็อกไปแล้วครึ่งบิล เหลืออีกครึ่งไม่ตัด แล้วไม่มีใครรู้ว่าจะนับสต็อกปลายเดือน

```

// lib/orders/close-bill.ts
import { prisma } from "@lib/prisma";
import { Prisma } from "@prisma/client";

export async function closeBillAndDeductStock(orderId: string, closedByUserId: string) {
  return prisma.$transaction(async (tx) => {
    const order = await tx.order.findUniqueOrThrow({
      where: { id: orderId },
      include: { items: { include: { menuItem: { include: { recipe: { include: { lines: true } } } } } } }
    } },
    });

    if (order.status !== "SERVED") {
      throw new Error(`ปิดบิลไม่ได้: ออร์เดอร์อยู่สถานะ ${order.status}`);
    }

    // รวมปริมาณวัตถุดิบที่ต้องตัดของทั้งบิล ก่อนยิงคำสั่งจริง
    const deductions = new Map<string, Prisma.Decimal>();
    for (const item of order.items) {
      const recipe = item.menuItem.recipe;
      if (!recipe) continue; // เมนูที่ยังไม่ได้ผูกสูตร ข้ามไปก่อน (ดูหัวข้อ "เริ่มจากฐานเรียนง่าย")
      for (const line of recipe.lines) {
        const perUnit = line.quantity.div(recipe.yieldQty);
        const total = perUnit.mul(item.quantity);
        deductions.set(
          line.ingredientId,
          (deductions.get(line.ingredientId) ?? new Prisma.Decimal(0)).add(total)
        );
      }
    }

    for (const [ingredientId, qty] of deductions) {
      await tx.ingredient.update({
        where: { id: ingredientId },
        data: { currentStock: { decrement: qty } },
      });
      await tx.stockMovement.create({
        data: {
          tenantId: order.tenantId,
          branchId: order.branchId,
          ingredientId,
          type: "SALE_DEDUCT",
          quantity: qty.neg(),
          orderId: order.id,
          createdBy: closedByUserId,
        },
      });
    }

    return tx.order.update({
      where: { id: orderId },
      data: { status: "PAID", paidAt: new Date() },
    });
  });
}

```

จุดที่ต้องระวังเพิ่มคือ ถ้าตัดสต็อกแล้วยอดติดลบ (ของในระบบไม่พอจริง ๆ) เวอร์ชันแรกสุดผมแนะนำให้ปล่อยให้ติดลบได้ แล้วรายงานออกมา ไม่ใช่บล็อกการปิดบิล เพราะร้านอาหารต้องปิดบิลได้เสมอไม่ว่าตัวเลขในระบบจะพังแค่ไหน — ลูกค้าจ่ายเงินไปแล้ว จะให้ไปติดอยู่ตรงหน้าจอ “สต็อกไม่พอ” ไม่ได้ ค่อยไปตามสาเหตุทีหลังจากรายงานส่วนต่าง

ของหมดกลางวัน (auto-86)

คำว่า “86” มาจากศัพท์ครัวอเมริกันที่แปลว่า “ของหมด เอาออกจากเมนู” ระบบที่ทำเองได้เปรียบตรงนี้มาก เพราะสามารถผูก `currentStock` ของวัตถุดิบหลักเข้ากับ `isAvailable` ของเมนูอัตโนมัติ ไม่ต้องรอพนักงานเดินไปกดปิดเมนูเอง

```
// lib/menu/auto-86.ts
export async function refreshMenuAvailability(tenantId: string, branchId: string) {
  const items = await prisma.menuItem.findMany({
    where: { tenantId, branchId, recipe: { isNot: null } },
    include: { recipe: { include: { lines: { include: { ingredient: true } } } } },
  });

  for (const item of items) {
    const lines = item.recipe?.lines ?? [];
    // เมนูทำไม่ได้ถ้ามีวัตถุดิบหลักตัวใดตัวหนึ่งเหลือไม่พอสำหรับ 1 งาน
    const canMake = lines.every((line) => {
      const perDish = line.quantity.div(item.recipe!.yieldQty);
      return line.ingredient.currentStock.gte(perDish);
    });

    if (canMake !== item.isAvailable) {
      await prisma.menuItem.update({
        where: { id: item.id },
        data: { isAvailable: canMake },
      });
    }
  }
}
```

ฟังก์ชันนี้ควรถูกเรียกทุกครั้งหลัง `closeBillAndDeductStock` (เฉพาะเมนูที่ใช้วัตถุดิบร่วมกับบิลที่เพิ่งตัด ไม่ต้องวนทั้งร้าน) ผลคือกึ่งแม่น้ำจานสุดท้ายถูกขาย เมนูกึ่งเผาจะหายจากหน้าลูกค้าและ POS ทันที ไม่ต้องรอใครไปกดปิด — นี่คือการที่ทำให้ร้านที่ทำระบบเองได้เปรียบเจ้าสำเร็จรูปหลายเจ้าจริง ๆ ถ้าทำถูก

นับสต็อกและส่วนต่าง

ตัวเลข `currentStock` ในระบบเป็นแค่ “ตัวเลขที่ควรจะเป็น” ตามสูตรอาหาร ไม่ใช่ของจริง เพราะของจริงมีเศษที่หก เหลือทิ้ง เผลอหยิบผิด หรือแม่ครัวดวงมือหนักกว่าสูตร ทุกร้านต้องมีรอบนับสต็อกจริง (physical count) แล้วเทียบกับตัวเลขระบบ

```
// lib/inventory/stock-count.ts
export async function recordStockCount(
  branchId: string,
  counts: { ingredientId: string; countedQty: number }[],
  countedByUserId: string
) {
  return prisma.$transaction(async (tx) => {
    const results = [];
    for (const { ingredientId, countedQty } of counts) {
      const ingredient = await tx.ingredient.findUniqueOrThrow({ where: { id: ingredientId } });
      const variance = new Prisma.Decimal(countedQty).sub(ingredient.currentStock);

      if (!variance.isZero()) {
        await tx.stockMovement.create({
          data: {
            tenantId: ingredient.tenantId,
            branchId,
            ingredientId,
            type: "COUNT_ADJUST",
            quantity: variance,
            createdBy: countedByUserId,
            note: `นับจริง ${countedQty} ${ingredient.stockUnit}, ระบบมี ${ingredient.currentStock}`,
          },
        });
        await tx.ingredient.update({
          where: { id: ingredientId },
          data: { currentStock: countedQty },
        });
      }
      results.push({ ingredientId, variance: variance.toNumber() });
    }
    return results;
  });
}
```

ส่วนต่างที่พบ (variance) ไม่ใช่แค่ตัวเลขสำหรับปรับสต็อก มันคือสัญญาณเตือนได้อีกชั้นหนึ่ง ถ้าวัตถุดิบตัวใดตัวหนึ่งมี variance ติดลบมากผิดปกติซ้ำ ๆ ทุกรอบนับ นั่นอาจแปลว่าสูตรอาหารตั้งปริมาณผิด (ตวงน้อยกว่าที่ทำจริง) หรือแย่กว่านั้นคือมีการหยิบออกไปโดยไม่ผ่านระบบขาย

food cost % — ตัวเลขที่เจ้าของร้านต้องการจริง ๆ

สุดท้ายของทั้งหมดนี้คือให้ได้ตัวเลข **food cost percentage** ต่อเมนูและภาพรวมร้าน สูตรพื้นฐานคือ ต้นทุนวัตถุดิบของเมนู ÷ ราคาขายของเมนู × 100

```
-- ต้นทุนต่อจานของแต่ละเมนู เทียบกับราคาขาย (คำนวณจากสูตรล่าสุด ไม่ใช่จากบิลเก่า)
SELECT
  mi.id,
  mi.name,
  mi.price_cents,
  SUM(ri.quantity / r.yield_qty * ing.avg_cost_per_unit) AS recipe_cost_cents,
  ROUND(
    SUM(ri.quantity / r.yield_qty * ing.avg_cost_per_unit) / NULLIF(mi.price_cents, 0) * 100,
    1
  ) AS food_cost_pct
FROM "MenuItem" mi
JOIN "Recipe" r ON r."menuItemId" = mi.id
JOIN "RecipeIngredient" ri ON ri."recipeId" = r.id
JOIN "Ingredient" ing ON ing.id = ri."ingredientId"
WHERE mi."tenantId" = $1 AND mi."branchId" = $2
GROUP BY mi.id, mi.name, mi.price_cents
ORDER BY food_cost_pct DESC;
```

Food cost % ทั่วไปที่ร้านอาหารใช้เป็นเกณฑ์คร่าว ๆ ก็ยังดี แต่ตัวเลข “เท่าไรถึงเรียกว่าดี” ต่างกันมากตามประเภทร้านและกลยุทธ์ราคา บทนี้จะไม่ฟันธงเปอร์เซ็นต์มาตรฐานให้ เพราะไม่มีอยู่ในคลังข้อเท็จจริงที่ยืนยันได้ — สิ่งที่เราควรทำคือ **แสดงตัวเลขให้เจ้าของร้านเห็นและเทียบกันเองระหว่างเมนู** ให้เขาตัดสินใจว่าเมนูไหนกำไรบางเกินไป

ทำไมถึงยากที่สุดในเล่ม และควรเริ่มจากพื้นฐานเรียบง่าย

ผมพูดตรง ๆ ว่าอย่าพยายามทำโมดูลนี้ให้ “สมบูรณ์” ตั้งแต่รอบแรก เหตุผลมีอย่างน้อยสี่ข้อ: (1) หน่วยแปลงกันวุ่นวาย ชื้อเป็นกิโล ใช้เป็นกรัม บางทีชื้อเป็นแพ็ค ต้องแตกแพ็ก่อนคำนวณ (2) สูตรซ้อนสูตรได้ (ซอสที่ทำเป็นล็อตใหญ่แล้วใช้กับหลายเมนู) ซึ่ง schema ข้างบนยังไม่รองรับกรณีนี้เต็มที่ ต้องขยายเป็น sub-recipe ที่หลัง (3) ของเสียจากการเตรียม (prep loss) เช่นปอกเปลือกกุ้งทิ้งไป 20% ไม่เท่ากับของที่ใช้จริงในงาน (4) แม่ครัวมือคนละแบบ ตวงไม่เท่ากันทุกครั้งจริง ๆ

พื้นฐานที่ผมแนะนำให้เริ่มคือ **เลือกผูกสูตรเฉพาะวัตถุดิบราคาแพงหรือหมุนไวจริง ๆ 15–20 รายการ** (เนื้อสัตว์ อาหารทะเล วัตถุดิบนำเข้า) ปลอยเมนูอื่นที่ไม่ได้ผูกสูตรให้ทำงานแบบเดิมคือพนักงานกดปิดเองเมื่อของหมด ระบบ auto-86 กับ food cost report จะครอบคลุมเฉพาะรายการที่ผูกไว้ก่อน แล้วค่อยขยายทีหลัง เมื่อร้านคุ้นกับ workflow การนับสต็อกแล้ว การพยายามผูกสูตรทุกเมนูตั้งแต่วันแรกคือสาเหตุอันดับหนึ่งที่โมดูลสต็อกของระบบ POS ที่ทำเองมักถูกทิ้งร้างหลังใช้ไปสองสัปดาห์ — คนตั้งสูตรหมดแรงตวงตัวเลขให้ครบก่อนที่ จะได้ใช้งานจริงด้วยซ้ำ

สั่ง Claude Code แบบนี้

ช่วยออกแบบและ implement โมดูล recipe + stock deduction สำหรับ POS ร้านอาหารนี้

โดยยึดกติกาต่อไปนี้เคร่งครัด:

1. ราคา/ต้นทุนเก็บเป็นจำนวนเต็มสตางค์ (Int) ปริมาณวัตถุดิบเก็บเป็น Decimal ห้ามใช้ Float หักคู่
2. การตัดสต็อกต้องอยู่ใน Prisma transaction เดียวกับการอัปเดตสถานะออร์เดอร์เป็น PAID เท่านั้น ห้ามตัดตอนสั่งอาหาร ห้ามทำเป็น background job แยก
3. ทุกการตัด/ปรับสต็อกต้องบันทึกลง StockMovement พร้อม type ที่ถูกต้อง เพื่อ trace ย้อนหลังได้
4. เขียนเฉพาะ schema.prisma ส่วน Ingredient/Recipe/RecipeIngredient/StockMovement กับฟังก์ชัน closeBillAndDeductStock ก่อน ยังไม่ต้องทำ auto-86 หรือ report

ใช้ plan mode ก่อนนะ schema จริง เพราะกระหนาบตารางที่มีอยู่แล้ว

สิ่งที่ต้องรีวิวใน output: (1) เช็คกว่า Claude ไม่แอบใช้ **Float** สำหรับต้นทุนหรือราคา (2) เช็คว่าการตัดสต็อกอยู่ใน **\$transaction** เดียวกับการอัปเดตสถานะออร์เดอร์จริง ไม่ใช่เรียกแยกกันสองครั้ง (3) เช็คว่ามันจัดการกรณีเมนูที่ไม่มีสูตร (ยังไม่ได้ผูก) โดยไม่ throw error จนปิดบิลไม่ได้ (4) ลองรันเทสกรณีสต็อกไม่พอ ต้องปิดบิลได้อยู่ ไม่ใช่ error

มุมมองเจ้าของร้าน

เจ้าของร้านมักคิดว่าระบบสต็อกคือ “กันขโมย” อย่างเดียว แต่ประโยชน์จริงที่วัดเป็นเงินได้ชัดกว่าคือสองเรื่อง: หนึ่ง — ลดการเสิร์ฟซ้ำเพราะของหมดกลางคัน (ลูกค้ารอนาน ยกเลิกออร์เดอร์ เสียชื่อร้าน) สอง — เห็นเมนูที่ “ขายดีแต่กำไรบาง” ซึ่งร้านจำนวนมากไม่เคยรู้จนกว่าจะมาคำนวณจริงจัง แต่ต้องเข้าใจว่าโมดูลนี้ต้องแลกกับ **วินัยของครัว** — ถ้าไม่มีใครในครัวยอมนับสต็อกตามรอบที่ตกลงกัน ตัวเลขในระบบจะเพี้ยนไปเรื่อย ๆ จนไม่มีใครเชื่อมั่นอีกต่อไป ระบบดีแค่ไหนก็ช่วยไม่ได้ถ้าไม่มีคนกรอกข้อมูลจริง แนะนำให้เริ่มนับสต็อกเฉพาะของแพง สัปดาห์ละครั้งก่อน ไม่ใช่ทุกวันทุกรายการ

❌ พลาดบ่อย / ✅ ทำแบบนี้

❌ **พลาดบ่อย:** ตัดสต็อกตอนออร์เดอร์ถูก “ส่งเข้าครัว” (สถานะ **PLACED**) เพราะคิดว่าเร็วกว่า ✅ **ทำแบบนี้:** ตัดตอนบิลถูกปิดจ่ายเงินแล้ว (**PAID**) เท่านั้น เพราะออร์เดอร์ที่ยกเลิกกลางทางไม่ควรตัดสต็อกไปแล้ว และถ้าตัดสองจุดจะตัดซ้ำ

❌ **พลาดบ่อย:** พยายามผูกสูตรอาหารให้ครบทุกเมนูในสัปดาห์แรกที่เปิดโมดูลนี้ใช้งาน ✅ **ทำแบบนี้:** เลือกวัตถุดิบแพงหรือหมุนไว 15–20 รายการก่อน ให้ทีมคุ้นกับการนับสต็อกจริง แล้วค่อยขยาย

❌ **พลาดบ่อย:** เก็บปริมาณวัตถุดิบเป็น **Float** เพราะ “แค่ตัวเลขปริมาณ ไม่ใช่เงิน คงไม่เป็นไร” ✅ **ทำแบบนี้:** ใช้ **Decimal** เสมอ เพราะระบบตัดสต็อกหลายพันครั้งต่อวัน ความคลาดเคลื่อนสะสมของ Float จะทำให้ตัวเลขเพี้ยนโดยไม่มีจุดเริ่มต้นให้ตามหา

❌ พลาดบ่อย: ถ้าตัดสต็อกแล้วติดลบ ให้ระบบ throw error บล็อกการปิดบิล **✅ ทำแบบนี้:** ปล่อยให้ปิดบิลได้เสมอ แล้วให้ตัวเลขติดลบไปโผล่ในรายงานส่วนต่างแทน เพราะลูกค้าจ่ายเงินไปแล้ว ระบบต้องไม่ขวางการรับเงิน

จำ 3 ข้อ การตัดสต็อกต้องอยู่ใน transaction เดียวกับปิดบิลเสมอ ไม่ใช่ตอนสั่งหรือ job แยก ห้ามใช้ Float ทั้งกับต้นทุน (สตางค์) และปริมาณวัตถุดิบ (Decimal) เพราะความเพี้ยนสะสมตามหาสาเหตุไม่ได้ เริ่มจากผู้สุত্রเฉพาะวัตถุดิบแพง/หมุนไวไม่มีรายการ อย่าพยายามทำให้ครบทุกเมนูตั้งแต่วันแรก

รายงาน ปิดกะ ปิดวัน



เจ้าของร้านคนหนึ่งเคยถามผมตอนส่งมอบระบบว่า “แล้วผมจะรู้ได้ยังไงว่าวันนี้ร้านกำไรหรือขาดทุน โดยไม่ต้องมานั่งเปิด Excel ทุกคืน” คำถามนี้เป็นเหตุผลที่บั้นนี้มีอยู่ ระบบ POS ที่ทำเองแล้วไม่มีรายงานที่ใช้งานได้จริง ก็เหมือนสร้างเครื่องคิดเลขที่ไม่มีปุ่มแสดงผล — ข้อมูลอยู่ในฐานข้อมูลครบ แต่ไม่มีใครเอาออกมาใช้ได้ทันเวลา

บทนี้เราจะพูดถึงสามระดับของรายงาน: **X report** (snapshot ระหว่างกะ), **ปิดกะ** (นับเงินสดจริงเทียบกับระบบ), และ **ปิดวัน / Z report** (สรุปยอดทั้งวันแล้ว “รีเซ็ต” ตัวนับสำหรับวันถัดไป) ปิดท้ายด้วยตัวเลข 5 ตัวที่เจ้าของร้านเปิดดูทุกเช้าจริง ๆ ไม่ใช่รายงานสวย ๆ ที่ไม่มีใครเปิด

X report กับ Z report ต่างกันยังไง

คำสองคำนี้มาจากยุคเครื่องเก็บเงินแบบกลไก (cash register) ที่พิมพ์ใบสรุปยอดออกมาเป็นกระดาษ ความหมายยังใช้ในวงการ POS ทุกวันนี้:

- **X report** = ดูปอดสะสม ณ ช่วงเวลาหนึ่ง **โดยไม่รีเซ็ตตัวนับ** ใช้เวลาไหนก็ได้ระหว่างวัน ดูกี่รอบก็ได้ ไม่กระทบข้อมูล เหมาะกับหัวหน้ากะที่อยากเช็กยอดกลางวันว่าตอนนี้นายไปเท่าไรแล้วโดยไม่ต้องรอปิดกะ
- **Z report** = สรุปยอดปิด (ปกติคือปิดวัน หรือปิดกะสุดท้ายของวัน) **แล้วรีเซ็ตตัวนับสำหรับรอบถัดไป** เป็นเอกสารที่ถือว่าเป็นทางการ ใช้กระทบยอดกับบัญชี และมักพิมพ์เก็บเป็นหลักฐาน

ในระบบที่เราสร้างเอง เราไม่มีตัวนับฮาร์ดแวร์แบบเครื่องเก่าที่ต้อง “รีเซ็ต” จริง ๆ (ข้อมูลอยู่ในฐานข้อมูลอยู่แล้วไม่หายไปไหน) แต่หลักการเดียวกันยังสำคัญ: **X report** คือ **query** แบบอ่านอย่างเดียว เรียกกี่ครั้งก็ได้ไม่ทิ้งร่องรอย ส่วน **Z report** คือ **action** ที่สร้างระเบียบถาวร (ปิดกะ/ปิดวัน) แล้วล็อกช่วงเวลานั้นไม่ให้แก้ไขข้อมูลย้อนหลังอีก

```
model ShiftClosing {
  id          String   @id @default(cuid())
  tenantId    String
  branchId    String
  shiftId     String   @unique
  closedByUserId String
  systemCashTotal Int    // ยอดเงินสดที่ระบบคำนวณจากบิลที่จ่ายเงินสด (สดางค์)
  countedCashTotal Int   // เงินสดที่นับจริงในลิ้นชัก (สดางค์)
  varianceCents Int      // countedCashTotal - systemCashTotal
  note        String?
  closedAt    DateTime @default(now())

  @@index([branchId, closedAt])
}

model DailyClosing {
  id          String   @id @default(cuid())
  tenantId    String
  branchId    String
  businessDate DateTime @db.Date
  closedByUserId String
  totalRevenueCents Int
  totalVatCents Int
  totalServiceChargeCents Int
  billCount   Int
  revenueByChannel Json    // { "QR": 123400, "POS": 456700 }
  revenueByPaymentMethod Json // { "CASH": 111100, "PROMPTPAY": 222200, "CARD": 333300 }
  closedAt    DateTime @default(now())

  @@unique([branchId, businessDate])
}
```

DailyClosing ล็อกด้วย `@@unique([branchId, businessDate])` เพื่อกันปิดวันซ้ำโดยไม่ตั้งใจ — และเป็นจุดสำคัญที่ **ปิดวันแล้วห้ามแก้ไขบิลของวันนั้นย้อนหลังอีก** ถ้าพบข้อผิดพลาดหลังปิดวัน ต้องออกเป็นรายการปรับปรุง (adjustment) ของวันใหม่ ไม่ใช่ไปแก้ตัวเลขเก่า เหมือนกับหลักบัญชีทั่วไป

ปิดกะ — นับเงินสด เทียบยอดระบบ ส่วนต่าง

ปิดกะเกิดขึ้นทุกครั้งที่มีการเปลี่ยนกะหรือจบวัน แก่นของมันคือเทียบ “เงินสดที่ควรมีในลิ้นชักตามระบบ ” กับ “เงินสดที่นับได้จริง ”

```
// lib/reports/close-shift.ts
export async function closeShift(shiftId: string, countedCashCents: number, userId: string) {
  const shift = await prisma.shift.findUniqueOrThrow({ where: { id: shiftId } });

  const cashOrders = await prisma.order.aggregate({
    where: {
      shiftId,
      status: "PAID",
      payment: { method: "CASH" },
    },
    _sum: { totalCents: true },
  });

  const systemCashTotal = cashOrders._sum.totalCents ?? 0;
  const varianceCents = countedCashCents - systemCashTotal;

  // ส่วนต่างเกิน 100 บาทถือว่าผิดปกติ ต้องมีหมายเหตุกำกับเสมอ
  const requiresNote = Math.abs(varianceCents) ≥ 10000;

  return prisma.shiftClosing.create({
    data: {
      tenantId: shift.tenantId,
      branchId: shift.branchId,
      shiftId,
      closedByUserId: userId,
      systemCashTotal,
      countedCashTotal: countedCashCents,
      varianceCents,
      note: requiresNote ? "PENDING_NOTE" : null,
    },
  });
}
```

ส่วนต่างที่นี้เป็นเรื่องปกติที่เกิดขึ้นได้เสมอ (ทอนผิด เศษสตางค์ปิดเศษ) แต่ต้อง **บังคับให้มีคนกรอกเหตุผล** เมื่อส่วนต่างเกินเกณฑ์ที่ร้านตัวเอง ไม่ใช่ปล่อยผ่านเงียบ ๆ — ตัวเลข **requiresNote** ในตัวอย่างนี้เป็นแค่ธงเตือนหน้า UI ต้องบังคับกรอกก่อนกดยืนยันปิดกะจริง

ปิดวัน — ยอดแยกช่องทาง ประเภทชำระเงิน VAT เมนูขายดี ชั่วโมงพีค

ปิดวันคือรายงานที่หนักกว่าปิดกะมาก เพราะต้องดึงหลายมิติพร้อมกัน งานนี้เหมาะกับการเขียน SQL ดิบตรง ๆ มากกว่าพยายามยัดทุกอย่างลง Prisma query ที่จะซ้อน **groupBy** จนอ่านไม่รู้เรื่อง

```

-- สรุปยอดขายแยกช่วงเวลา (ชั่วโมง/ปีค) ของวันที่กำหนด
SELECT
  date_trunc('hour', o."paidAt") AS hour_bucket,
  COUNT(*) AS bill_count,
  SUM(o."totalCents") AS revenue_cents,
  ROUND(AVG(o."totalCents")) AS avg_bill_cents
FROM "Order" o
WHERE o."branchId" = $1
  AND o.status = 'PAID'
  AND o."paidAt" ≥ $2::date
  AND o."paidAt" < ($2::date + interval '1 day')
GROUP BY hour_bucket
ORDER BY hour_bucket;

-- เมนูขายดีของวัน (ใช้ snapshot ชื่อ/ราคาใน OrderItem ตามกฎที่ 4 ไม่ join กลับไป MenuItem)
SELECT
  oi."itemNameSnapshot" AS menu_name,
  SUM(oi.quantity) AS qty_sold,
  SUM(oi."priceCentsSnapshot" * oi.quantity) AS revenue_cents
FROM "OrderItem" oi
JOIN "Order" o ON o.id = oi."orderId"
WHERE o."branchId" = $1
  AND o.status = 'PAID'
  AND o."paidAt" ≥ $2::date
  AND o."paidAt" < ($2::date + interval '1 day')
GROUP BY oi."itemNameSnapshot"
ORDER BY revenue_cents DESC
LIMIT 10;

```

สังเกตว่ารายงานเมนูขายดีดึงจาก `itemNameSnapshot` / `priceCentsSnapshot` ที่เราออกแบบไว้ตั้งแต่บทที่ 4 ไม่ join กลับไปที่ `MenuItem` ปัจจุบัน เพราะถ้าเมนูถูกลบหรือเปลี่ยนชื่อไปแล้ว รายงานย้อนหลังต้องยังแสดงชื่อ/ราคา ณ วันที่ขายจริง ไม่ใช่ค่าปัจจุบัน

ส่วน **VAT ที่ต้องนำส่ง** คำนวณจากยอดขายที่ผ่านมาแล้วตามลำดับที่ตั้งไว้ในบทที่ 10 (อาหาร → service charge → VAT) รายงานปิดวันแค่ดึงยอด VAT ที่ถูกคำนวณเก็บไว้ในแต่ละบิลมารวมกัน ไม่ต้องคำนวณใหม่ตรงนี้

```
// lib/reports/daily-summary.ts
export async function buildDailySummary(branchId: string, businessDate: Date) {
  const orders = await prisma.order.findMany({
    where: {
      branchId,
      status: "PAID",
      paidAt: { gte: businessDate, lt: addDays(businessDate, 1) },
    },
    include: { payment: true },
  });

  const revenueByChannel: Record<string, number> = {};
  const revenueByPaymentMethod: Record<string, number> = {};
  let totalRevenueCents = 0;
  let totalVatCents = 0;
  let totalServiceChargeCents = 0;

  for (const o of orders) {
    totalRevenueCents += o.totalCents;
    totalVatCents += o.vatCents;
    totalServiceChargeCents += o.serviceChargeCents;
    revenueByChannel[o.channel] = (revenueByChannel[o.channel] ?? 0) + o.totalCents;
    const method = o.payment?.method ?? "UNKNOWN";
    revenueByPaymentMethod[method] = (revenueByPaymentMethod[method] ?? 0) + o.totalCents;
  }

  return {
    billCount: orders.length,
    totalRevenueCents,
    totalVatCents,
    totalServiceChargeCents,
    avgBillCents: orders.length ? Math.round(totalRevenueCents / orders.length) : 0,
    revenueByChannel,
    revenueByPaymentMethod,
  };
}
```

ทำไมต้อง snapshot รายวัน ไม่ใช่ query สตทุกครึ่ง

ถ้าปล่อยให้หน้ารายงานย้อนหลัง (เช่น “เทียบยอดขายเดือนนี้กับเดือนที่แล้ว”) ไป query ตาราง **Order** สตทุกครึ่งที่เปิดหน้า ปีแรกอาจไม่รู้สึกละไร แต่พอร้านสะสมบิลหลักแสนหลักล้านแถว การ scan ย้อนหลังทั้งตารางทุกครั้งจะช้าลงเรื่อย ๆ และเปลืองโหนดฐานข้อมูลโดยไม่จำเป็น เพราะข้อมูลของ “เมื่อวาน” จะไม่มีวันเปลี่ยนอีกแล้ว

แนวทางที่ใช้กันคือ เขียน **DailyClosing** ทุกคืนหลังปิดวัน (สรุปตัวเลขที่คำนวณครั้งเดียวจบ) แล้วให้หน้ารายงานย้อนหลัง/เทียบเดือนไป query จาก **DailyClosing** แทน **Order** — ตารางนี้มีแถวแค่ 1 แถวต่อสาขาต่อวัน แม้ร้านเปิดสิบปีก็มีแค่ไม่กี่พันแถว เร็วกว่าการ aggregate จากตารางบิลจริงหลายเท่า ส่วนของวันนี้ที่ยังไม่ปิด ค่อยคำนวณสดจากตาราง **Order** ไปพลาง ๆ

5 ตัวเลขที่เจ้าของร้านเปิดดูทุกเช้าจริง ๆ

จากประสบการณ์ส่งมอบระบบให้ร้านหลายแห่ง สิ่งที่เจ้าของร้านเปิดมือถือดูตอนเช้าจริง ๆ ไม่ใช่รายงาน 20 หน้า แต่คือตัวเลขสั้น ๆ 5 ตัวนี้:

1. ยอดขายรวมเมื่อคืน เทียบกับค่าเฉลี่ยของวันเดียวกันในสัปดาห์ก่อน
2. จำนวนบิล และ ยอดเฉลี่ยต่อบิล (บอกว่าลูกค้าจ่ายน้อยลงหรือใช้จ่ายน้อยลง)
3. ส่วนต่างเงินสดจากการปิดกะ (ถ้าไม่เท่ากับ 0 คือมีอะไรผิดปกติที่ต้องถามพนักงาน)
4. เมนูขายดี 3 อันดับแรกของเมื่อคืน (ของหมดเร็วไปไหม ต้องเตรียมเพิ่มไหม)
5. ชั่วโมงพีค (ตารางพนักงานพอไหมช่วงนั้น)

หน้ารายงานที่ดีที่สุดที่ผมเคยทำให้ร้าน ไม่ใช่ dashboard ที่มีกราฟสวยที่สุด แต่คือหน้าเดียวที่แสดง 5 ตัวเลขนี้ พร้อมกันตั้งแต่บรรทัดแรกที่เปิดแอปมาเจอ

สั่ง Claude Code แบบนี้

ช่วยสร้างหน้ารายงานปิดวัน (daily summary) สำหรับเจ้าของร้าน โดยยึดกติกา:

1. ดึงข้อมูล "วันนี้ที่ยังไม่ปิด" จาก query สดในตาราง Order ส่วนข้อมูลย้อนหลัง (เดือนนี้ เทียบเดือนก่อน) ให้ query จากตาราง DailyClosing เท่านั้น ห้าม aggregate จากตาราง Order ย้อนหลังเกิน 1 วัน
2. เมนูขายดีต้องดึงจาก itemNameSnapshot/priceCentsSnapshot ใน OrderItem ห้าม join กลับไปที่ MenuItem ปัจจุบัน เพราะราคา/ชื่อเปลี่ยนได้
3. แสดง 5 ตัวเลขนี้เด่นที่สุดบนสุดของหน้า: ยอดขายรวมเทียบสัปดาห์ก่อน, จำนวนบิล+ยอดเฉลี่ยต่อบิล, ส่วนต่างเงินสดปิดกะ, เมนูขายดี 3 อันดับ, ชั่วโมงพีค
4. เขียน raw SQL สำหรับ query ที่ซับซ้อน (group by ชั่วโมง, group by เมนู) แทนการซ้อน Prisma groupBy หลายชั้น ขอ SQL query กับ Prisma service function ก่อน ยังไม่ต้องทำ UI

สิ่งที่ต้องรีวิวใน output: (1) เช็ค query ย้อนหลังไม่ scan ตาราง **Order** ตรง ๆ (ต้องผ่าน **DailyClosing**) (2) เช็คว่ามี WHERE กรอง **branchId** / **tenantId** ทุก query กัน data leak ข้ามสาขา (3) เช็ค timezone ของ **businessDate** — เทียงคีนของไทยกับ UTC ต่างกัน 7 ชั่วโมง ถ้า Claude ลืมแปลง timezone รายงาน “เมื่อคืน” อาจตัดวันผิดจุด (4) ลองรันกับข้อมูลว่าง (วันที่ไม่มีบิลเลย) ต้องไม่ error

มุมมองเจ้าของร้าน

รายงานพวกนี้ไม่ใช่ของหรูสำหรับร้านใหญ่เท่านั้น ร้านเล็กก็ต้องมี เพราะมันตอบคำถามที่เจ้าของร้านถามตัวเองทุกเช้าอยู่แล้วแค่ไม่มีเครื่องมือช่วย: “เมื่อคืนดีไหม”, “เงินหายไปไหม”, “พนักงานกะไหนทำงานเช้าเป่า” สิ่งที่ต้องเข้าใจคือรายงานพวกนี้แม่นยำได้เท่าที่ **วินัยการปิดกะของพนักงาน** แม่น ถ้าไม่มีใครนับเงินสดจริงจังกะ ตัวเลขส่วนต่างจะไม่มี ความหมาย และถ้าปิดวันช้าหรือลืมปิด รายงานเช้าของเจ้าของร้านก็จะว่างเปล่า — ระบบดีแค่ไหนก็แทนวินัยของคนหน้างานไม่ได้

✗ พลาดบ่อย / ✓ ทำแบบนี้

❌ **พลาดบ่อย:** ให้นำรายงานย้อนหลัง (เช่น เทียบยอดขายรายเดือน) query จากตาราง **Order** สดทุกครั้งที่เปิดหน้า ✅ **ทำแบบนี้:** เขียน **DailyClosing** snapshot ทุกคืนหลังปิดวัน แล้วให้รายงานย้อนหลังอ่านจากตารางนี้แทน

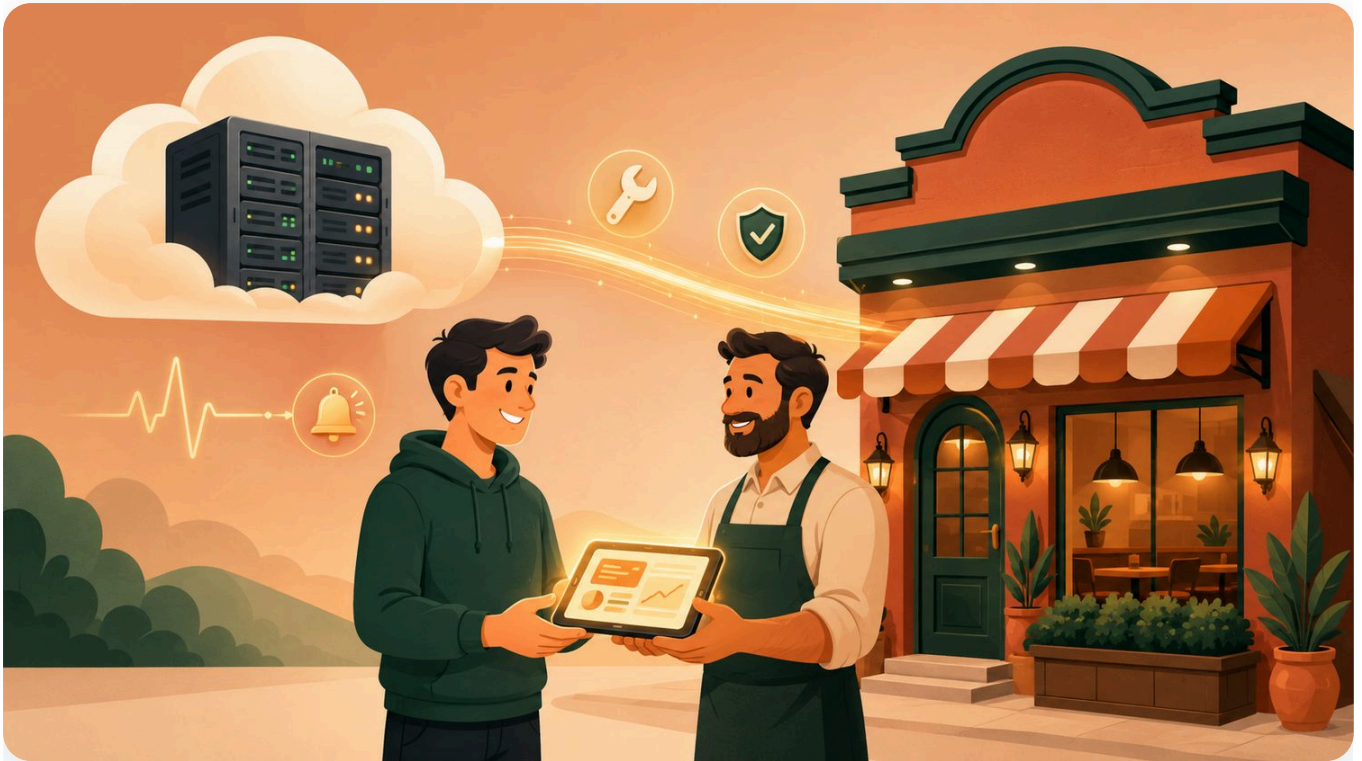
❌ **พลาดบ่อย:** ปล่อยให้ปิดกะที่มีส่วนต่างเงินสดผ่านไปเงียบ ๆ โดยไม่บันทึกเหตุผล ✅ **ทำแบบนี้:** บังคับกรอกหมายเหตุเมื่อส่วนต่างเกินเกณฑ์ที่ร้านตั้งไว้ ก่อนยืนยันปิดกะได้

❌ **พลาดบ่อย:** ให้แก้ไขบิลของวันที่ปิดไปแล้ว (หลัง **DailyClosing** ถูกสร้าง) โดยตรง ✅ **ทำแบบนี้:** ล็อกวันที่ปิดแล้วด้วย `@unique([branchId, businessDate])` ข้อผิดพลาดที่พบทีหลังต้องออกเป็นรายการปรับปรุงของวันใหม่

❌ **พลาดบ่อย:** รายงานเมนูขายดี join กลับไปที่ราคา/ชื่อปัจจุบันของ **MenuItem** ✅ **ทำแบบนี้:** ใช้ snapshot ราคา/ชื่อที่เก็บไว้ใน **OrderItem** ตอนขายจริง เพื่อให้รายงานย้อนหลังไม่เปลี่ยนตามราคาปัจจุบัน

จำ 3 ข้อ X report คือดูยอดได้เรื่อย ๆ ไม่รีเซ็ต ส่วน Z report/ปิดวันคือ action ที่ล็อกข้อมูลแล้วห้ามแก้ไขย้อนหลัง รายงานย้อนหลังต้องอ่านจาก snapshot รายวันที่คำนวณไว้แล้ว ไม่ใช่ scan ตารางบิลสดทุกครั้ง เจ้าของร้านต้องการแค่ 5 ตัวเลขทุกเช้า ไม่ใช่ dashboard ที่สวยแต่ไม่มีใครเปิดดู

Deploy และอยู่กับร้านจริง



โค้ดที่รันได้บนเครื่องคุณไม่ใช่ระบบ POS ระบบ POS ที่ใช้งานได้จริงคือโค้ดที่รันอยู่บนเซิร์ฟเวอร์ตอนเที่ยงวันเสาร์ที่ร้านแน่นทุกโต๊ะ เน็ตกระตุกแวนเดียว แล้วไม่มีใครในร้านรู้ตัวว่ามีอะไรผิดปกติเกิดขึ้นเบื้องหลัง บทนี้เป็นบทปิดเล่ม เราจะไม่พูดเรื่องฟีเจอร์ใหม่อีกแล้ว แต่พูดเรื่องที่ตัดสินใจว่าระบบที่คุณสร้างมา 15 บทที่ผ่านมาจะ “อยู่รอด” ในโลกจริงได้ไหม

เลือกโฮสต์: Vercel + managed Postgres vs VPS ของตัวเอง

ตัดสินใจเรื่องโฮสต์ไม่ได้แยกจากการตัดสินใจเรื่อง realtime ที่เราคุยกันในบทที่ 8 เลย เพราะข้อจำกัดทางเทคนิคมันผูกกันตรง ๆ: **Route Handler** ของ **Next.js** อัปเกรดเป็น **WebSocket** ไม่ได้ และ **Vercel** ถือ **TCP connection** ค้างไม่ได้ ส่วน **SSE** ใช้ได้ปกติบน **Vercel** เพราะเป็น HTTP request ธรรมดาที่ stream response กลับมาเรื่อย ๆ ไม่ใช้การอัปเกรด protocol

ถ้าคุณทำตามเล่มนี้แนะนำในบทที่ 8 คือใช้ SSE เป็นค่าเริ่มต้นสำหรับ KDS และ POS นั้นแปลว่า **Vercel + managed Postgres** (เช่นบริการ Postgres แบบจัดการให้ที่ทำงานร่วมกับ Vercel ได้) เป็นตัวเลือกที่ใช้งานได้จริง โดยไม่ต้องตะแคง infrastructure เอง deploy ทุกครั้งด้วย git push ไม่ต้องดูแล OS ไม่ต้อง patch security เอง

VPS ของตัวเอง (หรือ container บน cloud provider ที่คุณคุมเต็มที่) จำเป็นก็ต่อเมื่อคุณตัดสินใจยับยั้งใช้ WebSocket จริง ๆ (เช่นต้องการ two-way communication ถ้ามาระหว่าง client กับ server ที่ SSE ทำไม่ได้) หรือมี requirement อื่นที่ managed platform ทำไม่ได้ เช่นต้องรัน process พื้นหลังที่ทำงานต่อเนื่องยาว ๆ

โครงการต้นทุนที่ควรทำก่อนตัดสินใจ ไม่ใช่ตัวเลขราคาตายตัว (ราคาโฮสต์เปลี่ยนแปลงและต่างกันตามผู้ให้บริการ ต้องไปเช็คจริงตอนตัดสินใจ) แต่เป็นรายการต้นทุนที่ต้องเทียบ:

```
รายการต้นทุนที่ต้องเทียบ (Vercel+managed Postgres vs VPS เอง)
- ค่าแพลตฟอร์ม/compute ต่อเดือน (ตามการใช้งานจริง vs ค่าคงที่)
- ค่าฐานข้อมูล managed (รวม backup อัตโนมัติหรือไม่)
- ค่า bandwidth / egress
- ต้นทุนเวลาคน: ใครดูแล OS/security patch (VPS ต้องมี ไม่มีใน managed)
- ต้นทุนเวลาคน: debug เวลาระบบล่มกลางดึก ใครถูกปลุก
- ความสามารถ scale เวลาร้านเปิดสาขาเพิ่ม (multi-tenant ตามบทที่ 3)
```

สำหรับร้านเดียวหรือไม่ก็สาขา ผมแนะนำเริ่มจาก managed platform ก่อนเสมอ เพราะต้นทุนเวลาคนที่ต้องมานั่งดูแลเซิร์ฟเวอร์เองมักแพงกว่าค่าแพลตฟอร์มที่จ่ายเป็นรายเดือนมาก โดยเฉพาะทีมเล็กที่ไม่มีคนคอย on-call 24 ชั่วโมง

Migration บนร้านที่เปิดอยู่ — ห้าม downtime ตอนเที่ยง

ร้านอาหารไม่มี “ช่วงปิดปรับปรุงระบบ” แบบธุรกิจอื่น เพราะเปิดทุกวันเกือบทั้งวัน migration ฐานข้อมูลต้องรันได้โดยไม่ทำให้ระบบล่มกลางที่ลูกค้ากำลังสั่งอาหารอยู่ หลักที่ต้องยึดคือ **expand-contract pattern**: เพิ่มคอลัมน์/ตารางใหม่แบบ backward-compatible ก่อน deploy โค้ดที่ใช้มันทีหลัง ไม่ทำสองอย่างพร้อมกันในการ deploy เดียว

```
# ตัวอย่างขั้นตอน migration แบบไม่ downtime สำหรับเพิ่มคอลัมน์ที่ NOT NULL
# ขั้นที่ 1: เพิ่มคอลัมน์แบบ nullable ก่อน (deploy แยกจากขั้นที่ 2)
npx prisma migrate dev --name add_table_zone_nullable
# schema.prisma: zone String? (ยังไม่ required)

# ขั้นที่ 2: deploy โค้ดที่เขียนค่าลงคอลัมน์ใหม่นี้ทุกครั้งที่มีการสร้าง/แก้ไข record
# รอให้ record เก่าทั้งหมดถูก backfill ครบ (script แยกขั้นตอนโหลดค่า เช่นที่ 3)

# ขั้นที่ 3: ค่อยเปลี่ยนเป็น NOT NULL ในรอบ deploy ถัดไป
npx prisma migrate dev --name make_table_zone_required
```

```
# migrate deploy บน production ต้องรันแบบนี้เสมอ ไม่ใช่ migrate dev
npx prisma migrate deploy
# ตรวจสอบ pending migration ก่อนรันจริงทุกครั้ง
npx prisma migrate status
```

กฎที่ต้องจำ: ห้าม deploy migration ที่ลบคอลัมน์หรือเปลี่ยนชนิดข้อมูลแบบ breaking ในรอบเดียวกับที่โค้ดฝั่ง application ยังอ้างถึงคอลัมน์เดิม เพราะระหว่าง deploy จะมี instance เก่ากับใหม่วางพร้อมกันเสมอ

(rolling deploy) ถ้าคอลัมน์หายไปก่อน instance เก่าจะพังทันที ช่วงเวลาที่ปลอดภัยที่สุดสำหรับรัน migration หนัก ๆ ที่ต้อง lock ตาราง (เช่น backfill ข้อมูลจำนวนมาก) คือช่วงที่ร้านปิดหรือโหลดต่ำสุด ไม่ใช่กลางวัน หรือกลางมือเย็นเด็ดขาด

Backup และการกู้คืนที่ทดสอบแล้วจริง

Backup ที่ไม่เคยลองกู้คืนดู ไม่ใช่ backup — มันคือความหวังลม ๆ แล้ง ๆ ร้านที่ผมเคยดูแลมีเหตุการณ์ที่ต้องกู้ฐานข้อมูลจริงหนึ่งครั้ง และสิ่งที่ช่วยชีวิตตอนนั้นไม่ใช่การมี backup อัตโนมัติทุกวัน แต่คือการที่เคย ซ่อมกู้คืนบนเครื่อง staging มาก่อนแล้วรู้ขั้นตอนอยู่แล้วว่าใช้เวลาเท่าไรและพังตรงไหนได้บ้าง

```
# ตัวอย่างขั้นตอนซ่อมกู้คืนที่ต้องทำเป็นประจำ ไม่ใช่แค่ตอนมีปัญหาจริง
# 1. ดึง backup ล่าสุดมาอยู่บนฐานข้อมูล staging แยกต่างหาก (ห้ามกั้กับ production เพื่อ "ทดสอบ")
pg_restore --clean --if-exists -d pos_staging latest_backup.dump

# 2. รัน migration ที่ค้างอยู่ (ถ้า backup เก่ากว่า schema ปัจจุบัน)
npx prisma migrate deploy

# 3. เช็คความถูกต้องของข้อมูลสำคัญหลังกู้คืนเสมอ
psql pos_staging -c "SELECT count(*), max(\"paidAt\") FROM \"Order\" WHERE status = 'PAID';"
```

สิ่งที่ต้องบันทึกไว้เป็นเอกสารทุกครั้งที่ซ่อม: กู้คืนใช้เวลากี่นาที ข้อมูลสูญหายได้มากที่สุดกี่นาที (RPO) ระบบกลับมาใช้งานได้กี่นาที (RTO) ถ้าไม่เคยตอบสองคำถามนี้ได้ แปลว่ายังไม่รู้จริงว่าถ้าเกิดเหตุจะถูกกลับมาได้เมื่อไหร่

เน็ตหลุดกลางพีค — ทำอะไรได้บ้าง

นี่คือสถานการณ์ที่เกิดขึ้นจริงกับทุกร้านไม่ช้าก็เร็ว: เน็ตของร้านหลุดตอนเที่ยงคนแน่น ระบบ POS ที่พึ่งอินเทอร์เน็ตทั้งหมดก็ใช้ไม่ได้ทันที สิ่งที่ได้จริงมีสามระดับ เรียงจากพร้อมใช้เร็วสุดไปหายากสุด:

- พิมพ์ใบครัวสำรอง** — ตามที่วางไว้ในบทที่ 8 ว่า KDS ต้องมี fallback เป็นเครื่องพิมพ์ใบครัว ถ้าจอครัวหลุด (ซึ่งมักหลุดพร้อมเน็ตทั้งร้าน) ครัวยังทำงานต่อได้จากใบกระดาษ
- โหมดออฟไลน์แบบจำกัด** — POS หน้าร้านที่ cache เมนูไว้ในเครื่องล่วงหน้า ยังกดสั่งอาหารและคิดเงินสดต่อได้ชั่วคราว โดยข้อมูลออร์เดอร์จะถูก queue ไว้ในเครื่อง แล้ว sync กลับเข้าเซิร์ฟเวอร์เมื่อเน็ตกลับมา
- ให้พนักงานจดมือแล้วค่อยตามทีหลัง** — ทางเลือกสุดท้ายที่ทุกร้านต้องมีเป็น fallback แม้จะมีสองข้อบนแล้วก็ตาม เพราะไม่มีระบบไหนออฟไลน์ได้ 100% ในทุกกรณี

ผมต้องพูดตรง ๆ ตรงนี้: โหมดออฟไลน์เต็มรูปแบบ (ที่ทุกพีเจอร์ท างงานได้แม้ไม่มีเน็ตเลย แล้ว sync กลับอัตโนมัติแบบไม่มีข้อมูลชนกัน) เป็นงานวิศวกรรมที่หนักกว่าทั้งเล่มนี้รวมกัน ต้องรับมือกับ conflict resolution (สองเครื่องแก้ไขบิลเดียวกันตอนออฟไลน์พร้อมกัน) ต้อง sync แบบ eventually-consistent ต้องมี local database ในทุกเครื่อง POS ไม่ใช่แค่ browser cache ธรรมดา — StoreHub ที่กล่าวถึงในบทที่ 1 ถูกเคลมว่าทำงานตอนออฟไลน์ได้เป็นจุดขาย เพราะนี่คือของที่ทำยากจริง ไม่ใช่พีเจอร์ท ะเลืก ๆ ร้านที่ทำระบบเอง

ส่วนใหญ่ไม่คุ้มที่จะลงทุนสร้างโหมดยออนไลน์เต็มรูปแบบตั้งแต่วันแรก — เริ่มจากสามระดับข้างบนที่ implement ได้จริงในเวลาจำกัดก่อน

Monitoring และ alert ที่ควรมี

ระบบที่ล้มแล้วไม่มีใครรู้จนกว่าพนักงานจะโทรมาบ่นคือระบบที่ยังไม่พร้อมส่งมอบ อย่างน้อยที่สุดควรมี:

Monitoring ขั้นต่ำที่ต้องมีก่อนส่งมอบร้านจริง

- Uptime check: ping หน้า health endpoint ทุกไม่กี่นาที แจ้งเตือนทันทีถ้าตอบช้าหรือไม่ตอบ
- Error tracking: จับ exception ผีง server ทุกตัว ไม่ใช่แค่ log ตอนมีปัญหา
- Webhook failure alert: ถ้า webhook จาก payment gateway (บทที่ 11) ตอบ error ช้า ๆ ต้องรู้ทันที เพราะแปลว่าบิลค้างสถานะไม่อัปเดตทั้งที่ลูกค้าจ่ายเงินไปแล้ว
- Database connection pool: เตือนก่อนที่ connection จะเต็มจนรับ request ใหม่ไม่ได้
- ช่องทางแจ้งเตือนที่คนเห็นจริง (ไม่ใช่แค่ email ที่ไม่มีใครเปิด) — กลุ่มแชทที่ทีมดูแลอยู่ตลอด

จุดที่มักถูกมองข้ามคือ alert สำหรับ webhook ล้มเหลว เพราะบทที่ 11 บอกไว้แล้วว่า webhook ต้อง idempotent เนื่องจากถูกส่งซ้ำได้ แต่ถ้า endpoint รับ webhook พังไปเลย (ไม่ใช่แค่ช้า) บิลที่ลูกค้าจ่ายเงินจริงจะค้างสถานะ “รอชำระ” ในระบบตลอดไปโดยไม่มีใครรู้จนกว่าจะปิดวันแล้วยอดไม่ตรง

ส่งมอบให้ร้าน

โค้ดเสร็จไม่ได้แปลว่างานเสร็จ การส่งมอบให้ร้านจริงต้องมีอย่างน้อยสี่อย่าง:

- **เทรน 1 ชั่วโมงแรก** — จับมือทำจริงหน้างานในช่วงเวลาที่ร้านเปิดจริง (ไม่ใช่สอนตอนร้านปิด) ให้พนักงานลองกดของจริงต่อหน้าคุณ เพื่อจับปัญหาการใช้งานที่คุณคาดไม่ถึงตั้งแต่ชั่วโมงแรก
- **คู่มือ 1 หน้า** — ไม่ใช่เอกสาร 20 หน้าที่ไม่ใครอ่าน แต่เป็นกระดาษแผ่นเดียวติดข้างเครื่อง POS บอกขั้นตอนที่ใช้บ่อยที่สุดและเบอร์ติดต่อฉุกเฉิน
- **ช่องทางแจ้งปัญหา** — ต้องชัดเจนว่าเจอปัญหาแล้วติดต่อใครตอนไหน (เวลาทำการ vs นอกเวลา) และคาดหวังได้แค่ไหน
- **สัญญาดูแลรายเดือน** — ระบบ software ไม่ใช่ของที่ส่งแล้วจบ ต้องมีข้อตกลงชัดเจนว่าอะไรรวมอยู่ในค่าดูแลรายเดือน (แก็บ็ก, อัปเดตความปลอดภัย) และอะไรเป็นงานเพิ่มที่คิดค่าใช้จ่ายแยก (ฟีเจอร์ใหม่)

ถ้าจะขายระบบนี้ต่อให้ร้านอื่น

ถ้าเป้าหมายคือทำระบบนี้เป็นสินค้าขายได้หลายร้าน ไม่ใช่แค่ทำให้ร้านเดียว มีเรื่องต้องคิดเพิ่มตั้งแต่ตอนออกแบบ ไม่ใช่ค่อยแก้ทีหลัง:

- **Multi-tenant ให้ถูกตั้งแต่แรก** — ตามที่วางไว้ในบทที่ 3 ว่าแม้ร้านเดียวก็ควรมี **Tenant / Branch** เพราะเดิมทีหลังเจ็บบ่อยมาก ทุก query ต้องกรองด้วย **tenantId** เสมอ ไม่มีข้อยกเว้น

- เวอร์ชันของ schema และโค้ด — เมื่อมีหลายร้านใช้ระบบเดียวกันพร้อมกัน การ deploy ฟีเจอร์ใหม่ต้องไม่กระทบร้านที่ยังไม่พร้อมอัปเดต ต้องคิดเรื่อง feature flag ตั้งแต่ต้น
- ซัพพอร์ทหลายร้านพร้อมกัน — เวลาเกิดปัญหาที่ร้าน A ต้องรู้ทันทีว่ากระทบร้าน B, C ด้วยหรือไม่ ต้องมี logging ที่แยกตาม tenant ชัดเจนพอจะ debug ได้เร็วตอนมีหลายร้านพร้อมกัน
- ต้นทุนซัพพอร์ทต่อร้านที่เพิ่มขึ้น — ยิ่งขายได้หลายร้าน เวลาที่ต้องใช้ตอบคำถาม/แก้ปัญหา ก็เพิ่มตามจำนวนร้าน ไม่ใช่คงที่ ต้องคิดกระบวนการซัพพอร์ทที่ scale ได้ ไม่ใช่พึ่งตัวคุณคนเดียวตอบทุกร้าน

สั่ง Claude Code แบบนี้

ช่วยตรวจสอบระบบ POS นี้ก่อน deploy ขึ้น production ครั้งแรก โดยเช็กรายการต่อไปนี้:

1. หา migration ที่ค้างอยู่ทั้งหมด และเช็คว่ามีตัวไหนที่เป็น breaking change (ลบคอลัมน์ เปลี่ยนชนิดข้อมูล เพิ่ม NOT NULL โดยไม่มีค่า default) ที่ต้องแยกเป็น expand-contract
 2. หา endpoint ที่รับ webhook จาก payment gateway ทั้งหมด เช็คว่า idempotent จริง และมี error handling ที่ไม่ทำให้ request หายเงียบ ๆ ถ้าพัง
 3. หา query ที่ยังไม่มีกรอง tenantId/branchId ที่อาจเป็นช่องโหว่ data leak ข้ามร้าน
 4. เสนอ health check endpoint และรายการ alert ขึ้นดำที่ควรตั้งก่อนส่งมอบ
- รายงานเป็นรายการ พร้อมระบุไฟล์และบรรทัดที่ต้องแก้ ยังไม่ต้องแก้โค้ดจนกว่าจะรีวิวก่อนกันก่อน

สิ่งที่ต้องรีวิวใน output: (1) เช็คว่า Claude ระบุ migration ที่เป็น breaking change ได้ถูกต้องจริง ไม่ใช่บอกว่าทุกอย่างปลอดภัยเพราะไม่เข้าใจบริบท expand-contract (2) เช็ค endpoint webhook ทุกตัวจริง ไม่ใช่แค่ตัวที่เจोक่อน (3) เช็คว่ารายการ query ที่ขาด `tenantId` ครบจริง ให้ลอง grep เพิ่มเองด้วยถ้าไม่มั่นใจ (4) อย่าให้ Claude แก้ migration ที่รันไปแล้วบน production โดยตรง ต้องสร้าง migration ใหม่เสมอ

มุมมองของร้าน

การส่งมอบระบบให้ร้านไม่ได้จบตรงวันที่ระบบทำงานได้ครั้งแรก มันเพิ่งเริ่ม สิ่ง queเจ้าของร้านต้องเข้าใจตั้งแต่ก่อนตัดสินใจทำระบบเองคือ: ระบบนี้จะมึวันที่มันพัง (เน็ตหลุด เซิร์ฟเวอร์ล่ม บั๊กที่ไม่มีใครเคยเจอ) และคำถามสำคัญไม่ใช่ “จะพังไหม” แต่คือ “ตอนพังแล้วร้านยังขายของได้ไหม และมีใครรับผิดชอบแก้ไขทันเวลาหรือเปล่า” ถ้าคำตอบคือไม่มีใครดูแลต่อ ระบบสำเร็จรูปที่มีทีมซัพพอร์ทรออยู่ตลอดอาจเหมาะกับร้านมากกว่าจริง ๆ — เล่นนี้ไม่ได้บอกว่าทำเองดีกว่าเสมอ แต่บอกว่าต้องรู้ว่าตัวเองแบกรับอะไรอยู่

❌ พลาดบ่อย / ✅ ทำแบบนี้

❌ พลาดบ่อย: รัน migration ที่ลบคอลัมน์หรือเปลี่ยนชนิดข้อมูลในรอบ deploy เดียวกับโค้ดใหม่ตอนร้านกำลังเปิด ✅ ทำแบบนี้: แยกเป็น expand-contract — เพิ่มของใหม่ก่อน deploy โค้ดที่ใช้นั้น แล้วค่อยลบของเก่าในรอบถัดไปเมื่อไม่มีใครอ้างถึงแล้ว

❌ พลาดบ่อย: มี backup อัตโนมัติทุกวันแต่ไม่เคยลองกู้คืนดูจริงสักครั้ง ✅ ทำแบบนี้: ซ้อมกู้คืนบน staging เป็นประจำ บันทึกเวลาที่ใชัจริง (RTO) และข้อมูลที่สามารถสูญหายได้ (RPO) ไว้เป็นลายลักษณ์อักษร

❌ **พลาดบ่อย:** คิดว่าต้องสร้างโหมดย่อไฟไลน์เต็มรูปแบบตั้งแต่เวอร์ชันแรก เพราะกลัวเน็ตหลุด ✅ **ทำแบบนี้:** เริ่มจาก fallback สามระดับที่ทำได้จริง (ใบครัวสำรอง, โหมดย่อไฟไลน์จำกัด, จดมือแล้วค่อยตาม) ก่อนลงทุนกับ sync แบบเต็มรูปแบบ

❌ **พลาดบ่อย:** ส่งมอบระบบให้ร้านพร้อมเอกสารเทคนิค 20 หน้า แล้วถือว่าเทรนเสร็จแล้ว ✅ **ทำแบบนี้:** เทรนจับมือทำจริงหน้างาน 1 ชั่วโมงตอนร้านเปิด พร้อมคู่มือ 1 หน้าติดข้างเครื่อง POS

จำ 3 ข้อ เลือกโฮสต์ตามข้อจำกัด realtime ที่เลือกไว้แล้ว (SSE ไปกับ managed platform ได้ WebSocket ต้องมี infrastructure เอง) ไม่ใช่เลือกก่อนแล้วค่อยมาแก้ปัญหา Migration บนร้านที่เปิดอยู่ ต้องเป็น expand-contract เสมอ และ backup ที่ไม่เคยข้อมกู้คืนไม่ใช่ backup ที่ใช้ได้จริง ระบบต้องมี fallback เวลาเน็ตหลุด และการส่งมอบต้องรวมคนที่ดูแลต่อ ไม่ใช่แค่โค้ดที่ deploy สำเร็จ

แหล่งอ้างอิง

- Bank of Thailand / EMVCo — Thai QR Payment (ภาพรวมมาตรฐาน): https://en.wikipedia.org/wiki/Thai_QR_Payment
- แกะสเปก PromptPay QR (Blognone): <https://www.blognone.com/node/95133>
- promptpay-qr — JS library เจน payload PromptPay: <https://github.com/dtinth/promptpay-qr>
- promptparse — all-in-one JS library สำหรับ PromptPay & EMVCo QR: <https://github.com/maythiwat/promptparse>
- Bangkok Bank Developer Portal — QR Payment API: <https://apiportal.bangkokbank.com/en/api/qr-payment>
- Omise / Opn Payments — Pricing Thailand: <https://www.omise.co/en/pricing/thailand>
- Omise — Payment Gateway Thailand (docs): <https://docs.omise.co/home/thailand>
- Idempotency in Payment APIs (Stripe / Omise / 2C2P): <https://simplico.net/2026/04/04/idempotency-in-payment-apis-prevent-double-charges-with-stripe-omise-and-2c2p/>
- กรมสรรพากร — ระบบ e-Tax Invoice / e-Receipt: <https://etax.rd.go.th/>
- กรมสรรพากร — ประกาศเกี่ยวกับภาษีมูลค่าเพิ่ม ฉบับที่ 46: <https://www.rd.go.th/3394.html>
- ใบกำกับภาษีอย่างย่อ (ABB) ที่ถูกต้องตามกฎหมาย: <https://www.humansoft.co.th/th/blog/tax-invoice-abb>

- Next.js — Mutating Data (Server Actions): <https://nextjs.org/docs/app/getting-started/mutating-data>
- Streaming in Next.js: WebSockets vs Server-Sent Events: <https://hackernoon.com/streaming-in-nextjs-15-websockets-vs-server-sent-events>
- node-thermal-printer (ESC/POS ผ่าน Node.js): <https://github.com/escpos>
- PCI SSC — ข้อผูกพันของ e-commerce merchant ที่ outsource การรับชำระเงิน:
<https://www.dwt.com/blogs/privacy-security-law-blog/2025/03/pci-faqs-card-processing-ecommerce-merchants>
- เปรียบเทียบระบบ POS ร้านอาหารในไทย 2026 (Qashier):
<https://qashier.com/th/blog/2026/06/22/thailand-pos-compare/>